

PROGRAMMING LANGUAGE PRAGMATICS

Jason: 7/10

Jason: 7/10

PROGRAMMING LANGUAGE PRAGMATICS

Christos Dimoulas & Matthias Felleisen

©2024–2026 Christos Dimoulas & Matthias Felleisen

This work is licensed to the public under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 license (international):

<http://creativecommons.org/licenses/by-nc-nd/4.0/>

All rights reserved except as licensed pursuant to the Creative Commons license identified above. Any reproduction or other use not licensed as above, by any electronic or mechanical means (including but not limited to photocopying, public distribution, online display, and digital information storage and retrieval) requires permission in writing from the publisher.

This book was set in Scribble, Racket, and LaTeX by the authors.

Library of Congress Cataloging-in-Publication Data

Names: Dimoulas, Christos and Felleisen, Matthias.

Title: Programming language pragmatics
/ Christos Dimoulas and Matthias Felleisen.

Identifiers: LCCN 2017018384 | ISBN 9780262534802 (pbk. : alk. paper)

Subjects: LCSH: Computer programming. | Electronic data processing.

Classification: LCC QA76.6 .H697 2017 | DDC 005.1/2–dc23

LC record available at <https://lcn.loc.gov/2017018384>

10 9 8 7 6 5 4 3 2 1

Jason: 7/10

Jason: 7/10

Contents

	Preface	xv
I	The Study of Programming Languages	1
II	Syntax and Parsing	13
III	Semantics and Execution	33
IV	Syntax and Validity	53
V	Nested Blocks	71
VI	Structued Data and Abstraction	93
VII	Modules	129
VIII	Types	145
IX	Mixing Typed and Untyped Code	193
X	Programming Language Pragmatics	233

The Pragmatics Questions

1. Why should you study programming languages?
2. What is an executable model of syntax?
3. What is an executable model of semantics?
4. Why do languages demand variable declarations?
5. Why should programmers declare variables close to their use?
6. Why do languages support structured data and abstraction?
7. Why do languages come with modules?
8. Why do languages include a type notation and a type checker?
9. Why do some languages support a mix of typed and untyped code?
10. What happens as a program allocates more and more memory?

Detailed Contents

Prelude	xv
This Book and Its Path Through History from 1960 to 2010 . . .	xvi
Dear Instructor:	xvii
Dear Reader:	xix
I The Study of Programming Languages	1
1 The Pragmatics Question	1
2 Studying Programming Languages: Why	2
3 Studying Programming Languages: How	2
3.1 What Makes a Programming Language	3
3.2 Models of Programming Languages	4
3.3 Syntax: From Essence to Models	4
3.4 Semantic Models	6
3.5 The Covers of Books, or It's Just Semantics	7
3.6 Pragmatics	8
II Syntax and Parsing	11
1 Concrete Syntax is Mostly Irrelevant	11
2 BNF Grammars for Describing Grammars	12
3 The Parsing Process and the Parser Function	14
4 The Pragmatics Question	15
4.1 Design Choices	15
4.2 Costs and benefits	17
5 Parsing: In General	17
5.1 A Parser Returns an Abstract Syntax Tree	21
6 Parsing: An Example	22
7 What is a Model	27

8	<i>Bare Bones: a Parsing Project</i>	29
8.1	Project Tasks	29
III	Semantics and Execution	31
1	Meaning Matters	31
2	Abstract Machines: In General	32
3	The CS Abstract Machine: The <i>Sample-</i> Language	35
3.1	The States	35
3.2	Exceptional Cases	36
3.3	The Transition Function	37
3.4	An Implementation of the CS Machine for <i>Sample-</i>	42
4	The CSK Abstract Machine: A Second Example	46
4.1	The States	47
4.2	The CSK Transition Function for <i>Sample-</i>	47
4.3	But Machines Don't Search For the Next Instruction!	49
4.4	A Little Bit of Theory: The Equivalence of Semantics	50
5	<i>Bare Bones: an Abstract Machine Project</i>	51
5.1	The CSK Meaning of Conditionals and Loops	52
5.2	The Meaning of Numbers, Errors	53
5.3	Project Tasks	55
IV	Syntax and Validity	57
1	The Pragmatics Question	57
1.1	Design Choices	58
1.2	Costs and Benefits	60
2	Valid Syntax: In General	61
3	Valid Syntax: An Example	63
3.1	The Implementation	67
3.2	An Implementation Suitable for an IDE	70
4	Static Checks Eliminate Dynamic Checks	71
4.1	A Bit More Theory: Soundness of Validity Checking	71

5	<i>Declared: a First Complete Model</i>	73
5.1	Project Tasks	73
V	Nested Blocks	75
1	The Pragmatics Question	75
1.1	Design Choices	75
1.2	Costs and Benefits	76
2	The Syntax of Local Variable Declarations, Scope	77
2.1	Grammar and Parsing	77
2.2	Scope	78
2.3	Validity Checking	81
3	The CESK Abstract Machine: Local Declarations	82
3.1	The States	82
3.2	The Transition Function	85
4	Core	89
4.1	Why Core? Why CESK?	92
4.2	Project Tasks	93
5	Scope in the Real World	94
VI	Structured Data and Abstraction	95
1	The Pragmatics Question	95
1.1	Design Choices	96
1.2	Costs and Benefits	97
2	<i>Classy: the Syntax</i>	100
2.1	<i>Classy: the Grammar</i>	100
2.2	<i>Classy: Scope</i>	102
2.3	<i>Classy: Validity, the Easy Part</i>	103
2.4	Undecidable Problems and Validity	105
2.5	<i>Classy: Validity, the Undecidable Part</i>	107

3	<i>Classy: the Semantics</i>	108
3.1	Representation of Objects	108
3.2	CESK States for <i>Classy</i>	110
3.3	The CESK Transition Function: Adapting the Cases for <i>Core</i>	111
3.4	Creating and Checking Instances	112
3.5	Field Access	113
3.6	Field Mutation	115
3.7	Method Calls, the Old Way (1959)	117
3.8	Method Calls, the New Way (1971)	119
4	Project Tasks	121
5	Pragmatics: Developers Must Learn to Think Hard	122
5.1	Dynamic Extent vs Lexical Scope	123
5.2	Object Aliasing	125
5.3	Equality	127
5.4	Call-by-value Prevents Variable Aliasing	129
5.5	Dynamic Dispatch	132
VII	Modules	135
1	The Pragmatics Question	135
1.1	Design Choices	136
1.2	Costs and Benefits	138
2	<i>Module: the Syntax</i>	139
2.1	<i>Module: the Grammar</i>	140
2.2	<i>Module: Scope</i>	141
2.3	<i>Module: Validity</i>	144
	<i>Module: Validity Rules</i>	144
	<i>Module: Validity Checking</i>	145
3	<i>Module: Linking</i>	147
3.1	Project Tasks	148
4	Pragmatics: Computability vs Expressive Power	149
VIII	Static Types	153

1	The Pragmatics Question	153
1.1	Design Choices	156
1.2	Costs and Benefits	158
2	Types: the Syntax	161
2.1	Types: the Notation	161
2.2	Types: the Grammar	162
2.3	Types: Validity	163
3	Type Checking	164
3.1	Contextual Information and Conventions	165
3.2	Type Judgments	167
3.3	Rules for Deriving Type Judgments	167
3.4	Implementing a Type Checker	176
3.5	Project Tasks	180
4	Static Types Eliminate More Dynamic Checks	182
4.1	The Final Bit of Theory: Type Soundness	182
5	Pragmatics: Static Type Systems	184
5.1	The Costs and Benefits for the Language Creator	185
5.2	The Benefits for the Software Developer	187
5.3	How to Defeat a Programming Language	191
5.4	The Costs for the Software Developer	193
6	The Costs and Benefits of Types: The Warning	197
IX	Mixing Typed and Untyped Code	201
1	The Pragmatics Question	201
1.1	Design Choices	202
1.2	Costs and Benefits	204
2	A Type System for Untyped Languages	207
2.1	A Type System for Untyped Idioms	207
2.2	Integrating Typed with Untyped Code	213
3	Mixed: the Syntax	216
3.1	Mixed: the Grammar	217
3.2	Mixed: Validity	218

3.3	<i>Mixed</i> : Type Checking	218
3.4	A Note on Types and Semantics	220
4	<i>Mixed</i>: the TypeScript Semantics	221
4.1	Project Tasks	221
5	<i>Mixed</i>: the Typed Racket Semantics	222
5.1	Linking in Mixed-Type Setting	223
5.2	A CESK Machine Based on Proxies	224
5.3	Project Tasks	232
6	The Very Final Bit of Theory: Real Type Soundness	234
7	Pragmatics: Migratory Type Systems	234
7.1	The Two Semantics with Concrete Examples	235
7.2	Performance	237
7.3	Testing, Debugging, and Deployment	239
X	Memory Management	241
1	The Pragmatics Question	241
1.1	The Nature and Purpose of this Chapter	242
2	Design Choices	242
2.1	Memory from the Point of View of a Model, Take 1	243
2.2	Memory from the Point of View of a Model, Take 2	243
2.3	Manual vs Automatic Memory Management	244
2.4	Costs and Benefits	246
3	Pragmatics of Memory Management	250
	Postlude	xxiii
	Pragmatics in Action	xxiii
	Where to Go From Here	xxviii
	Acknowledgments	xxx

Prelude

Most undergraduates enroll in computer science majors because they wish to enter the workforce as software developers. Most undergraduate curricula in computer science come with a course on the principles of programming languages or compilers. The juxtaposition of the two preceding sentences should raise the question how these courses serve the enrolled students in the future that they envision.

This book answers this question with a novel take on the programming languages course. It presents a framework for evaluating the pragmatics of programming languages. *Pragmatics*, a term borrowed from natural-language linguistics, is the study of programming-language use in a work context. In other words, the book asks how a programming language's nature affects the working software developer during design, code creation, testing, debugging, performance evaluation, deployment, and many more such situations.

The seemingly related word "pragmatic" is an adjective and a synonym for "practical." The word "pragmatics" is a noun with technical meaning that is only loosely related to "practical."

An understanding of pragmatics presupposes a rudimentary understanding of the workings of a programming language. Following the typical classification, the book covers syntax and semantics. Like for a natural language, the *syntax* of a programming language consists of its alphabet, its vocabulary, and its grammar. It does include context-sensitive constraints between distant parts of a program, say a variable declaration and references to this variable. While developers often react to properties of syntax when they encounter a language, *semantics*—the meaning of code—is what really matters when it comes to pragmatics. This book uses abstract state machines to present semantics. Roughly speaking, these abstract state machines resemble the virtual machines that many language implementation use nowadays, though with much less-detailed instructions.

Generally speaking, the book avoids details with the use of executable models of languages, not real implementations. In science, a *model* distills reality to a collection of essential elements. As such they approximate reality and leave it to the reader to imagine how properties of the model apply to the real world. By its very nature, computer science can rely on *executable* models, that is, programs that run and that readers can experiment with. We consider experimentation important and encourage readers to branch out from our chosen path through the language landscape as they explore this oldest of all fields in computing.

It leaves this aspect to courses on compilers.

This Book and Its Path Through History from 1960 to 2010

The book consists of ten chapters, and they present one path through the vast landscape of programming language design. Roughly speaking the path starts with the simplistic languages of the late 1950s, which enabled programmers to write code consisting of assignment statements, conditionals, and loops. Considering that many introductory programming courses teach these parts of the chosen language first, this starting point should be familiar to all readers.

While the first three chapters introduce this language model plus the tools needed to understand the construction of models, each of the remaining seven chapters responds to a pragmatics question that the preceding chapters raise. A basic example concerns the use of variables. Like Python, this first model does not require programmers to declare variables before use. Although this design may look like a simplification for beginners, it has dire consequences for the programmer who accidentally misspells a variable name on the left-hand side of an assignment statement, causing test failures and imposing additional debugging work. It is precisely this kind of relationship that illustrates so clearly and so simply what pragmatics means. Hence we add variable declarations to the model and introduce the very first form of context-sensitive checking of syntax.

Another example of a question of pragmatics concerns data representations of information. The first model deals with just one form of data, rational numbers of a small range, which is historically accurate but unrealistic. Contemporary languages come with mechanisms for representing all forms of information as data so that programmers do not need to encode everything as a number. In response the book adds classes and objects to the model, which still make up the dominant tool of choice for most developers. This addition advances the story to the late 1960s and early 1970s.

Following the historical development, the book covers modules and type systems next. The first is a simplistic approach to separate code development and compilation, plus the specification of explicit dependencies. The second explains types as the most stringent form of context-sensitive constraints on the syntax of programs.

Over time, this path prepares readers for the study of a model of languages that showed up in the 2010s: TypeScript and Typed Racket. The key novelty is that both of these enable developers to combine typed and untyped pieces of code into a single program. While their type systems resemble each other, their semantics differs radically. As TypeScript's documentation says code can be understood as JavaScript after erasing the

The addition does not include inheritance, which is, for example, what we mean when we say the book uses models.

type declarations. By contrast, Typed Racket compiles type declarations into run-time checks to ensure that the interaction between typed and untyped code is semantically coherent. Such a setup—one syntax, two different semantics—is ideal for a comparative and multi-faceted case study of pragmatics.

The last chapter explains how the reader can apply pragmatics to professional life. Every student is going to encounter some languages in college and different ones as a professional. Indeed, professionals are unlikely to work with just one programming language for their entire careers. In other words, software developers must acquire new languages time and again. Experience tells us that they do so by harking back to the languages they know, and that means they compare and contrast languages. From this perspective, this book teaches readers a framework that helps them organize their understanding of languages:

organize language comparisons along the many steps of the software development process.

It yields workable and nuanced insights concerning the daily work of a software developer.

Dear Instructor:

If you choose to use this book as a text for a programming language course, let us make a number of suggestions concerning prerequisites and course organization.

Two *prerequisites* matter most. First, the students should come into the course with a reasonable understanding of how to design structurally recursive functions. To simplify the model of syntax, we use semi-structured data, specifically S-expressions, and present a recursive-descent approach to the implementation of grammars as parsers. The same principle applies to checking syntactic constraints, from declaration checking all the way to type checking. If students' background in the design of recursive functions is lacking, we urge you to spend the first few lectures, including the first homework assignment, on a "crash course" concerning this topic.

Second, our models use abstract state machines instead of interpreters. This choice avoids recursion in favor of a plain loop that iterates state transitions until a final state is reached. Furthermore, it equips students with an idea of, say, the Java Virtual Machine, or similar virtual machines, which are now common and which they need to grasp at an intuitive level as future professionals. The key to keeping the corresponding code concise is

The first two parts of How to Design Programs, Second Edition, MIT Press provide a suitable introduction, especially those sections whose title contains the word "design."

If you are more comfortable with interpreters, it should be straightforward to formulate the semantics exercises in terms of those instead of machines.

to use algebraic pattern matching, a feature that most contemporary languages support to some extent. Students should be comfortable with pattern matching before they work on the sophisticated models of the book:

- If you allow students to use the language of their choice, point them to the respective pattern-matching constructs, e.g. `switch` in Java or `match` in Python and Racket.
- If you choose a programming language for your students, keep pattern matching in mind. (Typed) Racket supports it in the most suitable manner; OCaml is a close second.

It is appropriate to include corresponding exercises with the first homework set.

As for the *organization of the course*, we make five major recommendations. First, the course calls for an interactive approach. We introduce each model modification or enhancement via a discussion of the question of pragmatics. To prepare students for these discussions, each milestone should include a short written answer to the corresponding question of pragmatics. One way to start the discussion is to ask a student to read this essay to the class and to challenge students to react to it. Alternatively, just open the floor for contributions based on the essays.

Second, as indicated, the course benefits from a semester long series of project assignments that build on each other. All chapters following chapter III contain a section that collects the relevant exercises to extend the model of the preceding one. We have arranged these exercises into both semester-long projects and quarter-long projects. The exercises serve two purposes: they deepen the students' understanding of syntax and semantics, and they are the basis of in-class code presentations.

Third, while in-class code presentations seem to consume a lot of lecture time, they also enrich a course on the pragmatics of programming languages in a significant manner. Whether all students use an instructor-chosen language or their language of choice, the presentations end up differing from each other. Time and again, these presentations will illustrate how the programmers work *against* the language or have to find work-arounds due to expressiveness gaps. It is the instructor's job to spot such problems and to point them out, because they help with discussions of pragmatics. Additionally, these presentations help students acquire and practice the terminology they need to explain their designs and to discuss programming languages.

You may also wish to have students vote on which semi-structured data to use. While S-expression comes with several advantages, JSON or XML work almost as well.

Fourth, our experience suggests that students welcome a free choice of language. Students feel empowered, and they get to see different approaches to the exercises in a variety of languages. This exposure to languages they do not know or do not even know about is often a wake-up call, anticipating what they will experience as professional developers, namely, that they have to read code in an unknown language. Furthermore, each language offers different levels of support, and hence, students will see many different ways of approaching the same problem.

One final recommendation concerns the chosen path and end point of our exploration through the language landscape. The initial model of the core language essentially serves as a warm-up for the ones with interesting features, such as classes, modules, type systems, and so on. By contrast, the choice of covering the pragmatics of TypeScript versus Typed Racket matches students' current needs. The former language has taken the professional world by storm, and the students should understand what kind of pragmatics thinking went into the design of this language. Although we expect that TypeScript is going to play this role for some years to come, the professional world is quick to change, and instructors should adapt.

In this spirit, we are going to experiment with different forks in the road from the 1960s to our time. Here are some natural points of departure. Our model of classes omits inheritance, but an instructor may develop and *work* with models that compare and contrast single inheritance with multiple inheritance. Such a project milestone can explain to students why single inheritance has become the default. Likewise, the book keeps the model of a module-oriented language as simple as possible, even though the idea of modules fascinated language researchers for a large part of the 1980s and 1990s. You may wish to enrich this model so that students get a sense of why Java added modules, too. Many other possibilities exist, and we are going to explore them and post our results on the web page that comes with this book.

Dear Reader:

You do not have to be a student to read this book. Experienced software developers can benefit from an understanding of the pragmatics of programming languages as presented here. What it does take is (1) experience with several programming languages and (2) a willingness to work through the book and its numerous exercises. The next time you tackle a project written in an unknown language you might just approach it with an improved and sharpened understanding of how to go about acquiring a sufficiently deep

appreciation of this new language, how it can assist with the various software development tasks and what it means to program *with* the language.

Jason: 7/10

Jason: 7/10

Jason: 7/10

Why should you study programming languages?

1 The Pragmatics Question

Software developers create software systems with programming languages and their tool chains. They edit code in an IDE, which, in turn, provides feedback as the code evolves. They read code and they rely on properties of the language to help them with this task; for example, a reference to a variable comes with a declaration or is marked as undeclared. They add test suites and run them; the IDE indicates which tests fail. They use failing test cases to start the bug locating process, using the debugger to set breakpoints so that they can see where execution goes wrong. They change the code to fix bugs once found. They measure the performance of systems, and if it is found wanting, they try to replace low-performing pieces with faster ones. During this entire process, software developers engage with the chosen programming language, and the question they should have is how well the language and its tools serve them in each of these situations.

Linguists have studied natural languages and their use in various contexts for even longer than programming language researchers have studied their subjects. They recognize this description immediately. They have a word for it: *pragmatics*—short for “use of language in context.” Linguists appreciate the central role that pragmatics plays in understanding a language and its role in people’s lives.

We consider pragmatics equally important for future software developers, and this text book presents a first take on programming language pragmatics. It follows the historic development of programming languages from the early days of programming to the early years of the 21st century. The focus is on the key elements of languages and how they facilitate the work with code: creating it, reasoning about it, deploying it. Let’s take a close look at our rationale first.

Our use of “pragmatics” slightly differs from the use in linguistics.

2 Studying Programming Languages: Why

When the weather app on your phone tells you in the morning that it is going to rain today with a 90% probability, you can't do much about the rain. Similarly, when a bank announces an increase in the price of money—also known as interest rate—your parents can't change the bank's mind just because they need to get a mortgage now. And when a manager tells a newly hired junior developer that the team's chosen programming language is PLX, then this new person can't just start writing code in some different language YPL, because it is fashionable right now.

But, in all of these situations, you can act consistently with the new information. As they say, there is no bad weather, there's only bad clothing. So, if the app says it's going to rain and you have to go somewhere on foot, you will grab a rain coat or an umbrella. And if your parents are really into buying a home with a mortgage, they will pay attention to what their mortgage broker says about expected rates, and they may accelerate their home purchase or abandon it. Put differently, learning and following basic rules of thumb helps people navigate life situations, small and large.

As a newly hired junior software developer you will need to use those skills to adapt to the choices that your team has already made. For example, they will have chosen a programming language. If you do not know the language, you need to learn it, and to pick up a language means acquiring a sufficiently solid understanding of its vocabulary, its grammar, and the meaning of those. While picking up the syntax is often easy, wrapping your head around the exact meaning of features and the role they play in various work situations is hard. So, if you enter professional life equipped with a framework on how to approach these two difficult subjects, you will have a head start. How to study these three aspects of a language then has to be our next topic.

3 Studying Programming Languages: How

Programming language researchers use models to study programming languages. In this, they follow standard scientific procedure. Roughly speaking, a model reduces an object of study to those properties that a researcher wishes to understand in depth—without eliminating properties that would invalidate any results for the actual object. Put differently, the goal is to answer questions about the object via the model so that these answers are of some value in the actual world.

One great example is physics. In many cases, a physicist can think about moving objects as if they were simple points, even though the object may experience friction due to its physical extent. Similarly, a system biologist may act as if DNA were just a sequence of “letters” (A, C, G, and T), but these letters are nucleotide bases with chemical and physical properties. Models are also of use in social sciences. Classical economists assume that human beings act rationally as participants in economic activities, and they do so with some success for macro-level predictions. But before we can discuss language models, we must first explain what the basic properties of languages are.

3.1 What Makes a Programming Language

A *programming language* is an artificial language. Like a natural language, a programming language has three properties that a programmer must understand to use it properly:

- *Syntax* consists of the vocabulary and the grammatical rules. These rules determine which sequences of characters form a “word” and which sequences of “words” are *well-formed* sentences. A syntax specification may also impose *validity* constraints on well-formed sentences before one of these is accepted as an actual program.
- *Semantics* assigns meaning to programs and, implicitly, to its pieces. This meaning is typically specified via a natural-language document; a few programming languages come with mathematical specifications, based on the research of “semanticists.” A software developer, however, has to accept whatever the chosen implementation of a language does when presented with a program. Language researchers tend to call the latter “behavior” to differentiate it from the specified and thus *desired* semantics of the language.
- *Pragmatics* is about the usage of a language in specific work situations. This wording covers the creation of programs, the reading of code, the search for bugs, and so on. Unlike syntax and semantics, pragmatics remains an imprecise term and the idea of pragmatics has received much less attention from researchers than syntax and semantics—even though it clearly matters most to the working software developer.

3.2 Models of Programming Languages

Over time, researchers have developed a number of language models:

- informal descriptions, using natural languages;
- logic and mathematics; and
- executable programs.

All of these share the key property of models, namely, that they do not cover the entire language to be studied. They tend to restrict the syntax to essential features, and they tend to formulate a semantics that deals with a wide range of cases, but not all of them.

Each of these approaches has advantages and disadvantages. While a natural language description of a programming language can be used for an entire language, it necessarily comes with gaps. By their very nature, natural languages tend to allow ambiguous formulations. A logical or mathematical model is usually extremely exact, but it tends to describe a small, even minuscule fraction of any given real language. An executable model of any aspect of a programming language has the advantage that it runs and that it is amenable to quick explorations for specific cases. If the source code is available, a programmer can even explore alternatives to the chosen design. At the same time, writing a program to understand a fragment of a programming language means that an understanding of one language partially depends on an understanding of another one. For this reason, this text uses executable models while borrowing elements from the mathematical side. This mix allows for making precise models without tying them too closely to the chosen implementation language.

Implementing executable models is also a good way to reinforce good programming habits.

3.3 Syntax: From Essence to Models

People are told not to judge a book by its cover, but beginning programmers, as well as experienced ones, routinely do. The “cover” of a programming language is the program text that developers enter into an IDE or that an AI agent reads and a programmer must comprehend. Different language creators make different assumptions about which “letters” make up the *vocabulary* of the words of programs. And, these assumptions have a direct impact on developers. Some opt to allow only sequences of alpha-numerical characters as the “letters” of the vocabulary; others accept that developers wish to use `fahrenheit->celsius` as the name of a temperature conversion function.

Similarly, language creators also make assumptions about how developers wish to compose words into sentences. It starts with simple expressions, that is, the *grammatical rules* that govern how individual words are composed into expressions. Some creators think that the infix notation of algebra, which only emerged over the past couple of centuries, elegantly generalizes to programming, where developers define new operators all the time. Thus, they allow developers to write

```
x -> f ++ k
```

and, implicitly, assume that all future readers of such expressions can easily decipher the order in which the two operations are evaluated. Others prefer to have developers spell out this order of operations with parentheses so that the same expression may look like this:

```
(++ (-> x f) k)
```

After all, all students in elementary schools know that parentheses clarify operator precedence. People also intensely disagree whether `;` is a statement terminator or separator; others want TAB characters to signify something about programs; and the parentheses lovers don't care because parentheses and a bit of white space make it all obvious.

A student of programming languages must ignore the specific differences of /syntax and focus on the commonalities. This focus is a step from concrete examples to general ideas, insights that should be useful across languages. Concerning syntax, language researchers figured out two such general insights by the 1960s:

1. The first one is a sufficiently general technique for describing vocabularies and grammars. In essence, this technique—called *Backus-Naur Form* (short BNF)—is a standard grammar for defining grammars.

A logical-mathematical way of expressing this idea is that a grammar consists of a collection of n equations in n variables. Unlike the equations we know from high school and college courses, solutions to these grammar equations are sets of terms.

2. The second concerns the description of syntax with respect to questions concerning semantics. Whether a language uses prefix, infix, or postfix notation for an addition expression is simply irrelevant. What matters is that (1) the description brings across that it *is* about addition, (2) how many operands there are, and (3) how to refer to the operands. This idea is referred to as *abstract syntax*.

Chapter II expands on these insights and provides concrete examples. It explains how these two insights are the basis of a straightforward implementation of syntax checking, that is, an executable model of syntax.

3.4 Semantic Models

English speakers are familiar with the phrase “it’s just semantics,” typically used to dismiss an argument because the differences seem trivial. But, people who know that “semantics” is a fancy word for “meaning” must be stumped by this usage. Of course, it’s all about “meaning” and developers are perfectly aware of this fact. They know that it matters whether

an array reference enforces the container’s boundaries.

If it doesn’t, an array reference can cause a program to segfault when it uses a negative index or an index that exceeds the length of the array. Alternatively, the lookup succeeds and retrieves random bits that masquerade as an array element—and the code may appear to terminate normally and output proper results. This kind of semantics has posed major security problems for decades. By contrast, a boundary-enforcing array reference signals a run-time exception for such use cases, preventing catastrophic misinterpretations. In short, semantics very much matters.

Inspired by the success of investigations into syntax, programming language researchers struggled for quite some time to come up with a simple mathematical framework for describing the meaning of programs. Here are the most well-known, in historical order:

- An interpreter is an executable model, realized via a recursive function from syntax to values in a suitable programming language.

Note The resemblance between interpreters and the meaning function of denotational semantics tends to mislead superficial readers. A *denotational semantics* consists of a *domain*, which is a set of mathematical elements that satisfies some algebraic and topological properties, and a function from syntax to the domain. While this meaning function looks like an interpreter, its definition utilizes plain mathematics so that translating it into a programming language requires care.

- An *abstract machine* consists of two pieces: (1) a (large) set of states and (2) a transition function that maps one state to another.

Instead of the sequence of all bits in a hardware machine where each feasible sequence of 0s and 1s describes a possible state, the an ab-

stract machine directly uses the syntax of the modeled language (expressions, statements) as instructions plus some auxiliary data structures. The syntax serves as instructions, the auxiliary data structures keep track of data. Some states are marked as initial, others as final.

The cases of the transition function correspond to machine instructions. For final states, the transition function is undefined.

Given a program, it is loaded into the machine, which puts it into an initial state. Then the transition function is applied until the machine is in a final state. This final state is unloaded to yield an “answer.”

- *Operational semantics* combines ideas from the preceding approaches.

None typically cover a complete syntax, only full implementations do.

Of these approaches, the interpreter—by definition—and the abstract machine approach lend themselves most easily to implementation. Implementing an interpreter or abstract machine also yields a first impression of how much work it will be to build a performant implementation. Most importantly, with an implementation, people can check whether programs behave in the expected manner; they can explore corner cases of behavior; and they can begin to answer questions about how software developers can use language elements in various work situations.

Chapter III introduces abstract machines, and chapter IV presents a refinement that can easily cope with other features. We choose abstract models because they make it easy to decouple the meaning of the studied language from the chosen implementation language.

3.5 The Covers of Books, or It’s Just Semantics

Given some understanding of syntax and semantics, we can explain with a single word why the former is less meaningful than the latter:

US English	German
billion	Billion

At first glance, the two words from two different natural languages seem to correspond to each other. So if someone were to offer you a billion pennies, you would probably not worry about which *syntax* the person uses.

It turns out, however, that the semantics of these words differs radically:

US English	German
billion	Billion
1,000,000,000	1,000,000,000,000

Imagine this. The *semantics* of the two words differ by a factor of 1,000, which even at the level of pennies, makes a serious difference.

Stop! In which natural language would you like to get this offer now?

When someone tells you next time "it's just semantics," ask for a Billion dollars.

This difference is what we refer to when we say "don't judge the language by its syntax." While the way you read and write the words of a program clearly affects your abilities as a programmer, an easy-to-grasp semantics of the syntax matters even more than the plain words. And how the combination relates to the productivity of developers in certain work situations is "pragmatics," the final topic of this first chapter.

3.6 Pragmatics

Programmers notice questions of pragmatics when they chaff against the language, and the problem they notice is neither one of syntax nor semantics. To make this concrete, consider the widely used syntax of $a[i]$, i.e., a reference into an array. Next recall the two possible meanings, formulated in terms of a machine controlled by program expressions and a memory:

- $a[i]$ checks whether i is between 0 and the upper boundary of a . If so, it retrieves the corresponding element; otherwise it stops the executing machine or interpreter and signals a problem.
- $a[i]$ directs the machine to retrieve the bits from the memory location whose address is $a + i$ if a is the address where the 0th element of the array is located. Notice how this computed address may or may not point to an actual element inside of the array. Once these bits from inside or outside the array are available, the machine continues with the next instruction.

While the syntax is the same, the semantics differs radically.

Over the past few decades, programmers have expressed frustration with both, depending on the kind of work they perform:

- During the editing of code, what matters is whether the sequence of characters entered into the IDE form a well-formed and valid sentence. Since the syntax is the same for both meanings, it appears that writing down an array reference isn't affected by the choice of meaning. But, now imagine a developer who works in a language that does not check the index and who wishes to ensure that $a[i]$ always points to a cell inside of the array. This programmer must work around the chosen semantic choice during the editing process, because the only

way to protect potentially faulty references is to insert a conditional that performs the appropriate checks.

- Next, consider a software developer tasked with measuring the performance of some code base. The developer runs the program and notices that it doesn't run as fast as expected. It turns out that the most frequently executed code consists of a nest of loops with many array references in the body of the innermost loop. In this case, it is quite possible that a boundary-checking implementation of array references causes the bottleneck. That is, when it comes to performance, a developer might wish the language didn't check array references.
- When the program is tested, a quality assurance engineer notices an unusual output. Although the program does not crash, its answers just seem off. The engineer may now question whether a non-checking array reference is going wrong and interprets the retrieved bits mistakenly as if it were an array element.

This developer would clearly prefer the bounds-checking variant of array references. Alternatively, the developer should inspect all array references and manually wrap them with boundary checks as needed. That is, a developer can take on the work of implementing the checked variant based on the unchecked one, if needed.

In sum, depending on the work situation, a programmer may chaff against whatever the language creator chose as the semantics of `a[i]`. Since the syntax is the same in both cases, it is not a syntax problem. And similarly, the language creator clearly specified the semantics of `a[i]`, so it isn't a semantics problem either. Instead, it is a problem of working with array references in various situations.

Models of syntax and semantics enable researchers to explore questions of pragmatics. Executable models help, because researchers can interact with them and rapidly explore alternatives. By comparing alternative models, a researcher can study their impact on various work situations. In particular, a researcher can now ask two different, but related sets of questions. The first set concerns the costs and benefits to the person who uses the language. The second one is of interest to the implementer of the language, that is, the person who builds a robust and performant implementation. The answers to the pragmatics question may suggest changes to syntax or semantics plus the use of information computed by an implementation.

Let's assume the implementer wishes to write a compiler for a language with array references. Here are critical considerations:

- A translation of a non-checking array reference is straightforward. Assuming a is represented as a memory address and i denotes an offset, the compiler should issue an instruction to load the bits at memory location $a + i$.
- Without any further information, the compilation for the alternative meaning differs sharply. The compiler cannot represent an array as just an address; it must store the size of the array somewhere. This number must be retrieved and compared to the i -defined range. If the comparison succeeds, the retrievable of the bits can proceed in a manner similar to the first case; otherwise the sequence of assembly instructions must jump to a place that reports the error.
- The phrase “without any further information” is suggestive. What if the compiler could “think” about the program and determine up front that some particular array reference will always succeed? That is, the generated code does not need to perform the bound checking. Next the author of the compiler must consider (1) how difficult it is to implement this “thinking,” (2) how safe this thinking is, and (3) how much it slows down the translation process.

For many decades, the decision almost always came down to what was easiest to implement—until people discovered that unchecked array references are a major source of safety and security flaws. In response, the work of checking array references shifted from the many software developers using the language to the few implementers.

We ignore the case when two forms of syntax have the same meaning, because we consider this one mostly a question of taste.

Questions of pragmatics aren’t always as clear-cut as the one for array references when one syntax has two different semantics. Design alternatives may require a different syntax and different semantics. Furthermore, there are many different work situations, not just those of editing code, observing performance, and locating bugs. For example, a student in a introductory programming course *is* in a work situation, a situation that radically differs from those just mentioned. Try to imagine some other unusual work situations.

The rest of the book focuses on those questions of pragmatics that concern the working developer. The expectation is that the series of examples help (future) developers create an informal framework for approaching programming languages and evaluating features of programming languages. Before we can get started though, we need to take a close look at the basic notions of syntax and semantics.

What is an executable model of syntax?

1 Concrete Syntax is Mostly Irrelevant

Different programming languages use different kinds of concrete syntax. To programmers, concrete syntax consists of sequences of Unicode characters, which they enter into the editor of an IDE. To organize this text into words, sentences, and even complete programs, language creators write down rules that explain which sequences of characters make up words, which grouping of words create sentences, and which sentences are complete programs.

Borrowing terminology from linguistics, researchers dubbed these rules (*context-free*) *grammars*. The word *parsing* denotes the process of recognizing sequences of characters as words, groups of words as sentences, and certain sentences as complete programs according to some grammar. An IDE employs a *parser*, which implements the parsing process, to inform programmers about potential violations of these grammatical rules.

Some programming language creators shift some of this parsing work to the programmer. One prominent example is the Lisp family, which includes Clojure, Common Lisp, ELisp, Racket, Scheme, and a few more. A Lisp programmer uses balanced pairs of parentheses, braces, and brackets (plus arbitrary white space) to separate words and to organize words into groups. The result is a “fully parenthesized expression.” Due to the nesting of parenthesized sentences, people refer to this kind of notation as a concrete syntax *tree*. That is, even though the text in the IDE editor consists of just sequences of characters, programmers perceive it as a tree.

Lispers have referred to these trees as *S-expressions* since the late 1950s. Contemporary software developers call them *semi-structured data*. The designers of contemporary data description notations, e.g., XML and JSON, have adapted this idea, because it greatly simplifies the parsing process

XML employs
`<letters> ... </letters>`
as parentheses.

The syntax of a programming language also comes with context sensitive constraints. Dealing with those is only a partially solved problem. See chapter IV.

without imposing too much of a burden on human readers. They employ brackets, braces, commas, and (named) parentheses to organize data.

Due to the large variety of concrete syntax, the study of context-free grammars and parsing techniques for those became its own research area in the 1960. By now, most people consider context-free grammars and parsing a solved problem. The key insight is that people can use a grammar to describe the grammars of programming languages. Furthermore, researchers recognized by the 1970s that the translation of these grammars into parsers is a computational process, realizable via so-called *parser generators*.

Hence, this book mostly avoids spending time on the (solved) problem of grammars and parsing so that it can cover topics related to semantics and how semantic choices affect the pragmatics of a language. Concretely, this book compromises by using S-expressions for the sample programs of its methods. While this choice forces readers to formulate these programs in an unusual concrete syntax, it greatly simplifies their implementations of parsers for the model languages.

Readers interested in parsing techniques should consult text books on compilers.

Additionally, the choice of S-expressions as input data enables us to explain the concepts of grammar and parsing in a concise manner, starting with the BNF grammars for describing language grammars. From there, it is straightforward to discuss the pragmatics of parsers, both in abstract terms and with concrete examples.

2 BNF Grammars for Describing Grammars

As the preceding chapter mentions, BNF (or one of its modern variants) is the most commonly used formalism to describe the concrete syntax of a programming language. Instead of describing BNF formally, we present just enough examples so that future readers can easily comprehend the syntax descriptions in the remaining chapters.

The primary element of a BNF grammar is a *production*, which has the following shape:

`Program ::= a`

The `::=` symbol separates the capitalized name that is being defined from its definition. Our convention is to use capitalized words for defined entities and lower case words plus other sequences of keyboard characters for pieces of code. People pronounce such a definition as “A Program is a”.

A typical grammar describes elements of a language’s syntax as not just one possible shape but several possible shapes:

```
Program ::= a | b | c
```

The BNF symbol `|` is pronounced as “or” and, when a production has this shape, the `::=` is pronounced as “one of.”

Next, the author of a grammar often needs to specify that a programmer may write down sequences of language elements. To this end, BNF grammars come with two forms of superscripts. The first one uses a star:

```
Program ::= a* b
```

It informs a programmer that a potentially empty sequence of `a`s followed by `b` is a complete `Program`. For sequences that must contain at least one element, BNF grammars employ a plus as a superscript:

```
Program ::= a+ b
```

Stop! Pronounce this grammar rule out loud.

For the syntax description of most languages, it is necessary to use several, mutually related productions:

```
Program    ::= Statement* Expression
Statement  ::= print(Expression);
Expression ::= 43
```

This collection of three production rules informs a programmer of three definitions:

1. A `Program` is a possibly empty sequence of `Statements` followed by one `Expression`.
2. A `Statement` consists of a four pieces: the word `print`, the character `(`, an `Expression`, and the character `)` followed by a `;` character.
3. An `Expression` is just the text `43`.

Given a bunch of productions, a programmer can follow their implied instructions to write code. For example, the following is a `Program` according to the above rules:

```
43
```

It consists of no `Statement` and the required `Expression`. Here is another one:

```
print(43); print(43); 43
```

This program consists of two `Statements`, followed by the one required `Expression`. And that is all there is to describing grammars with BNF as far as this book is concerned.

Our definitions leave it implicit that white space is ignored.

3 The Parsing Process and the Parser Function

While the first role of a BNF is to inform the future developer about the shape of well-formed code, its second role is to tell a language implementer how to parse any given text according to some production. For example, given the text

```
44
```

it is impossible to parse it according to the production for `Expression`, and therefore it does not belong to the set `Programs` either. Similarly,

```
print (44);
```

does not belong to the syntax specified by `Statement`, even though a lot of the characters of this text fit the pattern of the production. By contrast,

```
print (43);
```

is a legal `Statement` but not a legal `Program`.

In general, parsing some text essentially answers a yes-no question relative to some production:

- yes, the given text belongs to all the collection of all texts that the production describes;
- no, the given text does not belong to the collection of texts that the production describes.

Hence, the task of a language implementer is to create one parser per BNF production, which can answer whether some text belongs to the set specified by the production. The implementation of these parsers may have to proceed in parallel because well-designed parsers rely on each other in the same way as the BNF productions refer to each other.

At first glance, this suggests that a parser for some production has the following signature:

```
parseExpression : PlainText → Boolean
// does the given text belong to the collection of texts
// described by the Expression production
```

In words, the parser for the `Expression` production from the preceding section consumes some text and produces `true` if the given text belongs to the syntax of the production; otherwise, it returns `false` to indicate that the text somehow does not fit the description.

For this particular production, it is trivial to define this function:

```
parseExpression(text) =  
    true if the text consists of ``4`` followed by ``3``  
    false otherwise
```

But, a developer would be rather unhappy if, after entering 44 into the IDE, the parser would display “false” in response. Creating the kind of parser that developers appreciate takes a lot more effort, and this brings us to the first question of language pragmatics.

4 The Pragmatics Question

The parser is the first piece of a language implementation with which any consumer of a programming language interacts. Hence it suggests itself as a first example for a pragmatics analysis.

When we consider the pragmatics of a syntactic element of a language or a piece of its implementation, we must study what kind of information it provides to a human being in a specific work situation. In the case of a parser, we can easily perceive two such pairings:

- the consumer of the programming language entering text into the IDE and wishing to get fast feedback as to whether this text is well-formed according to the intended grammar production;
- the implementer of the programming language perceiving the parser as just one piece of the complete implementation or, in the case of this book, an executable model for syntax and semantics.

Every consideration of a question of pragmatics should spell out which pairing is under consideration, because the implication of making a design decision—picking one of several alternatives, such as the three signatures for parser functions—differs from pairing to pairing.

4.1 Design Choices

A developer who enters text into an IDE’s editor wants informative feedback. And this developer does not want to know *whether* a text fails to belong to the expected production; the question is *which part* of the code causes the failure. In other words, a developer wants a red squiggle under 44 if the IDE’s editor contains the following piece of code:

```
print(43); print(44); print(43); 43
```

After all, this code is not supposed to be a program according to the BNF production in section 2.

This practical concern suggests that `parseExpression` should return a (data representation of a) reason why some text fails to live up to the expectations of a particular production. Using Java's `Optional` type, a language implementer might propose the following alternative to the original signature of `parseExpression`

```
parseExpression : PlainText → Optional<A>
// where A is either
// - some data that represents a parsing failure, or
// - none, if parsing succeeds
```

Implementing this interface would satisfy the developer who creates the erroneous text from above. After this text is entered into the IDE, the IDE would consult the parser function, which would return the some variant. In turn, the IDE would interpret the data by marking up the faulty piece of text that the programmer expects to be a program.

The client could be its creator, an IDE, a complete language implementation, and more.

A language implementer, however, should still have doubts about the usefulness of such a parser. If parsing succeeds, the result is uninformative. Specifically, it does not share with the client of a parser for, say, the Program production, a number of interesting insights from the parsing process:

- which parts of the text belong to the Expression;
- which parts of the text belong to the Statement; and
- which parts of the text belong to the Program.

After all, the complete implementation of the language must process this text again to, say, determine its meaning. Clearly, knowing the answers to the above questions is useful in this bigger working context, too.

In short, a language implementer wants an informative result from a parser in either case:

```
parseExpression : PlainText → B or C
// where B is informative data in the error case
// and C is informative data in the success case
```

The question is what kind of data to pick for B and C, though before we consider this questions, let's reflect on two concerns that come up as part of all questions of pragmatics.

4.2 Costs and benefits

The costs and benefits of a design decision represent its most important implication. Here is a table for the specific case of a parser function:

Parser Variant	User	Implementer
Boolean	uninformative	straightforward
Optional<A>	informative	middling complexity, does not help with the rest of the implementation
B or C	informative	complex; useful for other parts of the implementation

It concisely summarizes how each person in a specific working situation benefits from a particular design alternative and indicates how much work it imposes on each person. For example, a user working with the first kind of parser must manually compare grammar productions and plain text to figure out why parsing fails. Similarly, the implementer has some benefit from implementing the most complex solution.

Summarizing the benefits and costs of the design alternatives in this manner is a helpful technique for making decisions concerning language pragmatics. While the third alternative offers advantages to both people in the case of parsing, such clear-cut cases are rare when it comes to programming language pragmatics. It is therefore critical that decision makers keep in mind the pairings that determine concerns of pragmatics: the human being and the many working situations that those find themselves in.

5 Parsing: In General

Let us now turn to the challenge of designing a data representation that is useful to both successful parsing processes and unsuccessful ones.

We start with an example that exists in almost all conventional languages. Over the past few decades, language creators have come up with a fair number of textual variants for assignment statements:

Algol, Pascal	Fortran, C	Scheme, Racket	R, OCaml
x := 1	x = 1	(set! x 1)	x <- 1

This very variety of textual notations for assignment statements is just one of many indicators that concrete syntax is a matter of taste, not pragmatics.

Like in clothing fashion, every decade has had its distinct preferences.

The key to identifying a data representation is to recognize that all basic assignment statements share a common structure. Here is an informal, but structured description of the elements of an assignment statement and their relationship to each other:

```

an assignment statement consist of two parts:
- a left-hand side,
  - which is a plain-text variable name
- a right-hand side,
  - which is a number in the presented examples
    but, in general, is an expression.

```

A programmer might draw this informal description as a graphical sketch on a white board; see figure 1. But also note that the distinct syntactic marker of the assignment plays no role in this description.

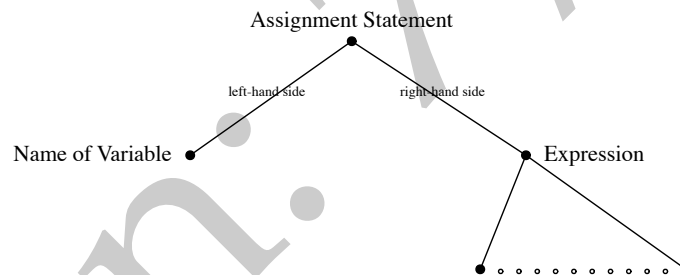


Figure 1: A tree presentation of the data representation for assignments

What this figure suggests is that an assignment statement can be understood as a node in a tree. Since one such tree abstracts many concrete-syntax shapes for the same kind of concrete-syntax element language researchers call them *abstract syntax trees* or just *ASTs*. In this example, the root node has two branches, each pointing to another node: one for the left-hand side and one for the right-hand side. While the left branch points to a leaf, namely, the name of the variable, the right one ends in a sub-tree, because the right-hand side is an expression and because the representation of an expression is likely to be a tree all by itself.

To make this last idea concrete, consider the concrete syntax for an addition expression. In Lisp, it is `(+ x 1)`; some old languages use postfix notation, `x, 1, +`; and conventional languages use the notation familiar from

grade school: $x + 1$. But just like assignment statements, all of these addition expressions share a common structure that resembles a tree shape:

```

an addition expression consist of two parts:
- an operand on the left,
  - which is another expression
- an operand on the right,
  - which is another expression

```

And again, the concrete addition syntax is absent.

Stop! Draw the tree for the following two assignment statements:

- (set! x (+ x 1)), what a Racket programmer writes to increase x by one;
- $x = x + 1$, what a Java programmer uses for the same purpose.

We can translate this tree idea into a collection of Java interfaces and classes. Take a look at the two columns of code in Figure 2. The left column displays the data representation of an assignment statement. While the name of the class signals intent, its two fields correspond to the two clauses of the informal description or the two branches of the tree in figure 1. The type specification for lhs implicitly informs a future reader of this code that variable names are represented as Strings.

<pre> class AssignmentStatement { String lhs; IExpression rhs; } </pre>	<pre> interface IExpression { } class VariableReference implements IExpression { String name; } class LiteralConstant implements IExpression { int i; } class Addition implements IExpression { IExpression left; IExpression right; } </pre>
---	--

Figure 2: A data representation for assignment statements and expressions

The right column is the data representation for simple expressions: variable references, literal constants, and additions. Since there are several

kinds of expressions that the rest of the program is going to treat in a uniform manner, the data representation uses a Java interface to give a common type to the various variants.

Stop! Reformulate the data representation from figure 2 in your favorite programming language. Use your data representation or the one in figure 2 to represent (set! x (+ x 1)).

Choosing such a tree-oriented data representation answers two of the questions of section 4 concerning *how* the parser succeeded in analyzing the text. Each node in the tree identifies a BNF production. Fields with atomic types in the corresponding structure represent the parts of the plain text that may vary, such as the name of a variable on the left-hand side of an assignment statement or the integer in a literal constant. The other fields of the structure represent the use of additional BNF productions.

The remaining question is what a parsing function should return when parsing fails. Since the pragmatics question about parsing identified the IDE and the user of the IDE as the recipient of the relevant information, the data representation must enable the IDE to mark up the pieces of text that do not match the given BNF.

<pre> Program ::= Statement* Expression Statement ::= print(Expression); Expression ::= 43 </pre>	<pre> class Program { Statement s[]; IExp r; } class Statement { IExp e; } interface IExp { } class Expression implements IExp { // there is one: 43 } class Error implements IExp { String badText; // data for the IDE } </pre>
---	---

Figure 3: A data representation for the grammar from section 2

Let's make this concrete by returning to the somewhat more interesting

example from section 2, starting with the following attempt:

```
print(44);
```

This text almost matches the Statement production. Specifically, it matches the prefix characters—`print(`—and the suffix characters—`);`. It is the `44` part of the given text that causes the parser implementation to fail when it tries to match it with the Expression production.

A bit of reflection suggests that the IDE needs to know:

- which part of the text the parser matches to the BNF production(s),
- which part causes the parser to fail.

Figure 3 displays one reasonably straightforward solution. On the left, it shows the original grammar from section 2. On the right side, the figure lists a mostly direct translation of this grammar into an AST representation. Each production is turned into a class, with fields to represent the pieces. For the statement class, this means a Statement array field for the potentially empty sequence of Statements and an IExp field for the final Expression. The Statement class contains one field; it represents the Expression that that parser expects to find.

Figure 3 also suggests what happens if the parser does not find an Expression according to the BNF. The interface IExp is implemented twice: once by the Expression class and once by an Error class. So, when the parser for the Statement production is confronted with `print(44);`, it defers to the parse for the Expression production, which returns an instance of Error instead of the Expression class. In other words, the parser's output remains a tree, but this tree representation contains nodes that indicate that a syntax error exists. When given an abstract syntax tree with an error node or even several such error nodes, the IDE can use the structure of the tree together with the data in the Error node to mark up text that can't be parsed.

In an actual implementation, the AST classes also contain line and column numbers so that the IDE can find the corresponding text.

Stop! Reformulate the data representation from figure 3 in your favorite programming language. Then use your data representation to represent `print(44);`.

5.1 A Parser Returns an Abstract Syntax Tree

Let's draw the general lesson from these examples. First, parsing plain text produces an abstract syntax tree. Second, in order to report failures in an informative manner, the AST representation is enriched with error nodes.

The signature for every parser function is essentially

```
parsePRODUCTION : PlainText → AST
```

To distinguish the success case from the failure case, we introduce the convention that *AST* is a tree that *might* contain an error node and that *AST-* denotes a tree without any error nodes. Hence a function *f* that consumes the result of a parser has the signature

```
f : AST- → ResultOfF
```

6 Parsing: An Example

An instructor may wish to choose an alternative form of semi-structured data for pedagogic reasons. All exercises can be reformulated in a straightforward way.

It is time to put all the pieces together and look at a fully-worked example. As the introduction to this chapter hints, this book simplifies the parsing task. The goal is to make these parsing exercises as easy as possible for the readers who *work* through the projects. The simplification consists of the choice of input and grammars. Instead of plain text, the parser presented here analyzes semi-structured data. Instead of arbitrary BNF grammars, the models of syntax in this book always identify a subset of the chosen set of semi-structured data. While XML and JSON are modern forms of semi-structured data, the idea is decades older and the original version—S-expressions—deserves the honor.

The word “S-expression” has two meanings here. On the surface, an S-expression is plain text, a sequence of characters. The sequence, though, starts and ends with parentheses (brackets, braces), and all nested open parentheses come with matching parentheses (brackets, braces). On the level of perception, we favor the implied tree-shape of this kind of text. The top-level text corresponds to the root of the tree, and every nested parenthesized piece of text represents a branch of the current level in the tree.

For our purposes, an *S-expression* is a fully-parenthesized term whose leaves are numbers and symbols:

```
An S-expression is one of:
-- Number
-- Symbol
-- (S-expression ... S-expression)
```

```
A Symbol is a non-empty sequence of keyboard characters
that is not also any number.
```

```
A Number is a double between -1000.0 and +1000.0 with at
most one digit to the right of the decimal period.
```

Stop! Reformulate this rigorous definition as a BNF.

Exercise 1. Pick a programming language in which you want to implement all remaining exercises.

Libraries for reading S-expressions are available for most programming languages. Check rosettacode.org for one collection of such libraries. If you find better libraries than those, feel free to share with your peers.

Create a program in your chosen language that can be run on the command line, reads an S-expression from standard input, and writes a count of all symbols to standard output.

Since most programming languages come with a core of expressions and statements, our first model of syntax—dubbed *Sample*—includes just those elements. To keep things simple, we also severely restrict the shapes of expressions and statements:

```

Program    ::= (Statement* Expression)
Statement  ::= (Variable = Expression)
Expression ::= 1.0
              | 2.0
              | 3.0
              | Variable
              | (Expression + Expression)

Variable   ::= consists of all non-empty sequences of
              alphanumeric characters, starting with an
              alphabetical letter of maximal length 10.

```

By BNF convention, =, +, 1.0, 2.0, and 3.0 are literal text. The capitalized words are just the defined sets, and nothing else.

Like the grammar in the preceding sections, the set of well-formed Programs consist of a potentially empty sequence of statements followed by one expression. There is only one kind of statement, namely, assignment statements that resemble those found in C, Java, and similar languages. Finally, the set of expressions comes with an infix-style shape for addition, variable occurrences, plus the numbers 1.0, 2.0, 3.0. Although it would be possible to define a BNF grammar that specifies variable names, we choose to use English instead for simplicity and to illustrate common practice.

Even simple BNF grammars call for the creation of examples as does *proper program design*, so here is one:

```

((x = 1.0)
 (y = 2.0)
 (z = (x + (x + y)))
 (z + 3.0))

```

Let's use the name `concrete` for this example.

The phrase “proper program design” refers to an understanding of programming as presented in How to Design Programs.

Clearly, `concrete` is an S-expression. The question is whether it satisfies the Program grammar. As before, we can verify this fact manually:

- `concrete` consists of four S-expressions between parentheses. Hence, we must check that the first three belong to the set of Statements and the last one to the set of Expressions.
- `(x = 1.0)` is a Statement, because `x` belongs to the set of Variables and `1.0` is obviously an expression.
- Verifying that `(y = 2.0)` and `(z = (x + (x + y)))` belong to the production for Statements proceeds in a similar manner.
- Finally, `(z + 3.0)` is an Expression:
it consists of `(` followed by a Variable followed by `+`, followed by `3.0`, and wrapped up by `)`.

Stop! Don't just accept it all. Double-check these claims, especially the one that say they are "similar" to others.

The goal is to design a parser function for the Program production of this grammar. First we must pick a programming language. Second we need to articulate an AST data representation. Third, we design parsers for all productions of *Sample* BNF.

Our favorite programming language for this purpose is Racket, and you may understand why after reading the next couple of pages. But, as exercise 1 says, you can solve all problems in almost any programming language that you like.

Figure 4 consists of two columns, which together specify our AST data representation. The left-hand column contains four groups of structure-type definitions:

- The first three groups correspond to the three productions of the BNF grammar of *Sample*.
- Since the Expression production describes three kinds of alternatives—numerals, variables, additions—the third group consists of a structure type that plays the role of an interface—`expr`—and three implementing variants—`num`, `var`, and `add`.
- The fourth and extra group contains a single structure-type: `err`. It exists to inject error nodes into the AST.

<pre> (struct prog [statements end]) (struct assg [lhs rhs]) (struct expr ()) (struct num expr (n)) (struct ref expr (name)) (struct add expr (left right)) (struct err (msg)) </pre>	<pre> #; {type Prog = (U Err (prog List<Stmt> Expr))} #; {type Stmt = (U Err (assg Symbol Expr))} #; {type Expr = (U Err (num N) (ref Symbol) (add Expr Expr))} #; {type Err = (err String)} </pre>
--	--

Figure 4: A Racket AST data representation for *Sample*

Since Racket does not impose type constraints on code, instances of `err` can be injected into an AST at any place.

The right-hand column consists of comments that use stylized English and Racket constructors to explain the relationship among the structure types. It tells a future reader of the code that, for example, an instance of `prog` always contains `List<Stmt>`, a list of `Stmts` in the first field and an instance of `Expr` in the second. Furthermore, the quasi-type `Prog` is a union that contains `Err` and all legal instances of `prog`, meaning a parser function for the `Program` production may return `(err "...")`.

Stop! Work out an S-expression that is *not* a member of the set that the `Program` production describes.

Stop again! Explain the remaining type-like comments on the right-hand side of figure 4.

Here is the expected AST for the concrete example:

```

(define ast-for-concrete
  (prog
   (list
    (assg 'x (num 1.0))
    (assg 'y (num 2.0))
    (assg 'z (add (ref 'x) (add (ref 'x) (ref 'y)))))
    (add (ref 'z) (num 3.0))))

```

It is an instance of `prog`, which contains a list of `assg` instances in the first field and an instance of `expr`—specifically an instance of `add`, which is a sub-type of `expr`—as the last one.

The notation leans on Typed Racket.

Figure 5 displays the entire program. The main function composes two computations: `read`, which reads an S-expression from the standard input, and `program->ast`, which parses this S-expression expecting a member of `Program`. The remaining functions correspond to the three productions in the BNF of *Sample* in a one-to-one fashion. Following section 5.1, the three functions are named to clarify their role in the parsing process. Each function consists of $n + 1$ conditional clauses, where n is the number of alternatives on the right-hand side of the production. The extra clause corresponds to a mismatch of the given S-expression and the parsed production.

```

#; { -> Prog}
;; parses the S-expression given on STDIN
(define (main)
  (program->ast (read)))

#; {S-expression -> Prog}
(define (program->ast txt)
  (match txt
    [(list s ... e)
     (prog (map statement->ast s) (expression->ast e))]
    [_ (err (~a "program expected, given " txt))]))

#; {S-expression -> Stmt}
(define (statement->ast txt)
  (match txt
    [(list lhs '= rhs) (assg lhs (expression->ast rhs))]
    [_ (err (~a "statement expected, given " txt))]))

#; {S-expression -> Expr}
(define (expression->ast txt)
  (match txt
    [1.0 (num txt)]
    [2.0 (num txt)]
    [3.0 (num txt)]
    [(? symbol?)
     (if (variable? txt)
         (ref txt)
         (err (~a "variable expected, given " txt)))]
    [(list left '+ right)
     (add (expression->ast left) (expression->ast right))]
    [_ (err (~a "expression expected, given " txt))]))

```

Figure 5: A Racket parser for the BNF grammar of *Sample*

Let's take a close look at the `program->ast` function:

- The function uses Racket's algebraic match form to check whether the

given S-expression is a list. Concretely, (list s ... e) matches txt if the latter is a list and contains at least one element, e. All other elements are collected in a potentially empty list called s.

- If txt matches a list that contains some potentially empty sequence named s and a last element named e—which represents the very basic aspects of the right-hand side of the Program production—program->ast instantiates prog;
- Otherwise the conditional's catch-all clause returns an instance of err to indicate that the S-expression cannot possibly belong to the Program production.

This last clause kicks in even if the set of S-expressions were extended with additional forms of data.

Stop! Try to explain the remaining two functions before reading on. What happens if parserStatement or parserExpression discover errors? Does program->ast still instantiate prog?

Of the remaining two functions, statement->ast is straightforward to explain, but expression->ast deserves an explanation. The match of expression->ast consists of five plus one clauses:

- The first five clauses correspond one-to-one to the right-hand side of the Expression production.
- The first three clauses are literal matches for 1.0, 2.0, and 3.0.
- The fourth clause takes care of the parsing of Variables.

Stop! The clause assumes the existence of a variable? function that enforces the constraints of the Variable production. Develop such a function in your favorite programming language.

- The last clause is a catch-all clause, which is used to report errors, that is, cases when the given S-expression should be a member of Expression but isn't.

And this is all there is to creating a parser—as far as this book is concerned.

7 What is a Model

Sample is our first model. Its syntax looks and feels a bit like a tiny kernel of programming languages you should have encountered so far, except for the many parentheses. It differs from these languages in many visible ways, but most obviously in that three numbers are the only form of literal data.

Assuming that *Sample*'s statements and expressions behave just like their corresponding constructs in languages you know, it is still possible to write representative, albeit somewhat convoluted programs.

Parsing is the only concern of pragmatics studied in this chapter, and therefore we must judge the models and its simplifications from this perspective. As the parsing program in figure 5 demonstrates, the model enables us to examine the essential ideas of parsing S-expressions for subset grammars:

1. A parser constructs an abstract syntax tree from a given input text.
2. The nodes in an AST represent the validated productions or the failure of some piece of text to satisfy an expected BNF production.
3. Failure is represented with an error node in the AST, which the IDE can use to highlight syntax problems.
4. The parser consists of as many functions as the BNF specifies productions, which each consumes some text expecting to find an instance of the corresponding production.
5. Each function consists of a conditional with as many branches as alternatives in the corresponding BNF production plus one for reporting errors.

To confirm the usefulness of the model, we need to check whether our conclusions would change in a significant way as we grow *Sample* to close the gap to a realistic 1960s language. Given the nature of BNF, growing *Sample* might mean adding alternatives to existing productions, adding productions, or both. Since the above five items do *not* rely on any of aspects of the actual *Sample* BNF, and since they are formulated generically in terms of alternatives and productions, the answer is clear: the model does not mislead as far as S-expression parsing is concerned.

Exercise 2. Add conditional and looping statements to the Statement productions of *Sample* and multiplication expressions to its Expression production. Would the parser program of figure 5 change in any way that violates the five conclusions about parsing?

Would the addition of (global) procedure definitions to *Sample* change your mind? A procedure definition abstracts over the literal (numeric) constants in statements. In the 1960s, sequences of procedure definitions preceded the sequence of statements in a Program.

8 Bare Bones: a Parsing Project

8.1 Project Tasks

Figure 6 presents the BNF grammar of the first project language. It extends *Sample* so that the syntax resembles the core of most contemporary programming languages, minus the parentheses. While a Program is still a sequence of Statements followed by an Expression, the set of Statements contains two new variants:

- (if0 Expression Block Block), a simple conditional, combining a conditional expression with two blocks of Statements; and
- (while0 Expression Block), a looping construct, combining a conditional expression with one block of Statements.

As always, BNF convention dictates that =, +, if0, and while0 are literal text. They all serve to distinguish one S-expression from another; the conventional terminology for such symbols is “keyword.”

```

Program    ::= (Statement* Expression)

Statement  ::= (Variable = Expression)
              | (if0 Expression Block Block)
              | (while0 Expression Block)

Block      ::= Statement
              | (block Statement+)

Expression ::= Number
              | Variable
              | (Variable + Variable)
              | (Variable / Variable)
              | (Variable == Variable)

Variables  ::= consists of all consists of all non-empty sequences
              of alphanumeric characters, starting with an
              alphabetical letter of maximal length 10, but
              excludes ``if0`` and ``while0``.

```

Figure 6: The concrete syntax of the *Bare Bones* language

The grammar in figure 6 introduces one innovation: Blocks. Both the if0 alternative and the while0 alternative of the Statement production refer to

Block, thus allowing programmers to use either one Statement or a non-empty sequence of Statements.

Finally, the set of Expressions comes with a few additions compared to those of *Sample*: many more numerical literals, a division expression, and a comparison expression. Note, however, that it also restricts the shape of Expressions in comparison to *Sample*. Thus, the *Bare Bones* language no longer contains the concrete example from section 6.

Stop! Determine where a parser for *Bare Bones* would flag the concrete syntax of concrete to inform the programmer of a grammar violation.

A *Bare Bones* programmer can easily work around this restriction with assignments to temporary variables. Here is the result:

```
((x = 1.0)
 (y = 2.0)
 (temporary = (x + y))
 (z = (x + temporary))
 (temporary = 3.0)
 (z + temporary))
```

In this case, the use of a single additional variable, *temporary*, overcomes the restriction. The nested Expression and the nested literal constant are lifted into the surrounding sequence of Statements and given a name: *temporary*.

Exercise 3. Your task is to design an AST data representation for *Bare Bones* and to implement a parser for this language in your favorite programming language.

The parser maps an S-expression to an instance of AST, an abstract syntax tree that may contain error nodes.

The complete program should read S-expressions from the standard input device and print either "okay" or "syntax error" to the standard output device, depending on whether the parser succeeds in constructing an AST.

Review the code with a peer who is not involved with your design.

What is an executable model of semantics?

1 Meaning Matters

The BNF of the *Sample* language tricks you into thinking that you know the behavior of programs, statements, and expressions. You might think that $(x = 1.0)$ sets the variable x to 1 and that $(x = (x + y))$ adds x to y and makes x stand for this sum. But, thus far, all we know is how to create a sequence of characters that belong to the set of Programs, Statements, and Expressions. We follow the rules of the BNF, and a parser for this BNF confirms that these sentences are well-formed. The point is that syntax seems to imply semantics, though without an accompanying specification, we can't know for sure.

As mentioned in the first chapter, semanticists study how to assign meaning to syntax. In general, their concern is to specify the semantics of any sentence that is well-formed according to some BNF production. So, for example, a semanticist might say that

- a literal constant is a value all by itself;
- an expression yields a value; and
- a statement updates a relationship between variables and values.

These English sentences suggest three insights about describing the semantics of a programming language: (1) we must choose a set of values; (2) we must state what operations on these values yield; and (3) we must have a mechanism for tracking which variables stand for which values.

Let's relate this abstract explanation to the real world of computers, starting from this "mechanism" just mentioned. A computer engineer calls this "mechanism" the machine's *memory*, also known as *store*. One way to imagine a store is as a collection of "boxes" that have a label—the name of

a variable—and that contain the value of the variable. A variable reference in an expression is a request to retrieve the number from the correspondingly labeled box. If an expression consists of an operator, say +, and two variables, the two corresponding numbers are sent to a co-processor that corresponds to the named operation. Now consider that this expression is on the right-hand side of an assignment statement. Once this co-processor delivers the result of this expression, the computer places this number into the box labeled with the name of the left-hand side of the assignment.

This concrete description of how to interpret a *Sample* Program has inspired people for a long time. People can easily visualize the execution of primitive statements and expressions in the context of a machine. For that reason, teachers use it in classrooms to illustrate what assignment statements do. However, this explanation does not scale to common features of existing languages. For example, it is extraordinarily difficult to explain classes and objects with this simplistic understanding.

Yet, the idea of a mechanical machine executing instructions inspires the solution: abstract machines. Roughly speaking, an abstract machine is a mathematical approach to describing machine-like instructions, but directly in terms of the syntax of a modeled programming language. Better still, these machines directly correspond to executable code, meaning people can implement them easily and experiment. This chapter introduces the basics of abstract machines. The next one expands on the basics a bit, so that the book can explain many of the features of contemporary languages.

2 Abstract Machines: In General

A semantic specification based on abstract machines starts from a grammar that defines the set of programs and a set of values, which are the meanings of programs run on the machine. For the description of the set of values, it might suffice to pick a set that people in computing accept, say the set of inexact numbers within certain limits, or it might require a small, separate BNF grammar.

Given the grammar and the chosen values, an abstract-machine definition consists of two parts: (1) a set of machine states and (2) a function from states to states, often called a *transition function*:

- The specification of a machine *state* always consists of the instructions of the program, though as an AST not plain text, plus “bookkeeping” data. Furthermore, the specification of this set identifies (1a) initial

states, which consist of the given program and some auxiliary data; (1b) final states, which cause the machine to halt; and (1c) intermediate states.

- The *transition function* maps states to states on a case by case basis. Each case corresponds to a piece of the remaining program elements, e.g., the next assignment statement or the next loop. We often refer to these cases as instructions.

Running a program on an abstract machine requires two additional functions: load and unload. The first one maps a program to an initial state. Once an initial state exists, the machine repeatedly applies the transition function to interpret the program's instructions, one at a time. When the transition function yields a final state, the machine halts. The second function interprets this final state as the answer of the given program.

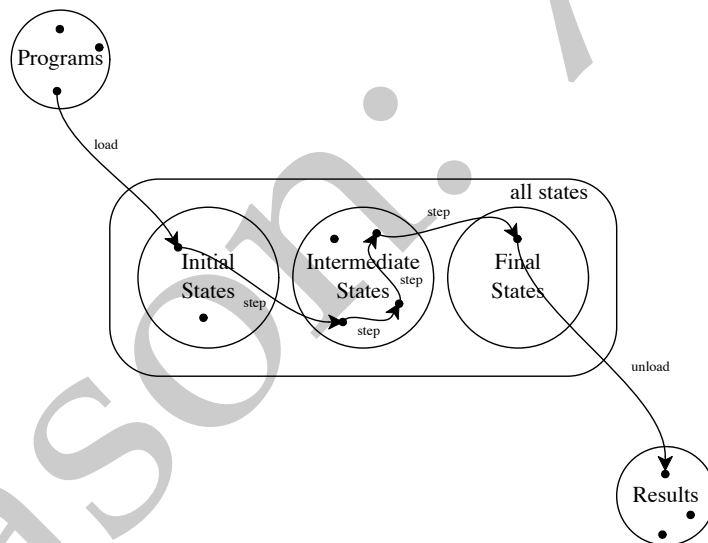


Figure 7: Executing programs on an abstract machine

Figure 7 presents a diagram that expresses this explanation graphically. The program is loaded, yielding an initial state. Then the transformation function is applied to the initial state, yielding an intermediate state. It is again applied to this state and so on, until the result belongs to the set of final states. By applying the unload function to this final state, we determine

the value of the program—that is, its meaning.

Although sets and functions come from mathematics, the key is that it is *executable* mathematics, meaning code. Here is a direct translation of figure 7 into a short procedure:

```
// determines the meaning of 'p'
Result runMachine(Program p) {
    State currentState = load(p);
    while (! (isFinal(currentState))) do {
        currentState = transition(currentState);
    }
    Value result = unload(currentState);
    return result;
}

State load(Program p) { ... }
State transition(State s) { ... }
Result unload(State s) { ... }
```

This pseudo-code employs a 1970s style procedural style, easily mimicked in every contemporary programming language. The `runMachine` function is the entry point. It composes three tasks via sequencing and a while loop: load the program; run a loop until the content of `currentState` belongs to the subset of final states; then unload the result from this final state.

Stop! Articulate this pseudo-code in your favorite language.

What this “story” and the figure fail to account for is the possibility of a misbehaving program. Think of an expression in a program that divides an integer by zero or that references an array element outside of its range. A few language descriptions, for example C’s, do not specify the semantics of such situations, leaving it up to the implementation to pick a behavior. Many others, say Java, describe such situations as exceptions and demand that implementations signal a problem.

*Let’s ignore exception
handlers for now.*

With abstract machines, it is straightforward to express this semantics. First, let’s enrich the set of final states with distinct *error states*, which bring the machine to a halt. Second, the `unload` function translates such error states into distinct answers. And that’s why the result type of `runMachine` is `Result`, not `Value`. The `Result` type represents the disjoint sum of `Values` and `Exceptions`:

```
type Result = Value | Exception
```

In short, the author of the abstract machine must not only pick a set of values but also a set of exceptions to assign meanings to programs.

3 The CS Abstract Machine: The *Sample*- Language

In a sense, section 1 precisely explains how programs in the *Sample* model of chapter II are executed. The goal of this section is to articulate this English description as a simple abstract machine, using a rigorous code-like formalism.

To keep things simple, let's assume that line in *Sample*, the addition expressions always have the shape $(x + y)$ for some variable names x and y :

```

Program    ::= (Statement* Expression)
Statement  ::= (Variable = Expression)
Expression ::= 1.0 | 2.0 | 3.0 |
               Variable |
               (Variable + Variable)

```

Let's call this the *Sample*- language. What we need next is a description of the set of values and states plus functions for loading a program into a initial state and for unloading a final state to get an answer; an examination of whether there are exceptional cases; and the transition function.

3.1 The States

The *Sample*- BNF allows programmers to write down exactly three literal constants: 1.0, 2.0, and 3.0. Furthermore, the programs may perform exactly one operation on these values: addition. All other statements merely move such values from one variable to another. Hence, it makes sense to say that the set of Values consists of just the inexact numbers found in computer hardware and that $+$ denotes their addition operation.

Following the introductory section of this chapter, a state obviously consists of two parts: the program's remaining instructions and the store. Semanticists call these parts *registers*, even if this usage isn't quite in conformance to how computer engineers apply it. Specifically, we say that the C register contains the remaining instructions and thus *controls* the execution of the machine; the S register represents the current content of the *store*. The name of the machine reflects this combination: CS machine.

A store is a collection of labeled boxes with values inside. If we were to use mathematics here, we could say it is a (finite) function from variable names to the set of Values. By contrast, a programmer would represent such a collection as some form of table, which many programming languages offer as primitive objects with relevant operations.

One task remains: identifying the three partitions of the set of states and defining the load and unload functions. Clearly, an *initial CS state* consists of

just a program p combined with an empty store, representing that no instruction has been executed. A mathematically inclined person may write this as a Cartesian pair:

$$(p, [])$$

A *final CS state* is reached when all instructions of a program have been evaluated, meaning the final expression is just a number:

$$(n, s)$$

where n is the number that results from evaluating the final expression in a *Sample*-program. Intermediate states are those that combine programs with instructions and any store.

Here are the function definitions for loading programs into the CS machine and unloading final states:

$$\begin{aligned} \text{load} &: \text{Program} \longrightarrow \text{State} \\ \text{load}(p) &= (p, []) \end{aligned}$$

$$\begin{aligned} \text{unload} &: \text{State} \longrightarrow \text{Result} \\ \text{unload}(n, s) &= n \end{aligned}$$

3.2 Exceptional Cases

Although the *Sample*-BNF supports only one operation on values, a programmer can still write faulty programs. Take a close look at these two programs, which are almost identical:

the “good” program produces a number	the “bad” program produces what?
<pre>((a = 1.0) (b = 2.0) (temporary = (a + b)) (c = (a + temporary)) (temporary = 3.0) (c + temporary))</pre>	<pre>((a = 1.0) (b = 2.0) (temporary = (a + b)) (c = (a + temprary)) (temporary = 3.0) (c + temporary))</pre>

Stop! Explain the difference.

The boxed variable reference in *bad* gives it away. A typo—note the missing letter—makes it impossible to evaluate the addition expression, because the store does not have a value for this variable.

Language creators have faced this problem for decades and have come up with two radically different solutions. One is to pick a random number for *temprary* and to continue the execution of the program. In our example, this alternative might cause the bad program to compute a truly strange looking result or a result “in the ballpark.” Another one is to let the

programmer know that the code refers to a variable without stored value. Clearly, pragmatics tells us that the second alternative is usually preferable, because it informs the programmer of the problem.

Following section 2, “letting the programmer know” means terminating the program execution by sending the machine to a finite state. In addition, this finite state should encode information about the problem that is useful to the programmer. For now, we indicate this idea by using Strings as exception indicators here and modify the unload function accordingly:

Result =	unload : State $\rightarrow$ Result
InexactNumber	
U	unload(n,s) = n
String	unload(error,s) = .. text ..

The question of pragmatics here is similar to the one concerning parsers in the preceding chapter. Here are three alternatives for the specific problem with the bad program:

1. “error”, that is, a string that is not a number;
2. “uninitialized variable ‘temporary’ encountered”, that is, a string that mentions the name of the problematic variable; or
3. “uninitialized variable ‘temporary’ encountered, see line 4”, that is, a string that also includes the location of the problematic statement.

Clearly, alternative 1 is the least amount of work for the language creator and the least amount of information for the IDE and the programmer who is using it. By contrast, alternative 3 requests the most work from the language creator. It also imposes a cost on the data representation of ASTs for *Sample-*: without line number information, the machine cannot construct such an informative error string. Then again, alternative 3 is obviously the most helpful one for the developer, because it delivers the most useful information. Finally, an IDE developer might benefit from a different form of exception information, e.g., structured data that enables it to highlight the statement and display an informative message.

3.3 The Transition Function

Now we’re in a position to define the transition function. Such a function always performs two tasks: (1) it determines the nature of the next instruction in the given state and (2) it creates a new state in response. This new state is similar to the old one, but with the interpreted instruction pruned.

Due to this characteristics, transition functions are typically expressed as tables that specify a number of “before” and “after” state cases. Computer engineers use similar tables, though for bit-level states.

executing a sequence of statements		evaluating return expressions	
Control	Store	Control	Store
execute $x = n$		evaluate y as return expression	
before: $((x = n)::\text{stmt}^* e)$	s	before: $([] y)$	s
after: $(\text{stmt}^* e)$	$s[x = n]$	subject to: y is defined in s	
		after: $([] n)$	s
		where $n = s[y]$	
execute $x = y$ (success)		evaluate y as return expression	
before: $((x = y)::\text{stmt}^* e)$	s	before: $([] y)$	s
subject to: y is defined in s		subject to: y is not defined	
after: $(\text{stmt}^* e)$	$s[x = n]$	after: error	s
	where $n = s[y]$		
execute $x = y$ (failure)		evaluate $y + z$ as return expression	
before: $((x = y)::\text{stmt}^* e)$	s	before: $([] (y + z))$	s
subject to: y is not defined		subject to: y and z are defined in s	
after: error	s	after: $([] k)$	s
		where $n = s[y]$	
		and $m = s[z]$	
		and $k = (+ n m)$	
execute $x = y + z$ (success)		evaluate $y + z$ as return expression	
before: $((x = (y + z))::\text{stmt}^* e)$	s	before: $([] (y + z))$	s
subject to: y and z are defined in s		subject to: y or z is not defined in s	
after: $(\text{stmt}^* e)$	$s[x = k]$	after: error	s
	where $n = s[y]$		
	and $m = s[z]$		
	and $k = (+ n m)$		
execute $x = y + z$ (failure)			
before: $((x = (y + z))::\text{stmt}^* e)$	s		
subject to: y or z is not defined in s			
after: error	s		

Figure 8: The CS transition function for *Sample-*

Figure 8 presents the transition function for *Sample-* with its intended semantics; see the first section. Its definition distinguishes nine cases: five

in the left column and four in the right one. Each case states a *before* condition on the given state and an *after* state specification:

- The *before* condition always specifies a specific program shape and a general store (s).

Some cases also subject this state pattern to additional conditions; for example, if the program shape contains a specific variable, the additional constraint may require this variable to have a value in the store.

- The *after* state specification explains how the next state is formed, when the given state satisfies the *before* condition. Concretely, it specifies the content of the registers:

- The C register typically contains some program code or an error, in which case the state is final and the machine stops.
- The S register always contains the given store or a revised store. A revision has the shape $s[x=n]$, denoting a store that is like s , but x associated with n .

This last sentence requires a bit of explanation because it doesn't look like mathematics. Recall that a mathematically inclined reader considers s a finite function from variable names to numbers. This same reader understands $s[x = n]$ as a *new* function s' , which works like this:

$$s'(z) = \begin{cases} s(z) & \text{if } z \neq x \\ n & \text{if } z = x \end{cases}$$

Some cases also come with *where* clauses. Each such clause defines the value of a variable. For example, $s[x]$ retrieves the value of x from the table named s . Similarly, $(+ m n)$ denotes an actual addition of two inexact numbers in an implementation language, here Racket, because $(m + n)$ looks like *Sample-* syntax.

The definition of the transition function uses the notation $a::\text{stmt}^*$, which is a list of statements where a is the first one and stmt^* denotes the remainder.

Note that the abstract machine uses concrete syntax. An implementation will use ASTs instead so that it doesn't have to account for ill-formed programs; see the next section.

While the left column of the figure contains the cases for executing assignment instructions, the right column concerns the evaluation of the final expression in a program. Let's take a close look at the first transition on the left, using the good program from the preceding subsection. The initial state

is a pair whose C register contains all of good and the S register the empty table. By matching this initial state with the *before* line of the transition, we get the following correspondence between the patterns in the table and actual pieces of code plus store tables:

abstract state description		concrete pieces of syntax, the actual store
(x = n)	====	(a = 1.0)
stmt*	====	((b = 2.0) (temporary = (a + b))) (c = (a + temporary))) (temporary = 3.0)) (c + temporary))
e	====	
s	====	[]

Pronounce ==== as “matches” in this table and the following ones. The first three lines match the pattern of the C register with the entire good program; the remaining line denotes the correspondence between s and the current content of the store register (S). In general, the tables in this section separate the patterns in the C register cases from the s pattern with a horizontal line.

Stop! Which program variable corresponds to x, which number to n?

Since the two patterns match, the transition function returns the *after* state of this same part of the table. This state is created by filling the pattern with the pieces of syntax and the store that the *before* match yields. Hence, C now contains the following syntax:

```
((b = 2.0)
(temporary = (a + b))
(c = (a + temporary))
(temporary = 3.0)
(c + temporary))
```

And S is set to the table [a = 1.0].

An application of the transition function to this new state proceeds in an analogous manner. Interpreting (b = 2.0) results in a shorter program in C and one new entry into the table that is in S.

The interpretation of (temporary = (a + b)) differs from the first two steps. The first case of the transition function does not apply. Instead the fourth case matches as follows:

abstract state description		concrete pieces of syntax, the actual store
(x = (y + z))	=====	(temporary = (a + b))
stmt*	=====	((c = (a + temporary)) (temporary = 3.0))
e	=====	(c + temporary)
s	=====	[a = 1.0, b = 2.0]

The guidance of the *after* part of the case tells a future reader that the machine must look up a and b in s, add the two retrieved numbers, and store the result as the value of temporary in S. Accordingly, the resulting state contains the program

```
((c = (a + temporary))
(temporary = 3.0)
(c + temporary))
```

in register C and [a = 1.0, b = 2.0, temporary = 3.0] in S.

Stop! Which case of the transition function is needed to interpret the state we just constructed? And what state results from this step?

If you worked through this riddle, you realize that the result is a state whose C register contains a program that consists of just an expression. The machine interprets such a state via the right-hand side of the table in figure 8. And here is how it matches the *before* pattern of the third case:

abstract state description		concrete pieces of syntax, the actual store
e	=====	(c + temporary)
s	=====	[a = 1.0, b = 2.0, c = 4.0, temporary = 3.0]

According to *after*, the resulting state is a final state, with 7.0 in the expression position of the control register. Hence, unload of this state yields 7.0 as the meaning of this program.

Note The third line of the store match comes with a surprise. It maps c to 4.0, a number that is *not* a valid piece of syntax. Like all major programming languages, the evaluation of expressions may create values that do not correspond to literal values from the BNF grammar. Here, the issue is merely due to some unreasonable constraint on the set of literal constants; in actual programming languages, this condition is a necessity.

Stop! Translate `good` into your favorite programming language, run it, and confirm the result.

Stop! What is the result of evaluating a program whose final expression is just a literal constant?

Let's next consider how the transition function would deal with the `bad` program from the preceding section. For the first three assignment statements, the transition function yields the same sequence of states as for `good`. The fourth intermediate state matches the last clause on the left, because the concrete variable corresponding to `z` is not defined in the store:

abstract state description		concrete pieces of syntax, the actual store
<code>(x = (y + z))</code>	====	<code>(c = (a + temporary))</code>
<code>z</code>	====	<code>temporary</code>
<code>stmt*</code>	====	<code>((temporary = 3.0))</code>
<code>e</code>	====	<code>(c + temporary)</code>
<code>s</code>	====	<code>[a = 1.0, b = 2.0, temporary = 3.0]</code>

Hence, the result of the transition function is (error, s) , a final state for any store s . Unloading this state informs the programmer that something went wrong during the interpretation of `bad`.

At this point, we can step back and rationalize the semantics of *Sample*- and all of our models. While the Statements in a Program perform the desired computation, its Expression's value determines the Program semantics. One way to understand this arrangement, is to view this final expression as the only way to print an output. There is no equivalent to reading an input; if the computation is to be run on a different input, the user of the model language must modify the Statements.

3.4 An Implementation of the CS Machine for *Sample*-

An implementation of an abstract machine requires three choices from the programmer: picking a data representation of the sets of values; a data representation for the store; and function definitions for `load`, `transition`, and `unload`. To run the machine, it is also necessary to port the code of `runMachine` from the preceding section to the chosen programming language.

This section presents the Racket implementation of the CS machine. Racket structure types serve as simple data representations of states:


```

(struct state [control store])

(define MT ... an empty table ...)

#; {type State = (U Initial InterM Final)}
#; {type Initial = [state Prog MT]}
#; {type InterM = [state Prog Store]}
#; {type Final = [state (U Value Error) Store]}
#; {type Value = InexactReal}

```

The first line declares the structure type; the second one introduces a useful constant. All other lines are type-like comments that explain how the structures are instantiated to represent initial, intermediate, and final states. In this example, the structure type and the comments directly encode the informal explanations into Racket. For example, according to the second comment line, an initial state combines a *Prog* with an empty table into one structure.

*See chapter II.6 for the
AST definitions of
Prog and prog.*

Like most languages, Racket supports a number of table-like data types. Since the details do not matter, a comment and two signatures suffice:

```

#; {type Store = [TableOf Symbol Value]}

#; {Store Symbol Value -> Store }
(define (extend table x v) ...)

#; {Store Symbol -> Value }
(define (lookup table x) ...)

```

Lastly, we add a structure type for reporting errors concerning references to undefined variables:

```

(struct err [message code])
#; {type Error = .. some informative data ..}

```

These data representations basically dictate how to define the load and unload functions:

```

#; {Prog -> State}
(define (load p)
  (state p MT))

#; {State -> Result}
(define (unload s)
  (match s
    [(state (prog '() n) s) n]
    [(state (err msg xtra) s) msg]))

```

```

#; {State -> State}
(define (transition a-state)
  (match a-state

    ;; execute the first assignment statement:
    [(state (prog (cons (assg x (num n)) stmt*) e) s)
     (state (prog stmt* e) (extend s x n))]
    [(and (state (prog (cons (assg x (ref y)) stmt*) e) s)
          (? is-y-defined-in-s?))
     (define n (lookup s y))
     (state (prog stmt* e) (extend s x n))]
    [(and (state (prog (cons (assg x (ref y)) stmt*) e) s)
          (? is-y-not-defined-in-s?))
     (state (err "undefined variable" y) a-state)]

    [;; BEFORE
     (and (state (prog (cons (assg x (add (ref y) (ref z))) stmt*) e) s)
          (? are-y-and-z-defined-in-s?))
     ;; AFTER
     (define n (lookup s y))
     (define k (lookup s z))
     (state (prog stmt* e) (extend s x (+ n k))))

    [(and (state (prog (cons (assg x (add (ref y) (ref z))) stmt*) e) s)
          (? is-y-or-z-not-defined-in-s?))
     (state (err "one of the variables is undefined" x y) s)]

    ;; evaluate the return expression:
    [(and (state (prog '() (ref y)) s)
          (? is-y-defined?))
     (define n (lookup s y))
     (state (prog '() n) s)]
    [(and (state (prog '() (ref y)) s)
          (? is-y-not-defined-in-s?))
     (state (err "undefined variable" y) s)]
    [(and (state (prog '() (add (ref y) (ref z))) s)
          (? are-y-and-z-defined?))
     (define k (+ (lookup s y) (lookup s z)))
     (state (prog '() k) s)]
    [(and (state (prog '() (add (ref y) (ref z))) s)
          (? is-y-or-z-not-defined-in-s?))
     (state (err "one of the variables is undefined" x y) s)]
    ...))

```

Figure 9: A Racket implementation of the CS machine

Stop! Did you notice the prog in the definition of unload? Take a second look at section 6 if it looks unfamiliar.

Compared to these two definitions, the definition of transition is far more interesting. Due to the already-mentioned algebraic match construct, Racket

still makes it easy to translate the cases from figure 8 into code, case by case. Figure 9 shows the complete Racket definition of transition. Following the abstract description, the function consumes a state and produces one. It uses `match` to determine which *before* condition of which case of figure 8 applies. Just like in the abstract definition in figure 8, the conditional in the function definition of figure 9 specifies the cases in an order-independent manner by imposing side-conditions on the patterns.

Let's take a look at one case to understand these side-conditions, specifically the boxed case in figure 9, which implements the case in figure 8 labeled "execute $x = y + z$ (success)". The match pattern, labeled BEFORE, consists of two parts, combined with *and*:

- The first one specifies an AST shape inside of a state structure that looks pretty much like the one in the mathematical specification.
- The second line imposes an aptly named predicate on the given state, ensuring y and z are defined in the otherwise unconstrained store s .

The code labeled AFTER constructs the result state in three steps. Using the lookup function for tables, the first two retrieve the values of y and z , respectively, from the store s in the given state. These retrievals succeed, because the BEFORE condition checked that the variables have values in s . Finally, the function instantiates state with (1) a program that no longer contains the interpreted assignment statement and (2) a new store that associates the sum of the two retrieved values with the left-hand side of the interpreted assignment statement. In short, the code for this case is a direct translation of the abstract formulation into Racket code.

Stop! How would you express this case of the transition function in your favorite programming language?

Every implementation effort of a transition function potentially suffers from two problems, both causing havoc unless the conditional of figure 9 is supplemented with one more case:

- A programmer can easily make a mistake with the translation of the *before* conditions in the various cases so that some instance of state does not match any of them.
- The specification of the *before* conditions in the table notation of figure 8 may fail to create a partition of the set of intermediate states, rendering even an exact translation into code buggy.

To catch such problems, a match-based implementation must come with a catch-all clause, which is what the dots (...) at the end of figure 9 indicate:

```
[_ (eprintf "stuck state (should never happen)")
  (state (err "stuck state (should never happen)" 's) s)]
```

The `_` patterns informs future readers of the Racket code that any given state is matched. In order to inform the user, the Racket function prints an error message and returns an error state to halt the machine gracefully.

A programming language researcher refers to states that match this pattern as *stuck states*. When running a machine ends up in a stuck state, the message must inform the user that the machine crashed—or, in the terminology of old computer scientists—that it seg-faulted. One major research thrust in the field is the search for methods that eliminate the possibility of stuck states.

4 The CSK Abstract Machine: A Second Example

For the meaning of
“same,” see
section 4.4.

Several different abstract machines can express the same semantics for one and the same syntax. By choosing one over the other, a language creator emphasizes a particular perspective. For example, the CS machine mirrors our hardware interpretation of *Sample-*. A language creator who presents this machine alludes to the programmers’ sense of hardware and compilation. Or, the language creator may expect that proving a property that holds for all programs, is best done with some different abstract machine.

On occasion, programming language researchers study a particular language and find that the chosen abstract machine isn’t a good choice for their objectives. In response, they formulate an alternative abstract machine. If they do, they are obliged to show that the two semantics define identical functions from Programs to Results; otherwise they don’t know whether the outcome of their investigations hold for the original combination of syntax and abstract machine.

This section presents the CSK machine for *Sample-*, an alternative for the *Sample* language. While the CS machine elegantly interprets one assignment statement after another, its definition intimately relies on the simple nature of programs. This tight relationship between simple program syntax and machine instructions makes it difficult to adapt this machine to models of functional or object-oriented languages.

4.1 The States

As its name suggests, the CSK machine comes with three registers. The C register still contains ASTs that control the machine's execution, and the S register continues to represent the association of variables with values. But, the C register of this new machine contains *only* the expression to be evaluated, not the un-interpreted part of the program. The latter is instead placed in the additional K register.

K is short for "continuation," the mathematical term for "ASTs still to be interpreted." But, since C is taken, semanticists use the letter "K" instead.

Let's turn these informal ideas into definitions that describe the states of the CSK machine as (inductively defined) sets:

$$\begin{aligned} \text{State} &= (\text{U Initial InterM Final}) \\ \text{Initial} &= \dagger \times [\] \times \text{Program} \\ \text{InterM} &= (\text{U Expression} \cup \dagger) \times \text{Store} \times \text{Program} \\ \text{Final} &= (\text{Value} \times \text{Store} \times \text{Expression}) \\ &\quad \cup \\ &\quad (\text{Error} \times [\] \times [\]) \end{aligned}$$

The set of Values remains the set of inexact numbers; Error is also the same as for the CS machine.

Stop! Explain these mathematical definitions of sets to a friend.

The novelty in these definitions is the $\dagger$ symbol, and we definitely need an interpretation for this symbol to make sense of these states. Since C contains the next expression to be evaluated, and since it isn't always obvious which expression it is, the CSK machine instead knows two modes:

- When $\dagger$ is in C, the CSK machine searches for the next expression that it must evaluate in the AST of the K register.
- When C contains an expression, it evaluates the expression; when the value is found, it is placed into the store, according to the instruction in K from which the expression originates.

4.2 The CSK Transition Function for *Sample-*

Figure 10 displays the definition of the CSK transition function in the same format as the one for the CS machine. The cases in the left column explain how the CSK machine searches for the next expression to evaluate. The right column defines the four cases of how the machine determines the value of an expression—if possible—and how it uses this value.

Consider the second case in the left column of figure 10. Its *before* constraint on the C register signals "search." The K register is supposed to contain a program that consists of a non-empty sequence of assignment

search for expression, place value			evaluating expressions		
Ctrl.	Sto.	Kont.	Ctrl.	Sto.	Kont.
search ends: return expression			evaluate a variable (success)		
before: $\uparrow$	s	([] e)	before: y	s	k
after: e	s	([] e)	subject to: y is defined in s		
			after: n	s	k
			where n = s[y]		
search ends: assignment stmt.			evaluate a variable (failure)		
before: $\uparrow$	s	((x = rhs)::stmt* e)	before: y	s	k
after: rhs	s	((x = rhs)::stmt* e)	subject to: y is not defined		
			after: error	[]	[]
value for assignment stmt.			evaluate an addition (success)		
before: n	s	((x = rhs)::stmt* e)	before: (y + z)	s	k
after: $\uparrow$	s[x = n]	(stmt* e)	subject to: y and z are defined in s		
			after: (+ m n)	s	k
			where m = s[y]		
			and n = s[z]		
			evaluate an addition (failure)		
			before: (y + z)	s	k
			subject to: y or z is not defined		
			after: error	[]	[]

Figure 10: The CSK transition function for *Sample-*

statements followed by an expression e . Since the assignment statement's right-hand side, rhs , is an expression whose value is needed, the next expression is found. Hence the case's *after* specification places rhs into the C register and leaves the other two registers alone.

Once the C register contains an expression, the evaluation starts. Consider the third case in the right-hand column. Its *before* condition demands that the C register contains an addition expression; it imposes no constraints on the content of S and K. The side condition, though, ties together the expression with the store. It requires that the store associates values with both variables in the addition expression. Assuming that the two condi-

tions hold, the *after* specification constructs a state with the sum of the two numbers in C, with the other two registers unchanged.

To understand how an assignment is executed, we need to inspect the third case in the left column. Its *before* line says that C contains a number (*n*) and K contains a program with a non-empty sequence of assignments. The number is the result of evaluating the right-hand side of the first assignment statement in this sequence, and that immediately explains the *after* specification. The number *n* becomes the new value of the left-hand side of the assignment statement, *x*, and the assignment itself disappears.

Stop! Explain the remaining cases to a peer.

Exercise 4. Add cases for `if0` and `while0` to figure 10:

- An `if0` statement needs the value of its sub-expression. If this value is 0.0, the “then” branch is executed next; otherwise the machine picks the “else” branch.
- As for `while0`, the machine executes the sub-statement as long as the evaluation of its sub-expression yields 0.0. If the sub-expression evaluates to some other number, the `while0` statement is removed from K.

For their syntax, see figure 6 but replace Blocks with plain Statements. Don’t peek ahead. The solution is presented in the next section.

Exercise 5. Your task is to implement the CSK abstract machine for *Sample-* in your favorite programming language.

Define `csk0`, a program that consumes an S-expression from standard input, parses it, determines its meaning via the CSK machine, and prints the meaning to standard output.

The output should be a number only when the parser succeeds and the machine runs to completion without discovering variables without a store association. If parsing the given S-expression fails, the error message should say “parser error.” If the machine stops in an error state, your program should print “run-time error.”

Challenge Can the `csk0` program run forever?

4.3 But Machines Don’t Search For the Next Instruction!

At this point, you might think that these CSK “machines” are anything but machines. You know that *real* computer hardware does *not* search for the next instruction to execute. Real hardware comes with a program counter that points to the next instruction to be executed. As one instruction gets done, the pointer is advanced appropriately.

While the CS machine acts in a way that resembles hardware, recognizing the relationship between the CSK machine and a computer is much more difficult. Here is an attempt. First, its K register does contain a sequence of instructions. Second, S still points to a store that is like a hardware memory. Finally, the CSK machine's C register mostly drives arithmetic instructions like an ALU on a CPU, meaning they compute sums; in other cases, it demands the retrieval and storing of numbers in memory.

A CSK machine differs from hardware in that it carefully explains how the program counter is modified. The machine leaves the right-hand side of an assignment statement in place while it evaluates the expression. If you solved exercise 4, you also know that the machine *copies* the loop to iterate, while compiled code on an actual hardware machine moves the program counter backwards.

Coming from a simple-minded, hardware-based understanding of execution, copying code feels counter-intuitive, but we have reasons to formulate such abstract machines. First and most importantly, such abstract machines provide an easily adaptable formalism for basically any kind of language feature. Here "easily adaptable" means that the machine can directly deal with ASTs; technically, it is often possible to deal with linguistic features via cases in the transition function. Second, it is straightforward to implement these machines in almost any programming language. While differentiating the cases of the transition function may take a bit more code in one language than another one, it remains an achievable task. Finally, the transition function doesn't make big leaps but performs small steps. An entrepreneurial reader may wish to equip the CSK machine with code for optionally displaying these steps, one at a time. Showing such steps is, to this day, some programmers preferred way of debugging.

4.4 A Little Bit of Theory: The Equivalence of Semantics

The addition of the CSK machine to our repertoire raises the kind of question that theoreticians in the field of programming languages often have to consider:

do the CS machine and the CSK machine define the same semantics?

And you may wonder what it even means for two semantics to be the same.

So suppose you write a program P in *Sample-*. You use your parser to make sure it is well-formed. If so, you get the AST of P and you proceed to apply `runMachineCS` function to this tree. When the final state is unloaded,

the result is either a number or an error. What if we proceeded in this manner but used the runMachine_{CSK} function instead?

Stop! Did you notice the subscripts? Do you know what they mean?

What you should expect is that the second process should produce the same outcome as the first one. Indeed, you should get the same outcome no matter which P you start from.

Since we can understand the runMachine functions as *mathematical* functions, we can turn this informal claim into a mathematical theorem about them as follows:

For all programs P in *Sample-*, $\text{runMachine}_{CS}(P) = \text{runMachine}_{CSK}(P)$. $\square$

A language theoretician would use mathematical methods (relations, induction) to prove this statement formally.

In this book, we are primarily concerned with pragmatics, not with theoretical-mathematical arguments that validate such relationships. Still, the relationship between a language's capabilities and work situations do depend on theoretical arguments on occasion. So when needed we mention these arguments but without presenting any of the mathematical material. Readers interested in this mathematical material may wish to check out traditional text books on programming languages.

5 Bare Bones: an Abstract Machine Project

Let's equip the *Bare Bones* BNF of section 8 in chapter II with a semantics. Programs in this BNF consist of sequences of Statements, followed by an Expression. This production is pretty much the same as for BNF of *Sample-*.

According to the second production of the *Bare Bones* BNF, a Statement is either an assignment statement, a conditional, or a loop. In other words, the *Bare Bones* syntax adds two kinds of Statements to *Sample-*. Conditionals and loops may contain Blocks. Such a Block is either a plain Statement or a non-empty sequence of Statements, marked with the keyword *block*.

Finally, a *Bare Bones* expression is either a literal numeric constant, a reference to a variable, an addition expression, a division expression, or a comparison expression. Each of those contains exactly two references to variables. As for three literal numeric constants (1.0, 2.0, and 3.0), we think we know the meaning of these literal constants but these constants actually deserve a separate discussion. See section 5.2.

In sum, *Bare Bones* syntax extends the syntax of the BNF of *Sample-* in two essential places:

- the production for Statement;

The CSK machine's transition function must cope with this extended production. Concretely, the extended transition function needs cases for using the value of expressions in `if0` and `while0` plus transitions that search for expressions in such statements.

- the production for Expression.

The machine's transition function must also come with cases that cover division and comparison expressions. While the former can appeal to a mathematical function or an operator from the implementation language, the latter needs some additional attention.

search for expression, place value			evaluating expressions		
Ctrl.	Sto.	Kont.	Ctrl.	Sto.	Kont.
pick then branch from if0			decide whether to run while loop (positive)		
before:	n	s	before:	n	s
subject to: n is basically 0.0			subject to: n is basically 0.0		
after:	↑	s	after:	↑	s
((if0 tst thn els)::stmt* e)			((while0 tst body)::stmt* e)		
((thn @ stmt*) e)			where o = (while0 tst body)::stmt*		
pick else branch from if0			decide whether to run while loop (negative)		
before:	n	s	before:	n	s
subject to: n is not close to 0.0			subject to: n is not close to 0.0		
after:	↑	s	after:	↑	s
((if0 tst thn els)::stmt* e)			(stmt* e)		
search for expression in if			search for expression in while		
before:	↑	s	before:	↑	s
after:	tst	s	after:	tst	s
((if0 tst thn els)::stmt* e)			((while0 tst body)::stmt* e)		
((if0 tst thn els)::stmt* e)			((while0 tst body)::stmt* e)		

Figure 11: The CSK transition function for *Bare Bones*: `if0`, `while0`

5.1 The CSK Meaning of Conditionals and Loops

Figure 11 adds six cases to the transition function of the CSK machine from figure 10. All other functions of the CSK machine can be re-used as is.

The left column of the figure presents the cases that deal with `if0` statements. First, when the expression evaluates to 0.0, the machine picks the `then` sub-statement to execute next. Since it doesn't know which expression is to be evaluated next, it initiates a search. Second, when the expression evaluates to a number other than 0.0, the machine picks the `els` sub-statement and searches for an expression in it.

The right column of figure 11 is about the cases that process `while0` statements. First, when the expression evaluates to 0.0, the machine executes the body sub-statement of the loop followed by the loop. To this end, it copies the `while0` loop and adds it to the front of `stmt*`, the remaining sequence of statements. Second, otherwise the machine discards the loop statement. The machine initiates a search for the next expression in both cases.

Both of these cases use the notation `b @ stmt*` where `b` is a Block. If `b` is a plain statement, it is added to the front of `stmt*`. Otherwise it is a sequence of statements, marked with `block`, and this sequence is prepended to `stmt*`.

A second look at the last cases in both columns confirms how searching for an expression terminates when the machine encounters a `if0` or `while0` statement as the first one in `K`. In both cases, the machine picks the sub-expression of the statements, because its value is needed to make a decision about what to execute next.

5.2 The Meaning of Numbers, Errors

Stop! Do you notice an unusual word in some of the cases in figure 11?

The unusual word is “basically” in two cases: the first in each column. While informal and even colloquial, this word indicates a problem and requests your attention. If 0.0 were a rational number, the specification of these cases would not deserve an extra sentence. But, as you might know from an introductory course on programming, computer hardware does not implement real numbers. It represents them as *inexact* numbers, generally known as IEEE floats or doubles. And, as you might also recall, it really is wrong to compare inexact numbers (doubles) for equality.

If it is wrong to compare some value to, say, 0.0, you might wonder what the implementation of such a transition function might look like. The key is to rely on some basic advice concerning inexact IEEE numbers:

To compare two IEEE doubles, check whether the difference between the two numbers is less than some small number ϵ .

Put differently, the implementation of the cases in figure 11 must use an auxiliary function that checks whether the value of an expression in a test

This standard is known as IEEE 754. Almost all programming languages therefore use this standard to represent numbers.

position is close to 0.0 or not. Otherwise it is impossible to implement the transition function in a way that yields predictable answers.

Exercise 6. Develop the function `areBasicallyEqual` in your favorite programming language. It consumes two doubles and determines whether their difference is smaller than a fixed constant ε .

Having cleared the first obstacle, we can turn to the interpretation of numeric expressions. Figure 11 specifies that the interpretation of an addition expression with an appeal to the addition operator from mathematics, which we all know and love. Sadly, when it comes to IEEE's inexact numbers, even addition of such numbers is not the addition we know from grade school.

Stop! Write a short program in your favorite language that adds two very large doubles and prints the result.

Assuming you choose a truly large number the result is `+inf.0`. Computer scientists refer to this phenomenon as *numeric overflow*, meaning the result is such a large number that computer hardware can't represent it. Similarly, the result of a division operation can be so small that it can't be represented; this is called *numeric underflow*.

In recognition of these problems with IEEE doubles, the grammars of this book include only small literal constants. Although this constraint does not prevent the creation of programs that cause overflow or underflow, it makes it rather hard. So, when you compare your executable models with those of your peers, don't write such test cases.

Division comes with yet another problem. In grade school we learn that division doesn't work when the divisor is 0. A mathematician says that division is a partially defined operation. Hardware people trick people into thinking that `(/ 1.0 0.0)` is `+inf.0`, too.

For pedagogic reasons, this book deviates from the IEEE standard for just this one case. Instead of allowing division by 0.0 to yield infinity, the CESK machine for *Bare Bones* follows grade-school practice and signals an error. The specification of this distinction consists of two cases in the CESK transition function. Figure 12 presents these cases. Each case comes with a side-condition labeled "subject to." In the first one the side-condition adds the constraint that the divisor is *not* 0.0, while the second case's side-condition expresses the opposite. The *after* part of the second case specifies a state whose C register contains an error element, which makes it a final state and causes the machine to stop.

Control	Environment	Store	Kontinuation
evaluate a division (success)			
before: (y / z)	e	s	k
subject to: z is not close to 0.0			
after: double-ieee-division-of-yn-by-zn yn zn e		s	k
where yl = e[y]			
and zl = e[z]			
and yn = s[yl]			
and zn = s[zl]			
evaluate a division (failure)			
before: (y / z)	e	s	k
subject to: z is basically 0.0			
after: error	e	s	[]

Figure 12: The CESK transition function for *Bare Bones*: division

5.3 Project Tasks

Exercise 7. The specification of the CSK machine’s transition function lacks a case for the comparison expression. Formulate this missing case in the style of figure 12.

Unlike in the case of addition and division expressions, there is no obvious mathematical operator for comparing two numbers. Indeed even your favorite programming language is unlikely to supply a corresponding method or procedure. After all, the comparison of two numbers must yield a number, because numbers are the only values in *Bare Bones*. For the abstract specification, define an appropriate case-based auxiliary function.

Since the point of a comparison is to make decisions—which branch of a conditional to evaluate or whether to continue the execution of a while loop—returning 0.0 for one of the two cases is also natural. We choose to say the comparison function yields 0.0 when the two given numbers are the same (in the sense of IEEE sameness). For the other case, choose an appropriate double that is not 0.0. Explain the rationale for these choices.

Exercise 8. Your task is to implement the CSK abstract machine for *Bare Bones* in your favorite programming language.

Define *csk*, a program that consumes an S-expression from standard input, parses it, determines its meaning via the CSK machine, and prints the meaning to standard output.

The output should be a number only when the parser succeeds and the machine runs to completion without discovering variables without a store association. If parsing the given S-expression fails, the error message should say "syntax error". If the machine halts in an error state, your program should print "run-time error".

Hint To help with debugging, the machine can also print additional error information to the error device of your operating system as it transitions to an error state.

If you completed exercise 5, this task represents a request to add six cases to the existing transition function.

Challenge Can the `csk` program run forever?

Why do languages demand variable declarations?

1 The Pragmatics Question

Programmers suffer when the chosen language does not support variable declarations. One of these languages is Python, whose core syntax is like *Bare Bones*'s, though without parentheses. Here is an example, using the model's syntax:

```
((title = 1.0)
 (offset = 0.0)
 ... dozens of lines: loops, conditions, functions ...
 ... possibly with assignment statements to 'offset' ...
 (titel = title + offset)
title)
```

Stop! Take a look at the boxed code, which highlights the problem. Has something like this happened to you when you wrote Python code, perhaps with a field assignment?

Unlike the problem of references to variables without value in an expression, which the model's semantics naturally deals with, the problem of assignments to misspelled variables goes undetected. Neither the *Bare-Bones* parser nor the *BareBones* semantics notice the problem. Hence, assuming this program does not suffer from bad variable references in the omitted lines, it is syntactically well-formed and computes a number according to the runMachine semantics. It just may not be the number that the program's creator desires.

If it is not a syntax problem and if it is not a semantics problem, the program exposes a problem of pragmatics. We defined pragmatics as a concern of language combined with work situations, and since we have identified assignments to misspelled variable names as a language concern, we must point to work situations where this issues creates an actual problem.

Some people mistakenly refer to such a wrong answer as a semantic error. It should be called a logical error.

So, imagine a quality assurance (Q/A) engineer whose task is to write black-box tests. Unless such tests force the value of *offset* to differ from 0.0, these test may never reveal that the program suffers from a misspelled variable name in an assignment statement. Once this software gets deployed, though, a user may trigger a program run with inputs that *do* expose the problem in an observable, and possibly damaging, manner. If this malfunction gets reported, the bug causes additional work for the on-call developer and it may cast blame on the Q/A engineer. In short, these typos can pose serious problems to Q/A engineers and users in languages without declarations. Let's consider solutions and the implications of such solutions.

1.1 Design Choices

Almost all contemporary programming languages make the declaration of variables mandatory. Every variable reference and every assignment to a variable must connect to a variable declaration. Indeed, language creators recognized the pragmatics problem in the late 1950s and came up with this solution by 1960: Since then, many language implementations check this property even before a programmer runs a program. Unless the property holds, implementations do not run the program at all. Nowadays, the implementation reports such problems to the IDE, which marks up occurrences of undeclared variables so that programmers can fix them quickly.

Generalizing a bit, two fundamental changes to the syntactic part of the model language answer this first question of pragmatics:

1. The language's syntax must include a facility for declaring variables.
2. The language must come with an explanation of what it means to relate a variable to a distant declaration.

Property 2 points out that more than parsing is needed to ensure that the syntax of a program satisfies all constraints. Even after a parser confirms that the program is *well-formed* syntax according to some BNF, we do not know whether all variable names in the program relate to a declaration. The problem is that a parser checks adherence to a context-free BNF, and a BNF cannot express context-sensitive constraints, which relate distance pieces of syntax to each other.

There are two ways to check constraints that a BNF cannot express:

- The *semantic* approach adds conditions to the relevant cases of the transition function of the underlying abstract machine. An implementation translates such conditions into run-time checks.

Let's make this concrete for the problem of undeclared variables on the left-hand side of an assignment statement:

	Ctrl.	Sto.	Kont.
value for an assignment statement (success)			
<i>before</i>	n	s	((x = rhs) :: stmt* e)
<i>subject to:</i>	x is a declared variable		
<i>after</i>	†	s[x = n]	(stmt* e)
value for an assignment statement (failure)			
<i>before</i>	n	s	((x = rhs) :: stmt* e)
<i>subject to:</i>	x is <i>not</i> a declared variable		
<i>after</i>	error	[]	[]

That is, a protocol between the parser and the machine-transition function supplies the latter with the set of declared variables. By inspecting this set every time the machine interprets an instruction that involves variables, it can terminate the program execution with an appropriate error message.

- The *syntactic* approach checks that the AST also satisfies properties not expressible via a BNF *before* the program gets run. Unlike a parser, these traversals keep track of the surroundings of sub-trees and can thus check properties that relate distant pieces of syntax.

Programming languages realize this approach in two different ways:

- An *internal* implementation consists of a AST traversal. If such an implementation-internal traversal discovers a constraint violation, the program does *not* get run.
- An *external* implementation consists of a separate tool in the language's ecosystem, often dubbed a *linter*. Since a linter is an external tool, a software developer has to opt into its use but can also ignore it and its reports.

For the specific case of variable-declaration checking, the traversal functions take an extra argument, called an accumulator. It represents the set of declared variables, and the functions can thus check every variable name against this set.

Having figured out the design choices, the next step of working out a solution to a problem of pragmatics is to analyze their (dis)advantages.

1.2 Costs and Benefits

Every design decision deserves a cost-benefit analysis, even a decision as simple as the one concerning variable declarations. In the case of programming language pragmatics, such cost-benefit analysis takes into account the working situations of the programmer that raised the question of pragmatics *and* the work that the decision imposes on the language creator. While shifting work from a lot of programmers to a few language creators is generally a good idea, it may become unreasonable when the decision is close.

As far as programmers are concerned, variable declarations come with some obvious positives and one negative for the programmer. On the positive side, variable declarations protect programmers from unwanted undeclared variable errors, but they do so in different ways:

- A semantic implementation catches the problem as the program is executed and possibly after bad things have happened due to outputs.
- A syntactic implementation comes in one of two forms:
 - The internal variant does not run invalid programs. It thus prevents bad effects. On the flip side, this strict checking imposes some extra work on the programmer, namely inspecting and fixing the spelling of all undeclared variables.
 - An external linter offers the flexibility of checking the program and running it even if the tool points to potential problems. But, since a developer can ignore the linter or its warnings, broken code may end up in a system.

On the negative side, the requirement to declare variables before use imposes a small amount of extra keyboarding on the programmer. On balance, programmers benefit, and the positives clearly favor the addition of variable declarations and syntactic checking.

As far as the language creator is concerned, variable declarations impose two costs. The first cost is an intellectual effort. In addition to a BNF specification, which determines when some text is a *well-formed* program, the creator is now responsible for articulating constraints governing distinct and distant pieces of code. If a given program satisfies these additional constraints, it is considered well-formed and *valid*.

The second cost of variable declarations and their accompanying constraints consist of extra implementation work. In the case of a semantic solution, the language implementation needs to collect the set of declared

variables and hand them over to the machine. In the case of a syntactic solution, the language implementer must realize an extra pass over the AST.

Note An AST traversal can assume that the program is well-formed (or the erroneous pieces are marked with distinct error nodes in the tree). Hence the validity checker does not have to rediscover which production is being inspected (and can ignore erroneous sub-trees). And this clarifies, why the parser's creation of an AST is so important. It reduces the implementer's work. So, while the introduction of ASTs may have appeared to complicate things at first, it should now be clear that it simplifies language implementations.

Given this cost/benefit analysis and the costliness of safety and security bugs, most language creators opt for syntactic validity checking. Indeed, modern programming language researchers attempt to shift ever more checking from the machine to AST traversal passes.

2 Valid Syntax: In General

As the preceding section suggests, syntactic validity checking is a function that consumes an abstract syntax tree. To work out the precise nature of its signature, let's start by recalling the generic signature of a parser:

```
parse : PlainText  $\longrightarrow$  AST
```

Its domain is, in general, plain text, typically read from an input device; its range is AST, which is the set of abstract syntax trees enriched with error nodes. The IDE uses the latter to inform programmers of syntax mistakes, that is, violations of the language's BNF.

If the parse function discovers a violation, it is unnecessary for the model to check whether the program comes with declarations for all variables. That is, we may assume that the validity checker consumes only the AST-subset of AST. The question is what this function should return. Just like parse, the validity checker may succeed and confirm that all variables mentioned in the program come with a declaration; or it may fail and discover a variable name without a declaration. In the failure case, programmers would once again want the IDE to highlight undeclared variables, meaning the AST should be marked appropriately:

```
// check that each variable name comes with a declaration  
validityCheck : AST-  $\longrightarrow$  AST
```

IDEs tend to report both syntax errors and validity problems at the same time.

Unfortunately, working out the top-level signature is the easy part; the difficult aspect of validity checking is due to the physical separation of variable declarations from variable occurrences, be that on the left-hand side of an assignment statement or inside of an expression. Following proper program design principles, `validityCheck` dispatches to auxiliary functions that deal with declarations, statements, and so on. And this implies that these auxiliary functions need to communicate which variables have been declared. We can express this relationship with two sketched signatures:

```
// check that variables in a piece a program are declared
validityCheckOfPieces :
  ASTforPiece- Set<Symbol>  → ASTforPiece

// determine the set of declared variables
declaredVariables :
  ASTforDecl- → Set<Symbol>
```

The first one says that checking pieces of a program—declarations, statements, expressions—requires knowledge about which variables have been declared. We use sets of variables to represent this knowledge. The second one is for a function that extracts this set of variables from a sequence of variable declarations.

```
(define (main)
  (define s (program->ast (read)))

  ;; STATIC
  (define parsed (program->ast s))
  (unless (plain-ast? ast-well-formed)
    (error "well-formed program expected"))

  (define valid (closed-program parsed))
  (unless (plain-ast? valid)
    (error "valid program expected"))

  ;; DYNAMIC
  (runMachine valid))
```

Figure 13: Language implementations consist of many passes

Now that we have parsers, validity checkers, and a machine, it is time to adapt the main function from figure 5 to cope with them all. Figure 13 explains how this composition works. The function reads an S-expression from the standard input device, followed by two traversals of the program:

parsing and validity checking. Each traversal is followed by a conditional that sends an error signal to the caller unless the result of the traversal is an abstract syntax tree without error nodes. If both of these traversals succeed, `main` uses `runMachine` to determine the result of the well-formed and valid program—either a number or an error message.

The figure also indicates a bit of terminology concerning language implementations, including executable models. Each such implementation consists of two parts: a *static* one and a *dynamic* one. The static part always comes in the shape of a series *passes*, each of which checks some properties and potentially computes information about the given program. Furthermore, pass $n + 1$ always consumes the results of pass n and relies on the properties it has checked. The dynamic part consists of just the semantics, that is, a piece of functionality that consumes well-formed and valid programs to determine its value. While our models use abstract machines, a proper implementation typically realizes this functionality via a combination of hardware and software.

Going forward, this book presents additional static passes and significantly richer abstract machines than the CS or CSK machine. In this chapter still, we also explain how static checks eliminate dynamic checks. Put differently, the construction of an abstract machine—or the code sent to hardware—may assume the success of the static passes and that the corresponding properties hold. Indeed, this is also true for the communication between the language implementation and the IDE; it can use information gathered from static passes to assist the programmer.

3 Valid Syntax: An Example

Let's illustrate the abstract explanation from the preceding section with an extension of the *Sample*-language that comes with variable declarations:

```
Program      ::= (Declaration* Statement* Expression)
Declaration  ::= (def Variable Expression)
Statement    ::= (Variable = Expression)
Expression   ::= 1.0 | 2.0 | 3.0 |
                  Variable |
                  (Variable + Variable)
```

The symbol `'def'` is a keyword, not a variable.

We refer to this language as *SampleD*. A *SampleD* program is a single S-expression, though it consists of up to three pieces: a potentially empty sequence of variable declarations, followed by a potentially empty sequence

Remember that keywords merely separate S-expressions that play one kind of role from those that play others.

of (assignment) statements, wrapped by a single expression. The addition of a production for Declarations is the truly new part. It uses one additional keyword: *def*, similar to the keywords in *Sample*. And like the grammar for *Sample*, the grammar for *SampleD* does not permit programmers to use *def* as a variable name.

Here are the two *Sample* programs from the preceding chapter modified to satisfy the Program production of the *SampleD* BNF:

well-formed, valid	well-formed, invalid
<pre>((def a 1.0) (def b 2.0) (def temporary (a + b)) (def c (a + temporary)) (temporary = 3.0) (c + temporary))</pre>	<pre>((def a 1.0) (def b 2.0) (def temporary (a + b)) (def c (a + temprary)) (temporary = 3.0) (c + temporary))</pre>

A comparison with the preceding chapter tells us that the first three assignment statements have been changed to variable declarations, which consist of a symbol and an Expression. The box around *temprary*, the misspelled variable inside the addition expression, indicates where an IDE should flag the code as invalid.

Stop! Formulate a well-formed and valid program whose sequence of variable declarations is empty.

Stop! Did you notice that it is a *variable declaration* that is invalid?

The right-hand column illustrates that a variable declaration may be well-formed but invalid. In the case of *SampleD*, a variable declaration consists of a name and an expression. The latter may refer to variables—and those variables should already be declared.

The word “already” points back to the word “sequence” (of variable declarations), which implies order. So, if a programmer writes

```
((def oneVariable 1.0)
 (def anotherVariable 2.0)
 (def andAThirdVariable (oneVariable + anotherVariable))
 ...)
```

then the validity checker inspects one such declaration after another. In this particular example, the first declaration checks out because there are no variables on the right-hand expression; the second one is valid for the same reason. Key is that each declaration contributes one variable to the set of declared ones. Hence, right before the validity checker inspects the third declaration the set of declared variables is {*oneVariable*, *anotherVariable*}. The checker’s next task is to look at the expression part of the third declaration,

which is an addition expression. Such expressions consist of two variable references separated by `+`. For each of these variables, the validity checker must clarify whether they are already in the set of declared variables. For this example they are, and therefore the checker blesses this sequence of declarations.

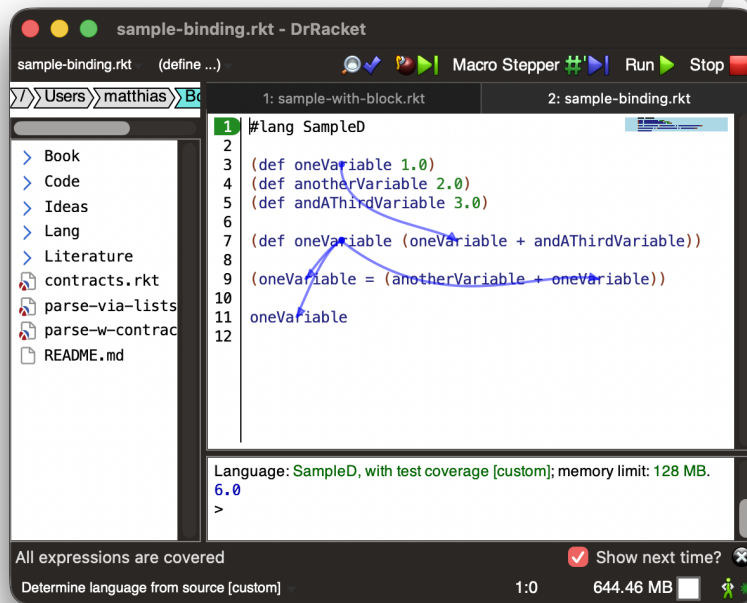


Figure 14: Binding in the *SampleD* language

Our explanation immediately raises the question whether it is acceptable that such a sequence contains *two* declarations for the *same* variable. Here is a corresponding modification of the running example:

```
((def oneVariable 1.0)
 (def anotherVariable 2.0)
 (def andAThirdVariable 3.0)

 (def oneVariable (oneVariable + andAThirdVariable))
 ...)
```

And the answer to this question is “it is up to the language creator.”

Some creators disallow duplicate variable declarations; others accept them; and in some languages one form of variable declaration is made for duplicate declarations and a different one prohibits them. For illustrative purposes, the *SampleD* language accepts duplicate variable declarations, and figure 14 illustrates with a screenshot what this means.

The program in the figure starts with the very sequence of declarations under discussion and adds an assignment statement plus a final expression. The arrows indicate how variable references are resolved. While the first declaration of `oneVariable` points to the occurrence in the expression of line 7, the second declaration—on line 7—points to three occurrences below: one on the left-hand side of the assignment, one on the right, and one in the final expression. The technical term for this relationship among variable declarations and variable occurrences is *binding*; that is, people say the occurrence inside the expression is bound by the first declaration and the occurrences on lines 9 and 11 are bound by the second declaration.

Note Like in mathematics, some languages come with recursive declarations or even several declarations that refer to each other. A recursive declaration simultaneously defines a name and uses the very same name inside the right-hand side. While such a self-referential definition makes sense for, say, a function, it would be pointless for variable declarations in a language like *SampleD*.

```

(struct prog
 [declarations
 statements
 end])

(struct decl [var rhs])

(struct assg [lhs rhs])
(struct expr [])

(struct num expr (n))
(struct ref expr (name))
(struct add expr (left right))

(struct err (msg))

#; {type Prog =
      (U Err
       (prog
        List<Decl>
        List<Stmt>
        Expr))}
#; {type Decl =
      (U Err
       (decl Symbol Expr))}
#; {type Stmt =
      (U Err
       (assg Symbol Expr))}
#; {type Expr =
      (U Err
       (num N)
       (ref Symbol)
       (add Symbol Symbol))}
#; {type Err =
      (err String)}
```

Figure 15: An AST data representation for *SampleD*

The screenshot is for an implementation of *SampleD*, with the *DrRacket* IDE tailored to this implementation via the first line.

3.1 The Implementation

Figure 15 displays the adaptation of our AST data representation to *SampleD*. It comes with one new structure type, *decl*, and a corresponding type-like comment. The *prog* structure type has one additional field, *declarations*, which is going to contain the AST of the declarations. Otherwise, the AST data representation is like the one in figure 4; a quick comparison may help.

```

#; {Prog- -> Prog}
(define (closed-body p- declared----)
  (match-define (prog d-* s-* e-) p-)
  (define d* (closed-decl* d-*))
  (define declared (decl->declared-variables d*))
  (define s* (closed-stmt* s-* declared))
  (define e (closed-expr e- declared))
  (prog d* s* e))

#; {List<Decl-> -> Set<Variable>}
(define (decl->declared-variables d-*) ...)

#; {List<Decl-> -> List<Decl>}
;; accumulate the declared variables, checked declarations
(define (closed-decl* d-*) ...)

#; {List<Stmt-> Set<Variable> -> List<Stmt>}
(define (closed-stmt* s-* declared) ...)

#; {Expr- Set<Variable> -> Expr}
(define (closed-expr e- declared) ...)

```

Figure 16: A validity checker for the *SampleD* language

The chosen AST representation drives the design of the validity checking functionality. Since there are five kinds of AST nodes, the validity checking functionality consists of (at least) five functions or methods: one per kind of AST. Furthermore, the function for programs is the entry point and can thus serve as an overview of the complete piece of functionality.

So take a look at the top-most function definition in figure 16. It uses the name *closed-prog*, because programming-language researchers say a “program is closed” if all variable occurrences refer to a variable declaration. Because this function consumes an error-free AST (*p-*), it suffices to match it against the only kind of structure instance of *Prog-*. The result of this match are the three pieces of an AST for a program: the potentially empty list of error-free declarations (*d**, the potentially empty list of error-free state-

ments (s^*), and the final, error-free expression (e). Each of these pieces is processed in turn, yielding ASTs that potentially contain error nodes when undeclared variables are discovered. The last line of `closed-prog` assembles these validity-checked pieces back into a complete `prog` node.

```

#; {List<Decl-> -> Set<Variable>}
;; collect the declared variables
(define (decl->declared-variables d-*)
  (for/set ([decl1- d-*])
    (decl-var decl1-)))

#; {List<Decl-> -> List<Decl>}
;; check and optionally annotate the declarations
(define (closed-decl* d-*0)
  #; {List<Decl-> Set<Variable> -> List<Decl>}
  ;; ACCU 'declared' is the declared variables between
  ;; 'd-*0' and 'd-*'
  (define (closed-decl*/accu d-* declared)
    (cond
      [(empty? d-*) '()]
      [else
       (match-define (decl x rhs) (first d-*))
       (cons (decl x (closed-expr rhs))
             (closed-decl*/accu (rest d-*) (set-add declared x))))])
  ;; -- IN --
  (closed-decl*/accu d-*0 (set)))

```

Figure 17: A validity checker for declarations in *SampleD*

Of the three processing steps of `closed-prog`, the one concerning declarations is the only interesting one. A sequence of declarations contributes not one, but two results to the overall check: (1) the declared variables and (2) the results of checking each declaration. For simplicity, we compute these two results with two functions; see figure 17.

While the definition `decl->declared-variables` is straightforward—it uses a `for/set` loop to collect the set of declared variables—the second function definition deserves an explanation. First, note the distinction between the input—`List<Decl->`, a list of error-free ASTs—and the result—`List<Decl>`, a list of ASTs that may contain error nodes. Returning such ASTs is necessary so that the IDE can inform programmers of the kinds of mistakes that the beginning of the section illustrates. Second, the `closed-decl*` function traverses the list. It checks each declaration relative to the given set of declared variables and then “loops” over the remaining declarations with a

set of declared variables that is enriched with the one just found.

Let's take a look at how `closed-decl*` goes about checking declarations. Its first step is to deconstruct the existing declaration into the variable and the right-hand side of the declaration. The latter is an expression that may contain zero, one, or two variables. The `closed-expr` function checks such expressions, that is, it ensures that all of these variables are declared and, if not, constructs an AST with appropriate error nodes. In turn, `closed-decl*` combines whatever AST `closed-expr` returns with the declared variable into a new AST: `(decl x rhs)`.

All remaining functions for validity checking in figure 16 are much more straightforward than the two concerning variable declarations.

Stop! Try to sketch the last three functions in the figure. They are specified via signatures and meaningful names.

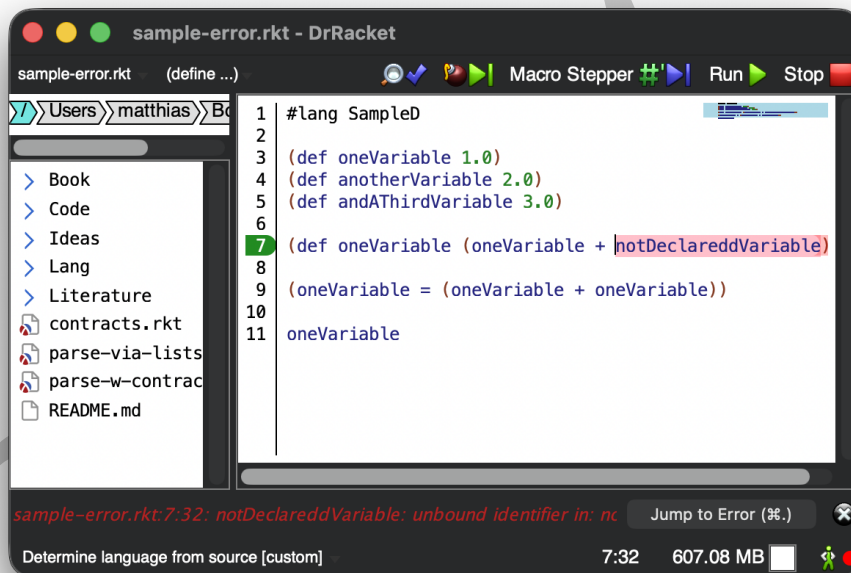


Figure 18: Undeclared variable errors in an IDE for the *SampleD* language

3.2 An Implementation Suitable for an IDE

Take a second look at figure 14. Arrows in this figure indicate which variable declaration binds which variable occurrence. Question is how an IDE can know where to anchor these arrows and where to point them to. Now take a first look at figure 18. It shows how an IDE for *SampleD* highlights a variable occurrence without a corresponding variable declaration. Again, the question is how an IDE can highlight this part of the program text. Finally figure 19 illustrates yet another case when an IDE clearly needs more than the plain AST data representation of this book: a name refactoring tool for the *SampleD* language.

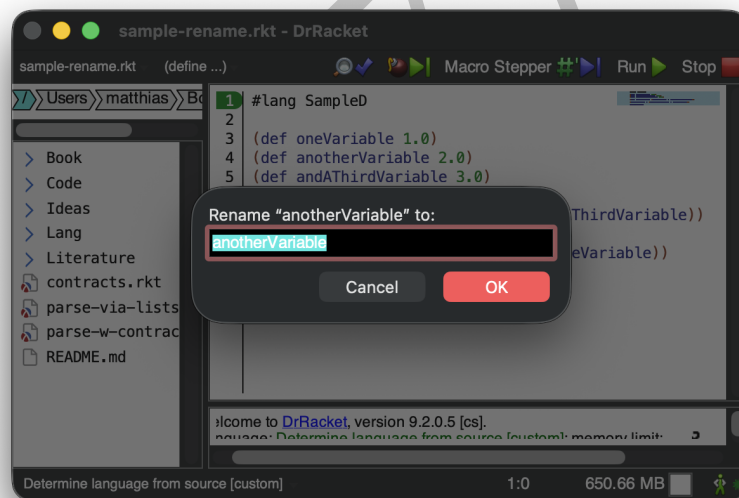


Figure 19: Name refactoring for the *SampleD* language

An IDE needs an abstract syntax representation that includes the data representations of the essential pieces of some program text *and* the source locations of all these pieces. For example, the DrRacket IDE for *SampleD* records the line and column for each piece of plain text in an AST node. In order to perform a decent job with drawing arrows between variables, the IDE also records the span of the names. Even plain compilers that wish to report syntax and validity errors in a productive manner need to include such information in their data representations.

Put differently, we once again expose the difference between a *model* of some aspect of languages and an actual *realization* of the same feature in the tool chain that software developers use. While the executable model suffices to gain an understanding of what, for example, validity checking means, you must keep in mind that it is just that: a model.

4 Static Checks Eliminate Dynamic Checks

Checking the validity of a *SampleD* program with declarations comes with the additional benefit of simplifying the machine. Recall the CS and CSK machine designs from the preceding chapter. In both cases, the machine has to have transitions for the cases when variables don't have a value in the store yet. Return to figure 10, and inspect the right column. It presents two pairs of transitions for evaluating expressions: each pair consists of a "success" and a "failure" case.

The cross-checking of variable declarations and variable occurrences *before* the program is handed to the abstract machine, eliminates the need for these case distinctions. All variable declarations precede all the program's statements and final expression. The expressions in the sequence of variable declarations may refer only to already declared variables. Finally, each variable declaration immediately associates a value with the declared variable, so that all variable references are guaranteed to have a value in the store register *S* of an abstract machine.

Figure 20 presents the CSK transition function for *SampleD* programs. It consumes well-formed and valid programs, whose variable declarations have been replaced by assignment statements. In other words, a simple rewrite of all *decl* nodes into *assg* nodes turns the AST into a program that can run on the simplified CSK machine.

Stop! Design and implement an AST traversal that performs this simple rewrite for your chosen data representation.

The next chapter shows that this approach to declarations does not work in general.

4.1 A Bit More Theory: Soundness of Validity Checking

The elimination of run-time checks demands trust in the validity checking idea and its implementation. From the perspective of theory, the relationship raises the following question:

does the validity checker eliminate all cases in which a SampleD program may try to retrieve a variable that does not have a value?

search for expression, place value			evaluating expressions		
Control	Sto.	Kont.	Control	Sto.	Kont.
search ends: return expression			evaluate a variable		
before: $\uparrow$	s	([] e)	before: y	s	k
after: e	s	([] e)	after: n	s	k
			where $n = s[y]$		
search ends: assignment stmt.			evaluate an addition		
before: $\uparrow$	s	((x = rhs)::stmt* e)	before: (y + z)	s	k
after: rhs	s	((x = rhs)::stmt* e)	after: (+ m n)	s	k
			where $m = s[y]$		
			and $n = s[z]$		
value for assignment stmt.					
before: n	s	((x = rhs)::stmt* e)			
after: $\uparrow$	$s[x = n]$	(stmt* e)			

Figure 20: The CSK transitions for well-formed, valid *SampleD* programs

More generally, you may briefly wonder whether any validity checking is “good checking.” The answer is obvious, of course. A validity checking function could just return the given AST without ado:

```
#; AST- → AST
(define (closed-program p-)
  p-)
```

And this kind of checker—unlike the one in figure 16—is unhelpful.

If we once again accept the CSK semantics as a mathematical function and also consider the validity checker in figure 16 as one, we can characterize their relationship as a mathematical theorem:

For all programs P in *SampleD*, if $\text{closedProgram}(P) = P$ and if $\text{runMachine}_{\text{CSK}}(P) = R$, then R is a *Value*.

Critically, *runMachine* does not signal an exception due to the absence of a variable from the store. A corollary is that the transition function without failure cases, as shown in figure 20, works correctly.

Language researchers borrow the word “soundness” from the study of formal logic to name this theorem. In general, every validity checker poses

the question whether it is sound, meaning whether it has a “good” relationship with the semantics of the language. Chapter VIII resumes the discussion of soundness theorems in a truly interesting context.

5 Declared: a First Complete Model

5.1 Project Tasks

Let’s expand on the *Bare Bones* project from section 5. That section’s *Bare Bones* language extends *Sample-* with conditionals and loops. Just as with the latter language, *Bare Bones* programs may contain variable references that have no value in the CSK store (of exercise 8).

```

Program      ::= (Declaration* Statement* Expression)

Declaration ::= (def Variable Expression)

Statement    ::= (Variable = Expression)
               | (if0 Expression Block Block)
               | (while0 Expression Block)

Block        ::= Statement
               | (block Statement+)

Expression   ::= GoodNumber
               | Variable
               | (Variable + Variable)
               | (Variable / Variable)
               | (Variable == Variable)

The set of Variables consists of all Names, minus keywords.

The set of GoodNumbers comprises all inexact numbers
(doubles) between -1000.0 and +1000.0, inclusive.

```

Figure 21: The concrete syntax of the *Declared* language

Figure 21 presents the BNF for *Declared*, a model language that extends the grammar for *Bare Bones* with variable declarations. Only the first two productions differ; the remaining ones are adapted as is.

The following exercises guide you through the process of adapting the executable model for *Bare Bones* to this new language.

Exercise 9. Your task is to modify your AST data representation for *Bare Bones* from exercise 3 and the corresponding parser to accommodate the variable declarations of *Declared*. Given an S-expression (representation in your chosen programming language), the parser delivers an AST that may contain error nodes.

Exercise 10. Design and implement a validity checker for *Declared* using the AST representation you chose for exercise 9.

Exercise 11. Design and implement a function that replaces all variable declarations with assignment nodes in a *Declared* program AST.

Exercise 12. Here is an adaptation of the theorem of section 4.1 to the *Declared* language:

For all programs P in *Declared*, if $\text{closedProgram}(P) = P$ and if $\text{runMachine}_{\text{CSK}}(P) = R$, then R is either a *Value* or a division-by-zero *Exception*.

Your task is to exploit this theorem for the adaptation of the CSK machine for *Bare Bones* to well-formed and valid *Declared* programs. Comment out all superfluous checks.

Exercise 13. Modify the main function from figure 13 so that it can run the entire process of parsing, validating, and determining the meaning of a *Declared* program. Use your solutions from exercises 9 through 12.

The output should be a number only when the parser and validity checker succeed, and when the machine runs to completion without discovering a division-by-zero problem:

- If parsing the given S-expression fails, the error message should say "syntax error".
- If the validity checker fails, the error messages should print just the name of the undeclared variable (in string quotes).
- If the machine halts in an error state, your program should print "run-time error".

In order to cross-check your implementation with one of a peer, make sure to construct tests that come with at most one undeclared variable.

Why should programmers declare variables close to their use?

1 The Pragmatics Question

Imagine yourself editing a *Declared* program of 10,000 lines. Okay, nobody should write such programs anymore, but in the past, programmers had to do just that. As a result, language creators had to confront a problem that these programmers encountered time and again. So again, imagine yourself editing this large *Declared* program and realizing that the values of two variables need to be swapped. Easy, you say—just use a temporary variable to hold the value of one, and so on. And this raises the question of where this temporary variable comes from.

A programmer can tackle this problem in one of two ways. The first one is to insert a new variable declaration with a distinct name, by navigating to the top of the program and then back to the place where the value swap is to be added. For a long program, this navigation effort might be significant. The second way is a mental exercise: the programmer figures out, and uses, the name of a variable whose current value is no longer needed. If the programmer is wrong about the need for the value, the computation is going to go wrong.

What this scenario describes is concern about the language in a particular work situation. The language forces the developer into a needless navigation or a potentially dangerous mental exercise. It sets up a program organization that provides no information to the programmer about variable names and uses. Let's take a look at how to solve this problem.

1.1 Design Choices

Some programmers work in a specific programming language only with one particular IDE. If we accept this combination as a common standard,

there are two different ways to address the problem:

1. add a linguistic construct to the programming language for declaring variables locally; or
2. equip the IDE with a linter (or several) that assist developers with the use and re-use of variables.

As you probably know from experience, programming language creators solved this problem with local variable declarations.

The question is why people chose the linguistic solution over the tools-based one. For the latter to work, all members of a project team would have had to use the tool. When this question came up in the 1960s and even as late as the 1990s, it was essentially impossible to impose this tool by force on the workflow of developers. But then again, the very idea that a tool *must* be used by every programmer implies that it should be an integral part of the language implementation, not a separate tool. Hence, it is the first solution that prevailed.

1.2 Costs and Benefits

Declaring variables locally has clear advantages for programmers. It first of all means a programmer does not have to remember all variables that are visible at a certain place. Just in case a local variable name is the same as one already in use, the first replaces the second. Next, it helps a programmer to declare variables when needed and for exactly the narrow region of text where they are needed. And in case, several programmers work on one large product whose code is in a single file, they don't have to worry about each others variable name choices.

Until recently, the entire TypeScript implementation consisted of a single file.

For the language creator, every change to a language raises questions about all of aspects:

- A change to a language's grammar induces a change to its parser.
- In the particular case of local variable declarations, such a change also demands a look at the validity checker to make sure it can deal with any new AST node due to changes to the parser.
- Finally, new language features may also require a revision of the semantics. While the CSK machine can deal with the global variable declarations of *Declared*, modeling the semantics of *local* variable declarations is best dealt with via a refinement of the CSK machine.

The remaining sections make these points concrete.

2 The Syntax of Local Variable Declarations, Scope

The design of a language extension starts with syntax. As the preceding section explains, syntax design has two aspects: (1) the BNF grammar and the parser; and (2) a validity check. Finally, the addition of local variable declarations to *SampleD* necessitates the introduction of a bit of terminology: “scope” or “the scope of variable declarations.”

2.1 Grammar and Parsing

Here is the grammar for a natural extension of *SampleD* with local variable declarations:

```

Program      ::= (Declaration* Statement* Expression)
Declaration ::= (def Variable Expression)
Statement    ::= (Variable = Expression)
               | (block Declaration* Statement+)
Expression   ::= 1.0 | 2.0 | 3.0 |
               Variable |
               (Variable + Variable)

```

The keyword ‘block’, like all keywords, is not a variable.

This grammar extends the one from chapter IV with one alternative for Statements: a declaration block. Such a block starts with a potentially empty sequence of variable declarations followed by a non-empty sequence of statements. Also notice that the symbol “block” is no longer available as a variable, because it is now used as a keyword. We refer to this language as *SampleN*.

Stop! Some languages allow the use of keywords as variable names. What do you think of this idea? Debate with a peer.

Stop! Note that the grammar allows arbitrarily deep nesting of block statements. Formulate a program according to this BNF that contains block statements nested five levels deep.

A parser for this final revision of *Sample* must recognize one more S-expression as a well-formed Statement. Once the parser recognizes a statement starting with block, it is going to construct an abstract syntax node from the sequence of declarations and statements.

The design of this abstract syntax node should account for the needs of the validity check and even the abstract machine that interprets it. From this perspective, a language creator has two choices:

- The first, obvious one is to develop a data representation for just these new kinds of nodes.

- A second one recognizes the similarity between a Program and a block statement. Both nodes contain a sequence of declarations and statements; a node for Program also includes an AST for the final Expression. One way to unify the two is to parse a block statement into a Program node that contains a distinct marker instead of an Expression AST.

Stop! Think through the pros and cons of each design alternative.

2.2 Scope

A programming language such as *SampleN* must come with a description of the scope of variable declarations. Language researchers introduced the word *scope* to refer to the region of text where a declaration binds occurrences of a variable name. Knowing this terminology, and understanding the concept, helps programmers present code in precise terms and assess the problems of programming languages they encounter.

Once a programming language creator has specified the syntax of the language, the next task is to specify scope. While this task sounds straightforward, it turns out that even the global declarations of the preceding chapter come with a hitch. Since two distinct variable declarations may use the same name, the region of the second one cuts a hole into the region of the first one. Take a second, close look at the screenshot of figure 14. It displays a program that declares the same variable—oneVariable—twice. The arrows indicate which declaration binds which occurrence of the same name, and they implicitly define the scope of each declaration:

- The first declaration's scope are lines 4 through the right-hand side of the declaration on line 7.
- By contrast, the second declaration's scope consists of lines 9 and 11.

The purpose of a block statement is to enable programmers to create such holes in scopes explicitly. Consider the following program, which contains one block statement with two variable declarations:

```
((def one 1.0)
 (def anotherVariable 2.0)
 (def andAThirdVariable 3.0)
 (def oneVariable (one + andAThirdVariable))
 (block
  (def anotherVariable (anotherVariable + oneVariable))
  (def x 3.0)
  (andAThirdVariable = (anotherVariable + x)))
 (oneVariable = (anotherVariable + oneVariable))
 oneVariable)
```

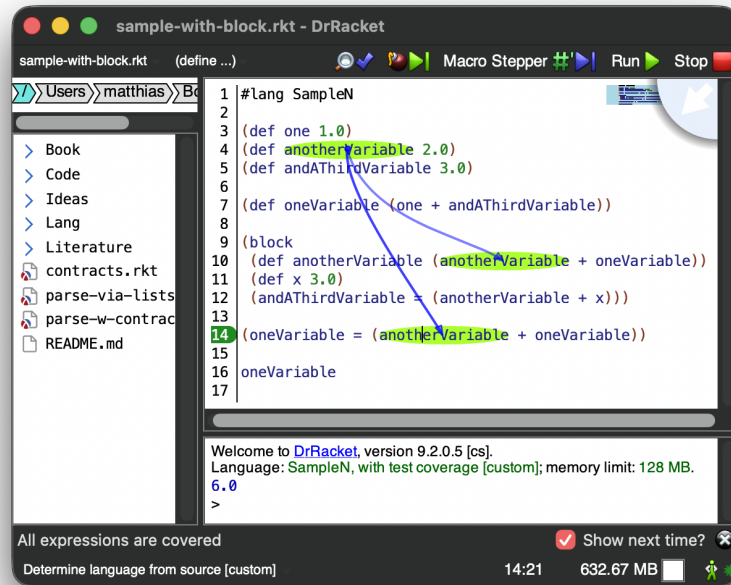
Figure 22: Blocks, scope, and binding in the *SampleN* language

Figure 22 displays the same program in one of *SampleN*'s IDEs.

One of the two local declarations introduces the same variable name as the second global declaration. Programming language people say the local declaration shadows the global one. But, the expression on its right-hand side refers to this name, which definitely raises the question of what the scope of each declaration is—and argues for using well-defined concepts and commonly understood terminology for this linguistic phenomenon.

Figure 22 clarifies the notion of scope for *SampleN* in a graphical manner. The scope of a variable declaration in a block statement consists of all declarations and statements that follow, meaning the right-hand side of a declaration is *not* included in the scope.

The two arrows originating from the global declaration of `anotherVariable` demonstrate a second point about scope. While one arrow points to the right-hand side of the shadowing declaration, the second reaches across the entire block statement. That is, a global declaration's scope may include all pieces of code outside of a nested block statement.

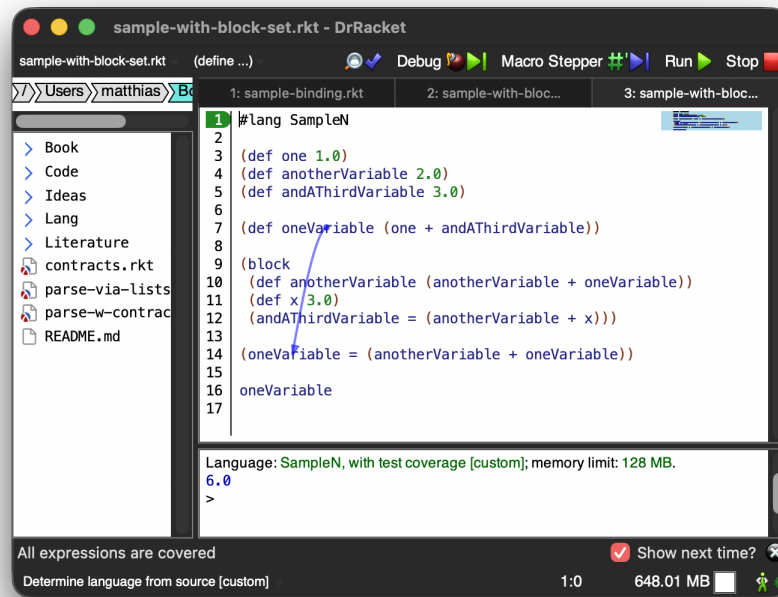


Figure 23: Blocks and assignment statements in the *SampleN* language

Finally, figure 23 makes another point concerning scope using the same program. The arrow points from the declaration of `andAThirdVariable` to the assignment statement *inside* of the nested block statement. It binds the left-hand side, meaning the variable whose value is to be changed.

What this discussion shows, is that an abstract machine must be able to deal with nested scopes. In other words, a variable may go in and out of scope as the machine evaluates the sub-expressions of a program and changes the association between variables and values. The next section explains the implications of this statement for the abstract-machine design. Before we revise the CSK machine, though, we need to briefly address validity checking for *SampleN*.

2.3 Validity Checking

The goal of validity checking remains the same: to ensure that all variable occurrences are associated with some variable declaration. Since the design of the validity checker in the preceding section follows fundamental design principles, adapting it to the revised BNF grammar is straightforward. The production for Statement is the only one that got modified. Hence, `closed-stmt` is the only function that must change.

Here is the adapted function:

```
#; { Stmt- Set<Variable> -> Stmt }
(define (closed-stmt s- declared)
  (match s-
    [(assg x rhs)
     (define x++ (closed-var x declared))
     (define rhs++ (closed-expr rhs declared))
     (assg x++ rhs++)]
    [(seq d-* s-*)
     (define d* (closed-decl* d-* declared))
     (define declared++
      (decl->declared-variables d-* declared))
     (define s* (closed-s* s-* declared++))
     (seq d* s*))])
```

Like the original, the function consumes an abstract syntax tree free of error nodes plus a set of declared variables. The former represents a single statement in *SampleN*. While the production for Statement in the preceding chapter has only one alternative on the right-hand side, the one from this chapter has two. Hence the adapted function employs a conditional to distinguish the two cases:

1. The first case is exactly the original code for this function.
2. The second, boxed one corresponds to the newly added alternative form of Statement.

This second case deconstructs the abstract syntax of a block (`seq`) into its two pieces: a sequence of error-free declarations (`d-*`) and a sequence of error-free statements (`s-*`). Since functions for checking the validity of sequences of declarations and sequences of statements exist, checking these pieces is delegated to those. The last line re-assembles the generated ASTs into a new representation of a block statement. If these auxiliary functions don't discover any errors, the given node and the returned one are identical; otherwise the latter contains error nodes so that the IDE and the compiler can inform the programmer of infractions. See figure 18 for an example.

Stop! The explanation comes with a glitch concerning `closed-decl*` and `decl->declared-variables`. Explain and fix.

3 The CESK Abstract Machine: Local Declarations

A semantics for local variable declarations should mirror the concept of scope in some ways. Graphically speaking, the semantics of binding must faithfully mirror the arrows of figure 22. So, unlike in the preceding chapter, it is no longer possible to run *SampleN* programs with nested block statements on the CSK machine by just changing variable declarations into assignment statements. Doing so would conflate distinct variable declarations for the same name.

For this reason, we instead transform the CSK machine into a model that gives a semantics to variable scopes. Giving meaning to scope as a part of the program execution in an abstract machine is a powerful solution. In particular, this *dynamic semantics* can accommodate local variable declarations that appear during execution and which could therefore be nested infinitely deep. While the syntax of *SampleN* does not enable such executions, revisions in subsequent chapters do. Hence, we opt for this second, dynamic approach. The new machine has been known as the CESK machine since the 1980s.

Note Compiler writers know how to continue with the CSK machine for *Core*. Since (1) the number of variable declarations (and their scopes) is finite and (2) the set of Variables is infinitely large, they know that an implementation can rename all variables so that their names are pairwise distinct. This renaming would have to respect the scope structure, of the given program, meaning names bound to the same declaration are replaced by the same new and distinct symbol. Once the renaming has taken place, declarations can once again be replaced by assignment statements. Language researchers call this approach a *static semantics* of scope.

3.1 The States

The CESK machine comes with four registers, just as its name suggests: C for control, E for environment, S for store, and K for continuation.

As before, the C contains expressions, including numerical literals that result from the evaluation of expressions, plus the special $\dagger$ token, which indicates that the machine is searching for the next expression to evaluate.

The new E register contains an association of variables with locations. More precisely, it maps just those variables to locations that are in scope for the evaluation of the expression in C. Assume that the set of locations is infinite, symbolic, and distinct from the set of Variables. The content of this register denotes the meaning of scope.

In contrast to the CS and CSK machines, the S register of a CESK machine associates locations with numbers. The number associated with a location is the current value of the variable that is mapped to this location.

While environments come and go, reflecting the evaluation of nested block statements, locations never disappear. But, it may no longer be possible to reach a location by starting from the variable names mentioned in the contents of the other registers. The period from when a location comes into existence until it becomes unreachable is the *dynamic extent*, or the *lifetime*, of a particular variable declaration in a program.

Chapter X explains how to remove such “dead” locations from the store.

Note At this stage, the indirection via locations is a convenience, not a necessity, because the scope of a variable coincides with its dynamic extent. It does reflect that the same name may refer to distinct variables with distinct values, even if only one of them is visible from the C register.

Finally, the K register no longer contains just a program, but stacks of program-like shapes combined with environments:

```
closure: (defn* stmtn* $) in en
...
closure: (def1* stmt1* $) in e1
closure: (def0* stmt0* expr) in [ ]
```

These combinations are known as *closures*. We refer to each layer in this stack as a *frame*.

The stack mirrors the nesting of block statements in the program. Each frame corresponds to a scope extension via the block’s local variable declarations. Its pieces represent the to-be-evaluated declarations and statements plus the environment in place before the machine descended into a nested block.

Stop! Think about our use of the word “environment” for the second piece in a closure. In what way is this use of the word highly appropriate?

Let’s take a close look at the shape of these frames, starting with

```
closure: (def0* stmt0* expr) in [ ]
```

It represents the part of the original program whose expressions (inside of the declarations and statements) haven’t been evaluated yet; the machine

needs to continue evaluating those when the frames atop of this one are evaluated. Each frame on top of the original program frame contains the \$ marker in the expression position instead of an expression:

```
closure: (defi* stmti* $) in ei
```

This marker signals that these frames represent a nested block without return expression. Its evaluation has been started yet contains expressions that are to be evaluated.

The search for expression remains a process driven by the contents of the K register. More precisely, the machine searches in the top-most frame until the sequences of declarations and statements are exhausted. Writing down the complete stack for these search is therefore unnecessary. It suffices to focus on the top-most frame, and it is useful to introduce a notation to formulate the cases of the transition function in a concise manner. So,

```
<< (defn* stmtn* $) >>
```

abbreviates the entire stack from the 0th layer to the n th one:

```
closure: (defn* stmtn* $) in en
...
closure: (def1* stmt1* $) in e1
closure: (def0* stmt0* expr) in [ ]
```

And,

```
<< (def0* stmt0* expr) >>
```

is short for

```
closure: (def0* stmt0* expr) in [ ]
```

Note how each frame contains an environment distinct from the environment in any other frame.

Partitioning this set of states into its three pieces follows the patterns from the preceding chapters. An *initial* CESK state has this shape, given some Program p :

```
†, [ ], [ ], (closure: p in [ ])
```

that is, it combines the search-for-an-expression marker with two empty association tables, and the full-program closure as the first and only frame in the continuation stack. The set of proper *final* CESK states has a number in the C register and an empty stack in K:

```
n, e, s, [ ]
```

For these final states, it doesn't matter what E and S contain. As before, the set of final states includes error states:

$\text{error}, e, s, []$

Recall that when an abstract machine transitions to an error state, `runMachine` stops and issues an informative message.

Stop! Choose a data representation for the states of the CESK machine in your favorite programming language.

Stop! Define load and unload functions for the CESK machine.

3.2 The Transition Function

One way to develop the transition function for the CESK machine is to adapt the CSK transition function to deal with the separation of the store into an environment and a store proper. Let's start with the transition for executing assignment statements:

	Control	Environment	Store	Kontinuation
search ends with right-hand side of assignment				
<i>before:</i>	$\dagger$	e	s	$\ll [], (x = \text{rhs})::\text{stmt}^*, r \gg$
<i>after:</i>	rhs	e	s	$\ll [], (x = \text{rhs})::\text{stmt}^*, r \gg$
value for right-hand side of assignment				
<i>before:</i>	n	e	s	$\ll [], (x = \text{rhs})::\text{stmt}^*, r \gg$
<i>after:</i>	$\dagger$	e	$s[xl = n]$	$\ll [], \text{stmt}^*, r \gg$
<i>where</i> $xl = e[x]$				

The *before* condition of the first case describes a search scenario when the next expression— rhs —has been found. Considering that the corresponding case in the CSK transition function does not involve the store, it is unsurprising that the *after* specification says that C contains rhs and otherwise the *after* state resembles the *before* state.

By contrast, the second case above involves the store in the corresponding CSK case, so the adaptation needs work. The *before* condition tells us that the value of the right-hand side of the assignment statement has been found; it is the number n . Intuitively, we understand that from now on going forward, the variable on the left-hand side of the assignment statement—symbolized with x —should be associated with this number.

But, environments associate variables with locations, and stores map locations to values. Hence, the transition function first determines the location of the variable via the environment:

$$xl = e[x]$$

and then creates a store that associates this location with n :

$$s[xl = n]$$

Stop! Why is $(x = rhs)$ missing from the *after* state description?

The adaptation of the rules for the evaluation of expressions proceeds in a similar manner:

Control	Environment	Store	Kontinuation
evaluate a variable			
<i>before:</i> y	e	s	k
<i>after:</i> n	e	s	k
<i>where</i> $yl = e[y]$			
<i>and</i> $n = s[yl]$			
evaluate an addition			
<i>before:</i> $(y + z)$	e	s	k
<i>after:</i> $(+ yn zn)$	e	s	k
<i>where</i> $yl = e[y]$			
<i>and</i> $zl = e[z]$			
<i>and</i> $yn = s[yl]$			
<i>and</i> $zn = s[zl]$			

When the CESK machine encounters a variable in C , determining its value proceeds in two steps: (1) find its location in the current environment (e) and (2) extract the value of this location from the current store (s). Similarly, an addition expression causes the abstract machine to retrieve the locations of the two variables and then to look into the store to obtain the actual value. Once these values are available, the transition function uses the addition operation from mathematics; an implementation would use the addition function for inexact numbers (doubles).

At this point, you should wonder how locations come into existence, and the answer can be found in figure 24. This figure displays all the cases of the CESK transition function related to variable declarations, both global and local. The first two cases explain how the transition function deals with variable declarations in the top-most stack frame:

	Control	Environment	Store	Kontinuation
search ends with def rhs				
before:	↑	e	s	⟨⟨ (def x rhs)::def*, stmt*, r ⟩⟩
after:	rhs	e	s	⟨⟨ (def x rhs)::def*, stmt*, r ⟩⟩
value for right-hand side of declaration				
before:	n	e	s	⟨⟨ (def x rhs)::def*, stmt*, r ⟩⟩
after:	↑	e[x = xl]	s[xl = n]	⟨⟨ def*, stmt*, r ⟩⟩
where xl = a new (relative to s) location				
search encounters nested block				
before:	↑	e	s	⟨⟨ [], (block d* s*)::stmt*, r ⟩⟩
after:	↑	e	s	push[cl, k]
where k = ⟨⟨ [], stmt*, r ⟩⟩				
and cl = closure: (d* s* \$) in e				
search exhausts nested block				
before:	↑	e	s	⟨⟨ [], [], \$ ⟩⟩
after:	↑	e1	s	pop[k]
where e1 = the environment in the top-most closure				
and k = ⟨⟨ [], [], \$ ⟩⟩				
search ends with the return expression				
before:	↑	e	s	⟨⟨ [], [], r ⟩⟩
after:	r	e	s	⟨⟨ [], [], r ⟩⟩
search ends with evaluated return expression				
before:	v	e	s	⟨⟨ [], [], r ⟩⟩
after:	v	e1	s	pop[k]
where e1 = the environment in the top-most closure				
and k = ⟨⟨ [], [], r ⟩⟩				

Figure 24: The CESK transition function for variable declarations

1. When the machine searches for the next to-be-evaluated expression and the sequence of declarations in the top-most closure is not empty, the right-hand side of the first declaration is transferred into C.

2. Once that expression is evaluated, the transition function allocates a new location, that is, a location not used in the current store s . Using this new location the *after* state specification tells us that
 - the new environment associates the left-hand side variable with the new location, and
 - the new store associates the location with the value (n) from the C register.

Note also that the machine switches into search mode again.

Stop! How would you implement “find a new location” in your favorite programming language? What does this implementation depend on?

The second pair of cases in figure 24 concerns local variable declarations, specifically nested block statements:

1. When a machine in search mode encounters a top-level stack frame whose sequence of declarations is empty and its sequence of statements starts with a block, it must continue the search inside this block statement. Hence the *after* state specification pushes a new frame on the stack. This frame combines the current environment with a quasi-program formed from the block statement. From here the search continues.

The \$ marker in the final-expression position indicates that this frame represents a nested block without return expression.

2. The CESK machine recognizes that a nested block’s instructions have been executed when the top-most stack frame contains an empty sequence of declarations, an empty sequence of statements, and the marker \$. If a machine state satisfies this *before* description, the transition function creates a state that pops the top-most frame. Concretely, this means
 - the environment of the top-most stack frame becomes the current environment in register E, and
 - then the top-most frame disappears.

Note that the machine remains in search mode.

In sum, the execution of a variable declaration causes a machine to allocate locations. Otherwise the use of variables just requires an indirection of looking into the environment first and then into the stack.

Stop! Design and implement the transition function in your favorite programming language using your data representations from the preceding subsection.

name(s) and term(s)	standing in for
c	a generic control
e, e1	generic environments
s, s1	generic stores
k	a generic continuation
cl	a closure
n, xn, yn, zn	numbers
x, y, z	program variables
xl, yl, zl	locations
r	an expression or $\uparrow$
rhs, tst	expressions
def*	a potentially empty sequence of definitions
stmt, thn, els, body	statements
stmt*	a potentially empty sequence of statements
e[x]	the location at variable x in environment e
s[xl]	the value at location l in store s
e[x = xl]	an environment like e, but x stands for xl
s[xl = n]	a store like s, but xl stands for n

Figure 25: Conventions for the CESK transition function for *Core*

Stop! Take a look at figure 25, which lists the notational conventions used to describe the transition function for the CESK machine. As you tackle this chapter's project next and the projects in the following chapters, keep this table in mind and consult it often.

4 Core

Figure 26 presents the syntax of *Core*. It represents the universally shared syntax of most widely used programming languages, adding local variable declarations to the syntax of *Declared*, or, equivalently, a conditional and a looping construct to *SampleN*.

```

Program      ::= (Declaration* Statement* Expression)

Declaration ::= (def Variable Expression)

Statement   ::= (Variable = Expression)
              | (if0 Expression Block Block)
              | (while0 Expression Block)

Block       ::= Statement
              | (block Declaration* Statement+)

Expression  ::= GoodNumber
              | Variable
              | (Variable + Variable)
              | (Variable / Variable)
              | (Variable == Variable)

The set of Variables consists of all Names, minus keywords.

The set of GoodNumbers comprises all inexact numbers
(doubles) between -1000.0 and +1000.0, inclusive.

```

Figure 26: The grammar of the *Core* language

A rather straightforward adaptation of the CESK machine for the *SampleN* language yields a semantics for *Core*. The states contain additional forms of syntax in *C* and *K*'s closures; and the transition function needs cases for the evaluation of these. Concretely, it needs to be equipped with cases for the evaluation of the two additional kinds of expressions and for searching the next expression in conditionals and loops.

Figure 27 gathers the cases for the transition function that describe how it deals with conditionals. The first case explains what happens when the machine is in search mode and encounters a conditional in the top-most frame of *K* as the next instruction. As the *after* specification shows, the conditional's test expression becomes the content of *C*.

Once the machine has evaluated this expression, it picks one of the two branches and discards the remainder of the conditional. The chosen branch becomes the first of the sequence of statements in the top-most frame of *K*, while the *C* register is set to $\dagger$, meaning the machine is back in search mode.

Stop! How does the machine distinguish the two cases?

The cases in figure 28 explain how the transition function treats *while0* loops. In a way, the three cases mirror the cases for conditionals, though

	Control	Environment	Store	Kontinuation
search for expression in if				
before:	↑	e	s	⟨⟨ [], (if0 tst thn els)::stmt*, r ⟩⟩
after:	tst	e	s	⟨⟨ [], (if0 tst thn els)::stmt*, r ⟩⟩
pick then branch from if0				
before:	n	e	s	⟨⟨ [], (if0 tst thn els)::stmt*, r ⟩⟩
subject to: n is basically 0.0				
after:	↑	e	s	⟨⟨ [], thn::stmt*, r ⟩⟩
pick else branch from if0				
before:	n	e	s	⟨⟨ [], (if0 tst thn els)::stmt*, r ⟩⟩
subject to: n is not close to 0.0				
after:	↑	e	s	⟨⟨ [], els::stmt*, r ⟩⟩

Figure 27: The CESK transition function for *Core*: conditionals

the generated states differ. Again, if the machine is searching for the next expression and encounters a looping statement as the first one in K , the loop's test expression is placed into C .

What happens when the value of the test expression is determined, is the interesting part. The first case—when the value is 0.0—tells us that the *after* state contains a newly synthesized statement in the top-most frame of K . In a conventional syntax, we might write this:

```
body; while0 (tst) body
```

That is, the *body* part of the *while0* statement becomes the first instruction of the sequence of instructions. Furthermore, the original *while0* statement is now the second one; after all, once *body* is executed the loop may have to run again. And, because *body* is a *Block* and there is no obvious expression to evaluate next, the machine is instructed to search for the next expression.

Stop! Take a close look at the second case in figure 28 to make sure you understand how it works.

Finally, the last case in figure 28 informs an implementer of what happens when the value of the test expression in a *while0* statement is not 0. The *after* state specification tells us that the *while0* statement is discarded

	Control	Environment	Store	Kontinuation
search for expression in while				
before:	†	e	s	⟨⟨ [], (while0 tst body)::stmt*, r ⟩⟩
after:	tst	e	s	⟨⟨ [], (while0 tst body)::stmt*, r ⟩⟩
decide whether to run while loop (positive)				
before:	n	e	s	⟨⟨ [], (while0 tst body)::stmt*, r ⟩⟩
subject to: n is basically 0.0				
after:	†	e	s	⟨⟨ [], body::(while0 tst body)::stmt*, r ⟩⟩
decide whether to run while loop (negative)				
before:	n	e	s	⟨⟨ [], (while0 tst body)::stmt*, r ⟩⟩
subject to: n is not close to 0.0				
after:	†	e	s	⟨⟨ [], stmt*, r ⟩⟩

Figure 28: The CESK transition function for *Core*: loops

from the top-most stack frame, leaving just the remainder of the sequence of statements (stmt*) in place.

4.1 Why Core? Why CESK?

Core can express all classic computations, meaning the partial-recursive functions of the Church-Turing hypothesis. Also, its syntax and semantics is one that all programmers immediately recognize. In this spirit, *Core* can serve as the foundation from which we investigate truly interesting questions of pragmatics.

This book focuses on pragmatics, not expressive power. Readers interested in the second topic may wish to check the literature.

While programmers may acknowledge that, in theory, *Core* is everything needed, none of them would want to develop a piece of software in this impoverished syntax or variants thereof. Software development requires much more than Church-Turing computability, both in terms of expressive syntax and pragmatics.

Also, the CESK semantics is easily adapted to any future extension with expressions and statements. Changing the set of states require a “re-direct” to the revised BNF productions. The actual work consists in adding cases to the CESK transition function:

- Every new kind of expression can show up in the C register, and therefore we need an evaluation rule for those occasions. Like the case for addition, these evaluation rules typically appeal to mathematical functions or operations from the chosen implementation language.
- For every new kind of statement, the transition function must be equipped with a rule for finding the next nested expressions to be evaluated and for how to use the value to change the K register once the value of the expression is found.

If a new kind of statement contains more than one sub-expression, it might be necessary to add several rules to the transition function.

Stop! How could the CESK machine deal with a for loop or a multi-pronged conditional such as a numeric switch statement?

In sum, *Core* and *CESK* jointly set us up for a proper and reasonably smooth investigation of the pragmatics of programming language concepts that actually affect software developers.

4.2 Project Tasks

Use your favorite programming language to solve the following exercises.

Exercise 14. Design an AST data representation for *Core*. Implement a parser for *Core* that maps an S-expression to an instance of AST.

Exercise 15. Design and implement a validity checker for the *Core* language. Remember that a validity checker consumes an AST *without* error nodes. Its result is the same AST if all variables are properly declared; otherwise it is an AST that contains error node that mark references to undeclared variables.

Exercise 16. Design a data representation for the states of the CESK machine for *Core*. Implement the load, transition, and unload functions. By combining these functions with the runMachine function (see section 2 of chapter III), you obtain a complete semantics for well-formed and valid *Core* programs.

Exercise 17. Adapt the main function from exercise 13 so that it can run the entire process of parsing, validating, and determining the meaning of a *Core* program. Use your solutions from exercises 14 through 16.

The output should be a number only when the parser and validity checker succeed, and when the CESK machine runs to completion without discovering a division-by-zero problem:

- If parsing the given S-expression fails, the error message should say "syntax error".
- If the validity checker fails, the error messages should print just the name of the undeclared variable (in string quotes).
- If the machine halts in an error state, your program should print "runtime error".

In order to cross-check your implementation with one of a peer, make sure that the integration tests come with at most one undeclared variable.

5 Scope in the Real World

This chapter presents the most common meaning of scope. It is often called "lexical scope," because each variable declaration connects to an easily identified region of program text. While the text may contain holes due to a re-use of a variable name, the direct association enables programmers to think about basic relationships among variables and the flow of values from one to another.

A few languages stick to lexical scope for their core syntax. C# and Java are examples. Many don't. JavaScript and Python come to mind. The former comes with deprecated linguistic constructs that make it extremely difficult for a reader to understand which name refers to which variable, not to speak of the flow of values from one variable to another. The latter does not know variable declarations. When a program assigns a value to a name, it is also not immediately clear where this name is visible. Thus, although both languages support mostly conventional syntax, determining scope and reasoning about value flow by just reading program text can be surprisingly hard.

Why do languages support structured data and abstraction?

1 The Pragmatics Question

Although the *Core* language can express all functions that Church and Turing consider computable, you would have a hard time writing a program that allows people to manipulate geometric shapes in the two-dimensional plane. Consider the simplest shape of all: a Cartesian point. You remember that you need two numbers to describe the location of Cartesian points: the x coordinate and the y coordinate. But, all you have as a *Core* programmer are plain numbers. So now you wonder whether

it is possible to represent the two coordinates of a Cartesian point with a single number.

While the answer is affirmative, it won't satisfy you. You will need a lot of code to combine the two numbers into one; you need even more code to extract the two coordinates from such a number; and you need a few other ways of manipulating such Cartesian coordinates.

Worse, when you look at a numeric constant in your program, you just don't know whether it represents this number or a coordinate. This observation has tremendous implications for code that must represent Cartesian coordinates in this manner. Imagine that some part of your code treats a single number as the representation of a coordinate while in reality the number represents a distance between two points. Since numbers come without any marker that classifies them as either plain numbers or representations of Cartesian coordinates, you cannot equip your code with appropriate checks. The best you can do is to create a rigorous and comprehensive test suite that ensures that everything works properly. And if it doesn't, looking for a bug will be extremely painful.

This scenario reveals another problem with the *Core* language, namely, the complete lack of an abstraction facility. Clearly, the operations for turning two coordinates into a single number and for extracting the x or y coordinates from a single number are going to show up all over a program. Even if we assume that they are short calculations—one-liners if we allow nested expressions—every maintainer of such a program must read the expressions instead of useful names and recognize their purpose. In short, *Core* lacks the ability to name expressions and re-use them. More generally, the language lacks a construct for abstracting identical or similar pieces of code into a single form and naming it.

In sum, any use of a language like *Core* raises the following two related questions of pragmatics:

- how can developers represent structured information as data, and
- how can developers abstract over and name repeated pieces of code so that it becomes possible to reuse them easily.

1.1 Design Choices

This discussion ignores chars, Booleans, and other essentially atomic forms of data similar to numbers.

Historically, language creators gave distinct, unrelated answers to these two questions. To permit a direct form of representation of structured information as data inside of a programming language, they added arrays (of numbers). Arrays have come in many forms and shapes: one-dimensional, vector-like data; two-dimensional ones, resembling matrices; 0-based, 1-based, and programmer-defined based arrays; and many more flavors. In essence, though, an array is little more than a slice of memory from the underlying hardware, possibly equipped with some additional data, such as length information.

As for naming pieces of code and abstracting over differences, language creators introduced procedures and functions. The latter word was chosen in recognition of the relationship between language design and mathematical notation where functions—like those from pre-algebra courses—name expressions and abstract over the differences among similar expressions. Of course, due to the imperative nature of *Core* and similar, historical languages, they rarely are mathematical functions.

Over time, the designers of programming languages realized that neither arrays nor plain functions met the needs of programmers concerning the representation of complex information. In response, two different solutions to these questions emerged:

- The first one is the idea of a class. A class is a blueprint for creating compound pieces of data, dubbed instances or objects. Each instance combines several pieces of named data with functionality. To name data in an instance, a class comes with fields, each essentially just a variable name. To equip instances with functionality, a class specifies methods, which work on instances plus additional data.
- The second one combines functions with structures, also known as records. Functions come with support such as pattern-matching for processing structures. If functions are also first-class values, it is possible to store functions in structures and to thus mimic objects.

At this moment in time, the first approach dominates the landscape of programming languages, though flavors of the second alternative are making a come back.

This chapter presents a model of the first approach, focusing on the most essential elements:

- Classes that have a fixed number of fields and methods.
- Instance creation takes place by naming a class and supplying exactly as many values as there are fields. The resulting object is a value, and it is distinct from all numbers and all already existing objects.
- All fields and methods are public, meaning a piece of code with access to an instance of the class can read any field, modify the content of any field, and call any method.

This sparse model—lacking even sub-classing and inheritance—is a good starting point to gain a first understanding of how the addition of classes to a language such as *Core* answers the two questions of pragmatics raised here. Additionally, even with all these simplifications, the model enables us to consider the rather dramatic increase in costs and benefits over *Core*.

1.2 Costs and Benefits

The addition of classes to a language such as *Core* comes with quite a number of pluses and minuses for the developer. As for the benefits, they are essentially the answers to the above questions:

- With classes, it becomes easy to formulate reasonably direct data representations for the information that a program is to process.

- Since class instances differ from numbers and each other, programming with classes also reduces the potential of conflating one piece of data for another. Coding becomes less error prone.
- A method is a natural mechanism for naming repeated pieces of code and abstracting over their differences. The parameters of a method are placeholders for values where the repeated pieces differed.

Moreover, the inclusion of methods in classes provides a design guide for code. Pieces of functionality that belong to a specific form of data should be bundled together with that data in a class. For instance, all pieces of functionality that operate on Cartesian points belong into the class that represents them. When developers strictly follow this design discipline, the resulting organization acts as a guide for both creators and future readers of the code.

Unsurprisingly, the addition of a complex sub-language such as a class system also injects some complexities into programming that a purely numerical language does not suffer from. Most importantly, programmers must grok a complex model of computation, meaning syntax and semantics, before they can truly take advantage of these features. Since a class-based language comes with a lot more syntax than say *Core*—meaning BNF productions and validity constraints—forming correct programs also becomes a more complex task. The semantics of a class-based language is also far more complicated than the one of *Core*:

- As a program instantiates a class, it allocates some space in the store for the values of the fields and the name of the class. Semantically, this space is a single piece of data. It can find its place in a variable or the field of another object; but it may also just be part of an expression that retrieves a field's value or calls a method.
- Since a program can point several different variables to the same object, changes to the object are visible from many different places.

Language researchers refer to this phenomenon as *aliasing*.

Programmers must learn to reason about the aliasing of objects, especially whether they want the effect in some situation or not. And if they don't want it, they need ways to avoid it.

- Furthermore, a programmer must become keenly aware of the distinction between variables and values. The life time of a variable corresponds to its lexical scope. By contrast, the life time of an object

depends on dynamic properties of the program. As long as any variable or field can refer to an object located in the store, programmers consider the object as being alive; otherwise it is dead.

Language researchers speak of the object's *dynamic extent*, i.e., the time between the birth of the object and its death. Semantically speaking, variables exist in environments, objects in stores.

Programmers must learn to consider dynamic extent, a far more complex notion than lexical scope, simply because it is a time slice of a program execution rather than a fixed region of text.

- In some way, a method call resembles the use of a function in pre-algebra, once the targeted method has been identified. It is this latter part of a method call that imposes work on the programmer. Since different classes may contain methods with the same name (yet different expression), the meaning of a method call is only clear when a programmer understands which object is the *target* of the call.

Language researchers dub this concept *dynamic dispatch*, because the actual method is identified only during program execution when the “dispatch” takes place.

A programmer must carefully think about method calls to ensure that the code points to the truly desired method. If a program comes with the classes *Artist* and *Cowboy* and both supply a method named *draw*, it is unlikely that these methods are meant to exhibit the same behavior.

In sum, while a class system clearly benefits programmers in many work situations, it also raises the bar for ensuring that a program performs the desired computation.

From the perspective of the language designer, adding classes to *Core* is a major undertaking. It means significant changes to the syntax, both the grammar and the validity constraints. It requires an extensive revision of the existing semantics plus the development of a semantics of classes: field access, field mutation, and method calls. All of these changes imply a good amount of implementation work: the parser, the validity checker, the compiler, and the run-time system. But, in the end, these additions represent tremendously helpful improvements for the working developers and therefore worth a careful exploration via a model.

The rest of the chapter introduces *Classy*, the syntax and semantics for a class-based extension of *Core*. With *Classy* in hand, we can discuss the complexities of class-based, object-oriented programming compared to rather simplistic model of computation from the preceding chapters.

Exercise 18. Imagine an extension of *Core* with structure-type definitions (records) and procedures. Which of the just presented benefits and costs of the class extension do/do not apply to this alternative?

Exercise 19. Consider an extension of the language proposed in exercise 18 so that functions are first-class values in addition to structures and numbers. In particular, assume that the fields of structures may contain functions. Which of the just presented benefits and costs of the class extension do/do not apply to this second alternative?

2 *Classy*: the Syntax

Adding the syntax of a class system to *Core* requires the addition of productions to the language's BNF grammar and the introduction of appropriate validity rules. While the first is reasonably straightforward, the second one exposes a critical tension inherent to language design: which parts of a program can be checked *before* program execution and which ones have to be checked *during* execution. Indeed, the tension goes back to the origins of computer science, concretely the theory of computation, which tells us that not all problems are solvable via algorithms.

2.1 *Classy*: the Grammar

Figure 29 presents the grammar of *Classy*, the class-based extension of *Core*. A *Classy* program consists of a possibly empty sequence of class definitions followed by the program's *body*. In a language such as Java, the latter corresponds to the main method of the main class. The body is like a *Core* program that may also contain expressions and statements related to classes.

The keyword `class` marks a class definition, which consists of an embedded parenthesis-delimited sequence of the names of fields followed by a possibly empty sequence of method definitions. In turn, a method definition comes with a parameter list—a sequence of variable names surrounded by parentheses—and a method body; the latter is again a sequence of variable declarations, statements, and an expression. This final expression computes the value that a method call returns.

Next, the Expression production enables object creation, access to a field, method calls, and object identification. Field modification, though, is a statement, because it has only an effect, not a result.

Class names, field names, method names, and method parameters are drawn from the same set as variables. The BNF productions use differ-

```

Program      ::= (Class* Declaration* Statement* Expression)

Class        ::= (class ClassName (FieldName*) Method*)

Method       ::= (method MethodName (Parameter*)
                  Declaration*
                  Statement*
                  Expression)

Declaration  ::= (def Variable Expression)

Statement    ::= (Variable = Expression)
                | (if0 Expression Block Block)
                | (while0 Expression Block)

                | (Variable --> FieldName = Expression)

Block        ::= Statement
                | (block Declaration* Statement+)

Expression   ::= GoodNumber
                | Variable
                | (Variable + Variable)
                | (Variable / Variable)
                | (Variable == Variable)

                | (new ClassName (Variable*))
                | (Variable --> FieldName)
                | (Variable --> MethodName (Variable*))
                | (Variable isa ClassName)

```

The set of ClassNames is the same as the set of Variables.

The set of FieldNames is the same as the set of Variables.

The set of MethodNames is the same as the set of Variables.

The set of Parameters is the same as the set of Variables.

The set of Variables consists of all Names, minus keywords.

The set of GoodNumbers comprises all inexact numbers (doubles) between -1000.0 and +1000.0, inclusive.

Figure 29: The grammar of the *Classy* language

ent (meta)names for these sets just to explain their various roles in a program. Also, these (meta)names indicate where it is possible to reference these variable names, a property that is a critical aspect of validity check-

ing and semantics.

*Some languages use
"self" instead.*

A second look at the grammar reveals a property of *Classy* that may surprise a student of computing. While in class-based languages “this” plays a special role in method bodies, it isn’t mentioned anywhere in the *Classy*’s grammar. Specifically, the word “this” is a reference to the concrete object on which the method is called; it is often dubbed the *target* of a method call. Syntax design of a class-based language can choose to make “this” special or to treat it as an ordinary variable that, semantically, has meaning inside of a method body. The design of *Classy* takes the second approach.

Exercise 20. Design an AST data representation for *Classy*. Implement a parser for *Classy* that maps an S-expression to an instance of AST. Re-use your solution from exercise 14 as much as possible.

2.2 *Classy*: Scope

Once a language creator has finished a (first draft of a) grammar, the next step is to specify the scope of names. That is, for each kind of name and name-binding construct, it is necessary to spell out the lexical regions in which these names are visible. The scope of a class’s name is the entire program. All *new* and *isa* expressions in a program may refer to any defined class. Next, the parameters of a method are visible in its body, that is, the following sequence of declarations, statements, and return expression. Following tradition, the method header introduces by default and implicitly the name *this*, whose scope ranges from the end of the parameter list to the end of the method.

Parameters, including *this*, are just variables. Hence, a programmer may declare a variable *this* inside of a method or use the name of a parameter for a declared variable. Doing so will shadow the implicit declaration of *this* and the parameter of the same name, respectively.

By contrast, *FieldNames* and *MethodNames* do not have a scope. That is, expressions and statements may use any of these names as an “index”—i.e. use the *->* notation—for a *FieldName* or a *MethodName* into an object. If the target object comes with an appropriately named field or method, the phrase has meaning; otherwise it doesn’t.

Given these scoping rules, let’s consider the two *Classy* programs in figure 30. Both come with one class definition named *C*, and both create one instance of this class in the program’s body. The program on the left uses *C* in the program’s body, and it refers to *this* as the target object in the *set* method. By contrast, the program on the right uses *C* inside the class

<pre> ((class C (f) (method set(p) (this --> f = p) 42.0)) (def one 1.0) (def obj (new C(one))) (def fld (obj --> f)) (obj --> f = (obj --> set(one))) (one = (obj --> f)) (one + fld)) </pre>	<pre> ((class C (f) (method make(p) (def this 66.0) (new C(this)))) (def one 1.0) (def obj (new C (one))) (obj = (obj --> make(one))) (one = (obj --> f)) one) </pre>
---	---

Figure 30: Two simple *Classy* programs

and in the program's body. Furthermore, it declares a variable named *this* immediately below the method header of *make*, thus making it impossible to refer to the target object in the remainder of the method's body.

Exercise 21. Draw arrows like those in figure 14 from all occurrences of variable names in figure 30 to their corresponding declaration according to the explanation of this section..

Exercise 22. Use your experience with object-oriented languages to explain the intuitive meaning of the two programs in figure 30.

2.3 *Classy*: Validity, the Easy Part

Since the design of *Core* avoids “undefined variable” errors, the question arises whether similar properties can hold for the *Classy* language. In other words, we are checking whether to impose validity constraints on well-formed *Classy* programs with the goals of

- providing feedback to the programmer during code creation, and
- simplifying the abstract-machine semantics of the *Classy* language.

As it turns out the design of the *Classy* language comes with two kinds of validity constraints that programmers would find useful and that language designers could use to simplify the semantics. The first kind concerns the definitions of classes, the major new component of the Program production of the BNF grammar. The second kind concerns the additional expressions and statements that enable the creation and manipulation of instances. Surprisingly, the two kinds of validity concerns radically differ

from each other, in nature and in ease-of-checking. While this section deals with the first, easy kind, the next one covers the difficult problems.

Let's start with the programs in figure 30. Both of them are clearly well-formed programs. Now contrast this first program with this one:

```
((class C (f) (method m(a) 42.0))
 (class C (f f)
  (method m(p p) 42.0)
  (method m(p) 21.0))
 (def one 1.0)
 (def instance (new C (one)))
 (def field      (instance --> f))
 (instance --> f = (instance --> m(one)))
 (one = (instance --> f))
 (one + field))
```

It defines two classes with the same name. Worse, the second, boxed class exhibits several other issues that every major programming language flags:

- The name *f* is given to two fields.
- Similarly, both methods in the second class definition are named *m*.
- And finally, the first method definition has two parameters named *p*.

Clearly, two class definitions with the same name make no sense, because the scope for each class name is the entire program, including the class definitions that precede its definition and those that follow.

Exercise 23. Explain in a similar manner why duplicate field names, duplicate method names, and duplicate parameter names make no sense.

Exercise 24. Explain why we admitted duplicate variable definitions in the same sequence of Declarations.

Turning these examples of meaningless program pieces into validity constraints is straightforward. The first one is about the global sequence of class definitions:

No two classes should have the same name.

The second one is all about constraints for code within a single class:

No two fields should have the same name, no two methods should have the same name, and no two parameters of a method should have the same name.

A well-designed implementation of a static checker should always follow the structure of its specification, and since there are two distinct set of rules, ours should employ two passes:

- one for enforcing that any pair of classes have distinct `ClassNames`; and
- another one for making sure that the various kinds of names in field declarations and method definitions, including each method's parameters, are distinct.

Once these validity constraints are successfully checked, we can think of the defined collection of `ClassNames` as a set. Indeed, it is just like the set of variables that are declared at the top of the program. And this suggests an analogous rule for the occurrences of `ClassNames`:

Each occurrence of a `ClassName` in the expressions of a program must refer to the name of one of defined classes.

Exercise 25. Design and implement a validity checker that enforces the validity rules for *Classy*. The checker consumes error-free ASTs from exercise 20; if it finds errors it annotates the AST appropriately.

Exercise 26. Add this rule to your validity checker.—The implementation follows the same pattern as checking that all variable references point to declared variables. Given a well-formed program, the set of classes is one direct component of its AST. This node contains the AST for all classes and, inside of those, their names. Hence, a static checker can enforce the rule by visiting all expressions and statements in the given program and checking that every occurrence of a `ClassName` is a member of the set of class names.

2.4 Undecidable Problems and Validity

Before we continue our exploration of validity constraints for *Classy*, we need to take a step back and consider the nature of computational problems. As it turns out, some problems are undecidable, and this idea is directly connected to the validity checking in language implementations.

Let's make this idea precise. A problem is a set S of data representations of information together with a subset $T \subseteq S$ of "positive" elements; all other elements are "negative.". Here information means programs in *Classy*, and ASTs are data representations of these pieces of information.

Such a problem is *decidable* if there is a total (recursive) function D from S to the Booleans such that for all $s \in S$

- if $D(s) = \text{true}$, then $s \in T$, and
- if $D(s) = \text{false}$, then $s \notin T$.

If there is no such total function D , the problem is *undecidable*.

Here is a first concrete example. Take the problem as the set S of all *Classy* ASTs, and the set T as those ASTs for which an evaluation on a suitably modified CESK machine terminates. People call this set the “halting problem”, and Turing proved that it is *undecidable*.

In the context of a class-based language, we get a second concrete example that is relevant to software developers. Let’s again start from the set of all ASTs as the set S , but take as T all those field-access nodes in the AST that an evaluation of suitably modified CESK machine reaches. If so, we could also consider T as the set such that the evaluation of $(o \rightarrow f)$ always succeeds. The question is whether such problems are decidable.

At this point, you might wonder whether D could use the CESK machine to determine whether a specific property holds for some *Classy* AST. In other words, why shouldn’t D run the given program on this CESK machine and observe how field references are evaluated?

Given your experience with running programs you know that program executions may not terminate. Using *Core*, a well-formed program may contain `(while 0 0. (x = x))`, and this fragment would cause it to loop forever. In short, if D relied on the CESK machine, it wouldn’t be a total function.

Stepping back, this brief excursion into computation theory relates validity checking to pragmatics. As a developer enters program text into an IDE, programming language tools such as the parser and validity checker are supposed to provide feedback. To be effective, these tools must always terminate and provide the IDE with an answer that can be rendered as feedback. Conversely, the problems that these tools address must be decidable. In particular, the developers want feedback *before*—and *without*—running the program.

Note Just because a problem is undecidable does not mean that we cannot determine whether $s \in T$ for some specific s . The key is that we cannot do so for all s . Researchers who investigate IDE tools occasionally exploit this “loophole.” They construct validity-checking tools that *attempt* to decide whether the code is valid and, if the answer isn’t available within a certain amount of time, it is ignored. We ignore this ambitious idea here.

2.5 *Classy*: Validity, the Undecidable Part

We can now return to the development of validity constraints for *Classy* and consider some problems that look more ambitious than those of section 2.3. Based on your experience with class-based programming languages, you might wonder whether the following validity problems are decidable and thus worth adding to a model of *Classy*:

- A (new C ($x_1 \dots x_n$)) expression may fail because C does not have n fields.
- A field access (target $\rightarrow$ f) may fail because target is not an object or there is no field f.
- A field modification (target $\rightarrow$ f = rhs) may fail because target is not an object or there is no field f.
- A method call (target $\rightarrow$ m($y_1 \dots y_k$)) may fail because target is not an object or there is no method m or m does not take k arguments.

While the first problem *is* decidable, the remaining three are undecidable.

Concretely, there is no total (recursive) function that consumes a program's AST and returns true if, say, a field access expression cannot fail. Computing this property would require inspecting the AST and determining exactly which objects target may ever denote. And this kind of problem is exactly one of the undecidable ones that people investigated in the 1930s and found undecidable.

A subset of this problem is decidable, however. When the target inside of a method body is *this* and there is no parameter and no locally declared variable named *this*, a validity checker could determine whether the object comes with a field named f. Similarly if the target of a field mutation or a method call inside of a method body is *this*, a validity checker could use the given AST to determine whether the named field or method is available.

Since these cases are rather special, and since the implicated phrases of *Classy* raise additional questions, we delay the treatment of such validity constraints. Technically speaking, chapter VIII covers the addition of a type system to our language models, which is a general technique for enforcing that only a subset of well-behaved programs is evaluated on the machine. Instead, here, we enrich the CESK semantics so that it comes with checks that determine whether a field access, a field mutation, or a method call is meaningful.

3 *Classy*: the Semantics

A modified CESK machine serves to explain the semantics of classes. Since the collection of classes does not change over time, it plays the role of a constant for the transition function. The load function can set up this quasi-constant so that the transition function can access it. As for the sets of control strings, environments, stores, and continuations, revising them is quite straightforward. In addition to the statements and expressions of *Core*, the ones for the modified CESK machine must account for the additional statements and expressions in *Classy*. Finally, a semantics for *Classy* must assign meaning to instances of classes, which implies that the set of values now comprises both numbers and data representations of objects.

3.1 Representation of Objects

The expression $(\text{new } C \ (x \ y \ z))$ expresses the intention to create an instance of C whose three fields are initially associated with the values of the variables x , y , and z . This information—the class combined with the field values—needs a representation in our model. Here are some possibilities:

1. The first five chapters present models that are completely mathematical. If we wanted to continue in this fashion, we would have to use a protocol for representing objects as bundles of locations in the store. For example, if a class has n fields, the representation of an instance could use $n+1$ slots in the store. One of them would contain the name of the class, and the remaining slots would contain the value of fields. In essence, this approach asks for implementing instances as immutable maps from fields to slots in the store. When the CESK machine evaluates a field-access expression, its transition function determines the field's slot in the store this map and then retrieves its value from there. Analogously, a field-modification statement results in a store update.
2. Another approach to extend the mathematical approach of the first five chapters is to use a vector-like concept from mathematics to represent an object—something like a Cartesian point—and to stick this representation into a single location. In practice, this idea means to pick some immutable data structure in the chosen implementation language and to stick instances of this data structure into the store. For a field-access expression, the CESK machine retrieves the

instance from its location and indexes the designated field; for a field-modification expression, it replaces the instance in the store with a new updated version.

3. In addition to these mathematical approaches, which are somewhat cumbersome to implement as executable models, a non-mathematical alternative is to exploit the imperative features of the chosen implementation language. By using a *mutable* data structure to represent the objects of *Classy*, modeling mutation becomes straightforward. To access and modify the fields of an instance, the machine simply accesses or mutates the underlying instance directly.

This book goes with the third alternative, emphasizing ease of implementation over mathematical reasoning.

Note This choice comes with some downsides. In particular, by tying the model to a mutable data structure in the implementation language, we must rely on our understanding of value aliasing in that language for the discussion of pragmatics in section 5. More generally, we can no longer interpret the model purely with our understanding of mathematics. Instead, we choose the so-called *meta-language* approach. That is, our model of semantics no longer just relies on neutral mathematical concepts, such as sets and functions, but, at least to some extent, on the details of our chosen implementation language. As John Reynolds pointed out a long time ago, this may inject elements of the implementation language into the modeled language. With the CESK approach, it is fortunately possible to avoid most of the problems in a natural way, and the rest of the book makes sure that the only point of “contact” remains the data representation of objects.

See “Definitional interpreters for higher-order programming languages” in ACM’s Annual Conference Record (1974).

Exercise 27. Design a data representation for instances of classes in your favorite programming language. Include the functionality for accessing and modifying fields.

Exercise 28. When the CESK machine evaluates a method call, it must retrieve the name of the class from the target object and the method’s AST from the class AST, which is contained in the given collection of classes.

To support this case of the CESK transition function, you will need a piece of functionality that retrieves the AST for a specific method when given a collection of classes, the name of a class, and the name of a method in that class.

3.2 CESK States for *Classy*

Let's revisit the contents of the CESK machine's registers:

- As for *Core*, control strings direct the CESK machine. The register contains one of four kinds of elements: an expression to be evaluated; the *value* from an expression evaluation; an error token, causing the machine to stop; or the $\dagger$ token, indicating that the machine is searching for the next expression to evaluate.

In *Classy*, however, the set of values includes *Core*'s numbers as well as data representations of objects. Several new forms of expressions operate on the second: *new*, *isa*, *target* $\rightarrow$ *f*, and *target* $\rightarrow$ *m*(...).

- Environments associate variables with locations. While the CESK machine for *Core* associates a new location with each variable declaration, the CESK machine for *Classy* acts likewise for each parameter of a method when it is called.
- Stores associate locations with *values*. Keep in mind that the set of *Classy* values includes data representations of objects.
- Continuations remain stacks of closures. Each closure consists of some instructions that remain to be executed and the content of the environment register applicable to the context of these instructions.

Figure 31 is an amendment to the notational conventions of figure 25. As you study the following subsections, consult these tables as needed.

As for the classification of the sets of states into initial states, intermediate states, and final states, the definitions remain the same. The set of initial states turns the variable declarations, statements, and return expression of a program into the first closure. Similarly, the final states are those that contain a value or the error token in the C register and an empty stack in the K register. All remaining states are intermediate states.

Exercise 29. Revise the data representation from exercise 16 for the states of the CESK machine for *Classy*. Also implement the load and unload functions:

- The load function must extract the collection of class definitions from the given program AST and make it available to the transition function.
- The unload function must deal with final states whose C register contains an object representation.

name(s) and term(s)	standing in for
C	a name for a class
f	a name for a field
m	a name for a method
o	a program variable that is supposed to denote an object
x*	a potentially empty sequence of program variables
v	values, include numbers and objects
v*	a potentially empty sequence of values
obj	an object in the implementation language
thisL	a location for this
xl*	a potentially empty sequence of locations

Figure 31: Additional CESK Conventions for *Classy* (also see figure 25)

3.3 The CESK Transition Function: Adapting the Cases for *Core*

The definition of the CESK transition function from chapter V almost works for *Core* statements and expressions within the *Classy* model language. But, given that the set of values now comprises numbers and data representations of objects, the cases do not apply as-is. Consider the simple case of an addition expression in the C register:

	Control	Environment	Store	Kontinuation
evaluate an addition				
before: (y + z)	e		s	k
after: (+ yn zn)	e		s	k
where	yl	= e[y]		
and	zl	= e[z]		
and	yn	= s[yl]		
and	zn	= s[zl]		

Since locations in the store map to numbers, this definition is a straightforward three-step procedure: retrieve the location of the two variables (yl, zl) from the environment; retrieve the numbers associated with the respective locations (yn, zn); and stick the sum of the two numbers (+ yn zn) back into

the C register. Now, however, the set of values comprises both numbers and objects, and as a result the addition may not always succeed.

Figure 32 presents an appropriate adaptation of the transition function, starting with two cases for addition expressions: one for success, when both variables refer to numbers in the current store; and one for failure, when at least one of them refers to an object. Beyond the cases for addition, figure 32 displays the case for a comparison expression. Like the success case for addition, it retrieves the values of the two variables and determines whether they are identical, which means

- when given two numbers, it checks whether the two numbers are close;
- when given two objects; it determines whether their data representations in the chosen implementation language are identical.

Otherwise, the result is 1.0.

All of the remaining rules of the CESK transition function from the preceding chapter work *mutatis mutandis*. For example, the cases for `if0` (or `while0`) statements remain the same, though the interpretation of the phrase “n is basically 0.0” shifts to the following:

if n is a number (i.e., not an object) and n is basically 0.0.

Exercise 30. Generalize the function `=0?` from exercise 16.

Exercise 31. Adapt all cases of the *Core* transition function from the preceding chapter so that they apply to the CESK machine for *Classy*.

3.4 Creating and Checking Instances

Figure 33 shows the transitions for creating instances of a class. While the validity checking ensures that C is a defined class, it does not guarantee that the new expression comes with the same number of values as there are fields in the class definition. If the two counts agree, the machine creates an instance of C and places it into the control-string register. Otherwise, the machine signals an error. See section 3.1 on what the data representation of an instance combines.

The third case in figure 33 concerns the `isa` expression. When such an expression shows up in the control-string register, the validity check of the syntax guarantees that C is the name of a class defined in the source text of the given program. The machine must determine whether the value is an instance of C, and, if so, deliver an appropriate value.

Again, this property could be checked but deferring this point to chapter VIII is the proper alternative.

	Control	Environment	Store	Kontinuation
evaluate an addition (success)				
before:	(y + z)	e	s	k
subject to: y and z denote numbers				
after:	(+ yn zn)	e	s	k
where	yl	= e[y]		
and	zl	= e[z]		
and	yn	= s[yl]		
and	zn	= s[zl]		
evaluate an addition (failure)				
before:	(y + z)	e	s	k
subject to: y or z does not denote a number				
after:	error	[]	[]	[]
evaluate a comparison				
before:	(y == z)	e	s	k
after:	rr	e	s	k
where	yl	= e[y]		
and	zl	= e[z]		
and	yn	= s[yl]		
and	zn	= s[zl]		
and	rr	= are-yn-and-zn-identical? yn zn		

Figure 32: The CESK transition function for *Classy*: addition, comparison

Exercise 32. Design and implement a function that consumes the name of a class and a sequence of values to create an instance. Re-use the solution of exercise 27.

Exercise 33. Design and implement a function that consumes any *Classy* value and a class C to determine whether (1) the value is an object and, if so, (2) it is an instance of C. As for the result of this function, see the discussion in exercise 16 and adapt as appropriate.

3.5 Field Access

In contrast to expressions involving class names, expressions involving field and method names need the transition function to check that every-

Control	Environment	Store	Kontinuation
evaluate a 'new' expression (success)			
before: (new C (x* ...))	e	s	k
subject to: class C has the same number of fields as in x* ...			
after: obj	e	s	k
where v*, ... = s[e[x*]], ...			
and obj = create instance of C using v*, ... for the values of the fields			
evaluate a 'new' expression (failure)			
before: (new C (x* ...))	e	s	k
subject to: class C does not have the same number of fields as in x* ...			
after: error	[]	[]	[]
evaluate an 'isa' expression			
before: (o isa C)	e	s	k
after: ans	e	s	k
where obj = s[e[o]]			
and ans = check whether obj is instance of C			

Figure 33: The CESK transition function for *Classy*: instances

thing matches. For field-access expressions such as $(o \rightarrow f)$ specifically, two situations may trigger faults if the machine doesn't recognize them first:

- The variable o may not refer to an object.
- While o may refer to an object, it may not have the specified field.

Since a field lookup can succeed only for objects with the desired field, the machine must signal an error in both situations.

The two cases for the transition function are presented in figure 34. The first case specified a *before* condition that includes “ o is an object and has the desired field.” Its *after* state consists of the value in the field—whose retrieval depends on your choice of data representation for objects—and the given environment, store, and continuation. As in other error cases, the second case in figure 34 places error into the C register and resets the remaining ones to initial values.

Exercise 34. Design and implement a function that consumes an object and the name of a field and that returns the value associated with this field

Control	Environment	Store	Kontinuation
evaluate a field reference (success)			
before: $(o \rightarrow f)$	e	s	k
subject to: o denotes an object and has the desired field			
after: ans	e	s	k
where $obj = s[e\ o]$			
and $ans = \text{get value of } f \text{ from } obj$			
evaluate a field reference (failure)			
before: $(o \rightarrow f)$	e	s	k
subject to: o does not denote an object or does not have the desired field			
after: error	[]	[]	[]

Figure 34: The CESK transition function for *Classy*: field access

in the given object. Keep in mind that due to the formulation of the transition function, it is guaranteed that the given object has the specified field. Re-use the solution of exercise 27.

3.6 Field Mutation

Like a variable assignment, a field-mutation statement consists of a left-hand side and a right-hand side, both of which must be evaluated. For the statement to make sense, the left-hand side must refer to an object with the desired field. The right-hand side expression must evaluate to a value, which ends up in the specified field *after* the machine has executed the statement.

Figure 35 displays the three cases of the transition function that determine the meaning of a field-mutation statement:

1. The first case states that the machine is in search mode ($\dagger$) and that the top-most closure of the continuation comprises a sequence of statements, starting with a field-mutation statement, and preceded by an empty sequence of definitions. The *after* state's C register contains the right-hand side expression, meaning the machine is supposed to evaluate it next. Once C contains the value of the right-hand side expression, the machine distinguishes two cases.

	Control	Environment	Store	Kontinuation
search ends a field assignment				
before:	$\dagger$	e	s	$\langle\langle [], (o \rightarrow f = \text{rhs})::\text{stmt}^*, r \rangle\rangle$
after:	rhs	e	s	$\langle\langle [], (o \rightarrow f = \text{rhs})::\text{stmt}^*, r \rangle\rangle$
execute a field assignment (success)				
before:	v	e	s	$\langle\langle [], (o \rightarrow f = \text{rhs})::\text{stmt}^*, r \rangle\rangle$
subject to: o denotes an object and has the desired field				
after:	$\dagger$	e	s	$\langle\langle [], \text{stmt}^*, r \rangle\rangle$
where $\text{obj} = s[e[o]]$				
and set f in obj to v				
execute a field assignment (failure)				
before:	v	e	s	$\langle\langle [], (o \rightarrow f = \text{rhs})::\text{stmt}^*, r \rangle\rangle$
subject to: o does not denote an object or does not have the desired field				
after:	error	$[]$	$[]$	$[]$

Figure 35: The CESK transition function for *Classy*: field mutation

- The second case specifies that the *before* state identifies o with an object in the store, specifically with an object that has the desired field. Its *after* state description says that the machine is back in search mode, that the object in the store is modified, and that the evaluation of the field-mutation statement is complete.
- The third case informs the reader of this specification that, if o does not denote an object or if it denotes an object without the desired field, the language implementation is supposed to signal an error.

Exercise 35. Design and implement a procedure that consumes an object, the name of a field, and a value. It modifies the named field of the object, so that it is associated with the given value. Keep in mind that due to the formulation of the transition function, it is guaranteed that the given object has the specified field. Re-use the solution of exercise 27.

Exercise 36. The semantics of field mutations evaluate the right-hand side of $(o \rightarrow \text{field} = \text{rhs})$ *before* checking whether the left-hand side refers to an object. Rewrite the cases so that the right-hand is evaluated only if the entire instruction may succeed. Can a programmer observe the difference?

3.7 Method Calls, the Old Way (1959)

In *Classy*, method calls are expressions that must satisfy certain conditions. before the machine can execute it. The target (*o*) must refer to an object; the object must have the desired method; and the number of parameters of the method must be the same as the number of arguments. If this is the case, the machine transitions to a state with new content in all registers:

- The environment register contains a table that associates the method's parameters and this with locations that the current store *s* does not associate with values.
- The store in the *S* register associates these newly allocated locations with the corresponding values. In particular, the location associated with this maps to the object itself.
- Finally, the machine pushes a new closure onto the continuation in the *K* register. This new closure combines the (AST of the) method's body with the environment of the method call.
- The control-string register contains $\dagger$, because the machine must find the next expression to evaluate. After all, the next expression to be evaluate is somewhere in the method's body (a sequence of definitions, statements, and a return expression).

See figure 36 for the complete specification.

It is for the method-call case of the transition function that the name of the class is included with an object. Given the object that *o* denotes, the machine can retrieve the class definition by using the class name. The desired method is located in the (AST of the) class definition, meaning the machine can both check the conditions of the *before* specification and extract the method's AST itself for the *after* state.

Exercise 37. Design and implement a function that consumes an object, the collection of all defined classes, and the name of a method, and that returns the (AST of) the specified method. Keep in mind that due to the formulation of the transition function, it is guaranteed that the given object has the named method. Re-use the solution of exercise 27.

The CESK transition function of *Classy* has just two rules for the evaluation of a method call. While the second merely describes the failure case, the first one specifies how the method's body becomes the driving instruction set in the top-most closure. What figure 36 does not explain, is how the machine “returns” from the evaluation of the method body.

The rule models the workings of method calls in two early successful attempts to systematic language design: Algol and Fortran.

Control	Environment	Store	Kontinuation
evaluate a method-call expression (success)			
before: $(o \rightarrow m(x^* \dots))$	e	s	k
subject to: o denotes an object ... has the desired method ... the corresponding number of parameters			
after: $\uparrow$	$e1$	$s1$	$\text{push}[cl, k]$
where obj	$= s[e[o]]$		
and $v^*, \dots$	$= s[e[x^*]], \dots$		
and $[para^*, body]$	$=$ the parameters & body of method m per the class of obj		
and cl	$=$ closure: body in e		
and $thisL$	$=$ a new (relative to s) location		
and $xl^*, \dots$	$=$ as many new (relative to s) locations as elements in $x^*, \dots$		
and $e1$	$= [][this = thisL][para^* = xl^*], \dots$		
and $s1$	$= s[thisL = obj][xl^* = v^*], \dots$		
evaluate a method-call expression (failure)			
before: $(o \rightarrow m(x^*))$	e	s	k
subject to: o does not denote an object or ... does not have the desired method ... takes a different number of arguments			
after: error	$[]$	$[]$	$[]$

Figure 36: The CESK transition function for *Classy*: method call

Consider the following schematic method-call:

$$(o \rightarrow m(a_1 \dots a_n))$$

Each a_i stands for an argument. Furthermore, assume o points to an object with a method m and that the method's definition looks like this:

$$(\text{method } m(p_1 \dots p_n) \boxed{\text{def}^* \text{ stmt}^* \text{ ex}})$$

One way to understand such a call is as a dynamic creation of a block:

```
(block
  (def this o)
  (def p1 a1)
  ...
  (def pn an)
  def* stmt* ex)
```

A close look shows that, if the keyword `block` is ignored, this block has exactly the shape of a *Core* Program, though its statements and expressions may refer to classes and object-oriented operations.

Stop! Use the transition function to convince yourself that once the synthetic variable declarations are evaluated, the CESK machine is in essentially the same state as if it had directly used the case for a method call from the preceding section.

At this point, it is clear why it is unnecessary to add a “return from method call” case to the transition function. If method calls conceptually create program blocks, a return from a method call is just like the evaluation of the last expression in such a block. Hence, the last case of the CESK transition function for *Core* applies.

Stop! Do take a close look at this case in figure 24.

While the CESK machines for *Classy* seems to resemble the one for *Core*, the two differ in a significant way. Given the finite nesting structure of *Core* programs, the K stack of the *Core* CESK cannot grow deeper than the deepest nesting of blocks in the given program. By contrast, since the body of a method may contain a call to itself, the K stack of closures can grow arbitrarily deep—just as the phrase “dynamic creation of a block” indicates.

3.8 Method Calls, the New Way (1971)

For some object-oriented design patterns, the arbitrary growth of the stack is a fatal flaw. These patterns ask the developer to align method calls with the “is a” and “has a” arrows in class diagram. But, doing so leads to method definitions whose return expressions create direct and indirect calls to each other. As a result, in a language implementation that follows the old implementation technique, the stack grows indefinitely and eventually hits its finite limit. That is, the old implementation technique fails developers in work situations where an object-oriented pattern would otherwise offer an established solution for how to structure their code.

In response to this observation, language creators developed an alternative “return from a block” instruction dating back to the late 1960s and early 1970s. Figure 37 presents this alternative case for the CESK transition function, dubbed *pitch*, which is short for proper implementation of tail calls. At a high level, the idea is simple: the machine pops the stack *before* it has evaluated the return expression.

At the detail level, this instruction needs a step-by-step explanation. Critically, since the top-most closure’s environment is associated with the method-call site, its association of variables with locations is likely a mis-

Read up on, say, the interpreter pattern presented in Gamma et al.’s Design Patterns.

Smalltalk and Scheme pioneered this approach to method calls.

	Control	Environment	Store	Kontinuation
search ends with the return expression (pitch)				
before:	$\uparrow$	e	s	$\langle\langle \ [], \ [], r \rangle\rangle$
after:	$r^\wedge$	$e1$	$s1$	$\text{pop}[k]$
where	$e0$	$=$ the environment in the top-most closure		
and	$z^*, \dots$	$=$ all variables of r		
and	$v^*, \dots$	$= s[e[z^*]], \dots$		
and	$r^\wedge$	$=$ rename all variables of r to unique names (relative to $e0$)		
and	$x^*, \dots$	$=$ all variables of $r^\wedge$		
and	$x1^*, \dots$	$=$ as many new (relative to s) locations as elements in $x^*, \dots$		
and	$e1$	$= e0[x^* = x1^*], \dots$		
and	$s1$	$= s[x1^* = v^*], \dots$		
and	k	$= \langle\langle \ [], \ [], r \rangle\rangle$		

Figure 37: The CESK transition function for *Classy*: pitch

match with the variables in the expression. Hence, the instruction synthesizes a new expression and environment as follows:

- It replaces all variables in the return expression r with new variables.
- It allocates locations for these new variables in s .
- It associates the new variables with these locations in the environment of the top-most closure.
- It populates the locations with the values from the environment in E before the content of this register is replaced.

Given the revised return expression, environment, and store, the machine can now pop the continuation and perform the evaluation of the return expression safely.

Note Language creators with an old mind set refer to pitch as “tail call optimization.” In reality the two ways of thinking about calls and returns are just distinct solutions to the same problem. The simplicity of the old solution convinced language creators in the late 1950s to choose it. But, by the early 1970s, others recognized that while the second approach imposes more work on the creators of an implementation, it simplifies the lives of working developers. Due to the reluctance to perform this work,

it took many language creators decades to follow the new way of thinking. By now, even the mainstream implementations of the C programming language (GCC, CLang) do so. Only Java and Python have stayed behind.

Exercise 38. Create an example of a call to a method whose body ends in an expression that calls the method again. Work through the machine transitions using both the old and the new implementation technique. Compare the stack depth after a couple of “iterations.” — Also see the exercises in section 4.

Exercise 39. Imagine a variant of *pitch* that merely pops the stack and evaluates the return expression in the environment of the top-most closure in the *before* state. Create an example that (1) differentiates this instruction from the correct one in figure 37 and (2) argue that the correct variant delivers the expected result.

4 Project Tasks

The exercises in the preceding two sections request a parser for *Classy*, several *Classy*-specific validity checkers, and an adaptation of the CESK machine to the new expressions and statements. It is now time to compose these pieces of functionality to a complete, executable model.

Exercise 40. Adapt the main function from exercise 17 so that it can run the entire process of parsing *Classy* program syntax, validating it, and determining its meaning. Use your solutions from exercises 20 through 39 in this chapter.

The output should be an error message if the program is ill-formed, invalid, or causes a constraint violation when run on the CESK machine for *Classy*:

- “syntax error” if the parser discovers any mistakes;
- “duplicate name error” if the validity checker encounters two classes with the same name, a class with two identical method names, or a class with two identical field names;
- “undeclared name error” if the validity checker encounters an undeclared `ClassName` or variable name; or
- “runtime error” if the CESK transition function stops due to a final state with an error in its C register.

direct tail call	indirect tail calls
<pre> ((class While (count) (method w() (def d (this --> count)) (def one -1.0) (def e (d + one)) (if0 e (e = e) (this --> count = e)) (this --> w ()))) (def u 3.0) (def w (new While (u))) (w --> w ())) </pre>	<pre> ((class While () (method w(other) (other --> w (this)))) (class Repeat () (method w(other) (other --> w (this)))) (def r (new Repeat ())) (def w (new While ())) (w --> w(r)) </pre>

Figure 38: Examples of direct and indirect tail-calling methods

If the final CESK state contains a number or an object as a final control string, main prints the number of “object”, respectively.

Exercise 41. Figure 38 displays two concrete *Classy* programs. The one on the left is said to use a tail-recursive method, because its final expression is a call to itself. By contrast, the program on the right employs *indirect tail calls*. Both methods come with return expressions that call a method in some given object, but not this.

Do these programs terminate on your modified CESK implementation?

Modify these programs so that they terminate in an error state after a non-trivial number of “iterations.”

Exercise 42. Equip your CESK machine implementation with code that measures and prints the depth of the stack in K after every step that the transition function takes.

Evaluate the program in figure 38 with a direct tail call on your modified CESK implementation. What do you observe?

Evaluate the program with indirect tail calls on your modified CESK implementation. What do you observe?

5 Pragmatics: Developers Must Learn to Think Hard

One major cost due to the addition of classes is that reasoning about a program’s properties becomes complex. With a model, software develop-

ers can pinpoint the reasons for the extra complexity; with an executable model, like the one for *Classy*, they can also run quick experiments to confirm their understanding of what happens.

This section presents examples that illustrate what makes logical reasoning in a language with classes complex: dynamic extent differs from lexical scope; object aliasing means a mutation in one place has a “long distance” effect; equality and parameter passing interact with aliasing in a non-trivial manner; and the meaning of field references and method calls are inherently dynamic properties.

5.1 Dynamic Extent vs Lexical Scope

When the CESK machine evaluates a variable declaration, it enters a specific lexical scope of the program text. It creates a new environment from the old one and an association between the variable and a new location. It then places this environment into the E register. As long as this new environment exists in the CESK machine—either in E or within a closure on the stack in K, the variable is alive. By design, this life time ends when the CESK machine finishes the evaluation of the lexical scope associated with the variable declaration.

By contrast, when the CESK machine encounters a new expression in the program text, it creates an object and places it into the C register. What happens after that depends on the remaining instructions in k. For example,

```
(x = (new C ()))
```

would assign the new object to an existing variable, meaning the object would be accessible as long as the variable is alive. But,

```
((new C ()) --> m())
```

would just call a particular method and then the object would cease to exist.

Equipped with this simple explanation, take a look at this example:

```
((class Counter (count)
  (method getCount() (this --> count)))
 (def u 3.0)
 (def v 42.0)
```

```
(block
  (def c (new Counter (u)))
  (v = c))
```

```
(v --> getCount()))
```

This expression is ill-formed in our restrictive syntax but well-formed in most object-oriented programming languages.

The boxed region is the lexical scope for the program variable *c*. Its initial value is an instance of class *Counter*. Otherwise the block contains a single assignment statement, which places the value of *c* into *v*, a program variable in the outer scope.

Thus, when the CESK machine encounters this nested block, it squirrels away the current content of *E* by pushing a closure onto the stack in *K* and then proceeds as follows:

1. The search inside the block places the new expression into *C* and instantiates the class to yield a new instance.
2. Once this instance is in *C*, the CESK machine allocates a location *cl* that is not in the current store *s*; associates *cl* with the new instance in an extended store; and adds an association between *c* and *cl* to the current environment. Let's call this new environment *ec*.
3. Using *ec*, the machine finds and evaluates *c*. The result of *s[ec[cl]]* is the new instance.
4. Now that the instance is back in *C*, the machine completes the assignment statement. It looks up *v*'s location in *ec*, yielding *vl*, and creates a store in which *vl* maps to the newly created instance of *Counter*.

At this point, the block's instructions are exhausted and the CESK machine pops the content of *K*. The result is that *ec* is no longer reachable in any way, but *vl* still contains the instance of *Counter*. Thus, calling *getCount* in the return expression succeeds and yields 3.0.

Stop! Why does it yield 3.0?

In this example, *ec* denotes the scope of the boxed block statement; to be even more precise, it denotes the lexical scope of the declaration of *c*. As the process description explains, the environment comes into existence and disappears. By contrast, the store keeps getting extended and never disappears. Hence, it is possible to assign the instance of *Counter* to a variable outside the scope of the nested block, a scope whose meaning is just *e*. Put differently, the instance is alive beyond the lexical scope defined by the box, because it is in the store.

Stop! When does the life of an instance end?

Note The dynamic extent of an instance is determined in an implicit manner. Most generally speaking, if at any point during an evaluation of a program it is possible to replace an instance in the store with a number, say

42.0, and the rest of the program evaluation does not change, the instance is *dead*. The earliest such point in a program's evaluation determines the life span of the instance; the time between the creation of the instance and this point is its *dynamic extent*.

One consequence of a finite dynamic extent is that the location associated with an instance could be reused for other values. Recognizing this property and getting the location ready for reuse is called *garbage collection*. The CESK machine, as presented, comes without a garbage collection mechanism. Hence, all instances remain in the store until the end of the program evaluation. If we limited the store to a finite size, evaluations would occasionally exhaust all available locations and fail due to this limit. Mathematical models of languages, including executable models, ignore such limits, but a programmer must keep them in mind. See chapter X.

5.2 Object Aliasing

In the preceding subsection, two variables briefly referred to the same object, though one of them immediately went out of scope. When two variables simultaneously refer to the same object, programming language people speak of *object aliasing*, and programmers must be keenly aware of aliased objects. After all, modifying the object via one variable reference means a reference via the second variable is going to notice the change.

Consider the following program:

```
((class Counter (count)
  (method getCount ()
    (this --> count))
  (method upCount ()
    (def crt (this --> count))
    (def one 1.0)
    (this --> count = (crt + one))
    0.0))

(def u 3.0)
(def c (new Counter (u)))
(def d c)

(u = (d --> upCount ()))

(c --> getCount ()))
```

Its three variable declarations create a single instance of Counter and give it two names: *c* and *d*. The following statement is really just a method call, because the value of left-hand side variable is not used again. Key is that

the right-hand side uses *d* to initiate the method call, not *c*. A quick look at the definition of *Counter* shows that this method call increases the *count* value of the object. But, when the return expression observes the value of *count* via *c*, it notices this change from 3.0 to 4.0.

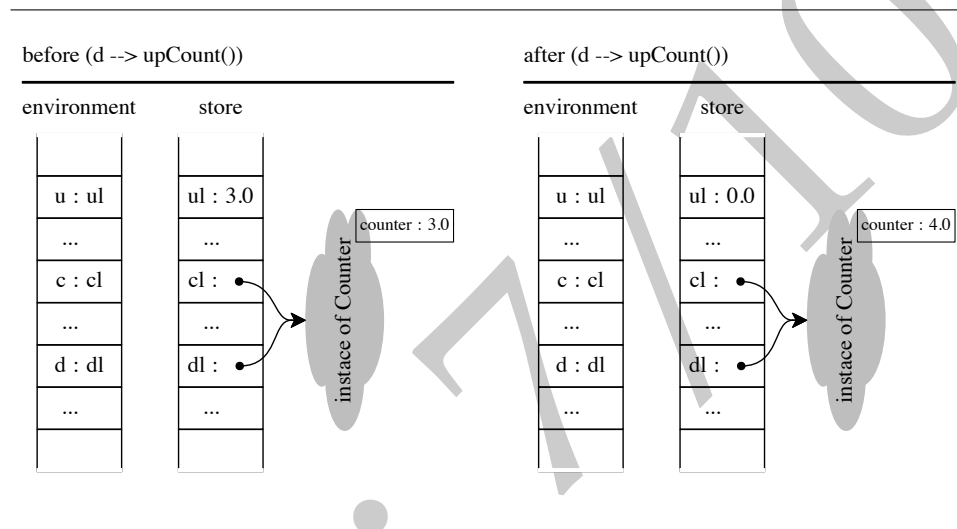


Figure 39: A diagram of object aliasing

A programmer with an understanding of the CESK semantics can think through the relationship among the variables, their locations, and the various objects. In this particular case, the environment for the program scope consists of three variables associated with three locations. The method call to *upCount* pushes this environment onto the continuation, together with the remainder of the assignment statements and the return expression. Once the CESK machine has determined the effects and result of the method call, this environment is restored and the assignment to *u* is executed. At this point, the machine has nothing left to do but evaluate the return expression of the program: (*c* -> *getCount*()).

Figure 39 shows how a programmer can imagine this state transition graphically. It consists of two diagrams: one of the *before* state and one of the *after* state—meaning before and after the evaluation of

(*d* --> *upCount* ())

Each diagram comes with two tables: the environment and the store. The environment associates variables with locations, and it doesn't change. The

The dots in the tables aren't necessary for the example. They indicate that environments for larger programs contain more entries.

store associates locations with values, which changes as needed. For example, the location `ul` maps to 3.0 in the *before* state and to 0.0 in the *after* state.

Stop! Why is this the case?

Each state diagram contains two arrows that point to a cloud: one from `cl`, the location for `c`, and one from `dl`, the location for `d`. The cloud represents the only instance of `Counter` that the sample program creates. This object's single field is displayed as a box near the top-right. In the *before* diagram, this box contains `counter : 3.0`; in the *after* state, it is `counter : 4.0`. Since both the locations for `c` and `d` point to the same object, the change is visible via references to either variable in the program. Hence, `(d -> getCount())` is 4.0.

Exercise 43. Turn this example of a *Classy* program with object aliasing into a unit test for `main` from exercise 40.

Exercise 44. Construct an example like the one from this subsection in your favorite programming language and observe the effects of aliasing.

5.3 Equality

The discussion of object aliasing directly points to another challenging aspect of object-oriented languages, namely, the question of when two values are equal. *Classy* has the ingredients to investigate this question, which has vexed developers for many decades. Let's consider three cases:

1. If a language's values include integers or some finite subset, equality is obvious. Mathematics tells us when two integers are equal.
2. It gets complicated when real numbers are added. As chapter III.5.2 points out, the data representation of reals causes serious problems when it comes to equality. Due to the inaccurate mapping of reals to IEEE doubles, comparisons of the latter demand an error tolerance. Two numbers are equal only *up to some small number ϵ* .
3. Mutable objects like those of *Classy* inject yet another dimension into this mix, namely, time. Two Cartesian-coordinate objects may contain the same numbers in their two fields at some point in time, but unless the language or the developers prohibits mutation of their fields, these objects may differ soon after.

Seen this way, immutable objects are equal if their field values are equal. researchers speak of *extensional equality* in this case.

At the other end of the spectrum, diagrams such as those in figure 39 provide a different answer: Given two mutable objects, they should

be considered equal if they are located at the same place in the store. Programmers speak of “pointer equality” in this case. Philosophically inclined language researches dub this notion *intensional equality*, because identical location means identical construction in all aspects. What developers have to ask then is whether

two mutable instances of a class should be considered equal
if they have equal field values *at some point during execution*.

In one case, giving an affirmative answer might be wrong, and in another case it might just be what a project needs.

class definition	program body
<pre> (class C (f) (method eq (other) (def old (this --> f))■ (def myf old) (def urf (other --> f))■ (def res 1.0) (if0 (old isa C) (this --> f = 42.0) (this --> f = (old + res)))■ (urf = (other --> f)) (myf = (this --> f)) (res = (urf == myf)) (this --> f = old) res)) </pre>	<pre> (def one 1.0) (def c (new C (one))) (def d c) (def e (new C (c))) (one = (c --> eq(d))) (d = (c --> eq(e))) (one + d) </pre>

Figure 40: Intensional (also known as) pointer equality

Based on an analysis such as the above, languages in the functional family typically support both operators by default. In contrast, creators of object-oriented programming languages opt to supply pointer equality as a built-in operator. Put differently, they delegate the definition of extensional-equality operators to developers. *Classy*’s `==` operator combines pointer equality with approximate equality on IEEE doubles. Occasionally they even come with a third one, dubbed an “always equal” operator, which combines extensional equality for immutable values with intensional equality on mutable ones—and its very name reminds developers that mutation injects time into their reasoning.

Exercise 45. Figure 40 shows how to implement (a primitive version of) intensional equality in *Classy*. The left-hand side of the figure displays a class with a single method, `eq`. This method consumes another instance of the same class and determines whether the two are intensionally equal.

Determine the result of this program. Explain.

Exercise 46. Design an extensional equality method for class *C* in figure 40 and add it to the class.

Create a program body that shows how two extensionally equal instances of *C* are intensionally distinct. Explain.

When code is run in parallel threads, this eq method won't function properly, which is why language implementation provide intensional equality as a basic, fast instruction.

5.4 Call-by-value Prevents Variable Aliasing

Another aspect of *Classy* that interacts with aliasing in a non-trivial manner is how it determines the values it hands to a method upon a call. Language researchers dub this transfer of a value as a parameter-passing mechanism. This particular one is referred to as *call-by-value*.

Note Students get to know this idea in (pre)algebra courses. If

$$f(x) = \dots x \dots$$

is a function definition, then a use of f , say $f(40+2)$, is evaluated by substituting 42 for x into the expression of the right-hand side of f 's definition.

Although it seems straightforward to use this same idea for method and function calls in programming languages, the designers of languages have experimented with a rather large variety of ideas over the past six decades: call-by-value, call-by-reference, call-by-copy-in-copy-out, and some more. The reason for this experimentation is that a straightforward adaptation of the mathematical concept faces two problems:

- The first one concerns the notion of what is passed, that is, the question of what programmers should consider an argument *value*. As you may have noticed, all of our semantic models—from the CS machine to the CESK machine—explicitly define the set of values and treat that set as the set of argument values. In *Classy*, the set of values comprises numbers and, unlike mathematics, data representations of mutable objects.
- The second one is due to the interplay between the kind of argument values in a language and assignment statements. Assignments for both plain variables or the fields of objects are present in most programming languages, but they do not exist in mathematics. Since

The designers of Algol and Haskell also experimented with when to determine the value of an argument. We ignore this dimension.

methods are added so that programmers can abstract over repeated pieces of code, the question arises whether methods should also abstract over code with assignments of either kind. The set of argument values determines when the answer is positive. In *Classy*, because objects are values, abstracting over field assignments is straightforward as the preceding sub-section shows.

Given this background, old timers of the software profession like(d) to confront job interviewees with the following puzzle:

Write a function in your favorite programming language that swaps the values of two given variables. Alternatively, explain why writing this function is impossible.

What they are getting at, is that there is a direct relation between the parameter-passing mechanisms of a language and aliasing. Specifically, when the set of arguments values includes variables (or better the store locations bound to variables), then a natural consequence is that it is possible to alias variables, just like objects, by passing them to methods arguments. If so, a method call would be able to change the value of a variable at the call site, and writing a “swap” method would be straightforward.

```
((class Counter ()
  (method upCount(x)
    (x = 42.0)
    0.0))

(def c (new Counter ()))
(def y 1.0)
(def uselessResult
  (c --> upCount (y)))
y)
```

env. for call

x : 1
this :

env. for body

y : 1
c : ...
u : 77.0

store

1 : •
...

Figure 41: What variable aliasing would look like, if it existed in *Classy*

A look at the CESK machine of *Classy* answers the puzzle in the negative. Every method parameter gets associated with a brand new location,

and each of these locations maps to the corresponding value in the argument list, independently of what the argument is. It is this transfer of values from one place to another that justifies the label “call-by-value.”

If a programming language uses call by value only, variable aliasing and its effects are impossible.

Now contrast call-by-value with an alternative that re-uses a location when the parameter is a variable. This alternative is referred to as call-by-reference. Figure 41 illustrates this idea with an example. The code on the left-hand side passes *y* (boxed) to *upCount*. In a call-by-reference world, the semantics would look up the location associated with *y* and point the parameter, *x*, to the same location. Therefore, locations are the argument values for variable arguments rather than the value that the location points to in the store. The right-hand side of the figure explains this situation with a diagram; technically, the diagram explains the environment and store right after the CESK machine begins its search for an expression in the body of *upCount*. It is easy to see that an assignment statement to *x* inside of *upCount* would change the value of *y*, too.

In Classy, all arguments to method calls are always variables (to simplify the semantics). Briefly imagine a relaxation of this constraint.

A programming language with some form of call-by-reference argument passing enables variable aliasing.

Just like with object aliasing, a programmer using a call-by-reference language may intend the aliasing of variables across method boundaries or may accidentally cause variable aliasing. In the first case, the code may just abstract over repeated variable assignment patterns, which is precisely what the question of old timers aims at. In the second case, the programmer may observe strange behaviors of the code. To assist client programmers and future maintainers, the creator of a method that uses call-by-reference should advertise any possible effects via a purpose-and-effect statement (method documentation), because a by-reference indicator alone does not suffice. Such a statement should help programmers with understanding aliasing effects.

Note Over time, programming language creators have recognized that the downsides of call-by-reference parameter passing outweigh the upsides. As a result many modern languages use call-by-value only.

The C family of languages, including C#, remains an exception, offering several by-reference parameter-passing mechanisms. In addition, C# supports forms of *in* and *out* parameters, inherited from the Ada language.

Old timers used to C-style coding may also make statements such as “objects are passed by reference” and “variables are passed by value.” Such statements simply show a lack of understanding of semantic models. Objects in a *Classy*-like language *are* semantic values and are mutable; no confusion of terminology is needed to explain these effects.

5.5 Dynamic Dispatch

Method calls and field access/mutation pose the problem of *dynamic dispatch*. Consider a schematic method call expression: $(o \rightarrow m(a \dots))$. The method name m is just that, a name. Without any knowledge of the target object o , a programmer cannot know which method’s code is going to be evaluated in reaction to this expression. Stated like this, the programmer faces the problem of reading the program’s code and deciding which objects o may refer to during program execution. Since we already know that asking this problem is undecidable, the programmer cannot rely on any IDE algorithm to figure out this question. In the worst case, the programmer must wait until the program execution reaches $(o \rightarrow m(a \dots))$.

<pre>(class Cowboy () (method draw() 1.0)) (class Artist () (method draw() 666.0))</pre>	<pre>(def a (new Artist ())) (def c (new Cowboy ())) (def x 0.0) (if0 (x = a) (x = c)) (x --> draw())</pre>
---	--

Figure 42: An example of dynamic dispatch

Figure 42 displays an old and well-known illustration of dynamic dispatch for methods. The class definitions of the program, displayed on the left, introduces Cowboy and Artist, both of which come with a method named draw. The names are to suggest that “drawing” for each is considered a rather different action. Here the methods return 1.0 and 666.0, respectively. The program’s body, shown on the right, creates an instance of each class, named c and a , respectively. Next a conditional statement selectively assigns one of these objects to x . Finally, the return value is determined by calling draw on x .

Key to this code snippet is the empty box in the test position of the conditional. In *Classy*, a programmer could create a random number generator,

and the conditional could check whether a generated number is greater or equal to some limit. In an actual class-based language, say Java, a programmer could use an expression to check whether the number of inches of rainfall today exceeded some given amount. In short, it is easy to see how the content of x is not determined until the conditional is evaluated, even for simplistic example like the one in figure 42.

Like the object aliasing aspect of parameter-passing, dynamic dispatch is desirable and potentially dangerous. It is desirable when a form of information needs a data representation that requires several classes, a so-called union type. When a programmer represents data with a union of several classes, all of them implement a method with the same name but different code. By contrast, dynamic dispatch poses a problem when the data flow and control flow of some code is opaque. In that case, the code may call a method with the correct name and unexpected behavior.

If you have chosen a class-based language for the projects of this book, you know that your data representation of ASTs is an example of this kind.

Note While a developer can warn clients about by-reference parameters in the documentation, it is impossible to do so for dynamic dispatch. Only disciplined code development can help a programmer with reasoning.

Exercise 47. Use the *Classy* program from figure 42 to formulate unit tests for `main` from exercise 40.

Exercise 48. Develop a *Classy* program that illustrates dynamic dispatch for field access. Use it to formulate unit tests for `main` from exercise 40. Re-use the relevant parts of the solution to the preceding exercises.

Jason: 7/10

Why do languages come with modules?

1 The Pragmatics Question

The models of the first six chapters approximately correspond to the programming languages people invented and used from the 1950s through the 1970s. Back then, the creation of software was mostly the task of a lonely programmer or small teams. In-person meetings established conventions and protocols that overcame weaknesses in programming-language support for separate development of components.

Imagine the following scenario in languages like *Classy*:

- All programmers need libraries beyond the basic ones that a sparse language provides. They may want basic data structure libraries or basic mathematical algorithms. Eventually someone creates such libraries, say in the form of classes, and they are added to the “batteries” that come with the language. The language implementation is then modified so that it *prepends* these libraries to the programmer-developed code as part of the loading process.
- A software architect designs a rather large software system and recognizes that this is way too large for the current team. So the architect asks the team to lead a collaborative effort with other teams. In that context, the team is charged with identifying the interfaces of components, say collections of yet-to-be-implemented classes, and to divide the work among its members team and other teams. For a time, all these people work separately, creating classes to meet the interface specifications and auxiliary classes to factor these implementations into reasonable chunks. The architect’s idea is to *compose* all these separately developed classes to set up and run one complete program.

If all this is done naively, such programs are sooner or later going to suffer from name clashes and other compatibility problems that inevitably arise in large-scale projects where teams manage complexity by splitting the work.

A team using a class-based language could work around this shortcoming with the use of naming protocols. Each member would have to systematically adhere to this protocol. The team might also agree to build a linter tool to check protocol adherence. However, any work situation that involves some kind of manual protocol, even when checked by a tool, is highly error-prone. Just imagine a successful team whose members quickly quit one after another. New members are brought on. Everyone needs to be trained in the use of the protocol and the linter. Soon someone is going to violate the protocol and cause havoc.

In our context, support for software development in a team means a language design that enables a team to divide up a system design into separately implemented and compiled pieces. Each of these pieces may contain a class or even several, and each of these pieces depends only on a few, explicitly mentioned pieces. When a language enables teams to organize code in this manner, it simultaneously avoids naming problems and enables the creation of separate (and separately compiled) libraries. In concrete terms, we imagine a mechanism for dealing with separately developed classes with explicit dependency declarations.

1.1 Design Choices

Programming language creators recognized the need for separate development as early as the late 1970s. Since then they have created a fair number of linguistic mechanisms to address the naming problem:

- The family of Lisp languages, and languages inspired by Lisp, started supporting namespaces as early as the 80s. For instance, in Java, the names of classes, variables, and fields exist in separate namespaces, and the Java implementation resolves the same name in these distinct spaces. Consider figure 43, which presents two classes: `Main` and `c`. While the latter also contains a field named `c` and uses `c` as a parameter name in the constructor, the former creates an instance of `c`, stores it in the variable named `c`, and dereferences field `c` via `c.c`.

Exercise 49. Compile and run the Java program in figure 43. It is going to print 66, which confirms that the Java implementation is capable of managing the various roles of the name `c`.

- Class-based languages added packages and inner-classes in the 90s. Both enable software developers to manage names, usually via the use of so-called fully qualified names. A package in Java is a container for related classes. For example, `java.util.Date` is a package that offers a generic data representation for dates and pieces of functionality for manipulating them. And, if dates are needed for SQL databases, developers are supposed to link in `java.sql.Date`, a package for representing and manipulating SQL dates.
- Around 1980, *Modula*, an aptly named language, introduced a distinct idea into the design space: modules. The addition of modules simultaneously enables separate development and name management.

Note By now, even Java offers modules. Its creators have understood that namespaces, classes, packages, and modules play distinct roles in different working situations.

```
class Main {  
    public static void main(String argv[]) {  
        c c = new c(66);  
        System.out.println(c.c);  
        return ;  
    }  
}  
  
class c {  
    int c = 2;  
    c(int c) {  
        this.c = c;  
    }  
}
```

Figure 43: Java: namespaces and fully qualified package names

This chapter presents a simple model of a module-based extension of *Classy*. Its purpose is to bring across how modules work, how they differ from classes, and how they assist software developers with their work in teams. For now, imagine the following general arrangement:

- We use the word *system* to refer to a collection of modules.
- A module contains and exports class definitions.
- A module may import classes only from already existing modules.
- Since it is about name management, a *Module* programmer should recognize the underlying class-based program without any problem.

Like all models, this one greatly simplifies actual module mechanisms. Most languages offer all kinds of conveniences on top of these basic aspects. But, once you understand the model presented here, you can work out the additional costs and benefits of them on your own.

1.2 Costs and Benefits

A system built from modules provides two major advantages to developers. The key is that each module isolates a developer's work from the work of others. In fact, reflecting that modules enable the decomposition of software construction, modern IDEs validate modules one by one without requiring a complete system and as the developers write code. Developers do have to list the dependencies of a module (so that the IDE validates those first), but beyond that, developers can focus on creating one module at a time. Similarly, compilers proceed on a module-by-module basis, and compile each module separately from others. As a result, modules act as the unit of stand-alone functionality for a code base. Just like developers construct code module by module, they can also experiment with and test it in the same manner.

To receive these benefits, developers must pay the costs for splitting a program into modules. The base cost is the notational overhead due to the lines of code that state the module-level dependencies. Fortunately, modern IDEs can assist developers with this task. The core cost though is that developers have to understand how to create valid modules, which can steepen the learning curve of a language. Concretely, developers must continue to respect all the existing validity constraints for the underlying language without modules and must learn what it means to formulate modules properly on top of that.

The picture differs for the language creator. It starts with the idea that modules are just textual containers that isolate one bunch of classes from others. As such, they are syntax without any meaning. Or, as implied so far, a system of modules imposes organization on an executable class-based program. To ensure that this program is well-formed and valid, however, the language creators have to work out appropriate validity constraints. Importantly, to unlock the benefits for developers, creators must make sure that the constraints for a system as a whole respect the decomposition of the system in individual, separate modules.

While these validity constraints aren't complex, the language implementer has to add extra enforcement code to the existing validity checker that mirrors the module-level validity constraints. Therefore, the imple-

menter should (re)structure and expand the checker for the underlying language of classes so that it validates each module separately. When the checker discovers violations of the constraints, it must put together a reasonable amount of information so that the IDE can help the developer with module-level reports.

Finally, the language implementer must supply a *linker*, a pass that unwraps the modules and creates a plain class-based program. Unwrapping means class definitions are directly exposed to each other. Since the introduction of modules is partly about freeing team members from having to adhere to naming protocols for classes, a naive unwrapping might cause undesirable name clashes. One way to eliminate this problem is to shape the module level so that such clashes cannot occur. If this is possible, the linker always succeeds in creating a well-formed and valid class-based program, meaning the tool itself does not impose any intellectual cost on the working software developer.

This arrangement also has a benefit for the language implementer. The linker is going to create a class-based program that can run on the existing CESK machine. In other words, the module system does not require any changes to the rather complicated and subtle machinery that assigns meaning to *Classy* programs and thus systems.

2 Module: the Syntax

Like all of our models, *Module* comes with the bare minimum of features:

- A module definition introduces a name and contains a *single class* definition, which is also visible to the outside—i.e. is *exported*.
- Each module may refer to classes from modules that precede it in the sequence of modules, that is, *import* existing modules.
- A *Module system* consists of a sequence of module definitions, followed by a *Core* program with references to some imported modules.

As usual, the model is small and omits many features of real-world languages that support modules. The goal is to understand the basics of how modules manage names and how they impose structure on a *Classy* program to enable separate code development.

2.1 *Module*: the Grammar

Figure 44 presents the BNF grammar for the syntax of *Module*. The first production shows how *Module* systems are created. They consist of a potentially empty sequence of modules, a potentially empty sequence of import specifications, followed by a *Core* program—referred to as the *system's body* in this chapter.

```

System      ::= (Module*
                  Import*
                  Declaration*
                  Statement*
                  Expression)

Module      ::= (module ModuleName Import* Class)

Import      ::= (import ModuleName)

The set of ModuleNames is the same as the set of Variables.
```

Figure 44: The grammar of the *Module* language

The second production introduces the rather simple module concept. A module has a name and contains (1) a sequence of import specifications and (2) a single class definition. The third and last production introduces import specifications. Each import names a single module; its purpose is to make the respective class definition available in the scope of the module.

Intuitively a module is a container that protects the class definition from name interference. Only imported *ClassNames* are visible, and exactly one *ClassName* is exported. Otherwise the code inside of a module must obey the grammatical rules of *Classy*.

A system consisting of modules also directly corresponds to a *Classy* program if we think of module names as prefixes for class names. Consider the sketch of a *Module* system on the left:

a <i>Module</i> system	the corresponding <i>Classy</i> program
(module M (class C ...))	(class M.C ...)
...	...
(module K (class C ...))	(class K.C ...)
...	...
(module L	
(import K)	
(class D ... C ...))	(class L.D ... K.C ...))
...	...
(import M)	
... C ... C ...	... M.C ... M.C ...

Module M as well as K contain a class definition named C. But, as the *Classy* program on the right shows, if we loosen the rules for formulating class names—by allowing the dot between letters—the translation is straightforward and resolves any doubts about which C is meant in which context. In short, our model of a module-based programming language satisfies all four elements of the description of *Module* spelled out in the introduction of this chapter, though as usual, in a highly simplified manner.

Java uses the \$ char instead of the dot for generated names. Many other languages come with similar conventions.

Exercise 50. Design an AST data representation for *Module*. Implement a parser for *Module* that maps an S-expression to an instance of AST. Re-use your solutions from exercises 14 and 20 as much as possible.

2.2 Module: Scope

The syntax of *Module* introduces module names and two different constructs that associate those with lexical regions: modules and imports. Surprisingly, even though both new constructs create similar-looking name-binding scenarios, their natural scoping rules differ significantly.

A look at the *Module* system production might suggest that sequences of modules should be treated like sequences of variable declarations. Concretely, if two modules have the same name, the second one in the sequence shadows (in the sense of chapter IV) the first one. But, this choice would violate the design rationale of *Module* in two ways:

- First, each module declares its dependencies on other modules. Since these dependencies impose at most a partial order on the set of modules, any linear arrangement respecting the partial order should make up a valid system. Hence the association between modules and lexical regions should not depend on their ordering, thus giving some freedom to whoever integrates the independently produced modules into a system.

- Second, a shadowing convention would invalidate a developer’s direct understanding of the modules as mere wrappers “name protecting” wrappers. Consider this sketch of a module sequence and its corresponding program:

<i>A Module system fragment</i>	<i>... and its “unwrapping”</i>
---------------------------------	---------------------------------

<pre>(module M (class C ...)) ... (module M (class C ...)) ...</pre>	<pre>(class M.C ...) ... (class M.C ...) ...</pre>
--	--

A straightforward unwrapping of this system yields an invalid *Classy* program. Conversely, if we wanted to go for this form of scoping, it would significantly complicate how developers comprehend modules.

Taken together, the two points clarify that a module declaration should associate the `ModuleName` with the text of the entire system.

By contrast, sequences of import specifications make a module’s export available *inside* of a module or for a system’s body. As such, an individual import binds the exported `ClassName` of its module in the module. The question is what happens when two distinct import in a sequence import the same `ClassName`. Here it makes sense to say that when two imports inject the same `ClassName` into a lexical region, the second name shadows the first one (in the sense of chapter IV. Expressed in terms of lexical scope, an import specification binds the corresponding `ClassName` in the remainder of the import sequence plus the module’s class definition or the system’s body. Of course, if the module defines a class whose name clashes with an imported one, the latter becomes inaccessible within the text of the class.

Exercise 51. Draw the lexical region for each binding occurrence of a name in the following program:

```
(module M (class C ... ))
...
(module K
  (import M)
  (class C ... C ... ))
...

(import K)
(import M)

... C ... C ...
```

The last point to clarify is how *ModuleNames*, *ClassNames*, and variables co-exist in *Module*. In *Classy*, a *ClassNames* is simultaneously somewhat like a variable and somewhat special. Consider this simple *Classy* program:

```
((class C ()
  (method m(C)
    (new C (C))))
 (def c (new C ()))
 (c --> m(c)))
```

Here *C* is the name of the only class, and it is the name of method *m*'s only parameter. Both uses are correct. Recall that the scoping rules of *Classy* demand only that for every expression such as *(new C ...)*, *C* is a defined class. Similarly, the rules of *Classy* require that a name used as an argument list of a constructor expression, say *(new ... (C))* refer to a defined variable. Since both of these constraints are satisfied in this example, the CESK machine for *Classy* can evaluate this program without any problems.

What this admittedly odd example shows is that the *Classy* model implicitly introduced Java-style namespaces: one for *ClassNames* and one for ordinary variables. Indeed, arguably the names of fields and methods form a third namespace, though in terms of scope this space is irrelevant.

Our *Module* model adds yet another namespace, namely, the names of modules. Fortunately, all these spaces are easy to keep separate because there are only a few places where most names may appear:

- *ModuleNames* set up a region when they are used in module definitions. They show up only in import specifications.
- *ClassNames* are used to name classes. References to these names show up in just two expressions: *new* and *isa*.
- The role of *FieldNames* and *MethodNames* is similar to the role for *ClassNames*. Unlike the latter, the names for fields and methods comes without any limitations on their scope.
- All other names, denote ordinary program variables.

The existence of separate spaces of names is a convenience, not a necessity. Our model would work equally well if all names existed in just one space.

Exercise 52. Draw the lexical region for each binding occurrence

```
((module A
  (class A (A)
    (method A (A)
      A)))

(import A)
1.0 )
```

Exercise 53. One aspect of managing names in *Module* is to allow developers to use the *same* name for different classes in distinct modules. Unlike in *Module*, though, existing programming languages allow programmers to import and *use* a class into a module even if it has the same name as one of the internally defined classes. They support this name resolution via a notation that distinguishes the imported class from the internal one. Design such a notation for *Module*.

2.3 *Module*: Validity

A validity checker enforces validity rules, which means scoping rules plus constraints on the choice of names. The point of *Module*, however, is that the validity checker enforces the rules on a module by module basis. Since module-by-module validity checking is *the* design rationale of our model, this section consists of two parts: the first one makes the *Module*-specific validity rules explicit, and the second explains how to turn these rules into a *Module*-appropriate validity checker.

Module: Validity Rules

One purpose of validity rules is to express scoping rules as explicit constraints. For the *Module* model, formulating those is straightforward based on the preceding subsection:

- Each module must be closed.
- The system's body must be closed.

Recall that “closed” means all name occurrences point back to a specific declaration or a method's parameter. Furthermore, all *ClassNames* refer to the module-defined class or an imported one.

The *Module*-specific scoping rules also imply naming constraints for the sequence of modules in a system and for all sequences of import specifications anywhere in the system:

- *No two modules in a system may have the same `ModuleName`.*
- *Every import specification must refer to a `ModuleName` that is already defined. This includes the import specifications in the body of the system.*

This collection of four validity rules represents everything needed for the new constructs introduced by the *Module* mode; for the constructs inherited from *Classy*, the pre-existing rules apply.

Module: Validity Checking

The translation of the validity rules into a module-by-module checker calls for a thorough understanding of how exporting modules and importing ones are related. In the world of actual languages, a module comes with an *interface*, which lists its exports. For our simple *Module* model, explicit interfaces aren't needed, because each module exports exactly one class name. That is, a *Module* interface is just an association of a `ModuleName` with a `ClassName`.

Every lexical region—a module or the system body—explicitly states which modules are imported. Each module import injects a `ClassName` into the scope. Hence, a checker can analyze a scope only if we provide it with a mapping from imported module names to their class names.

Exercise 54. An *export map* is an association of `ModuleNames` with `ClassNames`. Design a data representation for export maps in your chosen programming language. Your second task is to implement the functionality that consumes a sequence of well-formed modules and computes its export map.

Exercise 55. Design and implement a validity checker that enforces the validity rules for an individual module. It consumes two pieces of data:

- a well-formed module AST (see exercise 50);
- an export map, which represents the exports of the modules that pre-exist relative to the given one (see exercise 54).

The checker validates (1) that the module's class is closed and (2) that its imports refer only to already-existing modules. A simple-minded variant merely reports success (true) or failure (false). By contrast, an ambitious one is going to return an AST with all erroneous nodes marked up appropriately.

Exercise 56. Design and implement a validity checker that validates a the body of a *Module* system separately. The checker validates that the body

Chapter VIII adds types to the model, and in that context, a richer interface associates the `ModuleName` with a `ClassName` and the type of the class.

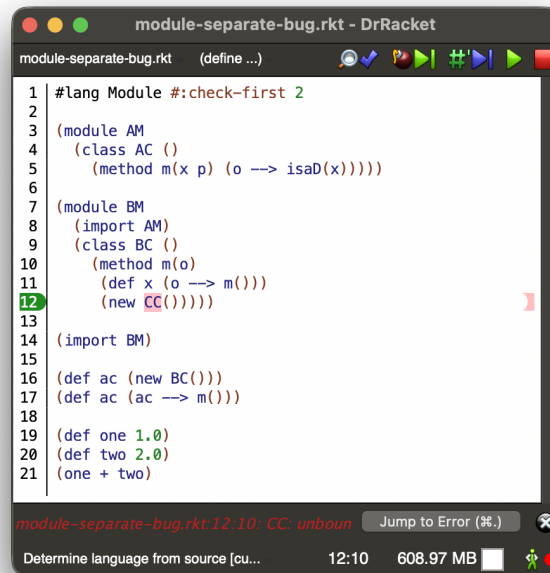


Figure 45: Separate validity checking with *Module* modules

is closed and that its imports refer only to already-existing modules. That is, it functions just like the checker of exercise 55.

Given a *Module* system, the checker of exercises 55 and 56 can check its pieces—the modules and the system body—in a random order. For example, it could be used to analyze the second module of a system separately. Figure 45 displays how an IDE might deal with such a scenario. The checker inside of DrRacket highlights only the undeclared class `CC` in the second module, even though the first one comes with an undeclared `o`, too, and ordinarily it would highlight this one first.

Exercise 57. Design and implement a validity checker that enforces the validity rules for complete *Module* systems. The checker consumes well-formed ASTs from exercise 50; if it finds errors it annotates the AST appropriately. Reuse the solutions of exercises 54 through 56, and don't forget to check that the given system satisfies the “no two identical module names” validity rule.

3 *Module: Linking*

The executable models of the preceding chapters fall into two categories: those related to syntax and those related to semantics. A parser or a validity checker belong to the first kind. A load function or a runMachine function belong to the second kind.

By contrast, a linker is neither a syntax-related tool nor a semantics-related one. It relates ASTs for one language, *Module*, to ASTs of a second one, *Classy*. The point of such a translation is to explain the meaning of the first kind of AST via the second kind. From the perspective of a language creator or a model maintainer, the linker avoids the need to revise the intricate workings of the CESK machine for *Classy*.

The functional purpose of a *linker* is to translate a well-formed and valid *Module* system into a well-formed and valid *Classy* program. As indicated already, this translation is accomplished via a two-step transformation:

- First, the linker replaces *all* occurrences of *ClassName* with their fully-qualified names:
 - If class *C* is defined inside of module *M*, then its fully-qualified name is *M.C*.
As mentioned in the preceding section, this name is illegal in the surface syntax, but since the linker operates on ASTs, we can accept this loosening of constraints on acceptable names.
 - The replacement of the defining *ClassName* is easy.
 - The replacement of any other *ClassName* occurrence must add the name of its exporting module, which is visible via an import specification.
- Second, the linker strips the module and import forms from the AST.

The resulting *linked* program is well-formed and valid because each module contains a single class, meaning any two class definitions introduce two distinct names.

Figure 46 displays a variant of the *Module* system from section 2.1. They differ in two regards: module *K* imports *M*, and the body of the system imports both *K* and *M* in that order. The snippet illustrates a couple of key properties of the linking process. First, every definitional name of a class is prefixed in the process. Second, if a module such as *K* imports a class that has the same name as the internally defined one, all occurrences of

a <i>Module</i> system	the corresponding <i>Classy</i> program
(module M (class C ...))	(class M.C ...)
...	...
(module K	
(import M)	
(class C ... C ...))	(class K.C ... K.C ...)
...	...
(module L	
(import M)	
(class D ... C ... D))	(class L.D ... M.C ... L.D))
(import K)	
(import M)	
... C ... C ...	... M.C ... M.C ...

Figure 46: Linking a *Module* system into a *Classy* program

this *ClassName* point back to the local one in the resulting program. By contrast, if the imported class has a different name, as is the case for module L, the imported class is visible via its fully qualified name. Finally, the system's body resolves class names according to the scoping rules concerning sequences of import specifications.

3.1 Project Tasks

Exercise 58. Design and implement a linker for *Module*. The linker should map an AST of a *Module* system to a *Classy* program.

Exercise 59. Adapt the main function from exercise 40 so that it can run the entire process of parsing a *Module* system, validating it, linking it into an object-oriented one, and determining its meaning on the CESK machine for *Classy*. Use your solutions from exercises 50 through 58.

The output should be an error message if the program is ill-formed, invalid, or causes a constraint violation when run on the CESK machine:

- "syntax error" if the parser discovers any mistakes;
- "duplicate module name" if the validity checker finds two modules with the same name;
- "import of non-existing module" if the validity checker discovers an import specification that does not refer to an already-existing module;

- “duplicate name error” if the validity checker encounters two classes with the same name, a class with two identical method names, or a class with two identical field names;
- “undeclared name error” if the validity checker encounters an undeclared `ClassName` or variable name; or
- “runtime error” if the CESK transition function stops due to a final state with an error in its C register.

If the final CESK state contains an object or a number as final control string, `main` prints “object” or the number, respectively.

4 Pragmatics: Computability vs Expressive Power

In the 1930s, Church and Turing settled on the hypothesis that all algorithmic languages can compute the exact same set of functions on the natural numbers, namely, the set of partial recursive functions. People such as Post, Smullyan, and some others tried to develop alternative computing systems, but all of them turned out to be equivalent to Church’s lambda calculus and Turing’s machines. Equivalent means that every algorithm in a computing system can be translated into an expression in an alternative one such that both compute the same function. From this perspective, all programming languages are the same.

Programmers know better, however. They instinctively argue that some languages are “better” for articulating some kinds of thoughts than others. The simplest reason might be that a particular platform (hardware, browser) does not support some language, so another one must be used to implement an algorithm. But, when a platform supports a range of languages, programmers may wonder which one lets them express an idea as “naturally” as possible.

This chapter offers a natural example to ask the question in a scientific manner, because the two models—*Module* and *Classy*—differ in just one way: the first has modules, the second doesn’t. Hence the question is whether one can express “more” computations than the other. If so, we have a first idea of how to judge languages systematically.

Stop! Write a *Classy* program that *Module* cannot directly express.

You may wonder what “directly” means in this context. Given our naive understanding of *Module* systems as *Classy* programs and given the

linker, which translate the former into the latter, the interpretation of “directly” should be straightforward: wrap each class definition in a module and add import specifications as needed.

Stop! Now that “directly” is understood, try again.

Here is one such program:

```
(class C () (method m (x) (x isa D)))
(class D () (method m (x) (x isa C)))
(def c (new C()))
(def d (new D()))
(c --> m(d))
```

Due to global scoping of class names, *C* and *D* can refer to each other in a valid manner. However, if we were to directly embed these classes into modules, the result would be a well-formed but invalid system:

```
(module M (class C () (method m (x) (x isa D))))
(module K (import M) (class D () (method m (x) (x isa C))))
(import M)
(import K)
(def c (new C()))
(def d (new D()))
(c --> m(d))
```

Since modules may import just those modules that precede them, it is impossible to resolve the (boxed) name *D* in *M*. Generally speaking, setting up mutually recursive classes is impossible in *Module*’s syntax.

All programming languages come with such weaknesses, and when a language exhibits a weakness, developers create workarounds. In this case, the workaround consists of the definition of a third module that comes with the desired functionality:

```
(module K (class C () (method m (x o) (o --> isaD(x)))))
(module L (class D () (method m (x o) (o --> isaC(x)))))



(module M
  (import K)
  (import L)
  (class CD ()
    (method isaC (x) (x isa C))
    (method isaD (x) (x isa D))))



(import K)
(import L)
(import M)

(def c (new C()))
(def d (new D()))
(def cd (new CD())) ;; <--- an instance of CD
(c --> m(d o))      ;; <--- m is modified to accept cd
```

Once the system is equipped with this synthetic module and class, an instance of the latter can be created and passed along to method calls on instances of the original classes, C and D.

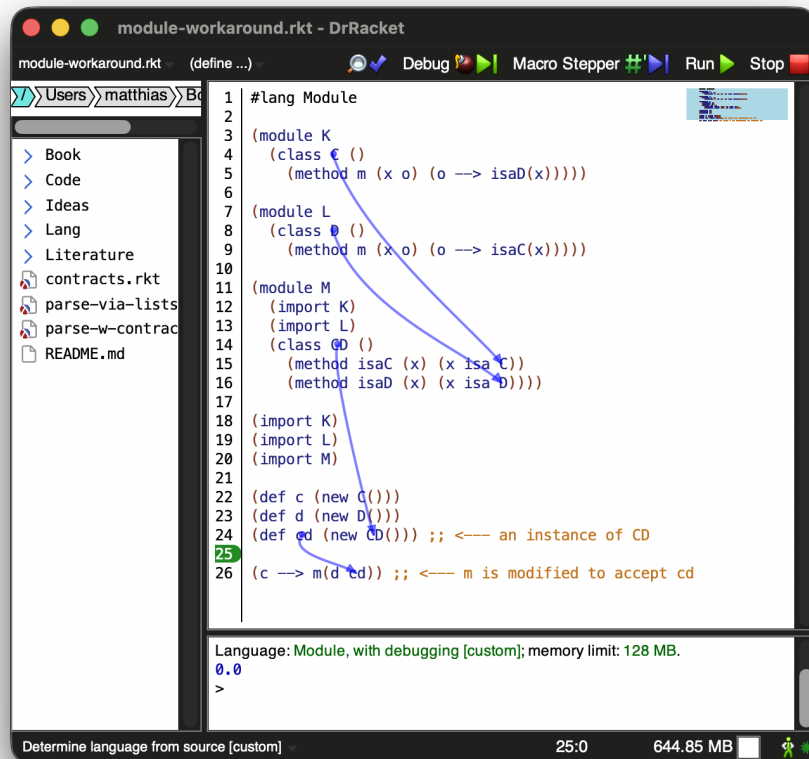


Figure 47: The indirections needed for the *Module* workaround

A second look reveals the obvious problem with such workarounds. Our way of overcoming the weakness of *Module* is to impose a usage protocol on the entire system:

- Someone must create a synthetic module that provides the needed functionality.
- The system's body must create an instance.

- Worst of all, the other team members must modify the method definitions and method calls so that, when needed, they accept and use instances of the synthetic module.

In short, the weakness of a programming language affects developers when they need a certain piece of functionality, and workarounds may impose a significant system-wide usage protocol with a non-trivial number of levels of indirections. See figure 47 for the latter.

The idea of comparing languages in this manner is due to Felleisen. It provides a well-founded argument for the rejection of the Church-Turing hypothesis as a relevant idea in the arena of software development. When Church and Turing claim that any two full-fledged languages are equivalent, they rely on translations that map code in one language to the other and vice versa. For their purposes, any translation from the *entire* space of functions between these languages is a good one. By contrast, Felleisen *restricts* the space of functions to those that require just local changes. This restriction corresponds to the idea that one team member cannot force others to change their code just because some system-wide protocol would fix a programming-language weakness.

For the purpose of understanding how programming languages support developers in certain work situations, Felleisen's notion of expressive power sets up a theoretical approach for the evaluation of the pragmatics information of a language construct. Sadly, it is a limited tool and further progress is needed to broaden this theoretical approach to pragmatics.

Why do languages include a type notation and a type checker?

1 The Pragmatics Question

While a 1970s programming language such as Smalltalk solves the problem of data representation and functional abstraction, it injects new problems into the work of software developers. As a Smalltalk program executes it may try to send a message to an object that does not come with a corresponding method. The language implementation's response is an error signal telling the user "method not found." Similarly, a program may try to retrieve the value of a non-existent field or modify such a field.

In the models of the preceding two chapters, these new problems manifest themselves as several novel failure cases in the CESK transition function:

- object creation, due to a mismatch between the number of arguments and fields;
- method call expressions, due to a mismatch between the number of arguments and parameters;
- method call expressions, due to a lack of the named method;
- field access and modification, due to a lack of the named field; and
- the addition and division operations.

This last failure case is due to the extension of the set of values with objects. In addition to the (floating point) numbers of the *Core* model, the value set of *Classy* consists of the union of numbers and objects. Since the model does not constrain what a variable can stand for, it is therefore possible that expressions such as $(x + y)$ or (x / y) have to deal with objects as arguments, which makes no sense.

```

(module Polar
  (class Polar
    (; the angle in degrees,
    ; counter-clockwise
    x
    ; the distance from (0,0)
    ; on the angled line
    y)
    ;; rotate 'this' point by ' $\delta$ '
    (method rotate ( $\delta$ )
      (def myX (this --> x))
      (def myY (this --> y))
      (def angle ( $\delta$  + angle))
      (new Polar (myX angle))))

(module Cartesian
  (class Cartesian
    (; the distance (going right)
    ; on the x axis from (0,0)
    x
    ; the distance (going up)
    ; on the y axis from (0,0)
    y)
    ; translate 'this' point by
    ; 'other' point interpreted
    ; as a vector
    (method translate (other)
      (def myX (this --> x))
      (def myY (this --> y))
      (def otherX (other --> x))
      (def otherY (other --> y))
      (def X (myX + otherX))
      (def Y (myY + otherY))
      (new Cartesian (X Y))))

(import Polar)
(import Cartesian)

(def one 1.4) ;; (sqrt 2)
(def half 45.0)
(def p (new Polar (half one)))
(def two 2.0)
(def c (new Cartesian (two two)))
(def d (c --> translate(p))
(def dx (d --> x))
(def dy (d --> y))

(dx + dy)

```

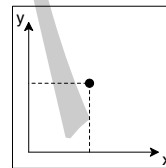
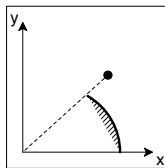


Figure 48: Logical mistakes due to a lack of an isa check

Run-time errors pose problems to software developers, and they can cause havoc after deployment. Imagine software for assisting the airplane pilots. When such a system displays “method not found” or “addition expects numbers, given an object” and goes down during, say take-off, the consequences could be catastrophic. Clearly, end users of software should experience as few run-time errors as possible.

The situation is worse than it appears: a system may terminate normally and output a number, without any indication that anything went wrong. For an example, consider figure 48. It displays a system in *Module* that consists of two independent modules (top) and a main program that relies on both (bottom). As the data interpretation comments in the module on the left explain, the module supplies a class for representing a point's coordinates in polar form; by contrast, the module on the right exports a class for representing point's coordinates in Cartesian form. Between the two rows are two diagrams that illustrate the data interpretation comments.

Let's take a look at the main program. It imports both modules, and it constructs an instance of each of the two classes:

- The first instance, named *p*, comes with polar properties 1.4 distance units and 45 degrees.
- The second instance, *c*, sits at (2,2), just as its coordinates say.

In terms of the Cartesian grid, *p* is near (1,1), because the root of $1^2 + 1^2$ is close to 1.4 and 45 splits a right angle in half. Interpreted as a vector, *p* points one step up and one step to the right.

Given these basics, the method call (*c* -> *translate*(*p*)) appears to have the goal of creating a point that is one step to the right and one step up of (2,2). So, we should expect that the new instance of *Cartesian* has the coordinates (3,3) and therefore outputs 6. But, when this program is linked and run on the CESK machine for *Classy*, it yields a rather strange result, namely, 50.4.

Stop! Explain why running the program in figure 48 yields 50.4.

A close look reveals the problem. The *translate* method expects to receive an instance of *Cartesian*, but the method call supplies an instance of *Polar*. In this admittedly silly example, the method call doesn't fail because other does have *x* and *y* fields. Hence, the retrievals of the respective field values succeed, the rest of the method computes essentially meaningless numbers, and the final expression creates an equally meaningless instance of *Cartesian*.

The question of pragmatics is that the language creator has shifted the burden of enforcing this design discipline to the many users of this language. Every software developer using a language like *Classy* must use checks such as (*other* isa *Cartesian*) at the entry to methods such as *translate*. If this check is omitted, the call may produce numbers and objects without any meaningful interpretation.

Arguably this kind of problem is worse than the run-time errors discussed first. When a program signals a run-time error, the developer gets some information about the program's problems via the error message. If

the language implementation is constructed to issue informative messages, the error signal cuts down the search space for the bug. By contrast, when a program outputs strange numbers, the developer might not find out until it is too late.

Deploying such code in real-world settings may cause serious problems. Consider the software for a pacemaker. If the software crashes due to a run-time error, the patient may die but a diagnosis may reveal that the software system issued an error message and trigger further investigations. If the software causes the instrument to put 500V instead of 5V on the “wire” neither a doctor nor a developer may suspect the software.

Language creators can accept the responsibility for the prevention of these run-time errors and logical mistakes. The key is to enrich the language with a form of validity checking that classifies how statements and expressions behave before they are run and that informs the developer of potential problems. In turn, the language creators may impose a different burden on the developer, namely, the task of massaging the code so that it satisfies these added validity checks.

1.1 Design Choices

By the 1990s, programming language creators had recognized that a static type system not only helped compiler writers to generate fast code but that a type system also helped software developers at almost all stages of the engineering process. Roughly speaking, a type system adds a notation to the language that enables programmers to state claims about statements and expressions. Thus, a piece of code may declare that a variable has a numeric type and therefore always represents a number in its lexical scope. To make sure that such claims are valid, a type system comes with a type checker. It is the task of the type checker to ensure that the types and the code are in sync throughout. For example, if a program states that some variable always stands for numbers but also contains an assignment that sets this variable to an object, the type checker must reject this program.

For the languages that *Classy* and *Module* model, creators developed three different approaches to type system design and implementation:

- nominal class types;

C++, Java, and similar languages identify the name of a class as the type of its instances. Thus, if two classes have identical fields and identical methods, their objects are still of a distinct type.

- structural class types;

Academic language research tend to favor the alternative idea that if two objects have the same fields (by name and type) and the same methods (by name and signature), they have the same type. That is, the structure of the class definition gives rise to its type, not its name.

- module-global type inference.

In the 1970s, researchers took structural types one step further. They noticed that a type checker could infer some type annotations if the programmer omitted them:

- A missing type could be replaced with a variable ranging over the set of types.
- The statements and expressions gave rise to equations over these type variables.
- Solving these equations could be done via the novel unification algorithm, a variant of Gaussian elimination.

Like for any system of equations, there are three possible outcomes:

- If there is *one* solution, the type variables can be replaced with plain types, and the compiler can generate conventional code.
- If there are *many* solutions, some type variables remain visible in the solution. Types containing such variables are called parametric polymorphic types.

Note The presence of type variables abstracts over pieces of the same statements and expressions with different types. As such, parametric polymorphism is occasionally desirable because it enables additional forms of reuse. To the implementer, it can be a problem, because it prohibits a straightforward choices of data representation and code generation.

- If there is *no* solution, type checking fails and the type checker tries to compose an type-error message. To this day, though, these systems tend to report such errors in terms that are often difficult to decipher for a software developer.

By the early 1990s, Luca Cardelli, a leader of the types research community, began to argue that when types are not written down, several software engineering steps suffer, starting with the systematic design

See "Typeful Programming," a DEC SRC research report (1989, 1993).

of data representations and their functionality all the way to documentation. Many others began to embrace this point of view and, as a result, *module-level* type inference experienced a decline in popularity. Modern languages use *local* type inference for individual expressions and statements instead.

The remainder of this chapter focuses mostly on the second idea, in preparation of the next chapter. It ignores module-level type and explains local type inference as an aside.

No matter which approach is chosen, the creator of a type system must confront the question whether to ensure that the type checker is supposed to be sound in the spirit of section 4.1 in chapter IV. While checking whether a variable in an expression is declared is essentially trivial, checking types is a rather complex task. Indeed, in some cases it is so complex that some language creators opt to “do their best” but make no effort to state or prove a theorem about the relationship between the type checker and the language’s semantics.

If the language creators aim for a mathematical soundness theorem, they tend to set up a formal *model* like those of this book, state the theorem for the validity checking and semantic functions of the model, and prove a relationship between the two with logical means. Critically, they do not work with the language implementation, which tends to support many more linguistic constructs than a model. As a result, even if the goal is to deliver a typed language that satisfies a type soundness theorem, the implementation may come with bugs that violate the theorem.

As for their connection to pragmatics, type soundness plays a role but perhaps not the critical role that language theoreticians ascribe to this theorem. Hence, this chapter presents the relationship between pragmatics and languages with both sound as well as unsound type systems. Readers interested in type soundness may wish to check out traditional text books on programming languages.

1.2 Costs and Benefits

Although the dominance of C++ and Java in the marketplace seemed to have settled the argument in favor nominal types, the emergence of TypeScript and the addition of similar type systems to other *Classy*-like programming languages resurrected the structural-class type idea. Both come with their own costs and benefits, and it is worth to compare those briefly.

From the perspective of a software developer and plain language user, the cost of using a type system comes in one form: a reduction in expressive power. A developer notices this loss in two ways: extensibility and safety/security. Any type system imposes restrictions on code. This statement is a tautology, because the very point of a validity checker is to eliminate code that *may* suffer from mistakes. Due to the undecidable nature of these problems, the emphasis is on “may” as in “a validity checker may reject code that, if type checking were disregarded, runs fine and computes the desired result.”

This decrease in expressive power affects developers in rather different ways, depending on whether they use a language with a nominal or a structural type system. Most importantly, programs in the latter remain extensible in ways that the former cannot accomplish. When one developer programs to a structural type, another developer can create a module with a structurally equivalent class that comes with a different name. The old code will work just fine with the new kinds of objects. By contrast, a software system in a nominally typed language forces the second developer to use the actual class to which the first one programmed; if this class isn’t accessible, the software system is not extensible.

If the type system also supports sub-typing, structural types make functionality even more reusable than nominal typing. As long as an object has the required fields and methods that some method demands, everything works out, even if the object has additional fields and methods.

On the flip side of extensibility, software developers are concerned with safety and security. With a nominal type system, a developer can specify that only instances of one particular class are welcome inside some piece of functionality. Given some knowledge about the workings of this class’s implementation, the developer knows that calling an instance’s method cannot do any harm. Contrast this situation with a structurally typed language. Just because the type signatures of two methods agree does not mean their behavior agrees. While one may compute the desired result, another one may raise an exception, violate a security policy, or perform any other undesirable effect. A developer can know how a method behaves only if its code resides in a particular known class.

This last concern, safety and security, shows that restrictions on the expressive power come with benefits in terms in reasoning about code. In this concrete context, while a type system restricts the space of well-formed and valid code, it empowers the developer to draw firm conclusion about the general behavior of code, not just about certain concrete runs. And this holds regardless of whether a language creator chooses a nominal or

a structural type system; a nominal type system just happens to be more restrictive than a structural one, and therefore it facilitates reasoning.

From the perspective of a language creator, the design and implementation of a typed language require significantly more work than that of an untyped language:

- The design of a type system involves a modicum of syntactic work, namely coming up with a notation for types.
- Next comes the development of type judgments and rules that establish such judgments. Theoreticians write down these rules with a notation borrowed from logic; language creators write them down in English so that language users can understand them and, critically, understand error messages from the type system.
- If the language creators aim for a soundness theorem about the type system, they need to make mathematical models of both the type system and the semantics, develop a relationship, and formulate an argument as to why this work scales to the full language. Reasonable language creators tend to leave this work to theoretical language researchers in academia.
- Finally, the validation pass must be implemented. Checking its compliance with the informal rules requires extensive testing work. Compliance means that the informal rules and the actual algorithms are in sync. And extensive testing is currently the only method for arguing compliance for a reasonably modern and large language. Furthermore, the implementation will inevitably come with errors that violate the specification. Hence the test suite must be carefully curated and maintained for this complex piece of code.

Choosing a nominal approach simplifies this work somewhat. Starting with the typing judgments and their rules, designing a nominal type system is simpler than designing a structural one. Just consider what it means for types to match: on one side, we have a name comparison and, on the other, a comparison of complex structures. Unsurprisingly, the translation of the respective rules into type checking code is more troublesome for structural systems when compared with nominal ones.

The final consideration concerns the semantics—the generated run-time code to be precise—of typed languages when compared to languages without type system. When code comes with explicit type specification for

program variables, a compiler can exploit this information for the generation of code, including the allocation of data structures, the move of data from one place to another, or the operations that manipulate the data. By contrast, a compiler for an untyped language must use approximation algorithms to infer comparable information about a given piece of code.

While this assistance for compiler writers motivated type systems in the early days of programming languages, the discovery of the soundness concept cast some doubt on the use of type information. If the validity checker for types is *unsound*, a compiler may generate run-time code that eventually moves bits from a string into an integer-typed variable or causes similar problems. As a result, the majority of language creators began to aim for type soundness, though to this day, some opt for designing a “usable” type system without consideration for soundness.

2 Types: the Syntax

Turning the *Module* model into a typed one mostly follows the design and implementation plan of the preceding section:

- The first step is to design a *notation* for types and to equip the existing grammar with type annotations.
- The second step concerns the meaning of “keeping code and types in sync.” Academic research has come to use *judgments* for this purpose, which provide a concise notation for expressing these relationships.
- Finally, the last step is to create a system of rules that explains how to judge, dubbed *judgment derivation rules*. Once type theoreticians noticed the relationship between the study of logic and the study of type systems, these rules have been stated as derivation rules also known as inference rules. Each rule states that some piece of code is in sync with its type and relative to its context if it is possible to make some other judgments. The former judgments are called *conclusions*, the latter *antecedents*.

The remaining subsections present these three steps, respectively.

2.1 Types: the Notation

Types classify the results and effects of expressions, statements, and so on. In the context of *Module*, there are two basic classifications:

```

Type      ::= REAL | Shape
Shape     ::= ((FieldType*) (MethodType*))
FieldType ::= (FieldName Type)
MethodType ::= (MethodName (Type*) Type)

```

Figure 49: *Types*: the Notation

- an expression may yield a number, or
- an expression may yield an instance of a class.

All other phrases merely play a role in creating one of these outcomes.

Accordingly a type notation must define a type as one of two possibilities: a literal symbol denoting numeric outcome (REAL) and a production describing an object, which we call a Shape. For the precise definition, see figure 49, which presents the BNF grammar of our type notation. Since an object consists of fields and methods, the Shapes that describe them consist of two parts: one describing the types of fields and another describing the types of methods. That is, a Shape describes the *structure* of an object.

Exercise 60. Explain what kind of objects the following Shapes describe:

```

( ((x REAL) (y REAL))
  ((distanceToOrigin () REAL)) )

```

Define two distinct classes whose instances intuitively have this type.

2.2 *Types*: the Grammar

The syntax of the *Types* model differs from *Module*'s in two ways. First, modules use the keyword `tmodule`, because we eventually wish to model a programming language that comes with typed and untyped pieces of code. Marking those with distinct keywords helps future readers. Second, modules in *Types* contain one more piece of code than those in *Module*: a Shape at the end, which specifies the type of the objects that the module's class can generate. See figure 50 for the BNF of *Types*.

Stop! Compare this BNF with the one of *Module* in the previous chapter.

You might wonder why this grammar does not come with typed variable declarations. In many languages, a developer would have to write something like `def x : int = 4` to declare an integer variable, but here, variable declarations come without types.

```

TypedSystem ::= (TypedModule*
                  Import*
                  Declaration*
                  Statement*
                  Expression)

TypedModule ::= (tmodule ModuleName Import* Class Shape)

```

Figure 50: *Types*: the Grammar

Following contemporary developments in programming languages, we instead have the type checker *compute* the missing type from the right-hand side of a variable declaration. In *Types*, computing this type is feasible and easy because the expression sub-language is quite simplistic. Language researchers refer to this process as *local type inference*. A fair number of contemporary programming languages incorporate local type inference for a number of cases.

Note Local type inference radically differs from the module-global inference mentioned in section 1.1. In our model, computing the type is always possible. In real-world programming languages, local inference rarely fails; when it does, generating an appropriate error message is straightforward, though repairing the expression so that inference succeeds rarely is.

Exercise 61. Design an AST data representation for *Types*. Implement a parser for *Types* that maps an S-expression to an instance of AST. Re-use your solution from exercise 50 as much as possible.

2.3 *Types*: Validity

The *Module* model comes with a number of validity rules. Adapting those rules to *Types* is mostly a matter of ignoring the types. However, these rules also induce a couple of natural constraints on types:

- Any two FieldNames in the Shape of a module must be distinct.
- Any two MethodNames in the Shape of a module must be distinct.

Clearly, these rules just correspond to the validity rules for a Class in the *Module* model. To simplify type checking, we impose a third rule:

The FieldNames in the class and those in the type of the class must show up in the same order.

It is easier to check this constraint than to type-check classes or new expressions with a loose ordering on field names. By contrast, it makes no difference whether methods and method types are supplied in the same order. Hence, to practice program design, consider examples that list methods and method types in distinct orders.

Exercise 62. Design and implement a validity checker that enforces the validity rules for *Types*. The checker consumes error-free ASTs from exercise 61; if it finds errors it annotates the AST appropriately.

3 Type Checking

A type checker is a validity checker, and it is the most sophisticated one. Language researchers have developed a distinct notation for specifying type checkers. This notation heavily leans on ideas and notations from the study of formal, also known as mathematical, logic.

Roughly speaking, a type checker *judges* each kind of phrase in a programming language. Since such phrases occur in syntactic contexts and since judging them usually needs information about this syntactic context, each judgment consists of at least two parts: information about the context and the piece of code. For some kinds of phrases, a judgment also includes a third part, namely an inferred kind of information.

Let's make this a bit more concrete in the context of *Types*. Consider the case of an expression, say $(y + z)$. Such an addition succeeds only if y and z are going to evaluate to a number; if either one of them evaluates to an object, the addition fails. Hence, the type checker needs a classification of variables, which originates from the context. Furthermore, if the expression successfully computes a result, it is going to be a number; the type checker can thus classify the expression as REAL. Here is the formal way of expressing this judgment, using $\vdash$ and $\Rightarrow$ as markers to separate the interesting pieces of information:

$$\{\text{type information about } y \text{ and } z\} \vdash (y + z) \Rightarrow \text{REAL}$$

Mathematics must be pronounced to be comprehended, and this “mathematical sentence” is best spelled out as “given type information about y and z from the program context of $(y + z)$, the type checker classifies the expression as REAL.”

Now consider an assignment statement, $(x = (y + z))$. In order to judge such a statement, the type checker needs to know the types of x , y , and z . The first one is needed to ensure a type-correct modification of x 's value; the last two are needed to check the addition expression. In contrast to expression checking, though, checking a statement does not produce any new type information. Hence the formal statement is simpler:

$$\{\text{type information about } x, y, \text{ and } z\} \vdash (x = (y + z))$$

Some judgments are plain wrong. Take $\emptyset \vdash (y + z) =_{\text{REAL}}$. This example states that without any information about the syntactic program context of the expression, the type checker should judge the expression as valid and classify it as `REAL`. Clearly, this specification would lead to an *unsound* validity check for types. So the question is how to come up with a system of *sound* judgments.

Like logicians, programming language researchers use so-called *judgment derivation rules* also known as *type inference rules*. We use the former term to avoid any confusion with local or module-global type inference:

In general, a derivation rule has the following shape:

$$\frac{\begin{array}{l} \text{some contextual information} \vdash \text{antecedent}_1 \text{ judgment} \\ \dots \\ \text{more contextual information} \vdash \text{antecedent}_n \text{ judgment} \end{array}}{\text{contextual information} \vdash \text{conclusion judgment}} \quad [\text{generic rule}]$$

The judgments above the horizontal line are called *antecedents*, the one below *conclusion*. If uses of the derivation rules can validate all of the antecedents, then the conclusion is validated, too.

An *axiom* is a derivation rule without antecedents. The first rule in figure 51 is one particular axiom. By convention, n is a numeric literal, and the type checker should classify it as such. The second rule in the same figure illustrates what a typical simple derivation rule looks like. Its antecedents extract from the contextual information that each of the variables has type `REAL`, and therefore the addition is going to produce a `REAL`, too.

3.1 Contextual Information and Conventions

Just like the specification of an abstract machine, the specification of a type checker relies on a fair number of notational conventions. The point of those is to present the specifications in a reasonably concise manner.

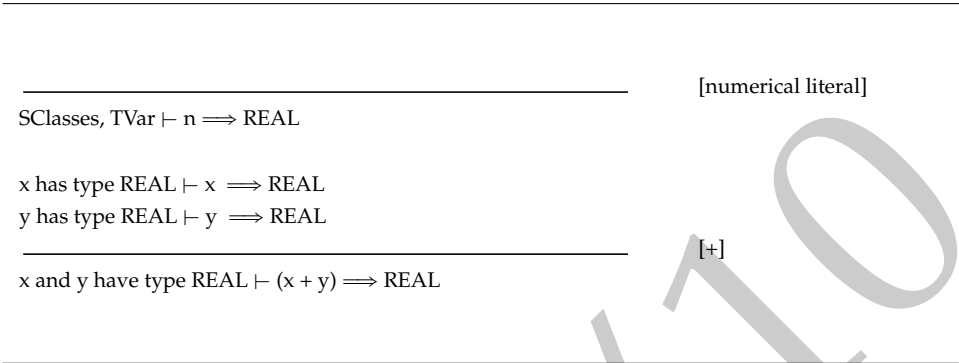


Figure 51: Two examples of judgment derivation rules

Figure 52 presents the conventions used here for both the statement of judgments and derivation rules. The first three name the most common pieces of contextual information: (1) sets of modules, (2) maps from ClassNames to Shapes, and (3) maps from variables to Types. The remaining ones are meta-variables for types and pieces of code. As you tackle this section’s project, keep figure 52 in mind and consult it as needed.

meta-variable	standing in for
Modules	a collection of modules
SClasses	Shapes of Classes, a mapping from ClassNames to Shapes
TVar	Types of Variables a mapping from Variables to Types
T	any type
S	Shape type
Tf	the type of a field
Tm	the type of a method
sys	a system
mod	a module
imp	an import specification
cls	a class
meth	a method

Figure 52: Type Conventions

3.2 Type Judgments

Figure 53 presents the collection of judgments that the type checker (of the *Types* model) makes. These judgments are grouped into several collections: (1) for systems-level pieces of code, (2) for class-level ones, (3) for the core of the language, which consists of declarations, statements, and expressions.

The entry point is the very first line, a judgment merely states that a complete system is valid. It relies on the three following judgments, which, in turn, validate (1) an individual module, (2) a sequence of import specifications, and (3) an individual import specification. The validation of a single module requires information the modules that precede it. A validation of import specification also relies on information about the existing modules, and it produces an *SClasses* mapping. The latter is part context and part result of an inspection of an individual import.

All other judgments follow this schema. They either just validate a piece of code, given some information about the syntactic context in which they appear. Or, they validate and produce additional information, which is often added to the context.

For an example, consider the judgment for validating a method. Its context consists of three pieces: (1) the mapping from *ClassNames* to *Shapes*; (2) the mapping from *Variables* to *Types*; and (3) the signature of the method. The last piece of information is needed so that the type checker can add the method's parameters and their respective types to the given *TVar* and compare the type of the return expression with the specified return type.

Stop! Jointly with a peer, pronounce the judgments in figure 53.

3.3 Rules for Deriving Type Judgments

Derivation rules make up the final part of the specification of a type system. As the preceding section explained, the derivation rules for *Types* come in two distinct flavors and there is a fair number of them.

Figure 54 displays the derivation rules for system-level pieces of code, excluding the rules for the system's body. The first one tells a programmer that a system is valid according to the type checker if $n+2$ antecedents hold:

- The n modules in the system are valid relative to the modules that precede each module. Naturally the first module has to be valid in an empty set of modules.
- Next, the sequence of imports setting for the system's body must be valid and produce an *SClasses* mapping.

read judgment as	the formal judgment
systems and modules	
a system is valid	$\vdash \text{sys}$
a module is valid	$\text{Modules} \vdash \text{mod}$
a sequence of imports produces an SClasses	$\text{Modules} \vdash \text{imp}_1 \dots \text{imp}_I \Rightarrow \text{SClasses}$
an import adds information to the given SClasses	$\text{Modules}, \text{SClasses} \vdash \text{imp} \Rightarrow \text{SClasses}$
classes and methods	
a class is valid	$\text{SClasses} \vdash \text{cls}$
a method is valid	$\text{SClasses}, \text{TVar}, ((T_d \dots) T_r) \vdash \text{meth}$
bodies of blocks, systems, and methods	
a program produces a type T	$\text{SClasses} \vdash \text{def}^* \text{stmt}^* \text{exp} \Rightarrow T$
a block is valid	$\text{SClasses} \vdash (\text{block} \text{def}^* \text{stmt}^*)$
declarations	
a sequence of declaration produces a TVar	$\text{SClasses}, \text{TVar} \vdash \text{def}_1 \dots \text{def}_d \Rightarrow \text{TVar}$
a declaration adds information to the given TVar	$\text{SClasses}, \text{TVar} \vdash (\text{def } x \text{ exp}) \Rightarrow \text{TVar}$
statements	
a sequence of statements is valid	$\text{SClasses}, \text{TVar} \vdash \text{stmt}_1 \dots \text{stmt}_n$
one statement is valid	$\text{SClasses}, \text{TVar} \vdash \text{stmt}$
expressions	
an expression produces a type T	$\text{SClasses}, \text{TVar} \vdash \text{exp} \Rightarrow T$

Figure 53: Type judgments

- And finally, the system's body must type-check, and its result type must be numeric.

This last antecedent illustrates how the development of such rules is somewhat whimsical. We could easily have allowed the system's body to return any type, but restricting it to REAL facilitates the comparison of implementations. Even real-world type systems incorporate such arbitrary decisions, though for different reasons than ours.

$\emptyset \vdash \text{mod}_1$ for all i from 2 to K : $\{\text{mod}_1, \dots, \text{mod}_{i-1}\} \vdash \text{mod}_i$ $\{\text{mod}_1, \dots, \text{mod}_K\} \vdash \text{imp}_1 \dots \text{imp}_I \implies \text{SClasses}$ $\text{SClasses}, \emptyset \vdash \text{def}^* \text{stmt}^* \text{exp} \implies \text{REAL}$	
	[system]
$\vdash (\text{mod}_1 \dots \text{mod}_K \text{ imp}_1 \dots \text{imp}_I \text{ def}^* \text{stmt}^* \text{exp})$	
$\text{Modules} \vdash \text{imp}_1 \dots \text{imp}_I \implies \text{SClasses}$ $\text{SClasses} [C : S] \vdash (\text{class } C (f \dots) \text{meth} \dots)$	
	[module]
$\text{Modules} \vdash (\text{tmodule } MN \text{ imp}_1 \dots \text{imp}_I (\text{class } C (f \dots) (\text{meth} \dots)) S)$	
$\text{SClasses}_0 = \emptyset$ for all i from 1 to I : $\text{Modules}, \text{SClasses}_{i-1} \vdash \text{imp}_i \implies \text{SClasses}_i$	
	[imports]
$\text{Modules} \vdash \text{imp}_1 \dots \text{imp}_I \implies \text{SClasses}_I$	
M is the name of mod in Modules C is the name of the class defined in mod S is the Shape of C in mod	
	[an import]
$\text{Modules}, \text{SClasses} \vdash (\text{import } M) \implies \text{SClasses} [C : S]$	

Figure 54: Typing rules for systems and modules

The second rule specifies how an individual module is validated. Its sequence of import specifications produce an *SClasses* mapping, which is then used to type-check the (one and only) class in the module. The scoping rules dictate that *SClasses* is then extended with an entry for *C*, the defined class. Doing so shadows any import of the same name, and it informs the type checker of the expected Shape of the *C*. It is a classic example of how syntactic context is incorporated into the contextual information to the left of the $\vdash$ symbol in a judgment.

Let's take the last two rules together. The third rule tells us that the type checker needs a function that turns a set of modules and a series of import specifications into a table from names of classes to their shapes. Starting with an empty table, the *I* antecedents gradually build up *SClasses*, one at a

time. The result of analyzing the last import specification, $SClasses_I$, is the result for the entire sequence of *imps*. Finally, the fourth rule specifies how the type checker must deal with a single import specification. Given the name of the module, it uses the given set of modules to retrieve the name of the class and its type: C and S , respectively. The pair is added to the given table, possibly replacing an existing entry for C , which again mirrors the scoping rules for imports.

$$\begin{array}{c}
 SClasses[C] = S \\
 S = (((f_1^T \text{ Tf}_1) \dots (f_g^T \text{ Tf}_g)) ((m_1^T \text{ Tm}_1) \dots (m_k^T \text{ Tm}_k))) \\
 f_1, \dots, f_g = f_1^T, \dots, f_g^T \\
 \{\text{NameOf}[\text{meth}_i], \dots, \text{NameOf}[\text{meth}_k]\} = \{m_1^T, \dots, m_k^T\} \\
 \text{for all } i \text{ from } 1 \text{ to } k: \\
 \quad SClasses, \emptyset [\text{this} : S], \text{Tm}_j \vdash \text{meth}_i, \text{ if } \text{NameOf}[\text{meth}_i] \text{ is } m_j \\
 \hline
 SClasses \vdash (\text{class } C (f_1 \dots f_g) \text{meth}_1 \dots \text{meth}_k) \quad [\text{class}] \\
 \\
 SClasses, TVar [y_1 : T^{a_1}], \dots, [y_n : T^{a_n}] \vdash \text{def}^* \text{stmt}^* \text{exp} \Rightarrow T_r \\
 \hline
 SClasses, TVar, ((T^{a_1} \dots T^{a_n}) T_r) \vdash (\text{method } m (y_1 \dots y_n) \text{def}^* \text{stmt}^* \text{exp}) \quad [\text{method}]
 \end{array}$$

Figure 55: Typing rules for classes and methods

Figure 55 presents the most complex derivation rule for *Types*. To check whether a class named C and its type are in sync, the type checker is given a class-shape table, which contains the type for the to-be-checked class. In this context, it proceeds as follows according to the antecedents:

1. It retrieves the Shape S of C from the given table.
2. As for fields, C and S must contain the same *sequence* of *FieldNames*.
3. As for methods, C and S must contain the same *set* of *MethodNames*.
4. Once these two judgments are satisfied, the type checker can deal with each method. Checking a method uses three pieces of context information: (a) $SClasses$; (b) a type-variable table, populated with an entry for *this*; and (c) the appropriate method signature from S .

Stop! Compare this explanation with the mathematical specification in the first rule of figure 55. Explain 4(b) to a peer.

The second derivation rule in figure 55 shows how to type-check methods. The type-variable table is extended with one entry per parameter with its matching type from the given method signature—assuming the two sequences are of the same length. Given these two tables, the type checker uses the [body] judgment from figure 56 to confirm that the type of the final expression equals the return type from the method signature.

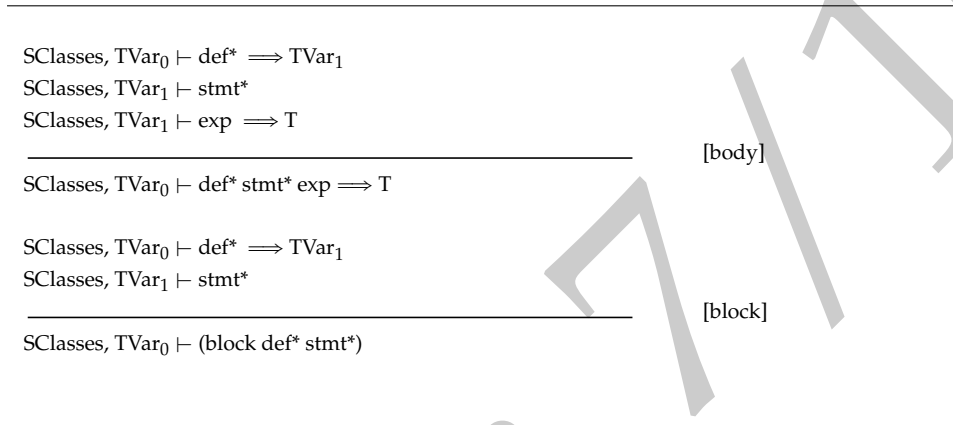


Figure 56: Typing rules for bodies of blocks, systems, and methods

A system's body has the same shape as a method's body; in comparison a block lacks the final expression. The derivation rules for these grammatical production resemble each other. As figure 56 shows, both require type-checking the sequence of definitions first, and doing so enriches the given type-variable table. Next up, this extended table is used to check the sequence of statements. In the case of system and method bodies, the type checker also type checks the final expression, and whatever type this check produces, also becomes the type of the complete body.

Type-checking a sequence of declarations demands two steps, too. First, the type checker needs to extend the given type-variable table with the types of the newly declared variables, one declaration at a time. The [declarations] rule in figure 57 specifies how this step works formally. Second, the type checker must ensure that the right-hand side expression has a type, which is then used as the type of the left-hand side variable. See the [one declaration] rule in the same figure.

Figure 58 displays the rules for sequences of statements and individual statements. Unsurprisingly, a sequence of statements is valid if each individual statement type checks. To get a sense of how individual statements

for i from 1 to d :	
$\text{SClasses}, \text{TVar}_{i-1} \vdash (\text{def } x_i \text{ exp}_i) \implies \text{TVar}_i$	
$\text{SClasses}, \text{TVar}_0 \vdash (\text{def } x_1 \text{ exp}_1) \dots (\text{def } x_d \text{ exp}_d) \implies \text{TVar}_d$	[declarations]
$\text{SClasses}, \text{TVar}_0 \vdash \text{exp} \implies T$	
$\text{SClasses}, \text{TVar}_0 \vdash (\text{def } x \text{ exp}) \implies \text{TVar}_0[x : T]$	[one declaration]

Figure 57: Typing rules for declarations

are type checked, take a look at the [assignment] rule. Its antecedent tells the implementer of the type checker that the expression on the right-hand side must type check and that its type must be the same as the type associated with the left-hand variable in the type-variable table.

Consider the derivation named [loop] next. The CESK machine runs the loop body if the value of the `tst` is 0.0; any other value causes the loop to terminate. In terms of types, the `tst` expression may have *any* type, and `bdy` must merely type-check as a statement.

Exercise 63. Languages such as Java demand that conditionals and looping constructs have Boolean-typed expressions in the analogous positions. Assuming *Types* came with a Boolean type plus `true` and `false` as literal constants of this type, how would you reformulate the [loop] and [conditional] rules in figure 58?

At this point, we have covered most derivation rules; the ones for expressions are missing. For the latter, it is best to proceed in two steps, starting with the expressions from *Core*, followed by those related to *Classy*.

Figure 59 covers the first set of rules: literal constants, variables, additions, divisions, and comparison. The first and the last rule are axioms, i.s. they validate that these expressions produce the type unconditionally. In both cases, the resulting type is `REAL`.

Exercise 64. Some typed programming languages do not implement unconditional comparison operators. Instead their type system ensures that both sides of `==` have the same type. Modify the [==] rule in figure 59 to model this kind of type checker.

for all i from 1 to n : $\text{SClasses}, \text{TVar} \vdash \text{stmt}_i$	
$\text{SClasses}, \text{TVar} \vdash \text{stmt}_1 \dots \text{stmt}_n$	[statements]
$\text{TVar}[x] = T; \text{SClasses}, \text{TVar} \vdash \text{rhs} \implies T$	
$\text{SClasses}, \text{TVar} \vdash x = \text{rhs}$	[assignment]
$\text{SClasses}, \text{TVar} \vdash \text{tst} \implies T$ $\text{SClasses}, \text{TVar} \vdash \text{thn}$ $\text{SClasses}, \text{TVar} \vdash \text{els}$	
$\text{SClasses}, \text{TVar} \vdash (\text{if0 } \text{tst } \text{thn } \text{els})$	[conditional]
$\text{SClasses}, \text{TVar} \vdash \text{tst} \implies T; \text{SClasses}, \text{TVar} \vdash \text{bdy}$	
$\text{SClasses}, \text{TVar} \vdash (\text{while0 } \text{tst } \text{bdy})$	[loop]
$\text{SClasses}, \text{TVar} \vdash \text{rhs} \implies T$ $\text{TVar}[o] = (((f_1^T \text{ Tf}_1) \dots (f_g^T \text{ Tf}_g)) (\text{Tm}_1^T \dots \text{Tm}_K^T))$ there exists an i such that $f = f_i^T$ and $\text{Tf}_i = T$	
$\text{SClasses}, \text{TVar} \vdash (o \rightarrow f = \text{rhs})$	[field mutation]

Figure 58: Typing rules for statements

The remaining three rules have straightforward explanations. Since the validity checkers ensure that all variable occurrences refer to some variable declaration, a variable always type checks and its type is the one from the given type-variable table. When it comes to addition and division, our *Types* model's semantics demands that both operands are numbers; the two derivation rules are set up accordingly.

Figure 60 covers the second set of derivation rules, those for expressions that deal with objects. A $(\text{new } C(\dots))$ expression is going to create an instance of C and therefore its type is the Shape of C —if validation succeeds. The antecedents of the [new] rule tell us that SClasses comes with a Shape type S for C , which in turn, specifies the number and the types of the class's fields.

A language such as Type Script would allow the addition of numbers and strings, and its type checking rules need to accommodate this semantics.

$\text{SClasses}, \text{TVar} \vdash n \implies \text{REAL}$	[numerical literal]
$\text{TVar}[x] = T$	
$\text{SClasses}, \text{TVar} \vdash x \implies T$	[variable]
$\text{TVar}[x] = \text{REAL}; \text{TVar}[y] = \text{REAL}$	
$\text{SClasses}, \text{TVar} \vdash (x + y) \implies \text{REAL}$	[+]
$\text{TVar}[x] = \text{REAL}; \text{TVar}[y] = \text{REAL}$	
$\text{SClasses}, \text{TVar} \vdash (x / y) \implies \text{REAL}$	[/]
$\text{SClasses}, \text{TVar} \vdash (x == y) \implies \text{REAL}$	[==]

Figure 59: Typing rules for arithmetic expressions

In order for `new` to succeed, the number of arguments must be the same as the number of fields, and according to the third antecedent, the types of the arguments to `new` must equal the types of the fields specified in `S`.

When it comes to the derivation rule for `isa` expressions, a language creator has to make a choice. Given `(o isa C)`, it is possible to determine the types of both `o` and `C`. Let's call them S_o and S_C , respectively. Based on this context, a type-system designer can choose from three alternatives:

1. to add $S_C = S_o$ to the antecedent;
2. to require that both are Shapes but not necessarily identical ones; or
3. to specify that S_C is a Shape without imposing any restrictions on `o`.

In the specific context of *Types*, alternative 1 make sense. Unless the two shapes are the same, the instance check is guaranteed to fail; it can succeed only if the two Shapes are the same *and* `o` is an instance of `C`, which cannot be decided by the type checker. All class-based languages come

$\begin{aligned} \text{SClasses}[C] &= S \\ S &= (((f_1^T \text{ Tf}_1) \dots (f_g^T \text{ Tf}_g)) (\text{Tm}_1^T \dots \text{Tm}_k^T)) \\ \text{TVar}[a_1] &= \text{Tf}_1, \dots, \text{TVar}[a_g] = \text{Tf}_g \end{aligned}$	
$\text{SClasses}, \text{TVar} \vdash (\text{new } C \ (a_1 \dots a_g)) \Longrightarrow S$	[new]
$\text{SClasses}[C] = S_C; \text{TVar}[o] = S_o$	
$\text{SClasses}, \text{TVar} \vdash (o \text{ isa } C) \Longrightarrow \text{REAL}$	[isa]
$\begin{aligned} \text{TVar}[o] &= (((f_1^T \text{ Tf}_1) \dots (f_g^T \text{ Tf}_g)) (\text{Tm}_1^T \dots \text{Tm}_k^T)) \\ \text{there exists an } i \text{ such that} \\ &\quad f = f_i^T \end{aligned}$	
$\text{SClasses}, \text{TVar} \vdash (o \rightarrow f) \Longrightarrow \text{Tf}_i$	[get]
$\begin{aligned} \text{TVar}[o] &= ((\text{FT}_1^T \dots \text{FT}_l^T) ((\text{m}_1^T \text{ Tm}_1) \dots (\text{m}_k^T \text{ Tm}_k))) \\ \text{there exists an } i \text{ such that } m = m_i, \\ &\quad \text{Tm}_i = ((\text{T}_1 \dots \text{T}_n) \text{ T}_r), \text{ and } \text{TVar}[a_1] = \text{T}_1, \dots, \text{TVar}[a_n] = \text{T}_n \end{aligned}$	
$\text{SClasses}, \text{TVar} \vdash (o \rightarrow m \ (a_1 \dots a_n)) \Longrightarrow \text{T}_r$	[call]

Figure 60: Typing rules for object-oriented expressions

with subtyping though, and a model with subtyping would undermine the rationale for the first choice.

Alternative 2 models a Java-like type check, even though the type system for *Types* is structural, in contrast to Java's nominal one. It is the one stated in figure 60, because it is widely used. By contrast, alternative 3 resembles what the type system and semantics of Type Script implements. As long as the type of *o* is remotely sensible, the type checker validates the expression and leaves it to the underlying machine to decide whether *o* is an instance of the given class.

The [get] rule comes with two antecedents: one for retrieving the type of *o*, the target object; another one for retrieving the type of the referenced field. Here, the first antecedent provides access to the names and types of the fields and methods, respectively. Since the only way for *o* to come with a shape type is by [class] and [new], we know that its shape comes with a

To bridge the gap between the type systems of Types and TypeScript, we would need to enrich the former with many more kinds of values and their types.

set of field names. Hence, the second antecedent may demand that one element of this set is equal to the `FieldName` in the expression. The result type of $o \rightarrow f$ is the type of the respective field, if it exists.

Exercise 65. Explain the last derivation rule, `[call]`, in the style of this section. **Hint** This rule combines aspects of `[new]` and `[get]`.

3.4 Implementing a Type Checker

In the context of a model, an implementation of the derivation rules should aim for a line-by-line transliteration of its antecedents and its consequence. With the use of auxiliary functions or methods in the chosen implementation language, reaching this goal is straightforward. Often these auxiliary methods or functions come in handy in several contexts.

Consider the `[system]` derivation rule from figure 54. While the preceding section correctly suggests $n+2$ antecedents, a programmer may identify four groups:

1. checking the base case, namely just the first module;
2. checking the remaining $K-1$ modules in the appropriate contexts;
3. extracting from the imports which classes and at which types they are available; and
4. checking the system's body and ensuring that its result type is `REAL`.

Recognizing this grouping of antecedents suggests how to organize the corresponding type-checking method for a complete system.

Figure 61 sketches a Java implementation of a type checking method for a complete system. Since the type checker of our model runs after the validity checker has confirmed a number of basic properties, the AST classes with the type checker methods represent well-formed and valid ASTs, hence, the name `WFandValidSystem`. The key method needed for each such class is `typeCheck`. As the purpose statement in figure 61 says, the method checks an instance of this system AST, throwing a runtime exception if it discovers a violation of the `[system]` rule itself or any of its antecedents.

Next take a look at the signature of the `typeCheck` method. It matches the format of the `[system]` judgment. For judgments such as this one, the method consumes no inputs, because the judgment has no contextual information to the left of $\vdash$; since it does not produce a result, the `typeCheck` method has

```

class WFandValidSystem {
    WFandValidModules modules = ...;
    WFandValidImports imports = ...;
    WFandValidBody    body = ...;

    // check the type of `this` complete system
    // EFFECT throw an exception if a violation is discovered
    void typeCheck() throws TypeError {
        this.modules.select(1).typeCheck(this.selectPreceding(0));

        for(i = 2; i <= modules.size(); i++) {
            this.modules.select(i).typeCheck(this.selectPreceding(i-1));
        }

        SClasses sc = this.imports.typeCheck(this.modules);

        Type actual = this.body.typeCheck(sc, new TVar());
        if (!actual.equals(TypesAST.REAL)) // the type for numbers
            throw new TypeError("wrong result type for system body");

        // -----
        return ;
    }

    // create a container from the first `i` modules in `this` system
    Container<WFandValidModule> selectPreceding(int i) {
        ...
    }
}

```

Figure 61: A Java-style implementation of the [system] derivation rule

a result type void in Java. By contrast, for judgments such as [imports] that do require information about the context and that do produce a result, the signature of the corresponding method specifies inputs and outputs types. In the case of `imports.typeCheck`, the method must have access to the (well-formed and valid) ASTs of all modules so that it can create a mapping from `ClassNames` to their types, and its result is going to be the mapping. The boxed expression in figure 61 displays an illustrative call to this method.

With all these ideas in place, matching the method body of `typeCheck` with the judgment derivation rule is straightforward. Like the rule itself, the method body consists of four pieces. The horizontal rule in a comment indicates the separation of the antecedents from the part of the conclusion that is the result of the judgment. Blank lines separate the four transliter-

ations of the antecedents from each other. Each block corresponds to one of the antecedents. The last one uses three lines to explicate the steps involved in the use of this judgment: (1) compute the actual, derived type; (2) compare it with the expected type; and (3) signal an error if this comparison fails. This last part is implied in judgment derivation rules—when antecedent judgments do not hold, the conclusion cannot hold either. An implementation may turn this implicit idea into an exception. In this example, it is the only exception that the method raises directly; all other exceptions would come from calls to the auxiliary type-checking methods.

While the method clearly mirrors the specification—the [system] rule—this programming style comes with a cost. Notice how the method calls the (local and private) `selectPrecedingModule` method as many times as it type-checks an individual module. The purpose of the method is to create a container from the first i modules in this instance of the `WFandValidSystem` class. Although implementing this method poses no problem, calling it creates an inefficiency—acceptable for a programmer who creates an executable model, but unacceptable for a developer who creates an efficient type checker for *Types*.

Exercise 66. Can the first two blocks of `typeCheck` in figure 61 be merged? If so, how? If not, why not?

Exercise 67. How would you go about eliminating the inefficiency of repeatedly calling `selectPrecedingModule`? How would this distort the relationship between the specification via a rule and its implementation?

Figure 62 sketches the implementation of the [imports] rule. Recall its purpose: it specifies how a sequence of imports creates an `SClasses` object, that is, a mapping from `ClassNames` to `Shapes`. As the purpose statement of the class says, the type-checking method for [imports] is located in the representation of such sequences. In Java, such a class may delegate to a collection class, say `ArrayClass`; this kind of detail does not matter here.

Like the judgment for [imports] itself, which assumes knowledge about all modules in the context of the imports sequence, the `typeCheck` method in `WFandValidImports` consumes one value: a container of modules. Its signature tells a future reader that it computes an instance of `SClasses`.

The method body of `typeCheck` consists of two blocks, each of which implements one of the two antecedents in the corresponding derivation rule. Also notice how these two pieces are of the same size as the corresponding antecedents. With the first block, the method creates an empty instance of `SClasses`. The second block of code uses a Java-style for each loop to iterate through the sequence of import specifications, type checking one at a time.

```

// represents a sequence of import specifications
class WFandValidImports {
    WFandValidImports imports = ...;

    // check the type of 'this' sequence of imports
    // EFFECT throw a runtime exception if a violation is discovered
    SClasses typeCheck(WFandValidModules modules) throws TypeError {
        SClasses sc = new SClasses();

        for(WFandValidImport oneImport : this.imports) {
            sc = oneImport.typeCheck(modules, sc);
        }

        // -----
        return sc;
    }
}

```

Figure 62: A Java-style implementation of the [imports] derivation rule

From the corresponding derivation rule, we know that the method for type checking a single import consumes two pieces of contextual situation—the collection of modules plus the mapping of from *ClassNames* to *Shape*—and returns such a mapping. Once the method has dealt with all imports, the full mapping, *sc*, is returned as the result of the method. This last part corresponds to the right of the $\Rightarrow$ arrow in the conclusion of [imports].

Exercise 68. While the method in figure 62 uses a for each loop, the one in figure 61 uses an index-based for loop. Could it use a for each loop? If so, re-factor the code; otherwise explain why not.

Figure 63 displays one last example of a *typeCheck* method that implements a derivation rule. It illustrates how type checking works for the data representation of a well-formed and valid expression. Following the pattern for the previous two examples, the method consumes some pieces of contextual information and produces a data representation of the *Type* class, because this is the shape of a generic expression type judgment.

The two blocks inside of the method body correspond to the two antecedents; the return statement realizes the confirmation to the right of the $\Rightarrow$ in the conclusion. Each of the two blocks retrieves the type of an operand of the addition expression and compares this type to *REAL*. Unless this comparison succeeds, the antecedent fails to hold, causing the method to signal an error.

```

// represents an addition expression
class WfandValidAddition {
    Variable leftOperand = ...;
    Variable rightOperand = ...;

    // check the type of 'this' expression
    // EFFECT throw a runtime exception if a violation is discovered
    type typeCheck(SClasses sc, TEnv tenv) throws TypeError {
        Type leftType = tenv.lookup(this.leftOperand);
        if (!leftType.equals(TypeAST.REAL))
            throw new TypeError("wrong argument type for + [left]");

        Type rightType = tenv.lookup(this.rightOperand);
        if (!rightType.equals(TypeAST.REAL))
            throw new TypeError("wrong argument type for + [right]");

        // -----
        return TypeAST.REAL;
    }
}

```

Figure 63: A Java-style implementation of the $[+]$ derivation rule

Exercise 69. Systematically design an abstraction for the two blocks of code that realize the antecedents so that each becomes just one simple line.

Exercise 70. Why is it guaranteed that the lookup method of *tenv* finds a type for the two Variables?

3.5 Project Tasks

At this point, we have everything in place to complete the model for *Types*: the parser for the *Types* language (exercise 61), the validity checker (exercise 62), and the ideas for translating the judgment derivation rules into code (for an object-oriented data representation of the syntax).

Exercise 71. Design a complete type-checking pass for *Types*. The implementation should adhere closely to the shape of the rule-based specification so that small changes to the specification directly translate to small changes in the code. Have your type checker signal an exception when it discovers a type violation. Try to formulate error messages that assist the *Types* programmer with the task of repairing type errors.

Exercise 72. An implementation of a type checker in an IDE cannot just signal an exception when it discovers a rule violation. Instead, it pretends

to repair the AST so that it checks and adds a node to the AST (or mutates an AST node) so that the IDE can highlight several such violations at once.

Design a variant of your solution to exercise 71 that lives up to this standard. When checking an expression fails, use the type *REAL*. Although this substitution is a simplistic repair, it gives you a first idea of how such an IDE-integrated type checker may work.

Exercise 73. Design and implement a pass that turns the AST of a well-formed and well-typed *Types* system into a *Module* AST so that the linker from exercise 58 can create a *Classy* program. The resulting program can then be run on the CESK machine.

Note Depending on your implementation language and your choice of AST data representation, your linker might already work for *Types* ASTs or you might find it easy to adapt the linker so that it works this way.

Exercise 74. Adapt the main function from exercise 59 so that it can run the entire process of *Types* programs, validating and type-checking them, linking the module-based program into an object-oriented one, and determining its meaning on the CESK machine for *Classy*. Use your solutions from exercises 61 through 73.

The output should be an error message if the program is ill-formed, invalid, or causes a constraint violation when run on the CESK machine:

- "syntax error" if the parser discovers any mistakes;
- "duplicate module name" if the validity checker finds two modules with the same name;
- "import of non-existing module" if the validity checker discovers an import specification that does not refer to an already-existing module;
- "duplicate name error" if the validity checker encounters two classes with the same name, a class with two identical method names, or a class with two identical field names;
- "undeclared name error" if the validity checker encounters an undeclared *ClassName* or variable name;
- "type error" if the type checker encounters a type violation; or
- "runtime error" if the CESK transition function stops due to a final state with an error in its *C* register.

If running the machine delivers a value, it will be a number. Why? The main function prints this number.

4 Static Types Eliminate More Dynamic Checks

Checking the types of a *Types* program simplifies the CESK machine. Concretely, it renders a fair number of run-time checks superfluous and thus eliminates the corresponding cases in the transition function. The purpose of these checks is to catch inconsistencies between operations and arguments. For example, consider the two cases for $+$ in figure 32: one if the value of both variables are numbers and one if either one of them is an object. For several other expressions, the CESK machine for *Classy* comes with pairs of success/failures cases for their evaluation.

The addition of types to the language and a type checker to the model *before* the program is handed to the abstract machine ensures that these consistency conditions never matter. Compare the cases for $+$ in figure 32 with the implementation of `typeCheck` in figure 63. Notice how `typeCheck` compares the type of each variable with `REAL` and how the side condition on the machine transition confirms the same fact. Since `typeCheck` is run *before* the program is (linked and) loaded onto the machine, the case-distinction is superfluous.

Exercise 75. Inspect the CESK transitions in figures 32 through 38 and create a list of conditions that distinguish success from failure cases.

With this list of conditions in hand, inspect the derivation rules in this chapter to determine which of these conditions type checking renders superfluous. Mark the corresponding specifications in figures 32 through 38.

4.1 The Final Bit of Theory: Type Soundness

The elimination of run-time checks from the CESK machine for *Classy* demands trust in the type checking rules and its implementation. From the perspective of theory, the relationship raises the following question concerning the relationship between type-checking on one hand and the CESK machine on the other:

does the type checker eliminate all cases in which a Types program may apply a computational operation to the wrong kind of value?

Here “computational operation” refers to those cases in the CESK transition function that come with success conditions:

- $+$ and $/$ are candidates, because both can process only numbers;
- `new` must match the number of arguments to the number of fields;

- $o \rightarrow f$ and $o \rightarrow f = e$ succeed only if o is an object *and* this object comes with a field named f ; and
- $o \rightarrow m(a \dots)$ needs three conditions to hold for a successful transition: (1) o is an object, (2) it has a method named m , and (3) the number of arguments agrees with the number of m 's parameters.

As far as the type checker is concerned, we need to adapt the basic idea from chapter IV.4.1: A bad type checker can accept ill-typed systems. Take a look at the following `typeCheck` method for a *Types* system:

```
class WFandValidSystem {
    ...
    void typeCheck() throws TypeError {
        return ;
    }
    ...
}
```

It does not inspect any of its pieces; instead it accepts the existing system AST as type correct. And this kind of type checker is not going to help a language implementer with the elimination of run-time checks.

Following the pattern of the preceding sections on theory, we can state a theorem that expresses how the type checker must relate to the CESK machine to be sound. To this end, we think of the CESK semantics as a mathematical function, and we also consider the type checker sketched in this chapter as a function called *typeCheck*. Using these conventions, the so-called *Typed Soundness Theorem* has the following shape:

Some people say "type safety theorem."

For all well-formed and valid systems S in the *Types* model, if *typeCheckSystem*(S) holds and if *runMachine_{Classy}*(P) = R , then R is either a number or a division-by-zero *Exception*.

Here P is the result of erasing types from S and linking the resulting untyped *Module* system to obtain a *Classy* program.

In other words, *runMachine* does not signal an exception due to most of the conditions listed in success-failure cases. A corollary is that the corresponding failure cases in the transition function are superfluous.

You may now wonder what such a theorem really means. As is, the statement concerns the mathematical nature of the judgment derivation rules and the CESK machine. Critically, it does not say anything about the *implementation* of the type checker or the CESK machine.

Seen this way, you may wonder whether a language implementer can truly rely on this theorem. Theoreticians tend to think so, because they

assume that implementers faithfully follow the specification. Practitioners know that the chosen implementation language or algorithmic concerns often necessitate a deviation from the specification. Unsurprisingly, they accept that implementations come with bugs, that is, differences to the specified behavior of either the type checker or the run-time machinery.

	Control	Environment	Store	Kontinuation
evaluate an addition				
<i>before:</i>	$(y + z)$	e	s	k
<i>after:</i>	$(+ \ yn \ zn)$	e	s	k
<i>where</i>	$yl = e[y]$			
<i>and</i>	$zl = e[z]$			
<i>and</i>	$yn = s[yl]$			
<i>and</i>	$zn = s[zl]$			

Figure 64: The CESK transition function for *Types*: simplified addition

Note Over the past two decades, researchers have figured out how to use proof assistants to create a proof and extract complete compilers for small programming languages. Research the *CompCert* project for ANSI C and the *Cake ML* effort for two examples.

Exercise 76. Figure 64 shows how an implementer who trusts the type checker may collapse the pair of cases in the specification of the CESK transition function into one. Use the result of exercise 75 to comment out those cases of your implementation of the CESK transition function that are superfluous due to the type system.

*These cases will be
needed again in the
next chapter.*

5 Pragmatics: Static Type Systems

The addition of a type system to a programming language raises one of the most complex questions of pragmatics. Assessing the costs and benefits for both consumers and producers demands a thorough analysis, covering several new aspects. We start with the simplest one, the cost/benefit analysis for creators.

5.1 The Costs and Benefits for the Language Creator

Designing a type system starts from a goal and must then work through the collection of linguistic features with this goal in mind. Consider the type system in this chapter, whose goal could be formulated as

analyze code to discover snippets that may cause CESK compatibility checks to fail and thus reduce the number of problems that can show up during execution.

At first glance, this goal just means checking that the variables in an arithmetic expression always stand for numbers. But, an arithmetic expression may appear inside of a method and involve the method's parameters. Hence it is necessary to consider methods, that is, how values flow from the system body *into* methods. Conversely, a variable declaration may contain a method call instead of a numeric constant, meaning the type system also has to analyze how values flow *out* of methods. More generally, every item in the list of exercise 75 forces a type-system designer to study which role each linguistic feature plays in moving such a constraint from the machine into the type system.

Once the type-system designer has written up the type judgments and the judgment derivation rules, the language implementer's task is to turn these rules into code. Section 3.4 illustrates with a few cases how this works, and it indicates how much work this translation of rules into code is. What it fails to show is the work involved in the design of auxiliary methods so that the `typeCheck` methods mirror the derivation rules as clearly as possible. Aiming for such an implementation is important because type systems evolve, and realizing such changes is simplest when the implementation is structured according to the specification.

All this work pays off when it comes to "selling" the language. Every compatibility check performed at run time slows down program execution. Conversely, the elimination of every check speeds up the execution. Furthermore, types assist language implementers in other dimensions too, including space-efficient representations of surface data as machine-level data and energy-efficient arrangement of instructions. In short, the creators of a typed programming language can "brag" more easily about its performance than those who create untyped languages.

Besides performance, language creators tend to advertise that type systems accelerate bug discovery. That is, the language implementation can inform a software developer about incompatible operations and values during the creation of code instead of when the program runs and the given

inputs force the code to exercise these operations. Given that such failed compatibility checks stop the program execution, accelerated bug discovery appears to provide another sales point for a language. After all, no developer wants the control software for an airplane to fail because it tried to add a number to an object.

Like all advertising, this last one needs a close look. Unless the language creators establish type soundness—both on the theory side and the implementation side—this claim is patently false. The problem is that

if a language does not satisfy the type-soundness property and if its semantics does not perform run-time checks to ensure compatibility between operations and values, the code may perform entirely nonsensical computations.

Take the example of an addition operation again and consider that both a number and an object are represented as a some fixed number of bits in an actual hardware machine. While the bit-level representation may distinguish numbers and objects, the machine's add instruction can, and will, add any two bunches of bits—except that this addition produces a meaningless result. In the best case, such meaningless calculations eventually result in so-called segmentation faults—but this may happen long after the add instruction used the bits of an object representation. In the worst case, the program execution terminates normally without much of a hint that something went wrong.

Establishing type soundness is a lot of work. Language creators typically perform this work in two phases. First, they develop mathematical models of the type checker and the semantics, like those presented in this book. Then they validate with a careful analysis of all cases in the semantics that the type system and the semantics are related in the desired manner for every case of the CESK transition function. Even if the language creators perform all this work of establishing theoretical type soundness and carefully translate the derivation rules into type-checking methods, the implementation may still suffer from bugs due to a number of reasons. Most importantly, theory work covers a model, and a key attribute of a model is its simplicity compared to the actual language.

Second, to close this gap between mathematical models and the full-fledged language, language creators also derive extensive test suites from the language syntax and the full-fledged type-system specification. These test suites must cover language constructs excluded from the model plus the corresponding judgments and derivation rules. Critically, these tests do

This specification is often formulated in structured English based on the type judgments and derivation rules.

not just focus on either the type checker or the machine; they must attempt to uncover bugs in the combination.

Clearly, this kind of work is extensive and complicated. Unsurprisingly, software developers discover problems with type-soundness claims long after a language implementation has hit the market. Language creators with the ambition of maintaining type soundness will react with new releases when such bugs are discovered. It should be equally unsurprising that many language creators do *not* bother with type soundness, leaving their consumers in the dark as to how safe the language is.

5.2 The Benefits for the Software Developer

A software developer faces many more work situations than those mentioned in the preceding chapters of this book. It starts with the design of software and goes all the way to its maintenance over time. Types affect almost all of these situations, so it is time to take an expansive look at the combination of those and types.

Figure 65 presents a table whose rows describe work situations and whose three columns list the three kinds of languages we can imagine: one without a type system, an ideal one with a sound type system, and a final one with unsound types. Each cell corresponds to a work situation combined with an approach to types in a programming language.

By filling in these cells, we get a comprehensive picture of the pragmatics of types. We start with just a few examples of row-specific and even cell-specific benefits. Consider these examples illustrative, and keep them in mind for the exercises at the end of this section.

Designing Code When developers design software, types assist with almost all steps, but definitely with the first and most important one: how to choose a data representation for the information that the code is going to process. A language of types describes the forms of available data, and it is only those forms that a developer can use to represent information. Once the data representation is chosen, the formulation of a data definition heavily relies on the type notation too (plus comments that interpret what each piece denotes).

In the context of a typed programming language—sound or unsound—a developer does not have much of a choice. The type notation and the type rules are baked into the language, and they have to be used as is. A developer who has to work with an untyped language should still choose to follow a type discipline, but ideally a discipline that fits the problem. If no

work situation	without	language with sound type system	with unsound
design code			
keyboard code			
— create			
— refactor			
— evolve			
— migrate			
testing			
debugging			
documentation			
deployed code			
— bug diagnosis			
maintenance			
— port to different PL			

Figure 65: Work situations and the role of types systems

such discipline exists, the developer can mix and match elements from type systems that apply. The rest of the software development process benefits from such choices as long as the developer performs mental usage checks.

The final question concerning this first row is whether a type checker is helpful. Only the right-most two columns in figure 65 combine a type system with a checker; the point of an untyped language is that programmers are allowed to violate the imagined rules of their type system—as long as they know what they are doing. Based on posts in a wide variety of forums, mailing lists, blog posts, and so on, it is clear that the vast majority of developers think that a type checker helps, though in all likelihood they conflate design with code creation inside of an IDE.

Creating Code The use of integrated development environments synthesizes code design—the creative act of thinking through the problem and coming up with a solution—with code creation—the act of keyboarding. Developers design as they keyboard, and they keyboard as they design. A typed, object-oriented setting supports keyboarding to the point where it assists with some part of the design—if the data representation is chosen carefully. If so, it often boils down to the act of choosing the proper menu item after entering “o.” into the editor.

Having clarified this much, we can turn to the question of what enables the IDE to offer a menu of possible completions for the “o.” sequence of keystrokes. And the answer is *the type system*, of course. The very moment when a programmer enters the dot, the IDE has determined the type of the o via some background computation. As this chapter shows, the type contains the names of all methods and fields for an object.

As a matter of fact, if the developer selects a method *m* to continue from “o.”, the IDE has type information about the shape of the method call. Using this information, the IDE can present a template for the method call that the programmer then fills in. In sum, having a type system is extremely helpful in this case of an object-oriented language, which partly explains the success of this kind of language.

The picture looks a bit different for typed functional or procedural languages. An IDE can still support the keyboarding act in this context, but the support is less helpful than for an object-oriented language. Most importantly, the IDE catches type errors as the programmer adds characters to the program text, and this form of alert is useful to prevent design and code creation errors.

Testing and Debugging Let’s skip some rows to analyze the pragmatics of testing and debugging. Here the three cells differ radically.

As noted, a typed language signals that an operation, say field access, is incompatible with the given value(s) during program creation, that is, as the programmer enters text into an IDE’s editor. This observation clearly separates the first cell from the other two. Separating the second cell from the third requires a reminder of the role that type soundness plays.

When a sound type system validates code, the programmer knows that most semantic compatibility checks cannot fail. In the context of *Types*, a programmer may experience only one kind of semantic error: division by zero. By contrast, when an unsound type system validates code, it is still possible that programmers run into semantic mistakes during testing. Recall, however, that a language semantics does not have to perform these run-time checks at all. A language creator can remove them with or without arguing type soundness. In this case, a program may terminate and output results as if every operation had worked properly.

It is rare that the semantics of an untyped language comes without compatibility checks.

From this point of view, a developer may observe the following outcomes during test runs in the context of an unsound type system:

- A test run may terminate with the correct result. While improbable, it is possible that the execution misapplied operations but that

the observable outputs do not give any hint that things went wrong. Clearly, this is the worst possible outcome, though luckily it is rare.

- A test run may terminate with an incorrect result. In turn, the developer must search for a mistake but does not know whether this is a mistake due to language semantics or due to program logic. The good news is that the developer is aware of a problem; the bad news is that the search space for the bug is large, due to lacking support.
- A test run may terminate with a segmentation fault, because a misapplied operation causes a (bit-level) violation of an operating system or hardware constraint. At this point, the developer knows that the program execution used a primitive computational operation with the wrong values. Sadly, the developer cannot know when this misapplication took place during execution nor to which operation in the source code it relates.

This list significantly simplifies the set of possible scenarios, but it suffices to make the point about dangers with unsound type systems.

This list of testing scenarios and related debugging work clarifies how type soundness pays off. Yes, establishing type soundness imposes a significant amount of work on the small team of language implementers, but at the same time, the vast number of language users benefit a lot.

Exercise 77. Analyze how a type system may assist programmers who have to modify existing code so that it can deal with new variants of data. In your experience with procedural and object-oriented programming, is this task easier in one or the other?

Exercise 78. Analyze how a type system may assist software developers who must modify a program so that it can compute additional results for the existing forms of data. In your experience with procedural and object-oriented programming, is this task easier in one or the other?

Exercise 79. C# and Java come with nominal type systems. By contrast, TypeScript has a structural system, similar to the one of our *Types* model. In your experience, is a nominal type system as accommodating as a structural one for the extensibility tasks described in exercises 77 and 78?

Exercise 80. Analyze how a type system may assist with documenting the interface of a software component.

Exercise 81. Does the existence of a type system help with the task of porting software from one language to another?

5.3 How to Defeat a Programming Language

Even without filling in all of the cells of the table in figure 65, you can see that a language with a sound type system comes with tremendous benefits in many working situations. From a perspective of pragmatics, such a language is the best choice. Experience demonstrates, however, that developers can miss out on all these benefits unless they work with the language and its tools, not against it.

To start with, a developer's choice of data representation matters most. Consider the "choose a data representation" exercises that show up in all of the chapter of this book. Every programming language offers an infinitude of choices. As demonstrated at the beginning of chapter VI, a developer could, in principle, encode any form of information as a number. While programmers typically do not choose numbers to encode information, they often do choose strings, even if the information is highly structured.

In the case of an abstract syntax tree, the represented information has two sources: the program text and the language grammar. While the first usually consists of either just a sequence of (Unicode) characters or a semi-structured collection of characters, the second is a highly structured form of information. A good data representation mirrors this information structure and accommodates pieces of the program text as needed.

Figure 66 displays a choice of data representation for *Module* ASTs. Here is what the creator of this data representation writes to represent an occurrence of the VariableName *x* in a *Module* system:

```
new AST("variable", new AST[0], "x")
```

Now imagine using this Java class to represent modules, import specifications, classes, methods, variable declarations, or an expression. So here is a representation of a *Module* system without modules:

```
AST xvar = new AST("variable", new AST[0], "x");
AST one  = new AST("number", new AST[0], 1.0);
AST x1[] = {xvar, one};
AST decl = new AST("declaration", x1, "");
AST de[] = decl;
AST st[] = new AST[0];
AST expr = xvar;
AST bd[] = {de, st, expr};
AST sys  = new AST("system", bd);
```

Stop! What information does this data represent?

Obviously, a data representation for ASTs such as the one in figure 66 fails a developer in many ways. Most importantly from the perspective of

```
// represents an AST node of Module
class AST {
    String kind;           // the BNF production 'this' represents
    AST[] pieces;          // ... its pieces
    String name = "";      // ... if it has a name
    double value = 666.0;  // ... if it represents a literal number

    AST(String kind, AST[] pieces) {
        this.kind = kind;
        this.pieces = pieces;
    }

    AST(String kind, AST[] pieces, String name) {
        ...
    }

    // no two modules have the same name
    AST noDuplicateModules();

    ...

    // all occurrences of a variable name refer to a declaration
    AST closed();
}
```

Figure 66: An ill-suited data representation of *Module* ASTs in Java

this chapter, it fails to take advantage of Java's type system in many different ways. For example, when the developer enters "xvar." into the IDE, the drop-down menu contains the entry "noDuplicateModules" because it is one of the many methods that AST implements. But invoking the noDuplicateModules method on the data representation of a variable makes no sense. Similarly, in order to check whether the modules of a system come with distinct names, the noDuplicateModules has to iterate over all pieces, although only the first elements of this array should represent modules:

```
AST noDuplicateModules() {
    for(AST ast: pieces) {
        if (ast.kind == "module") {
            ...
        }
    }
}
```

Furthermore, as the sketch indicates the method must compare strings to find out whether any particular instance of AST is a module. Only then

it make sense to check the name field of the instance and process it as a `ModuleName`.

Contrast this first data representation, with an approach that uses distinct Java classes to represent modules, import specifications, classes (those in *Module*), methods, variable declarations, and so on. In such a data representation, a system would come with one container that represents all of its modules. This container would be the only Java class that would need a `noDuplicateModules` method, meaning this method name would not pollute any drop down menus when the developer enters “someVariable.” into the IDE. And, it is easy to see how this method can uniformly access all of the modules’ names, without any further conditionals.

A good core college curriculum on programming focuses on the very principles that help you reap benefits from programming languages. Such a curriculum conveys an understanding of software development as a systematic design activity tailored to the underlying philosophy of several different languages. This approach naturally introduces students to IDEs that properly assist with both syntactic validation and program execution. If your college course didn’t not come with such a curriculum, the best course of action is to observe yourself (and others) interacting with the IDE, stop to reflect on these interactions, and experiment on how to get the best feedback from the IDE and the chosen language.

5.4 The Costs for the Software Developer

Benefits do not come for free. Most importantly, it is impossible to equip many well-behaved *Module* systems with types so that they can be run as *Types* systems. That is, *Types* comes with less expressive power than *Module*.

Figure 67 illustrates this point. Consider the *Module* code on the left side. This module defines a class, `Coordinate`, which comes with two fields, `x` and `y`, and one method, `move`. A mathematically inclined developer might have named this method `translate`, because it realizes a geometric translation of one point relative to another, interpreted as a vector. Critically, the method does not mutate this instance but creates a new one; it is a “functional” method.

Stop! Try to write down a *Shape* that describes `Coordinate` as a type.

Problem is, the types notation does not allow a programmer to refer to a *Shape* by name. In particular, a *Shape* for `Coordinate` would have to refer to itself, because `move` returns another instance of the class—meaning its return type is the *Shape* we are in the process of writing down. Technically speaking, language researchers say that the type system of the *Types* model

<pre> (module Cartesian (class Coordinate (; going right horizontally: x ; going up vertically: y) (method move (delta) (def myX (this --> x)) (def myY (this --> y)) (def deltaX (delta --> x)) (def deltaY (delta --> y)) (def nuX (myX + deltaX)) (def nuY (myY + deltaY)) (new Coordinate (nuX nuY)))))) </pre>	<pre> class Coordinate { double x; double y; Coordinate(double x, double y) { this.x = x; this.y = y; } Coordinate move(Coordinate delta) { double myX = this.x; double myY = this.y; double deltaX = delta.x; double deltaY = delta.y; double nuX = myX + deltaX; double nuY = myY + deltaY; return new Coordinate (nuX, nuY); } } </pre>
--	--

Figure 67: The loss of expressive power due to type constraints (1)

lacks recursive types.

The designers of type systems for real-world languages try to overcome this loss of expressive power with additional complexity. For example, Java's type system accommodates recursive class types. Take a look at the right side of figure 67, which displays a Java-style implementation of an equivalent class. As the boxed reference to `Coordinate` in the method's signature shows, the name of a class is in scope of all its method types. Hence, it is possible to equip a functional method such as `move` with a signature.

Exercise 82. Add a system body to the code on the left side of figure 67 so that you can run it in your implementation of *Module*. The example should enable you to observe an output related to the processing of a `Coordinate` via `move`.

Exercise 83. If you have experience with Java, complete the code on the right side of figure 67 in the same manner as your solution of exercise 82. Run the program and compare the output to the one of your *Module* system.

Support for generics—occasionally called parametric polymorphism—in typed class-based, object-oriented languages makes this point as directly as possible. While Java and any typed object-oriented language with a type system supports recursive types, it initially lacked generics. Indeed, it took

```

interface WithF<T> {
    WithF<T> f();
}

interface IList<T extends WithFC> {
    IList<T> map();
}

class Null<T extends WithF>
    implements IList<T> {
    public IList<T> map() {
        return this;
    }
}

class Cons<T extends WithF>
    implements IList<T> {
    WithF<T> one;
    IList<T> rem;
    Cons(WithF<T> one, IList<T> rem) {
        this.one = one;
        this.rem = rem;
    }

    public IList<T> map() {
        WithF<T> fone = this.one.f();
        IList<T> frem = this.rem.map();
        return new Cons<T>(fone, frem);
    }
}

(module Null
  (class Null ()
    (method map ()
      this)))

(module Cons
  (import Null)

  (class Cons (one rem)

    (method map ()
      (def one (this --> one))
      (def fone (one --> f()))
      (def rem (this --> rem))
      (def frem (rem --> map()))
      (new Cons (fone frem))))))

```

Figure 68: The loss of expressive power due to type constraints (2)

the Java team a decade to add this extremely useful feature to the type system. During this time, several researchers designed and explored a number of variants. In the end, the Java team selected an *unsound* alternative, mostly to stay backwards-compatible. By contrast, Microsoft's C# designers, who had also released the language without generics, opted for a sound alternative and modified the underlying virtual machine to accommodate this choice.

Figure 68 displays an example that demonstrates the need for generics and simultaneously documents another loss of expressive power. The left side of the figure presents an untyped *Module* system fragment. It consists of two modules, *Null* and *Cons*, which jointly create a linked-list data representation. Using the exported classes, a *Module* programmer can form lists that contain distinct kinds of objects, that is, instances from distinct classes.

As long as each of these classes supports a method named `f`, the programmer can also request a map over these lists. Here are two such classes:

```
(module F
  (class F (x)
    (method f()
      this)))

(module G
  (class G (x)
    (method f()
      (def x (this --> x))
      (def y 2.0)
      (x + y))))
```

Key is that the programmer knows that the result of this use of `map` is a list with the same kinds of objects, that is, instances of the respective classes. In the above example this would mean a list of `F`s and a list of `G`s, respectively. Since *Module* lacks a type system, however, the programmer cannot use the language's validity checkers to confirm this thought.

Neither could a Java programmer until 2004. Instead, the programmer would use Java's `Object` type to encode the fact that a list can accommodate all kinds of objects plus casts to confirm the result type. A cast tells the type system to accept a programmer's claim without checking it, and it informs the semantics to perform a run-time check. Hence, a pre-2004 Java programmer could confirm this thinking only with unit tests, not while editing the program in an IDE.

The right side of figure 68 displays a Java version of the *Module* fragment that uses generics. Critically, the programmer can express the thought "every class that has a method `f`" in the type system—see the boxed code—and the type system validates this thought across the entire program. In particular, it confirms that the `map` method inside of `Cons` may use `f`. Since the `IList` interface also specifies that the result of `map` is `IList<T>`, the type checker confirms that each application of `map` returns the kind of list on which it is invoked.

Technically, the type checker replaces the type variable `T` for each call to method with the type of the list elements.

Exercise 84. Using the two classes above, add a system body to the code on the left side of figure 68 so that you can run it in your implementation of *Module*. The example should enable you to observe an output related to the processing of these lists via `map`.

Exercise 85. If you have experience with Java, complete the code on the right side of figure 68 so that you can run the program and observe an output related to the processing of two distinct `ILists` via `map`.

While the addition of complex features to a type system solves one problem, it creates another one. As the complexity of a type system increases so does the slope of the learning curve. Yes, complex type systems enable developers to articulate their thinking about code as code and

have the type checker confirm it. But, language designers must acknowledge that doing so slows down the learning process, potentially frustrates novices, and occasionally injects new pitfalls into the language.

Consider Java's generics. On one hand, they replace the use of `Object` and casts with useful pieces of code. On the other hand, Java's generics break the soundness theorem that applied to its initial releases. As mentioned, this lack of soundness reduces the trust into the program's results. Fortunately in Java's case the problem remains a minor one.

Similarly, the replacement of `Object` and casts with generics requires a thorough understanding of a non-trivial type notation and the meaning of parametric type expressions. To get to the point where writing programs such as those in figure 68 comes easy, a developer is likely to get frustrated with efforts to get code to conform to the type system.

Exercise 86. A structural type system, such as the one of the *Types* model, greatly simplifies the extension of a code base with new forms of data and additional pieces of functionality. If some functionality exists, say `map`, that applies to any object with a method `f`, a programmer can re-use this functionality with new classes.

How can this reuse endanger the code's integrity at run time?

How does a nominal type system, such as Java's, prevent such uses?

The answers demonstrate that the design of a type system adds to the tension between the security of a piece of code and its extensibility.

6 The Costs and Benefits of Types: The Warning

Software developers began to understand the expressive power of type systems in the 1990s. They recognized that type systems enabled them to express intricate thoughts and have them checked; and they recognized that by doing so, sound type systems could thus guarantee the absence of certain run-time failures.

In response, academics decided to explore additional ways to add expressive power to type systems. They advertised types as theorems about code and type checkers as proof assistants that established the validity of these theorems. Enabling developers to state more theorems and prove them correct would eliminate more run-time problems. Deployed software would suffer from fewer problems. These researchers seemed to overlook, however, that powerful type systems create a steep learning curve and that in such settings, developers have to work so much harder to sync their code with their types.

As this chapter's cost-benefit sections clarifies, the design of a type system is not a mathematical exercise but an engineering task. Engineers know that they maximize benefits subject to resource constraints; or they minimize costs subject to safety constraints. Put differently, engineering is an optimization process. Engineers do not go for ideals, such as total safety.

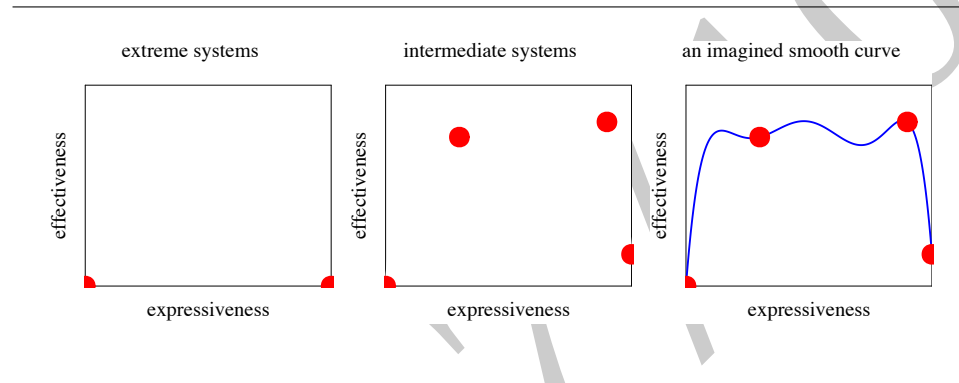


Figure 69: The Laffer curve of type systems

One way to illustrate this point is the use of the Laffer curve from economics. A Laffer curve records the benefits of some project over its resources. For simplicity, let's say the benefits of a type system is the prevention of bad behavior at run time, and let's similarly and equally naively say that the resource is the power of the type system to invalidate code.

While this setup is somewhat vague, it allows us to draw the first two points of the Laffer curve of types: see the left-most curve in figure 69. The point near the origin represents a "no-cost" type system that accepts every piece of code. Clearly such a type system comes without any benefits; all problems are discovered during execution, if at all. By contrast, the "extremist" type system on the right side of the graph rejects all programs. Although this guarantees that no program behaves badly at run time, it really means that no program can ever be run. Once again, it is appropriate to assign this type system a score of 0 for effectiveness. After all, effectiveness implies developers get programs to run easily and get them to behave well.

This chapter, and empirical evidence, shows that there are effective type systems with the kind of benefits that developers appreciate. Let's record this observation as red dots between the two extreme ones; see the graph in the middle of figure 69. For example, many programmers consider Java's

type system expressive and reasonably safe. Academics may instead point to OCaml's type system or Haskell's; both of these bestow more benefits on developers than Java.

If we now imagine effectiveness as a continuous function of expressiveness, as presented in the right-most part of figure 69, we can see that it may come with several local maxima over the full range. Given this understanding of the type-system design task, the goal of types researchers and type systems designers ought to be the discovery of such points. Or, as stated above, we should all try to find points on this curve that simultaneously assist working developers with mastering program design and yet ending up with reasonably safe code.

Jason: 7/10

Why do some languages support a mix of typed and untyped code?

1 The Pragmatics Question

Time and again, a start-up picks an untyped programming language, say JavaScript or Python, to get off the ground quickly. Rumor has it that such languages are good for the initial building process. Yet, as the company grows and recruits additional software developers, the lack of types or type expressiveness becomes noticeable.

People then recognize that typed languages facilitate “maintenance.” As the preceding chapter explains, type systems—whether they are sound or unsound—can assist developers in a range of work situations, including what is commonly called maintenance. Properly understood, maintenance includes every single work situation as well as the work of porting the software to a new platform. No matter what, types and type checking helps.

Without checked types, the on-boarding process becomes a painful exercise. New developers do not know which kinds of objects come with which methods; they have to look at the component’s informally documented interface or the code, because the IDE cannot display a matching menu. They do not know the signatures of these methods, and once again, the IDE cannot help with a method-call template. The informal nature of component interfaces poses another problem. Even though such an interface is an appropriate use of a comment, it still suffers from the usual problem with comments: they easily get out of sync with the code. When the underlying machine finally signals a mistake because some primitive computational operation fails, these new developers will have to search for the bug in their own code and *in the old code*.

Concretely, these problems show up in several shapes and forms. Most programmers know that JavaScript illustrates these problems well. Even with all the enhancements of the last two decades in place—especially the

addition of classes and modules, plus validation modes—developers struggle with adding functionality to its code base. Developers embrace its typed sibling—TypeScript—as soon as it appeared.

Another example is C++. It comes with a type notation and a type checker, though the former lacks expressive power and the latter is unsound. While the preceding chapter indicates in which work situations sound type checking helps, the type notation's lack of expressive is an altogether different problem. Since C++ is a language that permits developers to directly manipulate memory addresses—known as pointers—you might expect that the type system enables developers to articulate their thinking about pointers. For example, to avoid data race problems, the type notation should enable the creators of two concurrent components to articulate which one may modify the “pointed to” cell and which one may not. The C++ language lacks this power, however, while the new Rust programming language eliminates many of these gaps. Unsurprisingly, Rust is taking this systems development area by storm.

In short, the emergence of TypeScript and Rust demonstrate how language creators can address some of these questions of pragmatics to help working developers. The key is to come up with sibling language that (1) comes with an expressive type systems and (2) supplies an interface for the smooth integration of code in *both* languages. This chapter covers the relatively simple case of combinations that mix untyped and typed languages similar to the JavaScript-TypeScript pairing. For the case of strengthening type systems, say C++ and Rust, the interested reader should consult the relevant literature or blogs.

1.1 Design Choices

Language researchers recognized many decades ago how untyped code disadvantages developers as a code base grows. Since then, they have come up with a number of solutions. Here are three distinct approaches with different motivations:

- Going back to the 1980s, the Common Lisp language implemented an *optional typing* approach. A Common Lisp developer may declare types in various places, but they are essentially hints to the compiler and the IDE. In other words, they trust the annotations without checking and exploit them for various purposes.
- Historically speaking, the idea of *soft typing* came next. A soft-type system supplements an existing language syntax. It does not come

with a type notation that developers can use to write down claims about expressions. Instead, it infers types and presents the results in a notation that the soft-type system creator considers appropriate. A soft-type system may also come with tools that assist developers with exploring the meaning of type annotations. For example, when the system flags the arguments of a function call to be ill-typed, the developer may wish to understand where the arguments come from.

- Finally, *migratory typing* is the most recent and the most successful approach at the moment. Given some untyped language, language creators create a sibling language that is syntactically similar to the original one but comes with a type system. Indeed, once the type annotations are removed from a program in the sibling language, the result *is* valid code in the original language.

You may wish to consult the literature and the blogosphere for additional ideas in this realm.

What makes migratory typing so successful is a synergistic combination of factors. The adjective “migratory” tells us that it is all about moving from one place to another. First, language creators wish to enable a smooth migration of code from the untyped world to the typed one. When software developers notice that a growing code base suffers from a lack of types, they may wish to migrate any existing piece of code that needs to be modified. This piece might be a small expression or an entire module, but no matter what, the type system should pose as few obstacles as possible.

Second, when a language has been in use for decades, software developers have created a large body of re-usable components, frameworks, and libraries. They will want a simple way of accessing these untyped components from newly created typed code, and they want to make sure that this access is in sync with the type annotations. A common solution is to provide a mechanism for specifying type interfaces for untyped code in a *post hoc* manner.

Finally, language creators also apply the word “migrate” to the software developers themselves. Imagine developers who have spent many years or even decades working with the untyped language. Although they may not want to migrate existing and working code it is easy to imagine that they want assistance from a type system. Since these developers are used to certain idioms, one non-starter would be to force them to change their coding style just to accommodate the type checker. Once again, the type system must accommodate the idioms of the original, untyped language as much as possible, for the sake of working developers.

*Experienced
developers know that
touching code often
means enbugging it.*

The creators of Typed Racket and, a decade later, TypeScript recognized the critical nature of these factors and took them as objectives of their respective designs. Unsurprisingly, the resulting type systems resemble each other in some ways, and they differ in others. Both achieved the first and third objective in a reasonable manner, relative to the starting points. When it comes to the second one, however, the two designs differ radically.

When it comes to interactions between typed and untyped code, Typed Racket aims to satisfy the academic goal of achieving the highest degree of soundness. By contrast, the TypeScript documentation advertises that the language is essentially just JavaScript. More precisely, the two language designs implement two radically different semantic choices:

- The TypeScript approach is to validate that the types accurately describe the code in the typed component and, once this is done, to throw away the types. Clearly, this approach is *unsound*, because typed code interacts with untyped, and thus unchecked, code.
- The Typed Racket approach is to validate the typed component using the *post hoc* type specifications for an untyped component as given. It then compiles all *post hoc* types into run-time checks at the boundary between typed and untyped pieces. If these checks discover a non-conforming value, this value is a witness to a concrete violation of the specified type. To alert the developers, the machine raises an exception when the combined system is run.

Both of these solutions have distinct costs and benefits, for both software developers and language creators. It's time to consider those next.

1.2 Costs and Benefits

A migratory typing system shares the goals of a plain static type system, but it is clearly more complex than the latter. On the design side the complexity mostly shows up in the derivation rules, but, if a high degree of soundness is desirable, the translation from types to run-time checks requires quite some sophistication to keep the cost in check. On the usage side, the complexity appears in two forms. Even though the design of the type system aims to accommodate the idioms of the underlying untyped language as much as possible, developers still face a steep learning curve to get their code to pass the type checker. Furthermore, when developers impose a typed interface on existing untyped code, they invariably make mistakes, and someone has to cope and fix these mistakes later. Let's take

a close look at the two sides: the perspective of the developer and the language creator.

From the perspective of a software developer, a migratory type system means that each piece of code could live in either world: typed or untyped. As long as the developer works in the typed world, a migratory type system comes with all the same costs and benefits as a plain static type system: a decrease in expressiveness, support from the type system during most work situations, and so on. It all differs as soon as both languages are involved, and the difference has two major aspects: expressiveness and typed-untyped interactions.

The first difference between a migratory type system and a plain static one is about expressive power. While types restrict the expressions a programmer can use, a migratory typed system offers the best of both worlds in this regard. If a developer notices that the type checker cannot validate a particular kind of code, an escape to the untyped world solves the problem. Escape means, of course, that the developer must create an interface between the two pieces. But, unlike in the case of existing libraries or frameworks, in this case the creator of the typed code, the untyped code, and the interface between them is one and the same person. In short, the mixing of typed and untyped worlds via closely related programming languages offers a large benefit when it comes to the ease of turning ideas into working code.

The second difference concerns the interaction between typed and untyped code. If a developer wishes to use an untyped framework in typed code, it does not suffice to import it without ado. Otherwise the type checker does not know how to validate calls into the framework and returns from there, not to speak of higher-order exchanges. The developer must find or create a typed interface for the imports from the framework.

Since these useful frameworks have typically evolved over a long time, they tend to consist of large amounts of code, and their creators may be retired or even deceased. The combination of these two factors means that the creation of such typed interfaces is a significant amount of labor. Both factors also make it difficult to come up with *correct* type signatures for the exported pieces of functionality. As a result, such *post hoc* interfaces tend to come with numerous errors, that is, with types that fail to accurately and correctly describe the presumably well-tested and exercised framework code. For an illustration of this point, find the on-line repository called *Definitely Typed*, which is a collection of typed interfaces for JavaScript libraries that are used in TypeScript code. Visit some of the files to see how large they are. And check out the GitHub issues—closed and

open—to get a sense of how many errors people have already discovered.

Bugs in these types may manifest themselves long after the release of their containing interface (files and components). Let's make this precise. Suppose the framework supplies a function that supposedly returns an integer but, for the given inputs, actually produces a string. Nothing prevents such an event, because the type checker does *not* validate untyped code. What happens next depends on whether the combined language comes with (1) run-time checks at the boundary, (2) run-time checks in the underlying semantics, and (3) none of these. In all cases, the task of discovering the bug and locating it, is left to the developer who imports the framework—and this is usually neither the developer of the framework nor the developer of its typed interface. Obviously, the very possibility of imposing a corresponding debugging session on the developer must count as a potentially large cost.

From the perspective of the language creators, the cost/benefit analysis looks different. The creators must come up with judgment derivation rules that accommodate existing idioms and, if they want a sound type system, they must also develop a translation from types to run-time checks.

Before the language creators can even develop a type notation—not to mention derivation rules or type checkers—they must spend a good amount of time studying how programmers have used the untyped language in the past. These programmers don't just create code in a haphazard fashion, hoping for the best. With any training, experience, or both, they design code in a type-directed manner, just like the developers who use a typed language. But, they are not constrained by a specific type notation or by specific rules. Instead, they tend to think of sets of values and this thinking can be recognized as type-like reasoning. Hence the designers of migratory type systems must ask what “expressiveness” and “effectiveness” mean in this context, and how to balance the two. Unsurprisingly, this problem is by far the most complex language design problem mentioned in this book.

The semantic design choice—trust types or enforce them with run-time checks—also poses a significant problem for language creators. No matter what they choose, the result deeply affects the work of software developer:

- If they choose to throw the types away after validating them, they shift work to the software developers. The mix of typed and untyped code is going to cause mismatches between the types that specify a boundary and the values that cross the boundary. The continuing computations are erroneous in an insidious manner. Sometimes the

problem is easy to discover, sometimes it takes quite a while for people to recognize it. Since the unprotected boundary provides no information about the mismatch, the work of discovering where the mismatch is located, and how to fix it, is completely left to the developer.

- If the language creators choose to compile types to run-time checks, they clearly help developers with the discovery of bugs, but at the same time, they create a different problem for the working developers. Every run-time check costs time and thus reduces the performance of the program. Every run-time check may disable compiler optimizations, reducing performance even more. A software developer working on performance-critical code may not be able to tolerate this kind of performance loss.

In sum, the design and use of a migratory type system comes with potentially large costs and benefits. This chapter presents a model of both a sound and unsound system; doing so naturally builds on the preceding chapters. Once these designs are understood, we return to the discussion of the above points in section 7. By contrast, the design of an accommodating type system is (mostly) beyond the scope of this book; the next section supplies a taste of what it means, without explaining how the type system works.

2 A Type System for Untyped Languages

A migratory type system must satisfy two criteria: (1) It has to accommodate the idioms of the underlying, untyped language, and (2) it has to come with a mechanism for smoothly integrating untyped code into typed code and vice versa. The following two subsections illustrate the two ideas using Racket and Typed Racket for this presentation.

2.1 A Type System for Untyped Idioms

Inspecting Racket code repositories validates that Racket programmers mostly think of types as sets of values and that reasoning about expressions seems to rest of naive set theory. If they know a bit of advanced type systems, their signatures (in comments) may use an existential quantifier to hide a type inside of a region; they may resort to a universal quantifier to appeal to parametric polymorphism; they may add dependencies between domains

and ranges of functions; and so on. Typed Racket accommodates most of these ideas, at a mostly reasonable cost to the developer.

<pre>#lang racket ;; A TLC is one of: ;; - "red" ;; - "green" ;; - "yellow" #: {TLC -> TLC} (define (switch cc) (cond [(equal? "red" cc) "green"] [(equal? "green" cc) "yellow"] [(equal? "yellow" cc) "red"])))</pre>	<pre>#lang typed/racket (define-type TLC (U "red" "green" "yellow")) (: switch (TLC -> TLC)) (define (switch cc) (cond [(equal? "red" cc) "green"] [(equal? "green" cc) "yellow"] [(equal? "yellow" cc) "red"])))</pre>
--	---

Figure 70: Finite sets of values are types

The most basic way of “thinking of types as sets” reduces to an enumeration of a finite number of elements. Say a software developer’s charge is to develop a controller for a traffic light that at any moment illuminates either its red bulb, its green one, or its yellow one. Put differently, the state of the traffic light is equivalent to its illuminated color. This analysis suggests that a natural data representation of the traffic light is the finite set containing the three obvious strings: “red”, “green”, and “yellow”. As the left column of the figure 70 shows, the conscientious Racket developer writes down this choice using structured English. The module in the figure also comes with the switch function, which picks the bulb to be illuminated next.

The right column displays a conversion of this untyped module into a typed one. First, the programmer merely adds `typed/` to the language-specification line of the module. This line informs Racket and Racket-oriented IDEs that the rest of the code is supposed to be understood as Typed Racket. Second, the second use of a comment—the type signature of the switch function—also has an immediate and easy-to-understand representation in Typed Racket. Third, the body of the function definition remains the same, because the type checker can validate the function’s signature against the original code. That is, the type system allows a direct expression of data definitions as type definitions, and the type checker easily copes with function definitions for such simple examples.

A properly trained developer would replace these strings with constant names in both the types and the code.

Basic set operations make an appearance in almost every piece of code, and the type system accommodates the most important ones, as the next example demonstrates. Figure 71 presents two versions of the factorial function as a conscientious Racket programmer, or a properly instructed novice, would write them. On the left we see the factorial function in plain Racket. It comes with two appropriate comments: a signature and a mathematical purpose statement. While the function definition itself matches the proper mathematical definition, note that the module comes with a basic unit test (written as a submodule). The right column of the figure presents a conversion of the left-hand code into a Typed Racket module.

In Racket “!” really is a function name.

<pre>#lang racket ;; Natural -> Natural ;; n! = 1 · 2 · ... · n (define (! n) (if (= n 0) 1 (* n (! (- n 1))))) (module+ test (require rackunit) (check-equal? (! 3) 6))</pre>	<pre>#lang typed/racket (: ! (Natural -> Natural)) ;; n! = 1 · 2 · ... · n (define (! n) (if (= n 0) 1 (* n (! (- n 1))))) (module+ test (require typed/rackunit) (check-equal? (! 3) 6))</pre>
---	--

Figure 71: Set differences between types confirm case-based reasoning

This second example demonstrates additional points. First, the function definition does not have to change. As long as the programmer specifies a type that matches the function’s code, the code can often remain the same. Second, the signature (comment) is turned into a type specification that associates the name of the function `!` with its type `(Natural -> Natural)`. The type copies `Natural` from the unchecked signature, and the type checker successfully validates this type against the function definition. Finally, the programmer adds `typed/` to the `require` specification for the unit-test library, importing its typed variant.

A use of `Natural`—instead of the more conventional `Integer`—illustrates how programmers can reason in an untyped language. In the context of an ordinary statically typed language, a programmer would use `Integer` to specify the inputs and outputs of the factorial function, even though every discrete mathematics textbook says that it is a function on $\mathbf{N}$. A programmer who uses an untyped language can go along with the textbook, as the

signature on the left of figure 71 shows. The signature informs readers that `!` must be applied to natural numbers, and the creator of `!` must validate this constraint for the recursive call. Since `n` is larger than 0 in the `else` branch of the conditional, `(- n 1)` is still a natural number, meaning the application of `!` to this number is acceptable.

Racket's type checker reasons in a similar manner. The type system defines `Natural` as `0` or `n + 1` for an `n` in `Natural`. Furthermore, Racket's type system employs set-based reasoning via basic logical propositions. Thus, while `n` has type `Natural` in the conditional check, (1) it can focus on `0` when the outcome of the check is positive, and (2) it can subtract `0` from the type for the `else` case. Hence, by the time it checks the framed expression, it may conclude that (1) `n` is positive and (2) `(- n 1)` is still an element of `Natural`. In general, the type of a variable depends on where it occurs within a nest of conditionals and whether the type checker can convert these conditions into propositions that narrow down the specified type.

<pre>#lang racket #; (Shape = Circle or Square) (struct circle (radius)) #; (circle r) ; r is the radius ;; r is Real (struct square (side)) #; (square s) ; s, a Real, is ;; the length of its side #; {[Listof Shape] -> Real} (define (area-of-just-squares s) (define sq* (filter square? s)) (define ar* (map sq-area sq*)) (define sum (foldr + 0 ar*)) ;; return: sum) #; (Square -> Real) (define (sq-area x) (sqr (square-side x)))</pre>	<pre>#lang typed/racket (define-type Shape (U Circle Square)) (struct Circle ((radius : Real))) #; (circle r) ; r is the radius (struct Square ((side : Real))) #; (square s) ; s is its length (: area-of-just-squares ([Listof Shape] -> Real)) (define (area-of-just-squares s) (define sq* (filter Square? s)) (define ar* (map sq-area sq*)) (define sum (foldr + 0 ar*)) ;; return: sum) (: sq-area (Square -> Real)) (define (sq-area x) (sqr (Square-side x)))</pre>
--	--

Figure 72: Set-based reasoning generalizes to higher-order idioms

Figure 72 illustrates this principle with a higher-order example. The figure again presents two columns with untyped and typed code. The framed

expression within the typed module is the focus of this example.

Take a look at the untyped module. Its purpose is to represent and process shapes, concretely a mix of circles and squares. A structure definition is used to represent basic shapes; an informal data definition ties them together into (the set of) all Shapes. The main function, `area-of-just-squares`, consumes a list of Shapes and produces the area of all squares on this list. It employs several higher-order functions: `filter`, `map`, and `foldr`. Of these, the first one whittles the given list of arbitrary shapes down to a list of just squares. Hence, the programmer can use `map` to apply a function for computing the area of one square to all elements of this list—without any fear of accidentally applying the structure-selector to the wrong kind of struct.

Although this code does not contain any conditional *per se*, it nevertheless demands a form of conditional reasoning from the programmer. Technically, the reasoning lifts the hidden conditional from `filter` and combines it with `square?`, the given predicate. By combining the two, a programmer may conclude that the remaining occurrence of `sq*` in the definition of `area-of-just-squares` has a narrower type than `[Listof Shape]`, namely `[Listof Square]`.

Now take a look at the typed module on the right side. This conversion requires a bit more effort than for our first example, mostly due to the sloppy type definitions and signatures (in comments) of the untyped variant. For example, the first data definition (comment) refers to `Circle`, an undefined name. A future reader is presumably supposed to conclude that the structure definition of `circle` introduces this type. During the conversion of such a module into a typed one, a programmer must write down definitions that properly refer to each other, which explains the change in spelling of `Circle` and `Square`.

When it comes to the function definition of `area-of-just-squares` itself, the type checker again validates the existing code against the associated type without ado. It mimics the reasoning concerning the framed application of `filter`. Typed Racket's type for `filter` says "if the given predicate represent a subset *S* of the given list elements, then the result is a list whose elements belong to *S*." Here, `Square?` is a predicate that does describe a subset of `Shape`, namely the left half of the union, and therefore the resulting list contains nothing but instances of `Square`.

The last illustration exhibits how a programmer can equip a Racket program using classes and objects with idiom-appropriate type annotations. Like in Java or Python,

- classes serve as factories for objects.
- a class may extend another class, but determining which class it ex-

<pre>#lang racket (provide add-search) #; (Pos = (Cons Natural Natural)) #; (Opt = (Pos or #false)) #; (Opt* = ([Listof Opt] or #false)) #; ((Class [the-text (-> String)]) -> (Class [the-text (-> String)] [search (-> String Opt*)])) (define (add-search cls%) (class cls% (super-new) (inherit the-text) #; (String -> Opt*) (define/public (search s) (define txt (the-text)) (regexp-match-positions s txt))))</pre>	<pre>#lang typed/racket ;; add-mixin.rkt (provide add-search) (define-type Pos (Pairof Index Index)) (define-type Opt (U Pos #false)) (define-type Opt* (U (Listof Opt) #false)) (: add-search (All (a) ((Class #:row-var a [the-text (-> String)]) -> (Class #:row-var a [the-text (-> String)] [search (-> String Opt*)])))) (define (add-search cls%) (class cls% (super-new) (inherit the-text) (: search (String -> Opt*)) (define/public (search s) (define txt (the-text)) (regexp-match-positions s txt))))</pre>
---	--

Figure 73: Object-oriented idioms need more than set-based reasoning

tends is a semantic decision not a static one.

- a function may consume or return a (possibly locally defined) class. Racketeers call such functions *mixins*.

In other words, classes are first-class values in Racket.

Figure 73 presents a module that exports a simplified version of a library mixin. It consumes a class that allows users to manipulate text, e.g., a message box or a program editor. As its informal signature documents, it assumes that its argument comes with a method called `the-text` and that this method returns a string. The mixin derives a new class that supports all methods of the given one, plus a method that determines those positions, if any, where some pattern matches the string extracted with `the-text`.

By convention, names ending in % refer to classes.

Writing down the mixin's signature takes three auxiliary data definitions: one for positions (*Pos*) within strings; another one for optional positions, in case the search for a pattern fails; and a final one for an optional list of search results. The signature itself tells a future reader of this code that the given class must have at least a method named `the-text` and that its result still has the method plus a new one named `search`.

As the right column of figure 73 indicates, the signature is the focus of this example. The framed type signature formally expresses an implied property of the informal signature in the untyped code, namely, that the mixin cannot rely on, or override the meaning of, anything but the `the-text` method in the given class. Note, though, that it looks much more complex than the informal one on the left. Concretely it differs in two major ways:

- First, the `All` quantifier signals that the signature is parametric in some parts, and a reader must pay attention to where these parts are.
- Second, the quantified type variable, `a`, denotes a *row*, that is, any feature that a class type may mention: fields, methods, and more.

Put differently, turning the informal signature into a technically correct one takes both an understanding of a sophisticated notation and of universal row polymorphism. Given this type, the language can statically validate it for the typed set of modules and can generate run-time checks that guarantee the integrity of this statement during execution.

The presented examples are illustrative of Racket to Typed Racket conversions but do not cover the complete range. Once language creators have an initial design of a migratory type system, developers will ask for two kinds of refinements. First, some will argue that it requires too many changes to untyped code to appease the type checker or that it is impossible to appease it all but should be possible. Second, developers used to static type systems and their advantages may expect to find certain conventional type-system features that are incompatible with a migratory system. As a result, the development of a migratory type system is a never-ending refinement loop.

2.2 Integrating Typed with Untyped Code

Equipped with a sense of what it is like to program in Typed Racket, we can now address the issue of mixing typed and untyped code. Since Racket modules specify the language of their content, the creators of Typed Racket made the decision to “migrate” at the granularity of entire modules. This

decision implies that there are four kinds of module dependencies: the original untyped-on-untyped plus the new typed-on-untyped, typed-on-typed, and untyped-on-typed cases.

Of these four cases, only one poses a design problem to the type-system creators, namely, the case of importing an untyped module into a typed one. Since the type checker needs to know types for all names and since untyped modules do not supply types for their export, Typed Racket supplies one novel construct: `require/typed`, which, roughly speaking, enables programmers to attach types to imported names.

Next, importing a typed module into another typed module is straightforward. The first module attaches a type to each exported name, and the type checker can use this type for the validation of the second one. Conversely, since the type checker does not inspect untyped modules, exporting from a typed to an untyped merely means ignoring the types from the exports in the importing one.

Let's consider some concrete examples. Suppose a Typed Racket developer needs to create code that calls a factorial function. Further assume that in the decades prior to the creation of Typed Racket, the Racket team equipped its language with a discrete mathematics library that exports `!`, the factorial function. Here is how the developer would go about importing this one function from the untyped Racket library:

```
# lang typed/racket

(require/typed library/discrete-mathematics
  [! (-> Natural Natural)])

... (! 3) ...
```

This `require/typed` specification still refers to a specific module, but it also lists all names that are to be imported into this module's scope and, critically, their type. In this example, the type of `!` is the same as the one in the preceding section. Note, though, that a type of, say, `Natural -> Integer` might also work for this client module. That is, the type attached to imported names is up to the developer of client code, not the developers who created the library.

Things can get complicated, though, as the two columns of figure 74 show. On the left, this figure displays a simple data representation for squares and circles, like the one in the preceding subsection. Both structures are exported wholesale, meaning the respective predicates, constructors, selectors, and so on. On the right, the figure displays a typed client that recreates the `area-of-just-squares` function from figure 72.

<pre>#lang racket ; server.rkt (provide (struct-out square) (struct-out circle)) #; (Shape = Circle or Square) (struct circle (radius)) #; (circle r) ; r, the radius, ;; is Real (struct square (side)) #; (square s) ; s, Real, is ;; the length of its side</pre>	<pre>#lang typed/racket ; client (require/typed "server.rkt" [#:opaque Square square?] [#:opaque Circle circle?] [square-side (Square -> Real)]) (define-type Shape (U Square Circle)) (: area-of-just-squares ([Listof Shape] -> Real)) (define (area-of-just-squares s) ...) (: sq-area (Square -> Real)) (define (sq-area x) ...)</pre>
--	--

Figure 74: Requiring structures from an untyped module

A sophisticated `require/typed` specification bridges the gap between the two modules; see framed code. The first two imports name opaque types, `Square` and `Circle`, respectively. Each such opaque type corresponds to a predicate, here `square?` and `circle?`, respectively. The last import looks like the one from the first example, bringing the `square-side` selector into scope at type `Square -> Real`. By connecting the untyped and typed module with this `require/typed` specification, the developer can make the rest of the typed module look just like the code in figure 74 (right column).

The last example brings us back to the world of modules and classes. Figure 75 displays two modules: an untyped in the left column, a typed one in the right column. While the first one sketches a simplistic multi-line text editor, the second combines this text editor with the mixin from figure 73.

For the purpose of this chapter, the framed `require/typed` specification is the relevant part of this example. It brings the `text-editor%` class into scope of the typed module. Since the exporting module, on the left of the figure, is untyped, the importing module must assign a type to the class so that the type checker can validate the module's code. The type specifies the only field of `text-editor%` as a string-vector, and associates the two methods—`text++` and `the-text`—the expected signatures.

By contrast, the import `add-mixin` refers to a typed module. Hence it is unnecessary to use `require/typed` or to equip the imported name with a type.

<pre>#lang racket ; editor.rkt (provide text-editor%) (define text-editor% (class object% (super-new) (field #; [Vectorof String] (text (make-vector LINES ""))) #; Natural String -> Void ;; add 'c' to the end of ;; the 'at' line 'this' editor (define/public (text++ at c) (define a (vector-ref text at)) (define b (string-append a c)) (vector-set! text at b)) #; -> String (define/public (the-text) (string-join (vector->list text) " ")))</pre>	<pre>#lang typed/racket (provide searchable-text-editor%) (require add-mixin) (require/typed editor [text-editor% (Class [field (text (Vectorof String))] [text++ (-> Natural String Any)] [the-text (-> String)]))] (define searchable-text-editor% (add-search text-editor%))</pre>
--	--

Figure 75: Requiring a class from an untyped module

Instead, Typed Racket can retrieve the type of `add-mixin` from its exporting module. With both the editor and the mixin in scope, the module can create, and export, a searchable version of this text editor by applying the latter to the former.

Stop! Take a second look at figure 75. This example serves as inspiration for the model of the next section. Do not continue until you get a good sense of how this example works.

3 *Mixed*: the Syntax

Mixed is a model of modern languages with migratory type systems, such as Flow, Hack, and TypeScript. Like all of our models, it resembles it in others, and it deviates from reality in some ways. In order to build on the models from the first eight chapters, the migratory type system of *Mixed* exists at the module level, not inside of small pieces of code. Furthermore, instead of forcing *Mixed* programmers to write “interface modules,” like those found on TypeScript’s *DefinitelyTyped* repository, the model borrows

require/typed from Typed Racket. Although the result is seemingly simple, it still supplies the opportunity to investigate the major semantic design decision that creators of these language face.

The section presents *Mixed* following the outlines of the preceding sections: the grammar, the validity constraints, the extra judgment derivation rule, and a note on the relationship between types and semantics. The following two sections then present two starkly differing semantics for the syntax of *Mixed*.

3.1 *Mixed*: the Grammar

Figure 76 presents the essential productions of the BNF grammar of *Mixed*; for all other productions, consult figures 49 and 50. A mixed system consists of typed and untyped modules. The typed ones may import names from the untyped ones and vice versa. In order to facilitate implementations of *Mixed*, the new keyword `timport` marks how a typed module accesses the exports of an untyped module. Following the Typed Racket precedent, a `timport` specification includes the `Shape` that the type checker is supposed use to validate the importing module.

```

MixedSystem ::= (MixedModule*
                 MixedImport*
                 Declaration*
                 Statement*
                 Expression)

MixedModule ::= (tmodule ModuleName MixedImport* Class Shape)
               | (module ModuleName Import* Class)

MixedImport ::= (import ModuleName)
               | (timport ModuleName Shape)

```

Figure 76: *Mixed*: the Grammar

It is noteworthy how we can enforce certain contextual constraints via this BNF and the use of distinguishing keywords. First, `MixedImport` shows up only in the context of `tmodule`, i.e. an untyped module cannot impose a type on an imported class. Second, the imports for the system's body also belong to `MixedImport`, which implies that it is typed code. While this kind of “grammar hacking” accomplishes some basic validity checking, it cannot eliminate the context-sensitive validity checkers in general.

Exercise 87. Design an AST data representation for *Mixed*. Implement a parser for *Mixed* that maps an S-expression to an instance of AST. Re-use your solution from exercise 61 as much as possible.

3.2 *Mixed*: Validity

Even though the BNF forces programmer to use `timport` only in the context of `tmodule`, this constraint does not ensure that the given `ModuleName` refers to an untyped module. Similarly, a sequence of `MixedImports` could contain two different `timport` specifications for the same untyped module. In principle, this situation is acceptable but, to simplify the implementations of *Mixed*, we rule it out.

Technically, a *Mixed* system is subject to three structural constraints:

- The typed portions of a system must import untyped modules via a `timport` specification.
- The typed portions of a system may not import the same (untyped) module using two different `Shapes`.
- The typed portions of a system may not import a typed module using a `timport` specification.

Keep in mind that “typed portion” covers typed modules and the system’s body consisting of declarations, statements, and the final expression.

Exercise 88. Design and implement a validity checker that enforces the validity rules for *Mixed*. The checker consumes error-free ASTs from exercise 87; if it finds errors it annotates the AST appropriately.

3.3 *Mixed*: Type Checking

The point of creating mixed systems is to connect typed and untyped pieces of code; in the context of *Mixed*, this means connecting typed and untyped modules. Hence, a type checker must validate that the types and the code in the typed portions are in sync. Conversely, the untyped modules of the same system do not have to satisfy any type judgment rules.

Given this objective, the adaptation of the type judgment rules from chapter VIII is straightforward:

- The adaptation of the rule for type checking complete systems, see figure 54, must ignore untyped modules.

M is the name of mod in Modules	
C is the name of the class defined in mod	
Modules, SClasses $\vdash$ (timport M S) $\implies$ SClasses [C : S]	[an import]

Figure 77: Deriving the key judgment for mixed systems

- The import of an untyped module into a typed one calls for the additional type judgment rule of figure 77.

Summing up, the *Mixed* model is a rather small extension of *Types* from a syntactic point of view. In particular, the type system of *Mixed* does not accommodate the programming idioms of *Module*; instead, it imposes the same limitations on typed portions as the model of the preceding chapter.

(module A	(tmodule clientX	(tmodule clientY
(class A ()	(timport A	(timport A
(method id(x)	(()	(()
x)))	((id (REAL)	((id (((x REAL)) ()))
	REAL))))	(((x REAL)) ()))))
	...)	...)

Figure 78: Two types for the same exported class

Exercise 89. Figure 78 presents three modules in three columns:

- The untyped module A (left) exports a class named A.
- The typed module clientX (middle) imports A at one type.
- The typed module clientY (right) imports A at a different type.

Argue from the point of view of a *Mixed* programmer why the type checker should accept both timports or reject them.

Exercise 90. Adapt the type checker from exercise 71 to perform a complete type-checking pass for *Mixed*. The type checker should merely signal an exception when it discovers a type violation. Each such exception should come with an error message that assists the *Mixed* programmer with the task of repairing type errors.

3.4 A Note on Types and Semantics

A typed module can import an untyped module *U* at a Shape *S* that the type checker would reject if it were to check *U* equipped with *S*. Figure 79 illustrates this point with the same exporting module as figure 78, but with an importing module that assigns an inappropriate type. That is, the type checker would reject a typed version of module *A* if it attached the shape of the *timport* to class *A*.

```
(module A
  (class A ()
    (method id(x)
      x)))

(tmodule clientX
  (timport A
    ( ( )
      ( (id (REAL) ( ((x REAL)) () )) )))

  (class clientX ()
    (method di() (new A()))))

  ( ( )
    ((di ()
      ( ( )
        ( (id (REAL) ( ((x REAL)) () )) )))))
```

Figure 79: The wrong type for an exported class

Stop! Why would the type checker reject a typed variant of *A* if the framed type were used?

Exercise 91. The following code fragment completes the two modules of figure 79 into a full *Mixed* system:

```
(import clientX)
(def x (new clientX()))
(def a (x --> di()))
(def one 1.0)
(def atoo (a --> id(one)))
(atoo --> x)
```

Does this system type check? If so, explain how. Otherwise explain why the type checker rejects this system.

The example (figure plus exercise) raises the question of what should happen with a *Mixed* system that type checks even though the type assigned to a class imported from an untyped module is a mismatch. Answering this question means choosing a semantics and, by choosing a semantics, opting for certain choices of pragmatics. Unsurprisingly, different language creators give different answers to this question.

4 *Mixed*: the TypeScript Semantics

TypeScript advertises itself as a form of JavaScript. Concretely, its web page says “TypeScript becomes JavaScript via the delete key.” What this means, is straightforward. Once the type checker has confirmed the validity of the types with respect to the typed portions of the code, the TypeScript implementation traverses the given code, erases the types, and generates a plain JavaScript program.

Modeling TypeScript’s semantics with *Mixed* and *Module* is straightforward. Every *tmodule* of a *Mixed* system becomes a plain module; the type of the former simply disappears. Similarly, every *timport* specification becomes a *import* by erasing the type from the former. The resulting code is a well-formed and valid element of the *Module* model.

4.1 Project Tasks

Exercise 92. Your task is to implement the “erasure” pass, which turns a *Mixed* system into a *Module* system.

Exercise 93. Adapt the main function from exercise 74 so that it can run the entire process of parsing a *Mixed* program, validating it, type-checking its typed portions, erasing types to obtain a *Module* system, linking this system into an object-oriented one, and determining its meaning on the CESK machine for *Classy*. Use your solutions from exercises 87 through 92.

The output should be an error message if the program is ill-formed, invalid, or causes a constraint violation when run on the CESK machine:

- “syntax error” if the parser discovers any mistakes;
- “duplicate module name” if the validity checker finds two modules with the same name;
- “bad import” if one of the *Mixed*-specific validity constraints concerning imports is violated;
- “duplicate name error” if the validity checker encounters two classes with the same name, a class with two identical method names, or a class with two identical field names;
- “undeclared name error” if the validity checker encounters an undeclared *ClassName* or *variable name*;
- “type error” if the type checker encounters a type violation; or

- “runtime error” if the CESK transition function stops due to a final state with an error in its C register.

If the final CESK state contains a number or an object as a final control string, main prints the number of “object”, respectively.

Why can main print “object” for a *Mixed* program if its type is Number?

5 *Mixed*: the Typed Racket Semantics

In contrast to the industrial TypeScript team, academic researchers have spent two decades exploring different flavors of sound semantics of migratory type systems. The goal of these efforts is to notice when a value flows from an untyped portion of the code into the typed portion, or vice versa, but does not match the specified type.

For example, the *Mixed* system of exercise 91 and figure 79 is well-formed, valid, and blessed by the type checker. When run, the system’s body creates an instance of *clientX*, uses its *di* method to create an instance of *A*, and then accesses the type-checked field *x* in this latter object. Problem is, this object does not have any fields. If such a value flow takes place in a TypeScript program, it is unclear what is going to happen. Academic language designs aim to catch the kind of *type mismatches* that the import of *clientX* during execution sets up and then inform the programmer.

The stated goal allows many different implementations. Each implementation notices type mistakes at slightly different times. This section presents an implementation that approximates Typed Racket’s solution, but deviates from it in some ways. In essence, the Typed Racket solution is

- to check basic values, such as numbers or strings, when they flow from an untyped to a typed portion of the code or vice versa; and
- to wrap objects so that it becomes possible to check every access or mutation of the value according to the specified types.

In the spirit of keeping things simple, the semantic model of this section leverages the existence of the linker to manifest the boundaries between typed and untyped portions. Concretely, the linker is used to clone an untyped module if it is subject to a *timport* in the context of typed code. This clone comes with a type but is *not* type checked; after all, type checking takes place before the linker kicks in. Finally, the section presents a modified CESK machine, which creates simple proxy values when a class from a *tmodule* is instantiated. This proxy value monitors all interactions between

Typed Racket was the first language to implement a complete solution.

the object and its use contexts, and when a type mismatch is discovered, it causes the machine to halt execution.

5.1 Linking in Mixed-Type Setting

The goal of wrapping all instances of typed classes seems straightforward at first until we consider the import of *untyped* classes into typed modules. Although these classes do not come with a type, the importing module imposes one anyway, thereby implicitly creating a typed class whose instances may or may not live up to the type specification. Hence the semantics should wrap those instances, too, just to make sure they do not cause a type mismatch during execution. By implication, the language implementation must convey this information about these implicitly typed classes to the semantics.

Since the linker is the bridge between the type checker and the semantics, one natural solution is to modify linker so that it conveys the needed information. To differentiate this new linker from the one in chapter VII, we call it a *sound linker*. Its purpose is to translate a well-formed and valid *Mixed* system into a well-formed and valid *Classy* program, using the old linker pass plus two new ones:

1. For every occurrence of a *timport* specification, a *synthesis* pass generates a new typed module from the named untyped one:
 - For an occurrence of `(timport M Shape)` in module *K*, this pass creates a typed copy of *M* named *M.into.K* annotated with *Shape* and replaces the import specification in *K* with `(import M.into.K)`.
 - For an occurrence of `(timport M Shape)` in the system's body, this pass creates a typed copy of *M* named *M.into.Body* annotated with *Shape* and replaces the import specification in the system's body with `(import M.into.Body)`.

Constraint These names make sense only because this synthesis pass may assume the AST is well-formed and valid. It would be a problem if the system contained an untyped module named *Body*. So, *do not use Body as a ModuleName*.

2. A *typed-classes* pass transfers the types from *tmodule* ASTs to the ASTs of class definitions.

The modified CESK machine of section 5.2 will use these type annotations in class ASTs to monitor all of their instances.

Exercise 94. Design and implement the *synthesis* pass so that it adds the generated modules to the given *Mixed* system according to the specifications in this section. Keep in mind that the given AST is well-formed, valid, and type-checked.

Exercise 95. Design and implement the *typed-classes* pass according to the specifications in this section. It consumes the output of the *synthesis* pass from exercise 94.

Exercise 96. Adapt the linker from exercise 73 so that it works for the ASTs generated by the *typed-classes* pass exercise 95.

Note You might wonder why the model’s type checker cannot just validate that the types in *timports* work for the untyped modules. While this kind of fix would work for the *Mixed* model, doing so would create a completely false impression of how languages such as Typed Racket function. To keep our model reasonably honest, we present the chosen approach.

Stop! Why does Typed Racket not use this approach?

The creators of Typed Racket consider existing untyped code as “untouchable,” mostly because such code may exist in compiled form only. While it is not impossible to type check such code, the type checker would not be able to report conflicts between types and expressions in a sensible manner. Hence, programmers would have a hard time repairing the superimposed type or, worse, repair the code of the untyped class if they consider the type to be correct.

5.2 A CESK Machine Based on Proxies

A sound semantics for *Mixed* monitors instances of typed classes in the *Classy* program that the linker produces. Monitoring means inspecting values that flow into or out of objects via field access, field modification, or method calls. When some code retrieves the value of field *f* from an instance of typed class *C*, monitoring ensures that the value conforms to the type of *f* in *C*’s type. Similarly, when some code calls method *m* of an instance of typed class *C*, the monitor checks whether the arguments and the result conform to the domain part of *m*’s type signature in the type of *C*.

Let’s enumerate the three ideas that inform the adaptation of the CESK machine from chapter VI to *Mixed*:

1. An object is monitored if it is an instance of any typed class, type checked or not.
2. A monitoring semantics intercepts values as they flow into or out of a monitored object through any channel.

3. An intercepted value must conform to its type specification, meaning the semantic model must come with a *conformance* relation.

The remainder of the section introduces these ideas one at a time.

Note A moment's thought suggests that the second idea is too strict. Consider a method call whose target object is an instance of a type-checked class. The purpose of type checking is to guarantee that the method's code is in sync with its signature; in particular, the method returns a value that has the specified type, assuming the given arguments conform to their respective specifications. Hence, it should suffice to intercept only the arguments to such a method and to check their conformance. A similar argument could reduce the number of interceptions for field mutation and reference. While the *Mixed* model uniformly intercepts all values to keep things simple, the Typed Racket implementation reduces the number of interceptions as much as possible.

Your choice of data representation for objects in the CESK machine includes a reference to the class of which it is an instance. Hence, the machine can always determine whether an instance belongs to a typed class, because the sound linker adds types to the ASTs of classes. However, since the modified CESK machine must retrieve types frequently, it is best to combine objects with types when they are created from a typed class. Following the Typed Racket creators' terminology, we call this combination a *proxy*. The modified CESK machine considers proxies as values and monitors their interaction with other code.

Exercise 97. Design a data representation for proxy values, which combine objects with the types of their classes. See exercise 27 for the data representation of objects. Include the functionality for retrieving the types of fields, domain types of methods, and range types of methods.—Revise the definition of the set of values.

Figure 80 presents the machine transitions of the modified CESK machine for creating new instances from classes:

1. If the class comes without a type and the `new` expression supplies the correct number of variables as arguments, the machine retrieves the values of these variables and creates a regular object.
2. If the class is typed, the machine retrieves the values again and, additionally, makes sure that these values conform with the types of the class's fields. Instead of just creating an object, the machine forces

Control	Environment	Store	Kontinuation
evaluate a 'new' expression (untyped class, success)			
before: (new C (x* ...)) e		s	k
subject to: class C is untyped and has the same number of fields as in x* ...			
after: obj	e	s	k
where v*, ... = s[e[x*]], ...			
and obj = create instance of C using v*, ... for the values of the fields			
evaluate a 'new' expression (typed class, success)			
before: (new C (x* ...)) e		s	k
subject to: class C is typed and the values of x* ... conforms to its field types			
after: prx	e	s	k
where v*, ... = s[e[x*]], ...			
and obj = create instance of C using v*, ... for the values of the fields			
and typ = the type of class C			
and prx = make obj conform to type typ			
evaluate a 'new' expression (failure)			
before: (new C (x* ...)) e		s	k
subject to: class C is typed and			
... x* ... fails to conform to its field types			
... or is untyped			
... and x* ... is of incorrect length			
after: error	[]	[]	[]

Figure 80: The proxy-based CESK machine: creation

the object to conform to the class's type, which in this case means it combines the two into a proxy.

3. Finally, if neither condition holds, the CESK machine signals an error.

Given an object, a program can check whether it is an instance of a particular class. While the top half of figure 81 displays a transition rule that looks just like one from the original CESK specification (see chapter VI), it differs in a subtle but critical manner. And, understanding this difference takes some effort.

The *Mixed* system in the bottom half of figure 81 illustrates the subtlety of *isa* in this setting. While the left column displays the modules of this

Control	Environment	Store	Kontinuation
evaluate an 'isa' expression			
before: (o isa C)	e	s	k
after: ans	e	s	k
where obj	= s[e o]		
and ans	= check whether obj is instance of C		
the modules		the system body	
(module Untyped		(import Typed)	
(class U ()		(timport Untyped	
(method m(o) o)))		((	
(tmodule Typed		((m (Number)	
(timport Untyped		Number))))	
((		(def u (new U()))	
((m (Number)		(def t (new T()))	
Number))))		(t --> m(u))	
(class T ()			
(method m(o) (o isa U)))			
((			
((m ((() ((m (Number) Number))))			
Number))))			

Figure 81: The proxy-based CESK machine: inspection

Mixed system, the right one shows its body. Note how the framed timport specifications bring the untyped class U from module Untyped into the scope of Typed and the system’s body respectively—attaching the exact same type.

Stop! What do you expect as the outcome of this system? Although the system body seems to create an instance of U at the same type as module Typed expects, running the program yields 1.0, meaning that u is *not* considered an instance of U inside of method m in class A. Why?

With the introduction of synthetic modules and proxy instances created from their classes, the semantics also adds error behavior. That is, depending on context, an interaction with an instance of a class from a synthetic module may signal an error due to the superimposed type. In our running example, two distinct import specifications for Untyped can attach two distinct types. Hence, one instance may behave properly when it inter-

acts with a piece of code, while the other would signal an error during an interaction with the exact same piece of code. It is therefore necessary to distinguish instances of two classes from each other, even if the source of these classes is one and the same untyped module.

Exercise 98. Work out the *Classy* program that the (adapted) linker should produce. What would the names of *U* in *Typed* and the system's body be, respectively?

Exercise 99. Design the auxiliary function needed to determine whether some potentially proxied object is an instance of some class.

Control	Environment	Store	Kontinuation
evaluate a field reference (proxy)			
before: $(p \rightarrow f)$	e	s	k
subject to: p denotes a proxy and the value of the field conforms to its type			
after: v	e	s	k
where prx	$= s[e\ p]$		
and $[obj, fieldType]$	$=$ extract object and type of field f from proxy prx		
and u	$=$ get value of f from obj		
and v	$=$ make u conform to $fieldType$		
evaluate a field reference (failure)			
before: $(o \rightarrow f)$	e	s	k
subject to: o does not denote an object/proxy with the correct field property			
after: error	$[]$	$[]$	$[]$
where obj	$= s[e\ o]$		

Figure 82: The proxy-based CESK machine: field reference

Figure 82 presents a pair of new cases of the CESK transition function: the successful reference to a field of an object and the revised failure case. When the *C* register contains a field access expression, the machine also checks whether the value in the named field conforms to the type specified in the proxy. If so, it retrieves the value, enforces conformance, and places the conforming value into the *C* register. Regardless of the nature of the target, if the field is missing or the value in the field of a proxy does not match the specified type, the CESK machine signals an error.

Stop! Take a close look at the formulation of the precondition in the success case of figure 82 and the last line of the *after* state formulation.

While the precondition says “the value conforms to its type,” the *after* state uses the strong language of “make the field value conform to its type.” The second formulation indicates that conformance checking is not just an up or down vote on the relationship between a value and its specified type.

Indeed, checking whether some given value *conforms* to a type specification has *three* possible outcomes:

1. If the value matches the type, the value is returned.
2. If an object *might* live up to a type specification in the future, the conformance function creates an appropriate proxy from the object and the type.
3. Otherwise, the given value—a number, an object, a proxy—does *not* conform to the given type.

Figure 83 spells out the six scenarios via a two-dimensional table. The horizontal direction enumerates the three kinds of values that the conformance checker might encounter; the vertical one lists the two kinds of types.

if the value is:			
	a number <i>n</i>	an object <i>obj</i>	a proxy <i>prx</i>
REAL	use <i>n</i>	no	no
Shape <i>S</i>	no	combine <i>obj</i> and <i>S</i> into a proxy	if the type in <i>prx</i> is equal to <i>S</i> : use <i>prx</i> else: no

Figure 83: A value conforms to a type, if ...

Two of the cases in figure 83 deserve a detailed explanation:

- When the given value is a proxy and the type is some Shape, the conformance checker must compare types, i.e. the CESK machine checks types for equality at run time. This check happens only in the model; Typed Racket does *not* retain types beyond the type-checking step.
- When the given value is a plain object and the type is some Shape, the object may live up to the expectations. But obviously, it is impossible to know at this particular point whether the rest of the computation is going to interact with the object in an appropriate manner. Hence, the machine combines the object and the type into a proxy and continues from here.

The conformance could also check some “first order” properties, such as whether the set of method names are the same.

Control	Environment	Store	Kontinuation
execute a field assignment (proxy)			
before: u	e	s	$\langle\langle \text{[]}, (p \rightarrow f = \text{rhs})::\text{stmt}^*, r \rangle\rangle$
subject to: p denotes a proxy and the value of the field conforms to its type			
after: †	e	s	$\langle\langle \text{[]}, \text{stmt}^*, r \rangle\rangle$
where prx = s[e[p]]			
and [obj, fieldT] = extract object and type of field f from proxy prx			
and v = make u conform to fieldT			
and set f in obj to v			
execute a field assignment (failure)			
before: v	e	s	$\langle\langle \text{[]}, (o \rightarrow f = \text{rhs})::\text{stmt}^*, r \rangle\rangle$
subject to: o does not denote an object/proxy with the correct field property			
after: error	[]	[]	[]
where obj = s[e[o]]			

Figure 84: The proxy-based CESK machine: field mutation

Let's study field mutation next. Unlike field reference, which extracts a value from an object, field mutation injects one. According to our agreement, the CESK machine must intercept this value and inspect it, if the target object is proxied. Like for field references, this constraint requires one new case for the transition function and one modified one:

- If the target of a field mutation is a proxy and the value in the C register conforms to the field's type, then the CESK machine assigns a type-conforming value to the field of the object representation.
- If the field is missing or the value to be stored in the field of a proxy does not match the specified type, the CESK machine signals an error.

See figure 84 for these two cases of the revised transition function.

A method makes up a two-way street for interactions between an object and its context. On call, a method consumes values. On return, a method produces values. In both cases, these values have to conform to the types specified in a proxy.

Let's consider each direction separately, starting with method calls. The rules are spelled out in figure 85. A method call whose target is a proxied object may proceed if the arguments conform to the domain part of the

Control	Environment	Store	Kontinuation
evaluate a method-call expression (proxy)			
before: $(p \rightarrow m(x^* \dots))$	e	s	k
subject to: p denotes a proxy and the arguments conform to the method type			
after: $\uparrow$	$e1$	$s1$	$\text{push}[cl, \text{push}[rangeT, k]]$
where prx	$= s[e[p]]$		
and $[obj, domainT, rangeT]$	$=$ extract the object and the type of method m from prx		
and $tmp^*, \dots$	$= s[e[x^*]], \dots$		
and $arg^*, \dots$	$=$ make the $tmp^* \dots$ values conform to the $domainT$ types		
and $[para^*, body]$	$=$ the parameters & body of method m per the class of obj		
and cl	$=$ closure: body in e		
and $thisL$	$=$ a new (relative to s) location		
and $xl^*, \dots$	$=$ as many new (relative to s) locations as elements in $x^*, \dots$		
and $e1$	$= [[this = thisL][para^* = xl^*], \dots]$		
and $s1$	$= s[thisL = obj][xl^* = arg^*], \dots$		
evaluate a method-call expression (failure)			
before: $(o \rightarrow m(x^*))$	e	s	k
subject to: o does not denote an object/proxy with the correct properties for this call			
after: error	$[]$	$[]$	$[]$

Figure 85: The proxy-based CESK machine: method calls

method's signature. If these conditions hold, the CESK machine retrieves the proxy and extracts from this proxy the object, the domain types, and the range type. At this point during execution, the machine can make the arguments conform to the domain types. Using the resulting values, the CESK proceeds in a manner that is similar to regular method calls.

Stop! Take a close look at the *after* state of this case in figure 85.

Note how the K register of the *after* state contains a continuation that comes with *two* new frames. The top-most one is the usual closure. But the second from the top is just a type. It is at this point in the evaluation of a method call that the CESK machine knows to which return type a potential return value must conform, and it must store this type in the continuation so that conformance can be checked upon method return. And this brings us to the final two rules.

Exercise 100. Explain the second case in figure 85.

A method return takes place when the C register contains a value and

Control	Environment	Store	Kontinuation
returning from a proxied method call (success)			
before: v	e	s	$(\text{type: rangeT})::k$
subject to: value v conforms to rangeT			
after: rv	e	s	k
where $rv = \text{make } v \text{ conform to type rangeT}$			
returning from a proxied method call (failure)			
before: v	e	s	$(\text{type: rangeT})::k$
subject to: value v does not conform to rangeT			
after: error	$[]$	$[]$	$[]$

Figure 86: The proxy-based CESK machine: method returns

the top of the K register is a type. This arrangement is exclusively due to a call that targets a proxied object. Figure 86 displays the two corresponding cases of the revised CESK transition function:

- The first case deals with a return value v that conforms to the specified type. If so, the CESK machine forces v to conform and places the resulting value rv into C all while popping the continuation in K.
- The second case concerns failure. If the return value fails to conform to the specified type, the CESK machine signals a failure.

The introduction of a return-from-method frame into the set of continuation frames has a serious consequence. As section 4 in chapter VI demonstrates, a method that tail calls consumes no continuation space. For the proxy-based CESK machine, this property no longer holds. A self-calling method from a proxied object is going to grow the continuation one frame per call, meaning in real-world implementations such methods may run out of stack space.

5.3 Project Tasks

Exercise 101. Besides the new and revised transition rules presented in this section, the proxy-based CESK machine also needs a revised function for checking the structural equality of values. After all, the addition of proxies to the set of values clearly calls for a revision.

Revise your function for determining the structural equality of values so that it unwraps the objects hidden inside of proxies. See exercise 30.

Exercise 102. Revise the data representation from exercise 29 to accommodate proxies.

Exercise 103. When the proxy-based CESK machine encounters objects, they are instances of untyped classes. Hence interactions with these objects may go wrong just like they did in chapter VI.

Restore the run-time checks to the CESK machine that you eliminated in response to exercise 75.

Exercise 104. Revise and extend the transition function from exercise 40 so that it properly works with proxies.

Exercise 105. Adapt the main function from exercise 74 so that it can run the entire process of parsing a *Mixed* program, validating it, type-checking its typed portions, soundly linking the module-based program into a *Classy* one, and determining its meaning on the CESK machine with proxies. Use your solutions from exercises 87 and 88 and exercises 94 through 104.

The function should issue error strings in the following order:

- "syntax error" if the parser discovers any mistakes;
- "duplicate module name" if the validity checker finds two modules with the same name;
- "bad import" if one of the *Mixed*-specific validity constraints concerning imports is violated;
- "duplicate name error" if the validity checker encounters two classes with the same name, a class with two identical method names, or a class with two identical field names;
- "undeclared name error" if the validity checker encounters an undeclared `ClassName` or variable name;
- "type error" if the type checker encounters a type violation; or
- "runtime error" if the CESK transition function stops due to a final state with an error in its C register.

If running the machine delivers a value, it will be a number. Why? The main function prints this number. Why?

Exercise 106. Come up with an example system in *Mixed* that creates instances of type-checked and classes from untyped modules. Make sure these instances interact with each other.

6 The Very Final Bit of Theory: Real Type Soundness

Chapter VIII.4.1 explains which run-time checks in the CESK transition function are superfluous *if* the relationship between the type checker and the transition function satisfies a type soundness theorem. This chapter raises the question of whether such a theorem can hold for *Mixed* or the actual language implementations that *Mixed* models.

The answer may come as a bit of a surprise:

Language models that mix typed and untyped code are type sound to a degree, not in an absolute sense.

Put differently, “type soundness” theorems for modules like *Mixed* form a spectrum. Depending on what kind of conformance checks they impose, they catch some or all type mismatches and, if they catch them, they catch them at different times during program evaluation. The very fact that this chapter presents *two* semantics for *Mixed* is proof of this claim. While the semantics of section 4 checks *none* of the interactions between typed and untyped code, the semantics of section 46 checks *all* interactions.

One consequence of this lack of complete type soundness has implication for the run-time checks in CESK machines. They cannot be removed, which is why exercise 103 has you restore those removed in the preceding chapter. Another consequence concerns pragmatics. Run-time checks exist to catch type mismatches, which programmers introduce accidentally. And this brings us to the pragmatics of *Mixed* and its stark semantic choices.

They protect partial primitives such as division. We ignore those.

Note On reflection, you may realize that real language implementations are closer to the *Mixed* model than the models of the preceding chapters. All languages mix type-checked and unchecked code. For example, the latter come in the form of run-time libraries that access devices, and those libraries tend to use languages with few protective features. Hence the use of the phrase “type soundness to a degree.” For the two models in this chapter, it is the most realistic characterization a language researcher can provide, even if we ignore the distinction between models and implementations with bugs.

7 Pragmatics: Migratory Type Systems

The very same language with a migratory type system may come with a range of different semantics. An implementation can choose from two radically different options:

- to erase the types and just run the resulting untyped code; or
- to use run-time checks to enforce types at the boundaries between typed and untyped code.

Each choice of semantics answers questions of pragmatics differently and thus has major implications for the work of software developers.

Concretely, the key questions are why an implementer would choose

- to erase the types and thus fail to inform developers about type mismatches, or
- to use run-time checks and to impose the cost of these checks on the code that developers deploy.

Stop! Take a look at figure 87 and identify the work situations to which these questions allude.

work situation	programming with a migratory type system	
	with type enforcement	with type erasure
design code		
...	...	...
— migrate		
testing		
debugging		
deployed code		
— bug diagnosis		
...	...	

Figure 87: Work situations and the role of types systems

7.1 The Two Semantics with Concrete Examples

Before we discuss answers to these two questions, let’s consider an illustrative and comparable example from two real-world implementations of *Mixed*: TypeScript combined with JavaScript, and Typed Racket combined with Racket. While their type systems resemble each other, their semantics roughly correspond to the two presented here.

Figures 88 and 89 present the same program in the two combinations. Both figures display two columns, with one file each. The left column is a

typed module for managing (simplistic) bank accounts; the right one shows a naive client module. Each of these client modules interacts with the typed module in two ways:

- The first function call to `deposit` requests a type-correct addition of 100 to the current balance.
- The second function call applies `deposit` to " pennies!", a string. Obviously this application causes a type mismatch, because `deposit` expects a number or `Natural`, respectively.

bank.ts	client.js
<pre>// balance in \$\$s var blnc = 0; // add `amt` in \$\$s to `blnc` export function deposit(amt: number) { blnc += amt; } export function printBlnc() { console.log("balance: \$" + blnc); }</pre>	<pre>var Bank = require('./Bank.js'); // correct interaction Bank.deposit(100); Bank.printBlnc(); // incorrect interaction Bank.deposit(" pennies!"); Bank.printBlnc();</pre>

Figure 88: A TypeScript/JavaScript type mismatch

Stop! Inspect the two figures and make sure you understand the code and the two calls in the client module before reading on.

Following the presentation of the two semantics, the two combinations behave differently. While the TypeScript/JavaScript combination prints

```
balance: $100
balance: $100 pennies!
```

to the console, the Typed Racket/Racket system signals an error after printing the balance in response the first function call:

```
balance: $100
deposit: contract violation
```

bank.rkt	client.rkt
<pre>#typed/racket (provide deposit printBlnc) ;; balance in \$\$s (define blnc : Integer 0) ;; add `amt` to \$\$s to `blnc` (define (deposit amt : Natural) (set! blnc (+ blnc amt))) (define (printBlnc) (println (~a "balance: " blnc)))</pre>	<pre>#lang racket (require "bank.rkt") ;; correct interaction (deposit 100) (printBlnc) ;; incorrect interaction (deposit " pennies!") (printBlnc)</pre>

Figure 89: A Typed Racket/Racket type mismatch

```
expected: natural?
given: " pennies!"
in: the 1st argument of
  (-> natural? any)
contract from: bank.rkt
blaming: client.rkt
(assuming the contract is correct)
```

The error message deserves some explanation. It informs the developer about the type mismatch as the failure of “pennies!” to satisfy `natural?`, the predicate that enforces the type `Natural`. Furthermore, it explains which part of the function signature the value violates and which boundary between typed and untyped code causes the problem. Do note the last line, which is a warning to the developer and which is important when it comes to the rationale behind the choice of enforcing types.

Note In general, a mixed TypeScript/JavaScript program does not have to terminate properly when a type mismatch occurs. Like our CESK model, the JavaScript machine does perform some checks that protect primitive computational operations. Hence these checks may *eventually* catch a type mismatch because some inappropriate value flows into a run-time check, but they don’t have to.

As the example points out, it performs fewer checks and thus fails to “protect” addition from strings.

7.2 Performance

Performance matters. And every run-time check has the potential to affect a program’s performance.

After type checking, the TypeScript implementation compiles the given source code to plain JavaScript, mostly by stripping the types. Since users of JavaScript have experienced tremendous performance gains over the past decades, they consider it a performant language. If the TypeScript implementation were to add run-time checks, a developer's work might suffer in two ways:

- First, the translated TypeScript code would have to pay for the run-time checks during execution. The developer would have to measure whether the cost of these run-time checks is acceptable and, if not, conduct a performance-debugging session.
- Second, as is, TypeScript code carries performance "on its sleeves." An experienced JavaScript developer can "see through" the TypeScript source code and understand it in terms of the corresponding JavaScript code, including its performance. If the compilation injected run-time checks, this direct correspondence would be severely distorted.

*Plus readers who
remember
chapter VIII.4.*

Experienced language creators may object to this analysis. They understand that type-checking plus a soundness theorem enable language implementers to eliminate run-time checks that ensure the proper working of computational primitives. Conversely, they may wonder why type checking the typed portions of a mixed system does not yield similar benefits.

In the case of JavaScript, the answer is that since misbehaving untyped values may show up inside of TypeScript portions of mixed systems, the translation is cannot rely on type checking. Fortunately, JavaScript implementations typically observe the kind of values that flow through a program, recognize where it can eliminate the machine's checks, and re-compile the code accordingly while the program runs.

In the case of Typed Racket, the answer differs. The Typed Racket implementation *can* rely on the type checker because run-time check guarantees that the validated logical properties hold during program execution. As a result, programs written in *just* Typed Racket do benefit from type checking just like programs written in a statically typed language. Even mixed systems written in both Racket and Typed Racket exhibit improved performance on occasion.

In some situations, however, Racket/Typed Racket combinations exhibit steep performance losses. The creators of the language have reported slow-downs at the level of many orders of magnitude for object-oriented mixed systems, while mostly-functional code seems to suffer from a mostly

reasonable cost for the run-time checks. At this point in time, it is not clear whether these costs are intrinsic to languages with somewhat-sound migratory type systems or whether different implementations of the underlying untyped language can eliminate most of the cost of run-time checks. Research on this topic continues and the outcome is wide open.

7.3 Testing, Debugging, and Deployment

Discovering type mismatches matters. And every failure to discover a type mismatch or every delay in the discovery process affects the working software developer in several different situations, including testing, debugging, and system deployment.

As section 7.1 demonstrates, mixed TypeScript/JavaScript systems such as the one from figure 88 do not discover the type mismatch between the string "pennies!" and the type number. The result is an unexpected outcome. If this example were a test case, the developer would add an expected outcome and an automatic comparison, and the test-case failure would point out a problem.

What just this simple example shows then, are three obvious consequences for the developer's work:

- The lack of run-time tests shifts work from the language creator to the software developer. It becomes the task of the latter to write tests that catch type mismatches.
- A failing test case provides less information than the failure of a run-time check that identifies the source of the type mismatch and a witness value. It is now the developer's problem to find this boundary and ideally to determine which value failed to match the type.
- Finally, a type mismatch may also affect a system's behavior after it is deployed. In this case, it is the user who suffers. And, if this user reports the problem, the developers assigned to this bug have even less information than a failing test case. As a matter of fact, they will first have to develop a test case that reproduces the failure and then use this test case to start the search for the type-mismatch error.

Stop! Identify the cells in the table of figure 87 that match the three consequences. Can you think of additional consequences?

By contrast, the Typed Racket/Racket code from figure 89 signals an error as soon as it discovers the type mismatch accompanied by an informative message. At first glance, this message should assist a developer with

testing, debugging, and even with failures in deployed software. Research by the authors suggests, however, that these appearances are deceiving.

When a type does not match a value, it is possible that the type is wrong, the value is wrong, or both are wrong. And as it turns out, the information in the error messages seems to help in different ways, depending on what is the case:

- If it turns out that the type signature is mistaken, the information from Typed Racket's failing run-time checks seem to help developers quite a bit with the debugging process. As mentioned already, problems with *post hoc* type signatures are common in TypeScript's *Definitely Typed* code repository. So this research result points in a helpful direction.
- If the problem is due to a bug in the code that is equipped with types on a *post hoc* basis, the information seems less helpful. More precisely, the safety checks of Racket's version of a CESK machine already catch many type mismatch problems, and the information that they supply appears to be equally helpful.

In short, the pragmatics of testing and debugging in a language with a migratory type systems poses interesting research problems, and the outcome is wide open.

What happens as a program allocates more and more memory?

1 The Pragmatics Question

Real programs continuously request memory locations so they can store elements of data. Real computers come with a large but finite number of memory locations. Thus, when a program keeps requesting memory locations, it eventually exhausts the number of memory locations that the operating system is willing to allocate. If this actually happens, the operating system must terminate the program so that it does not harm other programs. The question then is whether a programming language can help developers with this problem and whether it should help.

Some programming languages indeed reclaim memory locations as the program runs. Every so often, the underlying machine analyzes the program's use of locations and somehow determines that some of them can be reused. That is, when some instruction requests memory after this analysis is done, the machine re-allocates locations that currently contain program data. The key is that the analysis ensures that this reuse has no visible effects on the remainder of the computation, or as we say the continuation.

The creators of other languages decided that it is the programmer's task to determine whether, and when, a memory cell can be reused for a different purpose. When a developer team must use such a language, it is responsible for instructions that request memory locations and that return them to the pool of available ones. In a sense, the team must conduct an analysis like the one mentioned in the preceding paragraph but while they design code and without (much) support from the IDE.

In short, language creators understand the difference between models and real systems, which comes with a finite amount of memory. They make a design choice as to whether they want to equip their language with a mechanism that helps programmers with memory management or not. We

And, colleges that teach with such languages must also teach analysis strategies for reclaiming memory.

want to understand the design alternatives that language creators face and the costs and benefits of each choice.

Note One special case, the Rust programming language, has gained some traction over the last few years. While it makes developers manage location requests on their own, its type system assists them with an analysis of location life times. For brevity, we mention this case to illustrate how wide the design spectrum is for just this one aspect of language design, but the remainder of the chapter ignores it.

1.1 The Nature and Purpose of this Chapter

This chapter differs from the preceding ones. Instead of presenting full models of memory management, it focuses on just the design choice, the cost/benefit analysis of the two major alternatives, and the pragmatics of memory management. As such, the chapter serves as an example of how a developer might conduct research on a question of pragmatics by diving into the literature and studying the role of a feature in the full spectrum of work situations—without building complete models.

A second purpose of this chapter is to highlight how prejudices can prevent people from reaching a rather obvious conclusion. Like all scientific communities, conflicting personalities push conflicting insights. This truism definitely applies to programming language research, and memory management serves as an illustrative example. Over many decades, programmers created and fostered a prejudice about automatic memory management. When a researcher finally performed the work of actually measuring the two major design alternatives, the results he found surprised him and everyone else. As a matter of fact, the results surprised people so much that the researcher had significant difficulties publishing them.

We would like our readers to understand this story so that they will approach future claims about programming language pragmatics with the necessary caution.

2 Design Choices

The CESK machine allocates locations for two distinct purposes: the declaration of variables and the creation of objects. While the former manifests itself in the actual instructions, the latter remains hidden due to our design choices in chapter VI. Furthermore, each of these two forms of allocation

imposes a different kind of cost on language creators and software developers. To clarify these points, the first two subsections present explanations of allocation for these two purposes. Based on these two, the remaining subsections sketch the available design choices and the long story of a cost/benefit analysis of these choices.

2.1 Memory from the Point of View of a Model, Take 1

Let us take a look at the problem in the context of our machine model of *Classy*. In the context of this model, we refer to memory as “store.” A store associates locations with values, and the machine requests locations in only one case: for entering a scope. Variables declared in this scope point to these locations and thus to their content. When execution exits the scope, none of the remaining program instructions can refer to these variables nor the corresponding locations.

Since a scope is region of program text with well-defined entry and exit points, we could modify the CESK machine to match the allocation of locations on entry with de-allocation actions on exit from the scope. Adding such de-allocation actions to the CESK transition function would (1) solve this part of the finite-memory problem and (2) complicate this pedagogic model of a machine. Hence we skipped this part.

From the 1950s through the mid 1990s, language creators considered this form of memory management the only proper form of *automatic* management. They implemented real-world languages so that the environment stack would be in sync with the blocks of locations allocated per scopes. Pushing on this stack on entry and popping from the stack on exit made this automated memory management fast. Everything else in terms of memory management was left to the programmer.

2.2 Memory from the Point of View of a Model, Take 2

In the past, “everything else” meant dynamic allocation and that, in the context of an object-oriented language meant the creation of objects.

For the semantic model of chapter VI objects in the CESK machine come with data representations of your choice in your favorite implementation language. An alternative, language-agnostic approach is to represent an object via a block of locations in the store of the CESK machine. Figure 90 presents this choice with a diagram. It assumes that a program defines a class *C* with three fields, and that the program creates an instance of the class via `(new C(x y z))`. Furthermore, *x*, *y*, and *z* stand for the values *f-v-1*,

f-v-2, and f-v-3, respectively. To accommodate such an object in the store, the machine allocates four locations: one for identifying the class C, plus three for its three fields. The meaning of the entire object—its value—is the location's name: P. For historical reasons, P is called a *pointer*.

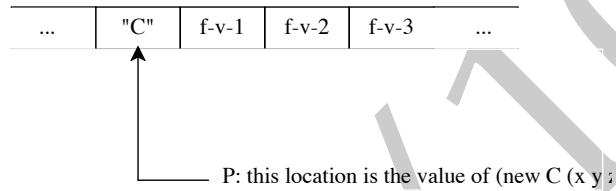


Figure 90: Allocating all pieces of an object in the store

Stop! Can you see a problem with this decision to identify the value with the location's name?

Descriptions of computers tend to say that the name of locations are numbers, natural numbers to be precise. Indeed, they identify the name with the number. None of the presented models rely on this choice, but if you made this choice for your implementation, you might end up suffering from it, if you were to implement this alternative way of representing *Classy* objects as data in your language.

As section 5.1 of chapter VI explains, the *dynamic* extent of an object—its lifetime—is not related to program text. By contrast to scope-based allocation, it is thus impossible to determine when to de-allocate these locations. It is due to this insight that language creators speak of *dynamic* allocation for objects and *static* allocation for scope.

Note Seen this way, the management of locations once again illustrates the nature of models. On one hand, the choice of never de-allocating locations greatly simplifies the semantic model. On the other hand, a juxtaposition with the real world's constraint of finite computer memories exposes it the truly unrealistic nature of the model.

2.3 Manual vs Automatic Memory Management

Language creators use two approaches to deal with dynamic allocations and de-allocations of locations^x. They refer to both approaches as memory management:

- Languages with *manual memory management* leave it to the software developer to cope with allocation and de-allocation of locations. They provide computational primitives for expressing both ideas, often called `malloc` and `free` or `constructor` and `destructor`, respectively.
- Languages with *automatic memory management* support de-allocation in a variety of ways. The two most important ones are the following:
 - A reference-counting arrangement associates another location with each object representation, namely, a count of how many times it is pointed to. When this counter reaches 0, the locations associated with the object become available for reuse.
 - Many modern languages come with a garbage collector, a software component that the underlying machine runs every so often to reclaim locations. Garbage collectors come in a conservative form, in stop-the-world form, in a concurrent form, and several others. Since its invention in the 1960s, garbage collection has become its own research topic, located somewhere between programming languages research and systems research.

In order to assess the costs and benefits of the approaches and to understand the history behind it all, we need to refine these overly concise descriptions.

To get started, let's define garbage location using the terminology of a CESK machine that uses pointers to represent *Classy* objects; remind yourself of the arrangement with a quick look at figure 90. A pointer `P` and its associated sequence of locations in the machine's store is *garbage* at a point during execution if replacing its content—the class of the object—with 666.0 does not change the behavior of the remainder of the program execution. Determining whether a pointer is garbage is an undecidable problem. But, if we can somehow show that a specific pointer in a CESK machine state is garbage, then it and its associated locations can be reused when the machine must allocate locations during the execution of the continuation.

Next, we need to gain an improved understanding of the store. Up to this point in the book, we treated the store as an association of locations with values. An alternative view is that the store is a directed graph, whose nodes are locations and whose edges are pointers. A classification of the nodes is key to understanding de-allocation:

- sinks are locations that contain numbers; and
- sources are accessible pointers to objects.

Based on this alternative view, language creators design garbage collectors as graph-traversal tools that compute an approximate set of garbage locations and thus enable the machine to reuse them.

Here is the precise idea. The store is divided into two parts: used now and free for future use. Every so often—yes, think randomly!—the CESK machine stops interpreting program instructions and instead runs the instructions belonging to a garbage-collection algorithm. Its first task is to find the sources of the graph, also called *roots*. To this end, it creates the set of all variables mentioned in the C register and in the domains of the environments in the E and K registers. The store in S associates every one of these variables with a location. And this set of locations makes up the set of sources.

Given the sources of the graph in S, the garbage collector traverses the graph and thus determines all *reachable* locations. This set contains those locations that the remaining computations *may* have to access. Notice the “may” in the preceding sentence, which emphasizes that this way of searching for non-garbage locations, also known as *live* locations, *approximates* the actual notion of garbage. One way to use the set of live locations is to copy their content into the free part of S, meaning the garbage locations are left behind and become the next free part of the store S. Thus, the algorithmic cost of this approach is a function of the number of live locations. Garbage locations do not impose any cost at collection time.

Stop! Can this graph traversal work if the set of locations overlaps with the set of numbers that show up in store locations?

Note An alternative way to find live location is to use inference of static types. This approach does not use a graph traversal, and it finds different sets of live locations than a graph traversal. Since this book does not cover type inference, we simply point out the idea and leave it at that.

The answer to the above challenge is “yes.” When the garbage collector reaches an interior node, it must check each of the content locations and decide whether it contains a numeric value or a location. Each of the latter must be added to the set of live locations and must thus become a quasi-source of the garbage-collection algorithm. Surprisingly this idea is critical for the cost-benefit analysis of memory management.

2.4 Costs and Benefits

Each form of memory management comes with its own costs and benefits and thus deserves a separate cost/benefit analysis.

That is, a graph traversal is a decision procedure for a weaker problem than the garbage location problem.

From an algorithm perspective, the cost/benefit analysis has some surprises in store:

- While the `free` function merely marks the locations associated with a pointer as available, the `malloc` function has to perform a significant amount of work. Since objects come in different shapes and sizes, a store becomes fragmented as locations in the middle of live locations get marked as available. As a result, when `malloc` is asked to place an object into the store, it must search for a block of available locations that can accommodate the given object. Fragmentation can get quite bad over time, and hence `malloc` may have to spend a significant amount of time on this search.
- At first glance, a reference counting schema imposes a low cost: increase the counter of object `o1` if some new or modified object `o2` points to `o1` and decrease the same counter when freeing of `o2` or a mutation eliminates the cross reference between the two. On modern hardware, though, these increases and decreases demand loading `o1` to get access to its counter, and such loading actions measurably slow down execution if, as is likely, `o1` or `o2` are located on distinct pages.
- Garbage collectors stop normal program execution to find the roots, traverse the graph, and create a new space of free locations. In the 1960s, these halts caused “embarrassing pauses” that observers easily noticed and made fun of. Nowadays hardware is sufficiently fast and the algorithms are sufficiently well developed that the pauses are barely noticeable. Even some real-time software systems—such as controls for missiles or rides in amusement parks—can rely on languages with automatic memory management.

Concurrent algorithms exist but are still rare.

Most developers are surprised to learn that garbage collection is the most efficient and effective choice in many situations. See below.

From a human perspective, the three design choices represent three different ways of distributing work between programming language creators and software developers:

- Providing `malloc` and `free` is an easy task for a language creator, but it does shift a lot of the intellectual work to software developers.

Concretely and as mentioned in the introduction of this section, the `malloc`/`free` combination forces developers to make decisions about the insertion of `malloc` and `free` into their code while the design programs. Unsurprisingly, this can cause major problems:

1. Using `free` too late.

If the software developer fails to use `free` on a location after it becomes garbage, the program is said to *leak memory*. That is, many seemingly allocated locations are actually garbage locations. Leakage can lead to slowdowns—see below—and may ultimately cause the operating system to shut down the underlying machine due to a lack of free locations.

2. Using `free` too early.

If the developer uses `free` on a location that is *not* a garbage location, the instruction creates a so-called *dangling* pointer. That is, some part of the continuation may refer to this location and, if it does, it retrieves the newly stored data instead of the old one. On actual hardware, this retrieval yields just a bunch of bits, which the machine may or may not be able to interpret as the old form of data. Either way, at the level of the chosen data representation, the computation results in nonsensical values. As such, they may turn into outputs that damage end-users, or they may cause havoc at some point during execution, imposing significant debugging work without any hints for the developer.

- Adding an accurate reference counter to each allocated object requires some additions to the functionalities of `malloc` and `free`. Even though these additions are relatively small, they do demand attention to detail and extensive validation.

In return, the software developers get some assurance that simple and hierarchical data structures get dealt with automatically. But, they still need to deal with cyclic object relationships on their own, because reference counting cannot cope with cycles.

- A garbage collector demands a serious amount of implementation work from the language creator. While a collector's essence consists of a straightforward graph traversal, making it performant requires a thorough understanding of the low-level data representations and good low-level coding and debugging skills.

When a programming language comes with a garbage collector, it mostly alleviates software developers from reasoning about allocation and de-allocation object-related locations. The exception is a developer who is seriously concerned with performance. In this case, it is imperative to keep track of how much memory the program al-

locates, because each allocation increases the need for a pause in the execution cycle to run the garbage collector at some point.

The two analyses indicate how much the choice of memory management can affect developers in various work situations. In order to answer the question of pragmatics, though, it also calls for more data concerning the performance of the various solutions.

Note: “Conservative” Part 1 Section 2.3 spells out two critical facts about garbage and automatic memory management. First, the problem of determining whether a location is garbage at any point during execution is undecidable. Second, a garbage collector must therefore over-approximate the set of live locations as it would otherwise create dangling pointers, endangering the integrity of the computation.

When an algorithm over-approximates, researchers call it *conservative*—except when it comes to garbage collection. See the next note.

When an algorithm over-approximates, logicians speak of “proof vs. truth.” As soon as a quantifier-based logic can express claims about the arithmetic of natural numbers with full addition and multiplication, the underlying truth is so complex that no proof system can discover them all. In a sense, this distinction is also the case with proving whether a store location is truthfully garbage or not.

This distinction appears many times in the theory of programming languages.

Note: “Conservative” Part 2 In the garbage-collection research community, the word “conservative” is reserved for the situation when a language creator has chosen to use the same set of bits for representing locations and natural numbers. All languages of the C family (C, C++, Objective C) and many others suffer from this problem. When a language creator has made such a design decision and implemented it, language researchers say the language is *non-cooperative*, meaning it does not cooperate with the garbage collector. In this situation, the collector faces two undecidable problems—whether a location is a garbage and whether a number is a data representation of a number—and therefore has to over-approximate the set of live locations even more than a cooperative collector.

This terminology and the collector are due to Hans-J. Boehm.

A collector that can cope with this situation is referred to as a *conservative collector*. The existence of conservative collectors plays a critical role in the assessment of the history and the pragmatics of memory management.

3 Pragmatics of Memory Management

Recall that pragmatics is about the support a language feature or tool offers to developers in various work situations. A pragmatics analysis calls for filling in a two-dimensional table such as the one shown in figure 91, a variant of the pragmatics table that several preceding chapters use. Here the feature columns specify manual memory management and its automatic pendant; the work situation column contains the usual entries.

work situation	manual memory mgt	automatic memory mgt
design code		
keyboard code		
— create		
— refactor		
— evolve		
— migrate		
correctness testing		
... and debugging		
performance testing		
... and debugging		
documentation		
deployed code		
— bug diagnosis		
— maintenance		
port to different PL		

Figure 91: Work situations and the role of memory management

To keep the analysis concise, let's focus on two rows in this table: "design code" and "performance testing and debugging." As the cost/benefit analysis indicates, a language without automatic memory management shifts programming work from the language creator to the software developer. The results of this shift are obvious, yet occasionally quite surprising.

When it comes to the *design of code*, the additional work for the developer is immediately clear. In principle, the developer may follow the usual code design guidelines, creating the code in a systematic manner first and ignoring the memory management problem during this step. Once the code exists, though, the developer must carefully analyze the life spans of all allocated objects in terms of the existing code. The goal is to insert free statements (or their analogues) to match the corresponding malloc (or

new) expressions. Critically, these *free* statements should mark locations as available neither too early nor too late, because, as explained, both create problems during program execution.

Now let's analyze the row using two different variants: the programs of beginners and the large code bases of teams of experienced developers. In the context of a small program, inserting *free* at the correct places seems to be a manageable task as the numerous introductory courses using C or C++ demonstrate. Experience teaches us, though, that (1) people often tend to ignore *free* in such courses; (2) beginners get these placements wrong and then struggle; and (3) if they do get them right, they draw the wrong conclusion about the complexity of the task.

Even experienced developers have a hard time keeping track of all the pointers that their large modules and packages allocate. The problem is that allocated objects flow from one module or package to another, and these various packages are created by distinct developers. Lacking checked notation for these flows of pointers, developers must use comments to document them; furthermore, as the code evolves over time, they are obliged to update the comments to make sure they remain correct. Even if these comments are always correct and properly observed, it is still up to each module or package creator to insert *free* at the proper place into the program text. Given the size of real-world code bases and the large quantity of allocated objects in software systems, it should not come as a surprise that developers lose track of when to free them. As a result, their programs leak memory, access dangling pointers, or both.

In response, companies have developed tools to detect such memory management problems but these tools are like band-aids, covering a deep wound. Put succinctly, when developers manage memory manually, it seriously complicates the design of functionality, overburdening both beginners and experienced developers with a bookkeeping task to which software is much better suited than human brains.

When it comes to *performance issues*, filling in the cells in the table of pragmatics becomes complicated due to the social nature of programmer communities. Indeed, the performance analysis of manual vs automatic memory management is just one more illustrative example of how a "consensus approach" to science fails people who must rely on it.

For several decades, the vast majority of language researchers and software developers maintained that programs using automatic memory management via garbage collection ran slower than equivalent programs using manual memory management. While they admitted that garbage collected

The second author is grateful to Will Clinger who interacted with Zorn and retold his story.

languages came with other advantages with regards to software development, they insisted that the critical nature of performance dominated all other aspects. Support for these claims consisted of anecdotes; actual evidence did not exist. Creators focused on languages with automatic memory management kept on working on their products, expecting that one day someone would debunk this claim.

In the late 1980s and 1990s Benjamin Zorn, then a faculty member at Colorado, began to investigate this claim systematically. He selected a small number of C/C++ programs, written by well-known and highly respected software developers. All of these programs came with carefully placed free statements, and over time, they had served their purpose well and in a robust and reliable manner. Zorn modified the run-time library for these programs in two ways:

- First, he turned `free` into an operation without effect.
- Second, he had `malloc` call a conservative garbage collector to find available locations when the consecutive block of such locations had been exhausted.

To evaluate the performance of the garbage collector, Zorn ran the original programs and the modified ones on the same inputs.

The results of Zorn's performance evaluation surprised him and everyone else. Even though the conservative collector cannot count on the language implementation for any assistance, the modified programs ran faster than the original ones. Indeed, the difference was so significant that Zorn did not trust the measurements at first. Once he had confirmed the measurements, he even worked on the original programs to improve their performance, because he expected that the reviewers of journals and conferences would not accept his work otherwise.

The interested reader may check out the story of Galileo whose scientific peers convinced the pope to suppress his support for the idea that the Earth rotates around the Sun.

Given that Zorn's evaluation was the first systematic one—eliminating all experimental variables other than `free` and `malloc`—you may expect that the scientific community welcomed his work. History shows, however, that the scientific community opposes such counter-intuitive results. And indeed, Zorn had a hard time publishing his results, due to opposition of his peer reviewers and their embrace of the established scientific consensus.

The attitude of the developer community changed only once Java successfully conquered IBM, the largest software company at the time. IBM's decision to create all new products in this new language affected the market more than any scientific evidence. One of IBM's teams had demonstrated that it could create software more easily with Java than C++ and

that the resulting product was robust, reliable, and performant. At the time, Java's implementation still used a conservative collector but was soon after equipped with a bespoke collector that exploited the language's many hints concerning locations in the store.

At this point in time, languages with automatic memory management are widely accepted and used in industry as well as in academia. Even many performance-critical systems use such languages, as already mentioned in the cost/benefit analysis. This is not to say that they come without any problems of their own or that they are embraced universally. People have created too many large systems with C/C++, and they cannot equip those with garbage collectors at a reasonable cost. Some hardware devices are also too small to accommodate a run-time system with a garbage collector. And for some use cases, a language without collector will do fine.

Finally, practicing software developers cannot lose sight of the cost of automatic memory management. Every explicit or implicit allocation of locations—via, say, `new` or a higher-order function—consumes available locations. Eventually the run-time system must pay for this consumption with a collection, an event that happens at essentially unpredictable, and possibly inconvenient, moments during execution.

The various problems of garbage collection show up every so often and cause developers to complain. Like all problems in the world of programming languages, they motivate researchers to continue their investigations of programming languages and their pragmatics. Because of such research, software developers should always keep an eye open to catch new development in programming languages.

Exercise 107. Pick another row in the table of figure 91. Discuss with a peer the pragmatics of automatic vs manual management for the two cells in the chosen row.

Jason: 7/10

Postlude

We have reached the end of one possible tour through the landscape of programming languages. The tour started with the basics from the 1960s, and it ended in the 2010s in chapter IX, with the emergence of languages with migratory type systems. Considering the complexity of this landscape, this tour is far from the only one that we could have taken. Chapter X illustrates this point; memory management becomes relevant as soon as a programming language comes with dynamic allocation such as the creation of objects from classes. Experienced instructors can easily cover this fork after working through chapter VI. As a matter of fact, they can explore many other forks in the road once this complex material on classes is mastered; almost every chapter offers several such opportunities.

Keep in mind, though, that the goal of this tour is not to belabor specific concepts that you find in programming languages. While we do not deny the importance of say, understanding a migratory type system and its pragmatics, our goal for you is different. The goal is to develop a framework that helps you assess the pragmatics of whatever language you will encounter over the course of your professional career or whatever new language feature you will confront, wish for, and discuss with colleagues—seriously or over a beer.

Pragmatics in Action

As chapter I.2 points out, software developers rarely get to pick the programming language for a project. They do join teams that work on an already existing, large code base. Hence the question is how an understanding of pragmatics helps these developers.

The short answer is that an understanding of the pragmatics of a programming language enables developers to make it and its ecosystem work *for* them. By contrast, a failure to understand pragmatics often makes programmers work *around* or even *against* the language.

The long answer requires us to go back to the key point of chapter I.2 and look at a work situation that has not shown up explicitly in tables such as the one in figure 91:

Developers often join a project team whose code base is written in a programming language that they do not know yet.

This developer could be a student on an internship or an experienced developer. No matter, the situation calls for learning a new language, and we all know how people go about this task:

When developers must acquire a new programming language, they tend to relate it to those they already know.

Given that people acquire a second natural language in the same manner, this statement should not come as a surprise.

So imagine programmers learning language L and basing it on their experience with language K. They will encounter three different situations:

1. a feature of L differs from the corresponding one in K;
2. K lacks a prominent feature of L, possibly for a good reason; and
3. L might benefit from a feature because it exists in K or because a developer's experience suggests it should be added.

Let's look at each of these situations in turn.

When Related Linguistic Constructs Differ First, a programmer leaving behind K to learn L is going to start with the kernel of the language. This kernel often resembles the *Core* model and also comes with several different forms of looping constructs, e.g. `do`, `for`, and `repeat`. Given this context, imagine a programmer who is familiar with, say, Java's `for` loops and encounters the following variable declaration in L:

```
Integer sum = for(+) (Integer x : Array[Integer] a) x;
```

At first glance, this `for` loop looks somewhat familiar, using the same header format as a so-called "for each" loop in Java. But, strangely enough its body is an expression—a variable reference—and its header has an extra clause: `(+)`. The suggestive name of the left-hand side variable and a moment's thought tells the newcomer to L that this loop returns a value, namely, the sum of all elements of `a`.

When a developer encounters such a construct, it raises at least two questions, namely, (1) whether it is possible to pass in any function instead of just `+` to accumulate values and (2) why loops return values when they don't in most languages. Expressed in terms of pragmatics, the first question becomes one about work situations, namely, in which work situations a developer should favor this new expressive power. A bit of experimentation will verify that this `for` loop is indeed quite powerful:

```
Integer product = for(*) (Integer x : Array[Integer] a) x;
```

Changing a single character turns the summation expression into a product expression. By implication, the second question really is how such looping constructs change a developer's thinking about, say, code design and perhaps maintenance tasks, to pick two rows in a table of pragmatics.

Stop! If you are curious about such loops, take a look at the Pyre language, our use of Racket's `for/set` in chapter IV.3.1 plus its many relatives in the language's documentation, and the Verse language of Epic Games.

When the New Language Supports an Unknown Feature Second, a programmer moving on from K to L may find that the two share a syntax, but surprisingly, L lacks K's *free* function, which was introduced in chapter X. After some digging, the programmer discovers that K comes with a new feature called "garbage collection," which is said to replace *free*.

Clearly, the programmer's first task is to find read up on garbage collection. Doing so may involve a semi-technical presentation such as chapter X, a blog post, or an edited collection of research papers. The second task is to understand the basic cost/benefit analysis behind this new feature from the perspective of a software developer. The third, critical task is to contemplate how this replacement feature affects the developers across all the work situations they encounter.

In the end, the programmer should end up with a table of pragmatics like the one in figure 91. The content of some of the cells will depend on the programming era and the programmer's willingness to dig out relevant information. As the preceding chapter spells out, insights about some of the cells in the table took a long time to emerge. For example, if a programmer had developed a table of pragmatics in the 1970s, the row concerning performance would have biased the decision in favor of going back to K; by contrast, the experiences of a large number of developers in the 1990s would have informed a programmer in the 2000s that garbage collection is a solid feature that helps programmers in almost all work situations.

Stop! Read up on the history of Java, how its design explicitly rejects certain C++ features, and how it got C++ developers to transition. Try to find literature on how the presence of a garbage collector in Java changed how C++ developers had to approach program design.

When the New Language Might Benefit from a Feature Finally, experienced and reflective developers may recognize that L could benefit from a feature of K, some other programming language, or even a completely new

one. To understand this point properly, let's consider two examples. While the first one is about pointers as presented in chapter X and found in older languages, the second concerns the reduction of expressive power due to the constraints of a static type system as presented in chapter VIII.

Some programming languages do not treat objects as values that react only to some definitive set of operations but as pointers, and they support arithmetic of pointers. Here "arithmetic" means that a pointer is a natural number and that programmers can write arithmetic expressions that involve pointers. You may now wonder how such pointer-arithmetic expressions could be useful. One prominent use of pointer arithmetic is to reduce the cost of method calls. Developers can check how the compiler lays out objects in memory and then perform arithmetic on the object's address to turn a method call into a fast, direct jump, which avoids the overhead of dynamic dispatch.

Stop! What could possibly go wrong with such computations? Once you have an answer, consider how you would fill in the rows of program design and code creation in a table of pragmatics.

Suppose *K* lacks a type system, and its syntactic twin, *L*, comes with a static type system, like the one presented in chapter VIII. A programmer moving from the first language to the second is going to notice many inconveniences. For example, the programmer can represent a list of students records in *K* with the same data structure as a list of instructor pay checks. Similarly, the programmer can use the same sort function in *K* to order the first list according to the students' GPAs and the second list according to the monthly gross pay. By contrast, once this programmer works with *L*, the same kind of program needs *two* distinct data structures to represent two lists with distinct element data and *two copies* of the sort method. An experienced programmer is likely to call for a revision of the type system to remove these forms of code duplication; in this day and age, the programmer will specifically appeal to the addition of generics to the type system.

Stop! Read up on the history of adding generics to Java.

One way to tackle such a situation is for experienced programmers to develop a model, like those presented in this book. As demonstrated, the model development is going to help with a cost/benefit analysis and, to some extent, is going to shed light on the feature's pragmatics. Another way to address such a problem is to articulate a feature request supported by a table of pragmatics. If a syntactically similar language exists and supports the desired feature, a table of pragmatics may include two columns; otherwise focusing on how a feature affects work suffices.

A developer who performs this work for a feature request may discover reasons why L's creators excluded the feature or may experience a warm welcome of the feature request:

- Pointer arithmetic can easily go wrong. Every programmer knows about off-by-1 calculations and how easy it is to make these mistakes during program design or code creation. When such a buggy calculation involves pointers, the hardware is going to retrieve meaningless bits—and doing so may end up producing inappropriate outputs or segfaults. Over the past few decades, many programmers have suffered from these bad outcomes. The creators of Java thoroughly investigated this pointer-arithmetic question, and their analysis of its pragmatics delivered a negative answer.
- By contrast, the creators of Java welcomed both models of generics and positive arguments based on pragmatics. Starting in the late 1990s, several research teams, some with working software developers on board, produced Java models with generics. Some models came as executable code, others in the form of operational mathematics. Due to commercial constraints, Java's creators picked the variant of generics that did not require any changes to the underlying (virtual) machine.

In either case, performing this kind of leg work for missing features is illuminating and rewarding at the same time.

Stop! The community around the Rust programming language established one of the most well-known set of rules for feature requests. A fair number of Rust's features are the result of such feature requests. Take a look at some of those requests, and identify the arguments based on pragmatics in these write-ups. While such requests do not come with the kind of tables presented here, you will find natural-language descriptions of some of the cells that correspond to work situations.

Where to Go From Here

Soon you will be, or perhaps you already are, a professional software developer. Almost every developer will end up in a conversation that calls for an evaluation of a programming language or a programming language feature. These conversations range from professional tasks to chance encounters at "meets ups" or "special interest groups" over a light dinner

and beer. No matter what, this book equips you with the intellectual tools to address such questions with an appropriate degree of seriousness.

The first point to make is that *every* question about programming languages, their features, and the tools in their ecosystem comes with a multi-dimensional answer. Conversely, it is impossible to answer such questions with a plain “yes” or “no.” At a minimum, the answer must involve a cost/benefit analysis as well as an analysis of the various work situations found in our tables of pragmatics.

The second point calls for a discussion of the costs and benefits of the language, the feature, or the tool under consideration. Keep in mind that costs and benefits apply to both sides: the creators and the users. It is easy to ask for “pie in the sky” without considering the cost. But, a considerate answer takes into account the work that someone else must perform on our behalf to create this “pie” and deliver it in the “sky.” Furthermore, a solid answer involves some creative thinking, namely, a consideration of alternatives and their comparable costs and benefits.

The final point concerns the pragmatics. While the construction of a complete table is often superfluous, identifying those situations where the impact of a language feature on the work of developers is most clearly visible. Our tables list all those work situations that are needed to assess the pragmatics of languages for our path through the landscape of languages. For other features, you may need to consider additional situations. The key is, however, that no single work situation should dominate the answer:

- If one participant in a conversation emphasizes performance, ask for a consideration of code design.
- To a participant who insists that some property of a language cuts down the search space for bugs, pose questions about the performance impact.
- No matter what, ask for examples, illustrations, systematic assessments, or even proofs. Don’t be satisfied with hand-waving.

By working through this book, its exercises, and its projects you have acquired the intellectual framework for evaluating the pragmatics of the most important tool for your work: the language with which you create software systems. Practice its use. Reach for it during these technical conversations you will inevitably have. And refine it and improve it as you learn more. Just don’t ever stop investing in this learning process.

Jason: 7/10

Acknowledgments

We gratefully acknowledge the many people who pushed back on our ideas of pragmatics over the past decades: Lukas Lazarek, Ben Greenman, Michael Ballantyne, Ryan Culepper, Cameron Moy, and Ben Lerner.

Thanks to Hans-J. Boehm and Will Clinger for sharing the background stories of conservative collection and Benjamin Zorn's performance evaluation of garbage collection.

The students in our courses who suffered through the first few drafts of this book, and especially those who provided feedback: Anfisa Bogdanenko, ...

Jason: 7/10